

לוגיקה בוליאנית – Boolean Logic

נתמקד בבניית משפחה של ציפים פשוטים שנקראים **שערים בוליאניים (Boolean Gates)**. כיוון ששערים בוליאניים הם מימושים פיזיים של **פונקציות בוליאניות (Boolean Functions)**, נתחיל מאלגברה בוליאנית. לאחר מכן, נראה איך ניתן לחבר שערים בוליאניים המממשים פונקציות בוליאניות פשוטות זה לזה כך שהם יספקו פונקציונליות של ציפים מורכבים יותר.

אלגברה בוליאנית – Boolean Algebra

באלגברה בוליאנית נשתמש בערכים בוליאניים (נקראים גם בינאריים) המסומנים בדרך כלל בתור on/off, yes/no, 1/0, true/false וכו'. אנחנו נשתמש ב-0 וב-1. פונקציה בוליאנית היא פונקציה הפועלת על קלטים בינאריים ומחזירה פלטים בינאריים.

ייצוג בטבלת אמת – Truth Table Representation: הדרך הפשוטה ביותר לתאר פונקציה בוליאנית היא למנות את כל הערכים האפשריים עבור הקלט של הפונקציה יחד עם הפלט של הפונקציה עבור כל קבוצה של קלטים. זה נקרא ייצוג של טבלת אמת.

דוגמה:

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

3 הטורים השמאליים מתארים את כל הערכים הבינאריים האפשריים עבור המשתנים של הפונקציה f . עבור כל אחד מ- 2^n ה-tuples האפשריים $v_1, \dots, v_n$ (בדוגמה $n = 3$), הטור הימני מתאר את הערך של $f(v_1, \dots, v_n)$.

ביטויים בוליאניים – Boolean Expressions: ניתן לתאר פונקציה בוליאנית ע"י שימוש בפעולות בוליאניות על משתני הקלט של הפונקציה.

האופרטורים הבוליאניים הבסיסיים בהם משתמשים בדרך כלל הם **"And"** (כאשר x And y הוא 1 כאשר גם x וגם y הם 1), **"Or"** (כאשר x Or y הוא 1 כאשר x או y או שניהם 1), ו-**"Not"** (כאשר $Not\ x$ הוא 1 כאשר x הוא 0).

נשתמש בנוטציות אריתמטיות עבור הפעולות הללו: $x \cdot y$ זה x And y , $x + y$ זה x Or y , ו- $\bar{x}$ זה $Not\ x$.

דוגמה: הפונקציה המתוארת בטבלת האמת מהדוגמה הקודמת שקולה לביטוי הבוליאני

$$f(x, y, z) = (x + y) \cdot \bar{z}$$

למשל, נעריך את הביטוי עבור הקלט $x = 0, y = 1, z = 0$: כיוון ש- $y = 1$, נובע כי $x + y = 1$ ולכן $1 \cdot \bar{0} = 1 \cdot 1 = 1$.

כדי להשלים את הוכחת השקילות בין הביטוי ובין טבלת האמת, צריך לחשב את ערך הביטוי עבור כל אחת מ-8 הקומבינציות האפשריות של הקלט, ולוודא שאכן מתקבל אותו הערך שמופיע בטור הימני של הטבלה.

ייצוג קנוני – Canonical Representation: ניתן לבטא כל פונקציה בוליאנית ע"י שימוש בלפחות

ביטוי בוליאני אחד שנקרא הייצוג הקנוני.

נתמקד בשורות בהן הערך של הפונקציה הוא 1. עבור כל שורה כזו, נבנה ביטוי שנוצר ע"י And-ים של משתנים או שלילתם המתאים לערכי הקלט של כל שורה. לאחר מכן, אם נוסיף Or בין כל הביטויים שנוצרו, נקבל ביטוי בוליאני השקול לטבלת האמת.

דוגמה: אם נתמקד בשורה השלישית של הטבלה מהדוגמה הראשונה, שבה ערך הפונקציה הוא 1, כיוון שערכי הקלט הם $x = 0, y = 1, z = 0$, נבנה את הביטוי $\bar{x} \cdot y \cdot \bar{z}$. באופן דומה, נבנה את הביטוי $x \cdot \bar{y} \cdot \bar{z}$ עבור השורה החמישית ואת הביטוי $x \cdot y \cdot \bar{z}$ עבור השורה השביעית. נקבל שהייצוג הקנוני של הפונקציה הבוליאנית המתוארת בטבלה הוא:

$$f(x, y, z) = \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z}$$

מסקנה: ניתן לבטא כל פונקציה בוליאנית ע"י 3 אופרטורים בוליאניים בלבד: Or, And ו-Not.

פונקציות בוליאניות עם 2 קלטים: מספר הפונקציות הבוליאניות שניתן להגדיר מעל n משתנים

בינאריים הוא 2^n .

לדוגמה, קיימות 16 פונקציות בוליאניות עם שני משתנים כקלט:

Function	x	0	0	1	1
	y	0	1	0	1
Constant 0	0	0	0	0	0
And	$x \cdot y$	0	0	0	1
x And Not y	$x \cdot \bar{y}$	0	0	1	0
x	x	0	0	1	1
Not x And y	$\bar{x} \cdot y$	0	1	0	0
y	y	0	1	0	1
Xor	$x \cdot \bar{y} + \bar{x} \cdot y$	0	1	1	0
Or	$x + y$	0	1	1	1
Nor	$\overline{x + y}$	1	0	0	0
Equivalence	$x \cdot y + \bar{x} \cdot \bar{y}$	1	0	0	1
Not y	$\bar{y}$	1	0	1	0
If y then x	$x + \bar{y}$	1	0	1	1
Not x	$\bar{x}$	1	1	0	0
If x then y	$\bar{x} + y$	1	1	0	1
Nand	$\overline{x \cdot y}$	1	1	1	0
Constant 1	1	1	1	1	1

לפונקציה Nand (הידועה גם בתור הפונקציה Nor) יש תכונה מעניינת: אפשר לבנות ממנה את כל אחת מהפונקציות Or, And ו-Not.

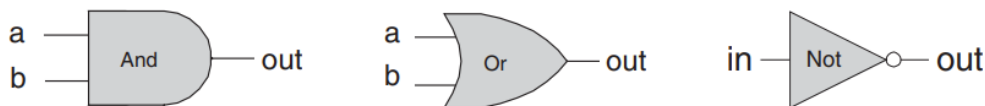
דוגמה: $x \text{ Or } y = (x \text{ Nand } x) \text{ Nand } (y \text{ Nand } y)$

כיוון שניתן לבנות כל פונקציה בוליאנית מ-And, Or ו-Not, נובע כי ניתן לבנות כל פונקציה בוליאנית מפונקציות Nand בלבד.

שערים לוגיים – Gate Logic

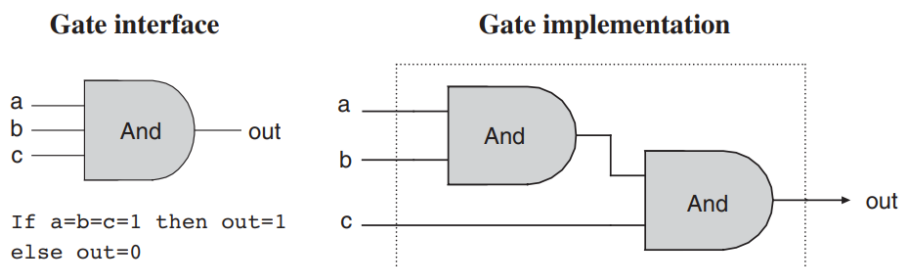
שער (Gate) הוא מכשיר פיזי שמממש פונקציה בוליאנית. אם פונקציה בוליאנית f פועלת על n משתנים ומחזירה m תוצאות בינאריות, לשער שמממש את f יהיו n כניסות ו- m יציאות. כאשר נשים ערכים כלשהם $v_1, \dots, v_n$ בכניסות של השער, השער צריך לחשב ולהחזיר כפלט את $f(v_1, \dots, v_n)$. השערים הפשוטים ביותר נקראים טרנזיסטורים (Transistors). אנחנו נשתמש במונח שער כדי לתאר ציפים פשוטים.

סימונים של שערים לוגיים בסיסיים:



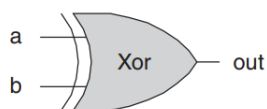
שערים פרימיטיביים ושערים מורכבים: כיוון שלכל השערים הלוגיים יש את אותה הסמנטיקה של קלט ופלט (כלומר הקלט והפלט הם 0 ו-1), אפשר לחבר שערים וליצור כך שערים מורכבים יותר.

דוגמה: נניח שאנחנו רוצים לממש את הפונקציה הבוליאנית $And(a, b, c)$. נשים לב ש- $a \cdot b \cdot c = (a \cdot b) \cdot c$, כלומר $And(a, b, c) = And(And(a, b), c)$. נוכל לבנות את השער הלוגי המתאים באופן הבא:

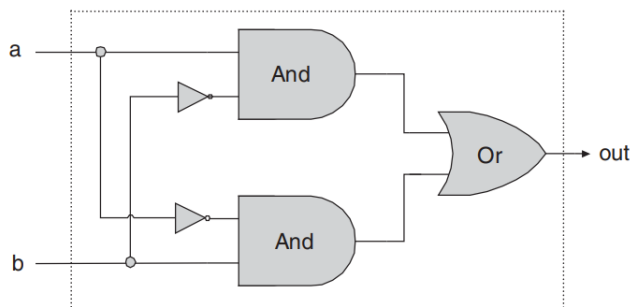


נשים לב שניתן להסתכל על כל שער לוגי משתי נקודות מבט שונות: חיצונית ופנימית. מימין, נוכל לראות את הארכיטקטורה הפנימית של השער, כלומר את המימוש. משמאל נוכל לראות את הממשק של השער, כלומר את ה-pins של הקלט והפלט כפי שהן נראות כלפי חוץ.

דוגמה – השער Xor: מתקיים כי $Xor(a, b) = 1$ כאשר $a = 1$ ו- $b = 0$ או כאשר $a = 0$ ו- $b = 1$. לכן, $Xor(a, b) = Or(And(a, Not(b)), And(Not(a), b))$, כלומר:



a	b	out
0	0	0
0	1	1
1	0	1
1	1	0



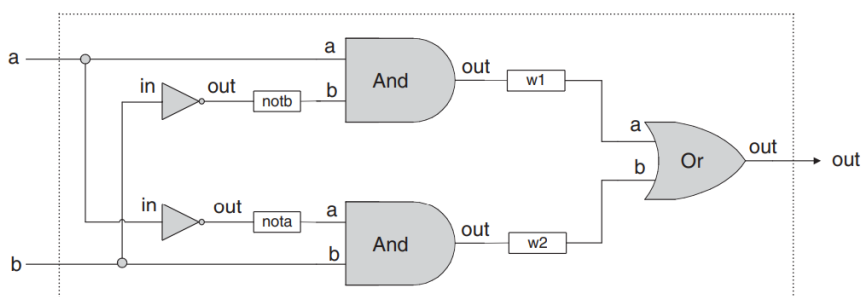
בהינתן ממשק של שער, נרצה למצוא דרך יעילה לממש אותו בעזרת שערים אחרים שכבר מימשנו, כאשר נרצה להשתמש בכמה שפחות שערים שאפשר.

Hardware Description Language (HDL)

נתכנן ונבנה ציפים ע"י כתיבת קוד HDL, כאשר כלי הנקרא hardware simulator משמש כדי לבדוק הציפים.

הגדרת ציפ ב-HDL מכילה header ו-parts. ה-header מפרט את הממשק של הציפ (כלומר את שם הציפ ואת השמות של ה-pins של הקלט והפלט). ה-parts מתאר את השמות ואת הטופולוגיה של כל החלקים (ציפים אחרים) מהם הציפ בנוי. כל חלק מיוצג ע"י הצהרה שמפרטת את שם החלק ואת הדרך בה הוא מחובר לחלקים האחרים. חיבורים של חלקים פנימיים מתוארים ע"י ציורה וחיבור של internal pins במידת הצורך.

דוגמה – בניית שער Xor:



<i>HDL program</i> (Xor.hdl)	<i>Test script</i> (Xor.tst)	<i>Output file</i> (Xor.out)															
<pre>/* Xor (exclusive or) gate: If a<>b out=1 else out=0. */ CHIP Xor { IN a, b; OUT out; PARTS: Not(in=a, out=nota); Not(in=b, out=notb); And(a=a, b=notb, out=w1); And(a=nota, b=b, out=w2); Or(a=w1, b=w2, out=out); }</pre>	<pre>load Xor.hdl, output-list a, b, out; set a 0, set b 0, eval, output; set a 0, set b 1, eval, output; set a 1, set b 0, eval, output; set a 1, set b 1, eval, output;</pre>	<table> <thead> <tr> <th>a</th><th>b</th><th>out</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	a	b	out	0	0	0	0	1	1	1	0	1	1	1	0
a	b	out															
0	0	0															
0	1	1															
1	0	1															
1	1	0															

Specification

שער Nand: נקודת ההתחלה של ארכיטקטורת המחשב שלנו היא שער Nand, ממנו נבנה את כל שאר השערים והצ'יפים. שער Nand מחשב את הפונקציה הבוליאנית הבאה:

a	b	$\text{Nand}(a, b)$
0	0	1
0	1	1
1	0	1
1	1	0

כאשר ה-API של הצ'יפ הוא:

```
Chip name: Nand
Inputs:    a, b
Outputs:   out
Function:  If a=b=1 then out=0 else out=1
Comment:  This gate is considered primitive and thus there is
          no need to implement it.
```

Not: שער עם קלט אחד הממיר את הקלט שלו מ-0 ל-1 ולהפך. ה-API שלו הוא:

```
Chip name: Not
Inputs:    in
Outputs:   out
Function:  If in=0 then out=1 else out=0.
```

And: פונקציית ה-And מחזירה 1 כאשר שני הקלטים שלה הם 1, ו-0 אחרת.

```
Chip name: And
Inputs:    a, b
Outputs:   out
Function:  If a=b=1 then out=1 else out=0.
```

Or: פונקציית ה-Or מחזירה 1 כאשר אחד מהקלטים שלה הוא 1, ו-0 אחרת.

```
Chip name: Or
Inputs:    a, b
Outputs:   out
Function:  If a=b=0 then out=0 else out=1.
```

Xor: פונקציית ה-Xor ("exclusive or") מחזירה 1 כאשר לשני הקלטים שלה יש ערכים הפוכים, ו-0 אחרת.

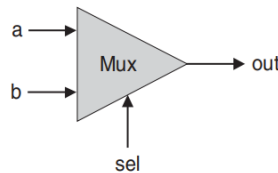
Chip name: Xor
Inputs: a, b
Outputs: out
Function: If $a \neq b$ then $out=1$ else $out=0$.

Multiplexor: שער בעל 3 קלטים המשתמש באחד מהקלטים שנקרא "selection bit" כדי לבחור את אחד משני הקלטים האחרים ולהחזיר אותו כפלט.

Chip name: Mux
Inputs: a, b, sel
Outputs: out
Function: If $sel=0$ then $out=a$ else $out=b$.

a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

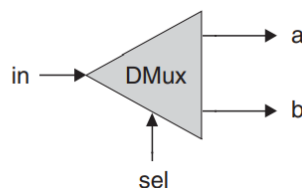
sel	out
0	a
1	b



Demultiplexor: מבצע את הפונקציה ההפוכה של multiplexor – מקבל קלט יחיד ו-selector-bit שלפיו הוא בוחר פלט מבין שתי אפשרויות.

Chip name: DMux
Inputs: in, sel
Outputs: a, b
Function: If $sel=0$ then $\{a=in, b=0\}$ else $\{a=0, b=in\}$.

sel	a	b
0	in	0
1	0	in



החומרה של המחשב אמורה לפעול על מערכים עם מספר ביטים הנקראים "buses". למשל, דרישה בסיסית של מחשב 32-ביט היא לחשב (bit-wise) פונקציית And של שני buses של 32-ביט. כדי לממש את הפעולה הזו, נוכל לבנות מערך של 32 שערי And בינאריים, כאשר כל אחד מהם יפעל בנפרד על זוג של ביטים. נוכל לעשות זאת בצ'יפ המורכב משני buses של 32-ביט בקלט ו-bus אחד של 32-ביט כפלט.

כאשר נרצה להתייחס לביטים בודדים ב-bus, נשתמש בסינטקס של מערך. לדוגמה: כדי לגשת לביטים בודדים ב-bus של 16-ביט שנקרא data, נסמן: $data[0], data[1], \dots, data[15]$.

16-Bit Not: שער Not של 16-ביט מקבל כקלט bus של 16-ביט ומבצע את הפעולה הבוליאנית Not על כל אחד מהביטים של הקלט:

```
Chip name: Not16
Inputs:    in[16] // a 16-bit pin
Outputs:   out[16]
Function:  For i=0..15 out[i]=Not(in[i]).
```

16-Bit And: שער And של 16-ביט מקבל כקלט שני buses של 16-ביט ומבצע את הפעולה הבוליאנית And על כל זוג ביטים:

```
Chip name: And16
Inputs:    a[16], b[16]
Outputs:   out[16]
Function:  For i=0..15 out[i]=And(a[i],b[i]).
```

16-Bit Or: שער Or של 16-ביט מקבל כקלט שני buses של 16-ביט ומבצע את הפעולה הבוליאנית Or על כל זוג ביטים:

```
Chip name: Or16
Inputs:    a[16], b[16]
Outputs:   out[16]
Function:  For i=0..15 out[i]=Or(a[i],b[i]).
```

16-Bit Multiplexor: בדיוק כמו ה-multiplexor הבינארי, רק ששני הקלטים הם 16-ביט וה-selector הוא עדיין ביט אחד.

```
Chip name: Mux16
Inputs:    a[16], b[16], sel
Outputs:   out[16]
Function:  If sel=0 then for i=0..15 out[i]=a[i]
           else for i=0..15 out[i]=b[i].
```

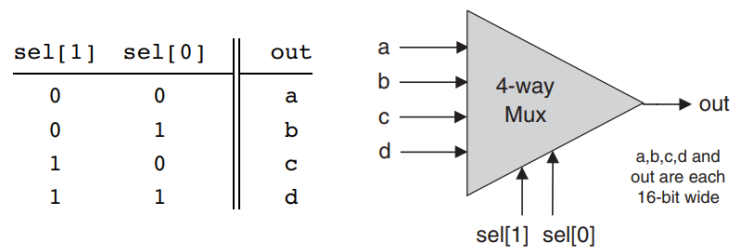
8-Way Or: מחזיר 1 כאשר לפחות אחד מ-8 הביטים בקלט הוא 1 ו-0 אחרת:

```
Chip name: Or8Way
Inputs:    in[8]
Outputs:   out
Function:  out=Or(in[0],in[1],...,in[7]).
```

m-Way n-Bit Multiplexor: בוחר אחד מתוך m ה-buses של n-ביט שהוא מקבל כקלט ומחזיר אותו כפלט. הבחירה מתבצעת לפי $k = \log_2 m$ ביטים הנקראים control bits. נבנה שתי גרסאות:

Chip name: Mux4Way16
Inputs: a[16], b[16], c[16], d[16], sel[2]
Outputs: out[16]
Function: If sel=00 then out=a else if sel=01 then out=b else if sel=10 then out=c else if sel=11 then out=d
Comment: The assignment operations mentioned above are all 16-bit. For example, "out=a" means "for i=0..15 out[i]=a[i]".

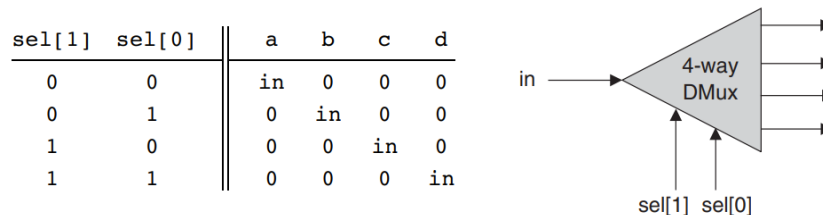
Chip name: Mux8Way16
Inputs: a[16], b[16], c[16], d[16], e[16], f[16], g[16], h[16], sel[3]
Outputs: out[16]
Function: If sel=000 then out=a else if sel=001 then out=b else if sel=010 then out=c ... else if sel=111 then out=h
Comment: The assignment operations mentioned above are all 16-bit. For example, "out=a" means "for i=0..15 out[i]=a[i]".



m-Way n-Bit Demultiplexor: מקבל קלט של n-ביט ובוחר מתוך m אפשרויות פלט של n-ביט לפי $k = \log_2 m$ ביטים הנקראים control bits. נבנה שתי גרסאות:

Chip name: DMux4Way
Inputs: in, sel[2]
Outputs: a, b, c, d
Function: If sel=00 then {a=in, b=c=d=0}
 else if sel=01 then {b=in, a=c=d=0}
 else if sel=10 then {c=in, a=b=d=0}
 else if sel=11 then {d=in, a=b=c=0}.

Chip name: DMux8Way
Inputs: in, sel[3]
Outputs: a, b, c, d, e, f, g, h
Function: If sel=000 then {a=in, b=c=d=e=f=g=h=0}
 else if sel=001 then {b=in, a=c=d=e=f=g=h=0}
 else if sel=010 ...
 ...
 else if sel=111 then {h=in, a=b=c=d=e=f=g=0}.



אריתמטיקה בוליאנית – Boolean Arithmetic

בפרק הזה נבנה שערים לוגיים המייצגים מספרים ומבצעים פעולות אריתמטיות עליהם. נתחיל מהשערים הלוגיים שבנינו בפרק הקודם, ונסיים בבניית Arithmetic Logical Unit (ALU) – צ'יפ מרכזי שמבצע את כל הפעולות האריתמטיות והלוגיות של המחשב.

מספרים בינאריים

בניגוד למערכת הדצימלית שהבסיס שלה הוא 10, הבסיס של המערכת הבינארית הוא 2.

כאשר אנחנו מקבלים תבנית בינארית כלשהי, למשל "10011", והתבנית הזו אמורה לייצג מספר integer כלשהו, הערך הדצימלי של המספר הזה יחושב באופן הבא:

$$(10011)_{two} = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 19$$

באופן כללי, יהי $x = x_n x_{n-1} \dots x_0$ רצף של ספרות. הערך של x בבסיס b , שנסמן אותו ע"י $(x)_b$, מוגדר באופן הבא:

$$(x)_b = (x_n x_{n-1} \dots x_0)_b = \sum_{i=0}^n x_i \cdot b^i$$

חיבור בינארי: חיבור של שני מספרים בינאריים מתבצע ע"י חיבור של כל שתי ספרות, מימין לשמאל, כפי שלמדנו ביסודי. ראשית, נחבר את שתי הספרות הימניות ביותר של שני המספרים הבינאריים. הספרה הימנית ביותר נקראת ה-**Least Significant Bit (LSB)** של המספר הבינארי. לאחר מכן, נוסיף את ה-carry bit של התוצאה (שהוא או 0 או 1) לסכום של זוג הביטים הבאים. נמשיך כך, עד שנחבר את שתי הספרות השמאליות ביותר. הספרה השמאלית ביותר של מספר בינארי נקראת ה-**Most Significant Bit (MSB)** שלו. אם יש carry של 1 לאחר החיבור האחרון, יש overflow. אחרת, החיבור מסתיים בהצלחה.

דוגמה:

0	0	0	1	(carry)	1	1	1	1	
	1	0	0	x		1	0	1	1
+	0	1	0	y	+	0	1	1	1
0	1	1	1	$x + y$	1	0	0	1	0
no overflow					overflow				

Signed Binary Numbers: במערכת בינארית עם n ספרות יש 2^n תבניות בינאריות שונות. כדי לייצג signed numbers במספרים בינאריים, נרצה לחלק את המרחב לשתי תתי-קבוצות שוות בגודלן – אחת עבור ייצוג של מספרים חיוביים, והשנייה עבור ייצוג של מספרים שליליים.

כדי לעשות זאת, נשתמש בשיטת ה-**2's Complement**: במערכת בינארית עם n ספרות, המשלים ל-2 של המספר x מוגדר באופן הבא:

$$\bar{x} = \begin{cases} 2^n - x & \text{if } x \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

דוגמה: במערכת בינארית של 5-ביט, הייצוג של המסלילים ל-2 של -2 או $minus(00010)_{two}$ הוא:

$$2^5 - (00010)_{two} = (32)_{ten} - (2)_{ten} = (30)_{ten} = (11110)_{two}$$

כדי לוודא שהחישוב נכון אפשר לראות ש- $(00000)_{two} = (00010)_{two} + (11110)_{two}$.

* הערה: בחישוב הנ"ל הסכום הוא למעשה $(100000)_{two}$, אבל כיוון שאנחנו במערכת בינארית של 5-ביט, אנחנו מתעלמים מהביט השישי השמאלי ביותר.

באופן כללי, כאשר מבצעים את שיטת המסלילים ל-2 למספרים של n-ביט, הסכום של $x + (-x)$ תמיד יהיה 2^n (כלומר 1 שאחריו יש n 0-ים).

דוגמה – מערכת בינארית של 4-ביט עם שיטת המסלילים ל-2:

Positive numbers		Negative numbers	
0	0000		
1	0001	1111	-1
2	0010	1110	-2
3	0011	1101	-3
4	0100	1100	-4
5	0101	1011	-5
6	0110	1010	-6
7	0111	1001	-7
		1000	-8

תכונות של מערכת בינארית של n-ביט עם שיטת המסלילים ל-2:

- המערכת יכולה לייצג סה"כ 2^n מספרים, כאשר המספר המקסימלי הוא 2^{n-1} והמספר המינימלי הוא -2^{n-1} .
- הייצוג הבינארי של כל המספרים החיוביים מתחיל ב-0.
- הייצוג הבינארי של כל המספרים השליליים מתחיל ב-1.
- כדי לקבל את הייצוג של $-x$ מתוך הייצוג של x , נעבור על הייצוג הבינארי של x משמאל לימין, כאשר בכל פעם שניתקל ב-0 נשאיר אותו כפי שהוא, עד שניתקל לראשונה ב-1 (כלומר ב-1 הראשון משמאל). לאחר מכן, נהפוך את כל הביטים שניתקל בהם (כלומר נהפוך 0-ים ל-1-ים ולהפך). באופן שקול, אפשר להפוך את כל הביטים של x ואז להוסיף 1 לתוצאה.

נשים לב שחיבור של כל שני signed numbers המיוצגים בשיטת המסלילים ל-2 הוא בדיוק כמו חיבור של שני מספרים חיוביים.

דוגמה: אם נרצה לבצע את פעולת החיבור $(-2) + (-3)$, נבצע את פעולת החיבור של המספרים הבינאריים $(1110)_{two} + (1101)_{two}$. תוצאת החיבור תהיה 1011 (לאחר שנתעלם מהביט של ה-overflow, כיוון שאנחנו במערכת של 4-ביט). ואכן, 1011 הוא הייצוג של המסלילים ל-2 של -5 .

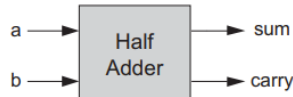
חיסור: בשיטת המסלילים ל-2, השלילה האריתמטית של מספר x היא החישוב של $-x$, והיא מתבצעת ע"י שלילת כל הביטים של x והוספת 1 לתוצאה. לכן, $x - y = x + (-y)$.

Specification

Half-Adder: חיבור שני ביטים, כאשר נקרא ל-LSB של התוצאה sum ול-MSB של התוצאה carry

```
Chip name: HalfAdder
Inputs:    a, b
Outputs:   sum, carry
Function:  sum = LSB of a + b
           carry = MSB of a + b
```

Inputs		Outputs	
a	b	carry	sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

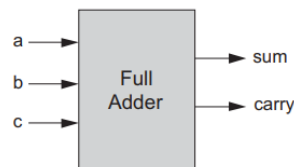


מימוש: נשים לב ש- $sum(a, b) = And(a, b)$ ו- $carry(a, b) = Xor(a, b)$, לכן המימוש נובע ישירות מהשערים הללו שבנינו בפרויקט הקודם.

Full-Adder: חיבור של 3 ביטים, כאשר באופן דומה ל-Half-Adder, מחזיר 2 פלטים – ה-LSB של תוצאת החיבור, וה-bit carry:

```
Chip name: FullAdder
Inputs:    a, b, c
Outputs:   sum, carry
Function:  sum = LSB of a + b + c
           carry = MSB of a + b + c
```

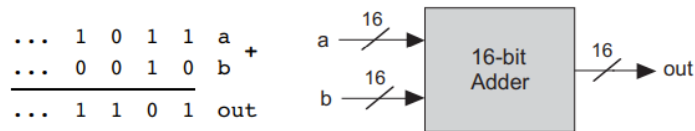
a	b	c	carry	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



מימוש: ניתן לממש את הציפ הזה באמצעות שני ציפים של Half-Adder ועוד שער אחד פשוט. אפשר גם לממש ישירות מבלי להשתמש בציפים של Half-Adder.

Adder: חיבור של שני מספרים של 16-ביט, כאשר נתעלם מ-overflow:

```
Chip name: Add16
Inputs:    a[16], b[16]
Outputs:   out[16]
Function:  out = a + b
Comment:  Integer 2's complement addition.
           Overflow is neither detected nor handled.
```



מימוש: ניתן לחבר שני signed numbers המיוצגים בשיטת המשלים ל-2 בתור שני buses של 16-ביט ע"י חיבור bit-wise מימין לשמאל ב-16 צעדים. בצעד 0 יתבצע החיבור של זוג ה-LSB, וה- carry bit יתווסף לחיבור של הביטים הבאים. נשים לב שבכל צעד יש חיבור של 3 ביטים. לכן, אפשר לממש את ה-Adder ע"י יצירת מערך של 16 ציפים של Full-Adder ופעפוע של ה-carry bits עד ל-MSB.

Incrementer: הוספת 1 למספר נתון של 16-ביט:

```

Chip name: Inc16
Inputs:   in[16]
Outputs:  out[16]
Function: out=in+1
Comment:  Integer 2's complement addition.
          Overflow is neither detected nor handled.

```

מימוש: ניתן לממש אותו באופן טריוויאלי מתוך Adder של 16-ביט.

Shift-Left: הפלט של הציפ הוא "left-shift" של הקלט – כל ביט בקלט זז מקום אחד שמאלה, ומכניסים ביט 0 חדש בתור הביט הימני ביותר. הפעולה הזו שקולה לכפל של הקלט ב-2.

Shift-Right: הפלט של הציפ הוא "right-shift" של הקלט – כל ביט בקלט זז מקום אחד ימינה, ומכניסים ביט חדש ששווה ל-bit sign בתור הביט השמאלי ביותר. הפעולה הזו שקולה לחלוקה של הקלט ב-2.

The Arithmetic Logic Unit (ALU)

ה-ALU של מחשב Hack מחשב קבוצה קבועה של פונקציות $out = f_i(x, y)$ כאשר x ו- y הם שני קלטים של 16-ביט, out הוא פלט של 16-ביט, ו- f_i היא פונקציה אריתמטית או לוגית הנבחרת מבין 18 פונקציות אפשריות. נודיע ל-ALU איזה פונקציה לחשב לפי הערכים של 6 ביטים שהוא מקבל בקלט הנקראים control bits.

כל אחד מה-control bits מנחה את ה-ALU לבצע פעולה בסיסית מסוימת. יחד, הם מאפשרים ל-ALU לחשב מגוון פונקציות שימושיות. למעשה, ה-ALU יכול לחשב $2^6 = 64$ פונקציות שונות.

טבלת האמת של ה-ALU:

These bits instruct how to preset the x input		These bits instruct how to preset the y input		This bit selects between + / And	This bit inst. how to postset out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x&y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x y

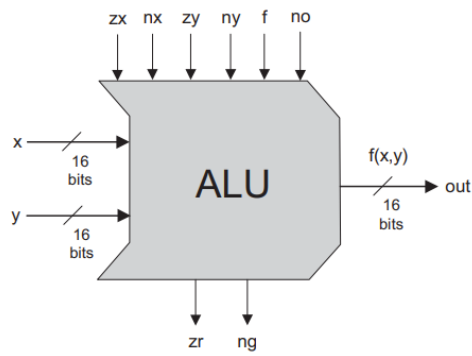
דוגמה: נתבונן בשורה המחשבת את הפונקציה $x - 1$. הביטים zx ו- nx הם 0, אז לא מאפסים או שוללים את הקלט x , והוא נשאר כפי שהוא. הביטים zy ו- ny הם 1, אז קודם מאפסים את הקלט y ואז שוללים אותו bit-wise. שלילה bit-wise של 0, כלומר שלילה של $(000 \dots 000)_{two}$, נותנת $(111 \dots 111)_{two}$ שהוא הייצוג במשלים ל-2 של -1. כיוון שהביט f הוא 1, הפעולה הנבחרת היא חיבור אריתמטי, ה-ALU יחשב את $x + (-1)$. לבסוף, הביט no הוא 0, לכן לא נשלול את הפלט אלא נשאיר אותו כפי שהוא. סה"כ, ה-ALU מחשב את $x - 1$, שזו הייתה המטרה.

ALU: הצ'יפ ייראה כך:

```

Chip name: ALU
Inputs:  x[16], y[16],      // Two 16-bit data inputs
         zx,                // Zero the x input
         nx,                // Negate the x input
         zy,                // Zero the y input
         ny,                // Negate the y input
         f,                 // Function code: 1 for Add, 0 for And
         no                 // Negate the out output
Outputs: out[16],          // 16-bit output
         zr,                // True iff out=0
         ng                 // True iff out<0
Function: if zx then x = 0      // 16-bit zero constant
         if nx then x = !x     // Bit-wise negation
         if zy then y = 0     // 16-bit zero constant
         if ny then y = !y     // Bit-wise negation
         if f then out = x + y // Integer 2's complement addition
           else out = x & y    // Bit-wise And
         if no then out = !out // Bit-wise negation
         if out=0 then zr = 1 else zr = 0 // 16-bit eq. comparison
         if out<0 then ng = 1 else ng = 0 // 16-bit neg. comparison
Comment:  Overflow is neither detected nor handled.

```



מימוש: השלב הראשון הוא ליצור מעגל לוגי שמשנה קלט של 16-ביט בהתאם לביטים zx ו- nx (כלומר המעגל צריך בהתאם לתנאים לאפס ולשלול את הקלט), והלוגיקה הזו יכולה לשמש כדי לשנות את הקלטים x ו- y וגם את הפלט out . כבר בנינו ציפים עבור And שמתבצע bit-wise וחיבור, לכן נשאר לנו לחשוב על לוגיקה שבוחרת שיניהם בהתאם לביט f . לבסוף, נצטרך לבנות לוגיקה שמשלבת את כל שאר הציפים ל-ALU הכולל.

לוגיקה רציפה – Sequential Logic

הצ'יפים הבוליאניים והאריתמטיים שבנינו בפרקים הקודמים חישובו פונקציות שתלויות בקלט שלהן, אבל הם לא יכולים לשמר מצב. כיוון שמחשבים צריכים לא רק לחשב ערכים אלא גם לאחסן אותם ולגשת אליהם, צריך שיהיו להם רכיבי זיכרון שיוכלו לשמור מידע לאורך זמן. רכיבי הזיכרון הללו בנויים מ-Sequential Chips.

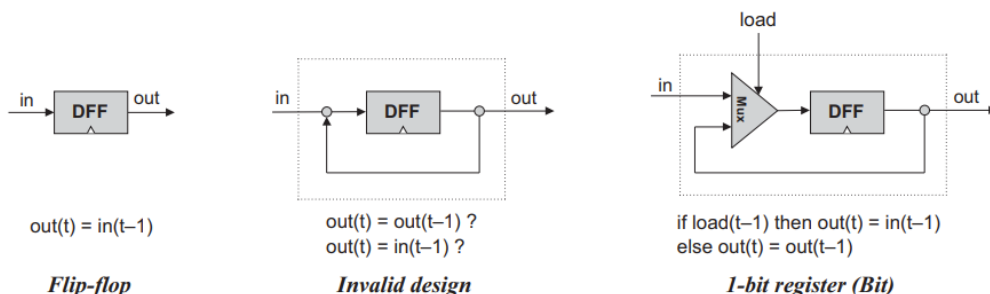
נשתמש בשערים רציפים הנקראים flip-flops כדי לבנות את כל רכיבי הזיכרון של מחשב מודרני. כדי לבנות צ'יפים ש"זוכרים" מידע, ראשית נצטרך שיהיה לנו מושג בסיסי לגבי ייצוג של זמן.

השעון: ברוב המחשבים, הזמן מיוצג ע"י שעון ראשי המספק רצף מתמשך של אותות מתחלפים. המימוש שלו מבוסס על מתנד שמחליף באופן מתמשך בין 2 מצבים: 0-1, low-high, tick-tock וכו'. הזמן שחולף בין תחילת "tick" ועד סוף ה-"tock" נקרא cycle, וכל cycle של השעון הוא יחידת זמן אחת. המצב הנוכחי של השעון (tick או tock) מיוצג ע"י אות בינארי, והוא משודר לכל הצ'יפים הרציפים במחשב.

Flip-Flops: הרכיב הרציף הבסיסי ביותר במחשב הוא flip-flop. אנחנו נשתמש בגרסה שלו שנקראת Data Flip-Flop (DFF). הממשק שלו מורכב מקלט של ביט יחיד ופלט של ביט יחיד. בנוסף, ל-DFF יש קלט של שעון שמשתנה כל הזמן בהתאם לאות של השעון הראשי. שני הקלטים (המידע והשעון) יחדיו מאפשרים ל-DFF לממש התנהגות מבוססת-זמן: $out(t) = in(t - 1)$, כאשר in ו- out הם ערכי הקלט והפלט ו- t הוא ה-cycle הנוכחי של השעון. במילים אחרות, ה-DFF מחזיר בפלט את ערך הקלט מיחידת הזמן הקודמת.

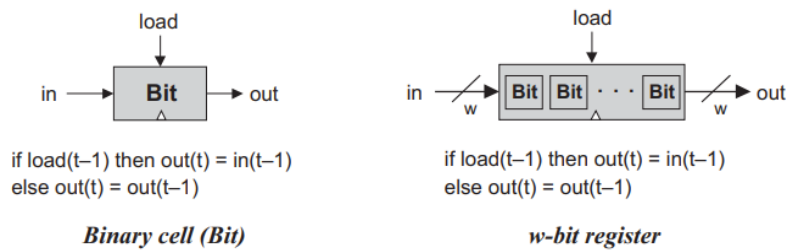
רגיסטרים (אוגרים) – Registers: רגיסטר הוא רכיב אחסון שיכול "לאחסן" או "לזכור" ערך במשך הזמן, המממש את התנהגות האחסון הקלאסית $out(t) = out(t - 1)$.

נוכל לממש רגיסטר באמצעות DFF שמקבל בתור קלט את הפלט של יחידת הזמן הקודמת:

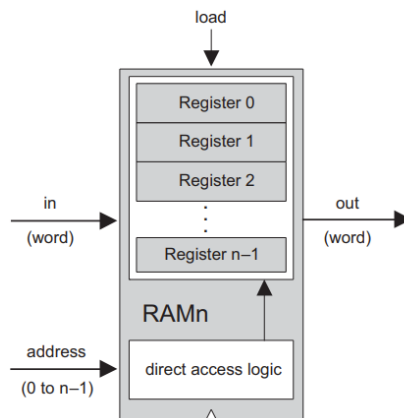


האופציה האמצעית אינה תקינה, כיוון שלא ברור האם הקלט מתקבל מה- in או מה- out . לכן, נשתמש ב-Mux, כאשר ה-select bit יהפוך להיות load bit של הרגיסטר: אם נרצה שהרגיסטר יאחסן ערך חדש, נוכל לשים את הערך הזה בתור ה- in ולקבוע את הערך של ה-load bit להיות 1, ואם נרצה שהרגיסטר ימשיך לאחסן את הערך שלו עד להודעה חדשה נוכל לקבוע את הערך של ה-load bit להיות 0.

כעת, נוכל לבנות בקלות רגיסטרים רחבים יותר, באמצעות מערך של רגיסטרים של ביט אחד. היוצרים רגיסטר שמחזיק ערכים המורכבים ממספר ביטים:



זיכרון – Memory: כעת, נוכל להמשיך ולבנות קטעי זיכרון מכל אורך רצוי. ניתן לעשות זאת ע"י ערימה של רגיסטרים כדי ליצור יחידת Random Access Memory (RAM). הדרישה של ה-RAM היא שנוכל לגשת באופן ישיר לכל מילה בזיכרון (כלומר לכל רגיסטר) בזמן קבוע.

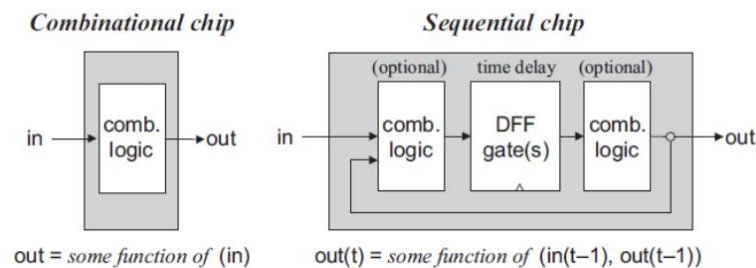


ראשית, נקצה כתובת ייחודית (מספר בין 0 ל- $n-1$) עבור כל מילה ב-RAM שמכיל n רגיסטרים, ולפי הכתובת הזו ניתן יהיה לגשת למילה. שנית, בנוסף לבניית מערך של n רגיסטרים, נבנה שער לוגי שבהינתן כתובת j מסוגל לבחור את הרגיסטר שהכתובת שלו היא j .

לסיכום, רכיב RAM מקבל 3 קלטים: $data$, כתובת $load$, וכתובת $load$. הכתובת מפרטת לאיזה רגיסטר ב-RAM צריך לגשת ביחידת הזמן הנוכחית. במקרה של פעולת קריאה (כאשר $load = 0$), הפלט של ה-RAM יהיה הערך שברגיסטר הנבחר. במקרה של פעולת כתיבה (כאשר $load = 1$), הרגיסטר הנבחר יאחסן את הערך החל מיחידת הזמן הבאה.

Counters: שבב רציף שהמצב שלו הוא integer שערכו גדל בכל יחידת זמן, ומשפיע על הפונקציה $out(t) = out(t-1) + c$ כאשר $c = 1$. ל-Counters יש תפקיד חשוב בארכיטקטורות דיגיטליות, לדוגמה CPU מכיל program counter שהפלט שלו הוא הכתובת של הפקודה הבאה שצריכה להתבצע בתוכנית הנוכחית. אפשר לממש שבב Counter באמצעות שילוב של לוגיקת קלט/פלט של רגיסטר סטנדרטי ולוגיקה קומבינטורית של הוספת קבוע. בדרך כלל, ל-Counter תהיה פונקציונליות נוספת כמו איפוס הספירה, טעינה של בסיס ספירה חדש או הקטנת הערך במקום הגדלתו.

כל השבבים שתיארנו עד כה הם רציפים. שבב רציף הוא שבב המכיל לפחות שער DFF אחד, באופן ישיר או לא ישיר. שערי ה-DFF מעניקים לשבבים רציפים את היכולת לשמר מצב (כמו ביחידות זיכרון) או לפעול על מצב (כמו ב-Counters). ניתן לעשות זאת ע"י יצירת לולאות פידבק בתוך השבב הרציף, כיוון שהפלט בזמן t לא תלוי בעצמו אלא תלוי בפלט בזמן $t - 1$. לעומת זאת, ב-Combinational Chips השימוש בלולאות פידבק הוא בעייתי כיוון שהפלט יהיה תלוי בקלט שתלוי בפלט, כלומר הפלט תלוי בעצמו).

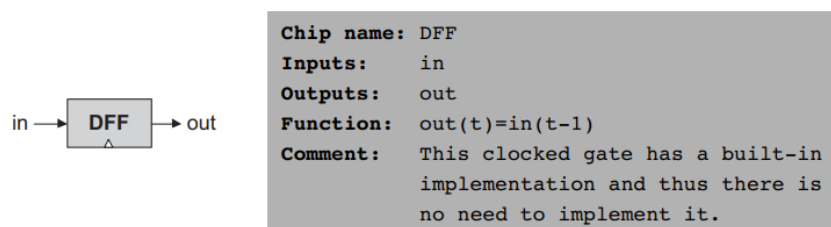


הפלט של Combinational Chips משתנה כאשר הקלט שלהם משתנה, בלי קשר לזמן. בניגוד לכך, כיוון ששבבים רציפים כוללים DFF, מובטח שהפלט שלהם ישתנה רק בנקודת המעבר מ-cycle אחד של השעון ל-cycle הבא, ולא ישתנה בתוך ה-cycle עצמו.

נניח שאנחנו רוצים שה-ALU יחשב את $x + y$ כאשר x הוא הערך של רגיסטר קרוב ו- y הוא הערך של רגיסטר RAM רחוק. עקב אילוצים פיזיקליים, האותות החשמליים שמייצגים את x ואת y יגיעו ל-ALU בזמנים שונים. כיוון שה-ALU הוא Combinational Chip, הוא לא רגיש לזמן, ולכן הוא מחשב באופן רציף כל מידע שהוא שנמצא בקלטים שלו. לכן ייקח זמן עד שה-ALU יתייצב ויחזיר את התוצאה הנכונה עבור $x + y$, ועד אז הוא יחזיר ערכי זבל. כיוון שהפלט של ה-ALU תמיד מנותב לשבב רציף כלשהו (רגיסטר, מקום ב-RAM וכו'), לא באמת אכפת לנו. רק נצטרך לוודא, כאשר אנחנו בונים את השעון של המחשב, שאורך cycle של השעון יהיה קצת יותר ארוך מהזמן שלוקח לביט לעבור את המסלול הארוך ביותר משבב אחד לשבב אחר בארכיטקטורת המחשב שלנו. בדרך זו נוכל להבטיח שבזמן שהשבב הרציף מעדכן את המצב שלו (בתחילת ה-cycle הבא של השעון), הקלטים שמתקבלים מה-ALU הם תקינים.

Specification and Implementation

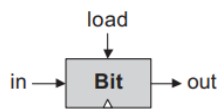
Data-Filp-Flop: הרכיב הרציף הבסיסי ביותר, בו נשתמש כדי לעצב את כל רכיבי הזיכרון. לשער DFF יש קלט של ביט אחד ופלט של ביט אחד:



כל השבבים הרציפים במחשב (רגיסטרים, זיכרון ו-Counters) מתבססים על מספר שערי DFF המחוברים לאותו שעון ראשי. בתחילת כל cycle של השעון, הפלטים של כל ה-DFFs במחשב מחויבים לקלטים שהיו להם במהלך יחידת הזמן הקודמת. בכל שאר הזמן, לקלטים של ה-DFFs אין

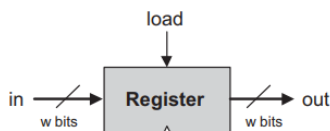
השפעה מיידית על הפלטים שלהם. כל שבב שמכיל שער DFF וכל השבבים שתלויים בו, יהיה גם כן תלוי בזמן. השבבים הללו נקראים רציפים לפי ההגדרה.

Registers: רגיסטר של ביט אחד, שנקרא לו Bit, מאפשר לאחסן ביט אחד של מידע (0 או 1). הוא מקבל כקלט ביט של מידע וביט load שמאפשר לכתוב לתא, ומחזיר כפלט את המצב הנוכחי של התא (הערך מיחידת הזמן הקודמת):



Chip name: Bit
Inputs: in, load
Outputs: out
Function: If load(t-1) then out(t)=in(t-1)
 else out(t)=out(t-1)

ה-API של השבב Register הוא זהה, רק שהקלט והפלט שלו הם 16-ביט:

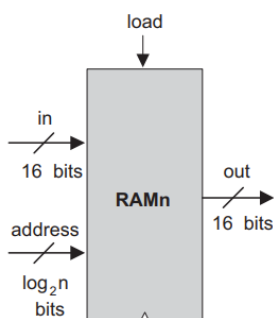


Chip name: Register
Inputs: in[16], load
Outputs: out[16]
Function: If load(t-1) then out(t)=in(t-1)
 else out(t)=out(t-1)
Comment: "=" is a 16-bit operation.

לשבבים Bit ו-Register יש את אותה ההתנהגות עבור קריאה/כתיבה: כדי לקרוא תוכן של רגיסטר, נבדוק מה הפלט שלו. כדי לכתוב מידע חדש d לרגיסטר, נשים את d בתור הקלט in ונעדכן $load = 1$. ב-cycle הבא של השעון, הרגיסטר יתחייב לערך המידע החדש, והפלט שלו יהיה d מאותו הרגע.

מימוש: המימוש של Bit מופיע בתחילת הפרק. הבנייה של שבב Register של w -ביט מרגיסטרים של ביט אחד היא ישירה. כל מה שצריך לעשות זה לבנות מערך של w שערי Bit ולתת את הקלט load לכל אחד מהם.

Memory: יחידת RAM היא מערך של n רגיסטרים של w -ביט המאפשרים גישה ישירה. מספר הרגיסטרים n ורוחב כל רגיסטר w נקראים הגודל והרוחב של הזיכרון. אנחנו נבנה את השבבים הבאים, כאשר הרוחב של כולם יהיה 16-ביט: RAM8, RAM64, RAM512, RAM4K ו-RAM16K. לכולם יש את אותו ה-API:



Chip name: RAMn // n and k are listed below
Inputs: in[16], address[k], load
Outputs: out[16]
Function: out(t)=RAM[address(t)](t)
 If load(t-1) then
 RAM[address(t-1)](t)=in(t-1)
Comment: "=" is a 16-bit operation.

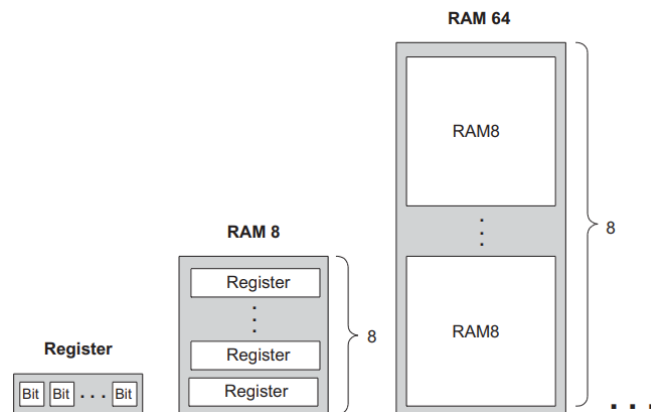
The specific RAM chips needed for the Hack platform are:

Chip name	n	k
RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

כדי לקרוא את התוכן של רגיסטר מספר m , נשים את m בתור הקלט $address$. הלוגיקה של ה-RAM תבחר את רגיסטר מספר m , שהפלט שלו יהיה הפלט של ה-RAM. זו פעולה שלא תלויה בשעון. כדי לכתוב מידע חדש d לרגיסטר מספר m , נשים את m בתור הקלט $address$ ואת d בתור הקלט in , ונעדכן $load = 1$. הלוגיקה של ה-RAM תבחר את רגיסטר מספר m , וה- $load$ bit יאפשר כתיבה אליו. ב- $cycle$ הבא של השעון, הרגיסטר הנבחר יתחייב לערך החדש שלו (d), והוא יהיה הפלט של ה-RAM.

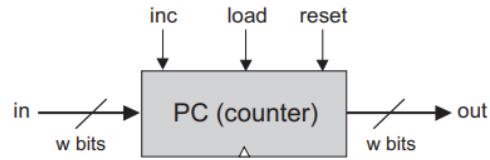
מימוש 8-Register Memory (RAM8): נבנה מערך של 8 רגיסטרים, ונרצה לבנות לוגיקה שבהינתן ערך $address$ ישים את הקלט in ברגיסטר הנבחר, כלומר בוחרת את הרגיסטר המתאים ומעבירה את ה- out שלו לפלט out של RAM8.

מימוש n-Register Memory: ניתן לבנות באופן רקורסיבי מיחידות קטנות יותר של זיכרון. נשים לב שניתן לבנות RAM של 64 רגיסטרים ממערך של שמונה שבבי RAM של 8 רגיסטרים. כדי לבחור רגיסטר ספציפי, נשתמש בכתובת של 6 ביט, למשל $xxxyyy$, כאשר ה-MSBs שלה xxx יבחרו את אחד משבבי ה-RAM8, וה-LSBs שלה yyy יבחרו את אחד מהרגיסטרים ב-RAM8. הנבחר. ל-RAM64 צריכים להיות מעגלים לוגיים שמשפיעים על ה- $addressing$ scheme.



Counter: הממשק שלו דומה לממשק של רגיסטר, רק של-Counter יש שני $control$ bits נוספים, $reset$ ו- inc . כאשר $inc = 1$, ה-Counter מגדיל את הערך שלו בכל $cycle$ של השעון כך ש- $out(t) = out(t-1) + 1$. אם נרצה לאפס את הערך של ה-Counter (כלומר שיהיה 0), נגדיר $reset = 1$. אם נרצה לאתחל את ה-Counter עם ערך d כלשהו, נשים את d בתור הקלט in ונגדיר $load = 1$.

```
Chip name: PC // 16-bit counter
Inputs:   in[16], inc, load, reset
Outputs:  out[16]
Function: If reset(t-1) then out(t)=0
          else if load(t-1) then out(t)=in(t-1)
          else if inc(t-1) then out(t)=out(t-1)+1
          else out(t)=out(t-1)
Comment:  "=" is 16-bit assignment.
          "+" is 16-bit arithmetic addition.
```



* דוגמה: נניח שיש לנו שבב Counter שמכיל את הכתובת של הפקודה הבאה שהמחשב צריך לבצע. ברוב המקרים, ה-Counter יצטרך להגדיל את עצמו ב-1 בכל cycle של השעון ובכך יגרום למחשב לעשות את הנדרש. במקרים אחרים, נרצה להיות יכולים לעדכן את ה-Counter ל- n , ואז הוא ימשיך את הספירה שלו משם עם $n + 1$, $n + 2$ וכך הלאה. נרצה גם את האפשרות לאפס את ה-Counter כדי להתחיל מחדש את ביצוע התוכנית.

מימוש: Counter של w -ביט מורכב משני רכיבים מרכזיים: רגיסטר רגיל של w -ביט, ו-Combinational Logic המשמשת לחישוב פונקציית הספירה ולקביעת הערך של ה-Counter לפי 3 ה-control bits שלו.

שפת מכונה – Machine Language

שפת מכונה היא פורמליזם מוסכם והיא מיועדת כדי לתכנת תוכניות low-level כסדרה של הוראות מכונה. ע"י שימוש בהוראות הללו, אפשר לתת פקודות למעבד כך שיבצע פעולות אריתמטיות ולוגיות, ייגש לערכים בזיכרון ויאחסן ערכים בזיכרון, יזיז ערכים מרגיסטר אחד לאחר, יבדוק תנאים בוליאניים, ועוד. המטרה של שפת מכונה היא ביצוע ישיר בפלטפורמת חומרה נתונה ושליטה מלאה בה. נתחיל בהיכרות כללית עם שפת מכונה, ולאחר מכן ניתן פירוט של שפת המכונה Hack (בגרסה הבינארית ובגרסת האסמבלי).

שפת מכונה – Machine Language: פורמליזם מוסכם הנועד כדי לתמרן זיכרון באמצעות שימוש במעבד ובקבוצת רגיסטרים.

זיכרון – Memory: המונח זיכרון מתייחס לאוסף של מכשירים שמאחסנים מידע והוראות במחשב. מנקודת המבט של המתכנת, לכל הזיכרונות יש את אותו המבנה: מערך רציף של תאים ברוחב קבוע הנקראים words או locations, כאשר לכל אחד מהם יש כתובת ייחודית. לכן, מילה (המייצגת פריט מידע או הוראה) מפורטת ע"י הכתובת שלה. נתייחס למילים באמצעות שימוש בסימונים השקולים: $Memory[address]$, $RAM[address]$, או $M[address]$.

מעבד – Processor: המעבד, Central Processing Unit (CPU), הוא מכשיר שיכול לבצע מספר קבוע של פעולות אלמנטריות (אריתמטיות, לוגיות, גישה לזיכרון ו-control). האופרנדים של הפעולות הללו הם ערכים בינאריים שמגיעים מרגיסטרים וממקומות נבחרים בזיכרון, והתוצאות של שלהן (הפלט של המעבד) יכולות להיות מאוחסנות ברגיסטרים או במקומות נבחרים בזיכרון.

רגיסטרים – Registers: כיוון שגישה לזיכרון היא פעולה איטית, לרוב המעבדים יש מספר רגיסטרים שכל אחד מהם יכול להחזיק ערך יחיד. כיוון שהם נמצאים במעבד, הם מהווים זיכרון מקומי שהגישה אליו מהירה, ומאפשרים למעבד לתמרן מידע והוראות מהר. זה מאפשר למתכנת להקטין את השימוש בפקודות גישה לזיכרון, ולהגביר את קצב ביצוע התוכנית.

תוכנית בשפת מכונה היא סדרה של הוראות כאשר כל הוראה מורכבת מ-0 ימים ו-1 ימים, למשל ההוראה 1010001100011001. כדי להבין את משמעות ההוראה הזו, נצטרך לדעת מהם "חוקי המשחק", כלומר מהן ההוראות. למשל, יכול להיות שכל ההוראות מורכבות משדות של 4-ביט: השדה השמאלי ביותר מתאר פעולה של ה-CPU, ושאר השדות מייצגים את האופרנדים.

כיוון שקוד בינארי לא פשוט לקריאה, שפות מכונה מוגדרות גם באמצעות קוד בינארי וגם באמצעות mnemonics (תווית סמלית שהשם שלה מרמז על משמעותה). למשל, אפשר להחליט שהקוד 1010 מיוצג ע"י ה-add mnemonic, ושנתייחס לרגיסטרים באמצעות הסמלים R0, R1, R2 וכך הלאה.

פקודות – Commands

פעולות אריתמטיות ולוגיות: כל מחשב צריך לבצע פעולות אריתמטיות בסיסיות כמו חיבור וחסור, ופעולות בוליאניות בסיסיות כמו שלילה bit-wise, bit shifting ועוד. לדוגמה:

```
ADD R2,R1,R3 // R2←R1+R3 where R1,R2,R3 are registers
ADD R2,R1,foo // R2←R1+foo where foo stands for the
                // value of the memory location pointed
                // at by the user-defined label foo.
AND R1,R1,R2 // R1←bit wise And of R1 and R2
```

גישה לזיכרון: קיימות שתי קטגוריות של פקודות גישה לזיכרון. הראשונה – ביצוע פעולות אריתמטיות ולוגיות על מקומות נבחרים בזיכרון. השנייה – פקודות טעינה (load) ואחסון המשמשות להעברת מידע בין רגיסטרים וזיכרון. רוב המחשבים מאפשרים:

- גישה ישירה – Direct Addressing: הדרך הנפוצה ביותר לגשת לזיכרון היא ביטוי של כתובת ספציפית או סמל שמתייחס לכתובת ספציפית באופן הבא:

```
LOAD R1,67 // R1←Memory[67]
// Or, assuming that bar refers to memory address 67:
LOAD R1,bar // R1←Memory[67]
```

- גישה מיידיית – Immediate Addressing: הגישה הזו משמשת כדי לטעון קבועים לרגיסטרים – במקום להתייחס לשדה מספרי בכתובת, נטען את הערך של השדה עצמו לרגיסטר באופן הבא: `LOADI R1,67 // R1←67`

- גישה עקיפה – Indirect Addressing: הכתובת של התא הרצוי בזיכרון לא מקודדת בפקודה. במקום, הפקודה מפרטת מקום בזיכרון שמחזיק את הכתובת הרצויה. לדוגמה: נתבונן בפקודה $x = foo[j]$, כאשר foo הוא משתנה שמתאר מערך, ו- x ו- j הם integers כלשהם. כאשר מצהירים על המערך foo ומאתחלים אותו בתוכנית ה-high-level, הקומפיילר מקצה מקטע בזיכרון כדי להחזיק את המידע של המערך, והסמל foo מצביע על כתובת הבסיס של המקטע הזה.

Flow of Control: תוכניות יכולות להיות מבוצעות בסדר לינארי, כלומר פקודה אחת פקודה, אבל הן יכולות לכלול "הסתעפויות" למקומות שונים מהפקודה הבאה. זה יכול להיות בשביל חזרה (קפיצה אחורה להתחלה של לולאה), התניה של ביצוע (אם תנאי בוליאני הוא false אז לקפוץ למקום אחרי בלוק ה-"if-then"), וקריאה ל-subroutine (קפיצה לפקודה הראשונה של מקטע קוד אחר). כדי לתמוך בפעולות הללו, כל שפת מכונה צריכה לדעת לקפוץ למקומות נבחרים בתוכנית, גם באופן מותנה וגם באופן לא מותנה. באסמבלי, אפשר לתת למקומות בתוכנית שמות סימבוליים ע"י שימוש בסינטקס שמסמן תוויות. פקודות של Unconditional Jump מפרטות רק את הכתובת של היעד. פקודות של Jump Conditional חייבות לפרט בנוסף תנאי בוליאני.

High-level

```
// A while loop:
while (R1>=0) {
    code segment 1
}
code segment 2
```

Low-level

```
// Typical translation:
beginWhile:
    JNG R1,endWhile // If R1<0 goto endWhile
    // Translation of code segment 1 comes here
    JMP beginWhile // Goto beginWhile
endWhile:
    // Translation of code segment 2 comes here
```

Hack Machine Language Specification

מחשב ה-Hack הוא מכונה של 16-ביט, המורכב מ-CPU, שני מודולים נפרדים של זיכרון המשמשים עבור זיכרון של הוראות וזיכרון של מידע, ושני התקני I/O שממופים לזיכרון – מסך ומקלדת.

Memory Address Spaces: יש שני מרחבי כתובות נפרדים – instruction memory ו-data memory. שניהם ברוחב 16-ביט ויש להם מרחב כתובות של 15-ביט, ולכן מספר הכתובות המקסימלי של כל זיכרון הוא 32K מילים של 16-ביט. ה-instruction memory הוא לקריאה בלבד.

רגיסטרים – Registers: יש שני רגיסטרים של 16-ביט הנקראים D ו- A . אפשר להפעיל על הרגיסטרים הללו הוראות אריתמטיות ולוגיות כמו $A = D - 1$ או $D = !A$ (כאשר המשמעות של "!" היא פעולת Not על 16-ביט). הרגיסטר D משמש לאחסון ערכי מידע, ו- A משמש גם כרגיסטר של מידע וגם כרגיסטר של כתובות. כלומר, A יכול להכיל ערך מידע, כתובת ב-data memory או כתובת ב-instruction memory.

הרגיסטר A מאפשר גישה ישירה ל-data memory. כיוון שההוראות ב-Hack הן ברוחב של 16-ביט והכתובות הן 15 ביטים, אי אפשר לתת הוראה אחת הכוללת גם כתובת וגם פעולה כלשהי. לכן הוראות של גישה לזיכרון ב-Hack פועלות על מיקום מפורש בזיכרון שנסמן בתור " M ". לדוגמה: $D = M + 1$. כדי למצוא את הכתובת הזאת, הקונבנציה היא ש- M תמיד מצביע על המילה בזיכרון שהכתובת שלה היא הערך הנוכחי שנמצא ברגיסטר A .

דוגמה: אם נרצה לבצע את הפעולה $D = \text{Memory}[516] - 1$, נצטרך הוראה אחת כדי להגדיר את הערך של הרגיסטר A להיות 516, כאשר ההוראה הבאה תהיה להגדיר $D = M - 1$.

בנוסף, הרגיסטר A מאפשר גישה ישירה ל-instruction memory. ה-jump instructions ב-Hack לא מפרטות כתובת ספציפית, והקונבנציה היא שכל קפיצה תמיד תהיה אל ההוראה שנמצאת במילה בזיכרון שהכתובת שלה נמצאת ברגיסטר A .

דוגמה: אם נרצה לבצע את הפעולה goto 35, נשתמש בהוראה אחת כדי להגדיר את הערך של A להיות 35, והוראה נוספת כדי לקודד את פקודת ה-goto מבלי לציין כתובת. שתי ההוראות הללו ברצף גורמות למחשב להביא את ההוראה הנמצאת ב- $\text{InstructionMemory}[35]$ ב-cycle הבא של השעון.

הפקודה @value: כאשר $value$ הוא מספר או סמל המייצג מספר, הפקודה הזו מאחסנת את הערך המפורט ברגיסטר A .

דוגמה: אם sum מצביע למקום 17 בזיכרון, אז גם $@sum$ וגם $@17$ יגרמו לכך ש- $A \leftarrow 17$.

דוגמה: נניח שאנחנו רוצים לחבר את כל המספרים בין 1 ל-100 תוך שימוש בחזרה על פעולת החיבור (לולאה). נתבונן בפתרונות אפשריים ב-C וב-Hack:

C language

```
// Adds 1+...+100.  
int i = 1;  
int sum = 0;  
While (i <= 100){  
    sum += i;  
    i++;  
}
```

Hack machine language

```
// Adds 1+...+100.  
@i      // i refers to some mem. location.  
M=1     // i=1  
@sum    // sum refers to some mem. location.  
M=0     // sum=0  
(LOOP)  
@i  
D=M     // D=i  
@100  
D=D-A   // D=i-100  
@END  
D;JGT   // If (i-100)>0 goto END  
@i  
D=M     // D=i  
@sum  
M=D+M   // sum=sum+i  
@i  
M=M+1   // i=i+1  
@LOOP  
0;JMP   // Goto LOOP  
(END)  
@END  
0;JMP   // Infinite loop
```

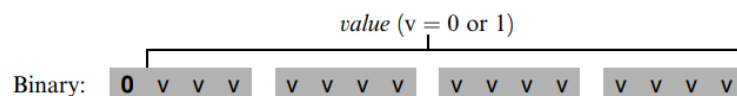
נשים לב שעבור כל פעולה שיש בה מיקום בזיכרון נדרשות שתי פקודות ב-Hack: אחת עבור בחירת הכתובת עליה נרצה לפעול, ואחת עבור פירוט הפעולה הרצויה.

שפת ה-Hack מכילה שתי הוראות גבריות: address instruction שנקראת גם A-instruction, ו-compute instruction שנקראת גם C-instruction. לכל הוראה יש ייצוג בינארי, ייצוג סימבולי, והשפעה על המחשב (שנפרט אותה כעת).

The A-Instruction

ה-A-instruction משמשת כדי לשים ברגיסטר A ערך של 15-ביט:

A-instruction: @value // Where value is either a non-negative decimal number
// or a symbol referring to such number.



ההוראה הזו גורמת למחשב לאחסן את הערך המפורט ברגיסטר A.

דוגמה: ההוראה @5 השקולה להוראה הבינארית 0000000000000101, גורמת למחשב לאחסן את הייצוג הבינארי של 5 ברגיסטר A.

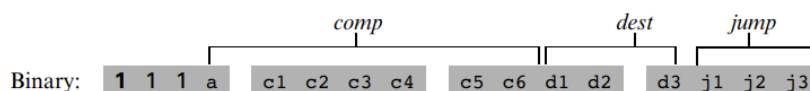
ה-A-instruction משמשת עבור 3 מטרות שונות:

1. מספקת את הדרך היחידה להכניס קבוע למחשב.
2. מכינה את הקרקע עבור C-instruction עוקבת שמטרתה לפעול על מקום מסוים ב-data memory, ע"י השמת הכתובת של המקום הזה ברגיסטר A תחילה.
3. מכינה את הקרקע עבור C-instruction עוקבת שמטרתה קפיצה, ע"י טעינת הכתובת של יעד הקפיצה לרגיסטר A.

The C-Instruction

ה-C-instruction היא פירוט העונה על 3 שאלות: מה לחשב, איפה לאחסן את הערך שחושב, ומה לעשות אחרי זה. יחד עם ה-A-instruction, הפירוטים הללו מגדירים את כל הפעולות האפשריות של המחשב.

C-instruction: *dest=comp;jump* // Either the *dest* or *jump* fields may be empty.
 // If *dest* is empty, the "=" is omitted;
 // If *jump* is empty, the ";" is omitted.



הביט השמאלי ביותר הוא הקוד של ה-C-instruction שהוא 1. שני הביטים שאחריו לא בשימוש. שאר הביטים מייצגים 3 שדות שמתאימים לשלושת החלקים של הייצוג הסימבולי של ההוראה. ההוראה הסימבולית היא מהצורה *dest = comp; jump*, כאשר השדה *comp* קובע ל-ALU מה לחשב, השדה *dest* קובע היכן לאחסן את הערך שחושב (כלומר את הפלט של ה-ALU), והשדה *jump* מפרט תנאי קפיצה – איזה פקודה להביא ולבצע אחרי זה.

The Computation Specification: ה-ALU של ה-Hack מחשב מספר קבוע של פונקציות על הרגיסטרים *D*, *A* ו-*M* (כאשר *M* מייצג את *Memory[A]*). הפונקציה שצריך לחשב מפורטת ע"י ה-a-bit ו-6-bits מהם מורכב שדה ה-*comp* של ההוראה. 7 הביטים הללו יכולים לקודד 128 פונקציות שונות, אבל רק 28 מהן מתועדות במפרט של השפה:

(when a=0) <i>comp mnemonic</i>	c1	c2	c3	c4	c5	c6	(when a=1) <i>comp mnemonic</i>
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

דוגמאות: ניזכר שהפורמט של C-instruction הוא *111a cccc ccdd djjj*. אזי:

- אם אנחנו רוצים שה-ALU יחשב את $D - 1$, כלומר את הערך הנוכחי שיש ברגיסטר *D* פחות 1. נוכל לעשות זאת ע"י ההוראה **1110 0011 1000 0000**.
- כדי לחשב את הערך של $D | M$, נשתמש בהוראה **1111 0101 0100 0000**.
- כדי לחשב את הקבוע -1, נשתמש בהוראה **1110 1110 1000 0000**.

The Destination Specification: אפשר לאחסן את הערך שמחשבים בחלק ה-*comp* של ה-C-instruction במספר מקומות, כפי שמפורט ב-3 הביטים של חלק ה-*dest* של ההוראה:

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

הביט הראשון אומר האם לשמור את הערך שחושב ברגיסטר A, הביט השני אומר האם לשמור את הערך שחושב ברגיסטר D, והביט השלישי אומר האם לשמור את הערך שחושב ב-M (כלומר ב-Memory[A]). אפשר לשמור את הערך ביותר מאחד מהם, ואפשר גם לא לשמור באף אחד מהם.

דוגמה: ניזכר שהפורמט של C-instruction הוא *111a cccc ccdd djjj*. נניח שאנחנו רוצים שהמחשב יגדיל ב-1 את הערך של *Memory[7]* ולשמור את התוצאה ברגיסטר D. ניתן לבצע זאת ע"י ההוראות הבאות:

```
0000 0000 0000 0111    // @7
1111 1101 1101 1000    // MD=M+1
```

ההוראה הראשונה גורמת למחשב לבחור את הרגיסטר בזיכרון שהכתובת שלו היא 7, וההוראה השנייה מחשבת את הערך של $M + 1$ ושומרת אותו גם ב-M וגם ב-D.

The Jump Specification: שדה ה-*jump* ב-C-instruction אומר למחשב מה הדבר הבא שהוא צריך לעשות. קיימות שתי אפשרויות: המחשב יכול להביא את ההוראה הבאה בתוכנית ולבצע אותה (שזו ברירת המחדל שלו), או שהוא יכול להביא הוראה שממוקמת במקום אחר בתוכנית ולבצע אותה. במקרה השני, נניח שברגיסטר A יש את הכתובת אליה אנחנו קופצים.

הקביעה האם תהיה קפיצה או לא תלוי ב-3 ה-j-bits של שדה ה-*jump* ובערך הפלט של ה-ALU (שחושב בהתאם לשדה ה-*comp*). ה-j-bit הראשון אומר האם לקפוץ במקרה בו הערך שלילי, הביט השני אומר האם לקפוץ במקרה בו הערך הוא 0, והביט השלישי אומר האם לקפוץ במקרה בו הערך חיובי. נקבל 8 תנאים אפשריים לקפיצה:

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

out מציין את הפלט של ה-ALU (כתוצאה מחלק ה-*comp* של ההוראה), ו-*jump* אומר להמשיך את ביצוע התוכנית החל מההוראה שהכתובת שלה נמצאת ברגיסטר A.

Logic

```
if Memory[3]=5 then goto 100
else goto 200
```

Implementation

```
@3
D=M      // D=Memory[3]
@5
D=D-A    // D=D-5
@100
D;JEQ    // If D=0 goto 100
@200
0;JMP    // Goto 200
```

ההוראה האחרונה JMP ; 0 גורמת לקפיצה לא מותנית. כיוון שהסינטקס של C-instruction דורש תמיד חישוב כלשהו, ניתן הוראה ל-ALU לחשב את 0 ונתעלם ממנו.

* הערה: כפי שראינו, ניתן להשתמש ברגיסטר A כדי לבחור מקום ב-data memory עבור C-instruction עוקבת שמשתמשת ב- M , או כדי לבחור מקום ב-instruction memory עבור C-instruction עוקבת שיש בה קפיצה. לכן, כדי למנוע שימושים סותרים ברגיסטר A , נדאג ש-C-instruction שיכולה לגרום לקפיצה (כלומר הוראה עם j-bits שאינם כולם 0-ים) לא תכיל רפרנס ל- M , ולהפך.

סמלים – Symbols

פקודות אסמבלי יכולות להתייחס למקומות בזיכרון (כתובות) ע"י שימוש בקבועים או בסמלים. קיימים 3 סוגי סמלים באסמבלי:

1. סמלים מוגדרים מראש – Predefined Symbols: תת-קבוצה של כתובות RAM מיוחדות

המכילה כתובות שניתן לגשת אליהן באמצעות סמלים מוגדרים מראש:

- רגיסטרים וירטואליים – Virtual Registers: כדי לפשט את התכנות באסמבלי, הסמלים $R0$ עד $R15$ מוגדרים מראש ומתייחסים לכתובות 0 עד 15 ב-RAM בהתאמה.
- מצביעים מוגדרים מראש – Predefined Pointers: הסמלים ARG , LCL , SP , $THIS$ ו- $THAT$ מוגדרים מראש להצביע לכתובות 0 עד 4 ב-RAM בהתאמה. נשים לב שלכל אחד מהמקומות הללו בזיכרון יש 2 תוויות. למשל, לכתובת 2 אפשר להתייחס בתור $R2$ או בתור ARG .
- I/O Pointers: הסמלים KBD ו- $SCREEN$ מוגדרים מראש להצביע לכתובות 16384 (0x4000) ו-24576 (0x6000) ב-RAM בהתאמה, שהן כתובות הבסיס של המסך ושל המקלדת בזיכרון.

2. תוויות – Label Symbols: סמלים שהמשתמש מגדיר כדי לסמן יעדים של פקודות $goto$.

מצהירים על תווית באמצעות הפקודה " (Xxx) ", שמשמעותה היא שהסמל Xxx מייצג את המקום ב-instruction memory בוא נמצאת הפקודה הבאה בתוכנית. ניתן להגדיר תווית פעם אחת בלבד, וניתן להשתמש בה בכל מקום בתוכנית, אפילו לפני השורה בה היא הוגדרה.

3. **משתנים – Variable Symbols:** לכל סמל Xxx שהמשתמש הגדיר ומופיע בתוכנית אסמבלי, והוא אינו מוגדר מראש או מוגדר במקום כלשהו בתוכנית ע"י פקודת " (Xxx) ", נתייחס בתור משתנה, והאסמבלר יקצה עבורו כתובות ייחודיות בזיכרון, החל מהכתובת 16 RAM-ב (0x0010).

קלט ופלט

אפשר לחבר לפלטפורמת Hack שני התקנים חיצוניים – מסך ומקלדת. שניהם מתקשרים עם פלטפורמת המחשב דרך memory maps. זה אומר שאפשר לצייר פיקסלים על המסך ע"י כתיבת ערכים בינאריים למקטע בזיכרון המזוהה עם המסך, ובאופן דומה ניתן "להקשיב" למקלדת ע"י קריאה מהמקום בזיכרון המזוהה עם המקלדת.

מסך: מחשב ה-Hack כולל מסך שחור-לבן הבנוי מ-256 שורות של 512 פיקסלים בכל שורה. התוכן של המסך מיוצג ע"י memory map של 8K שמתחילה בכתובת 16384 ב-RAM. כל שורה במסך הפיזי, החל מהפינה השמאלית העליונה, מיוצגת ב-RAM ע"י 32 מילים עוקבות של 16-ביט. לכן הפיקסל בשורה ה- r מלמעלה ובעמודה ה- c משמאל ממופה לביט ה- $c\%16$ (בספירה מה-LSB ל-MSB, כלומר מימין לשמאל) של המילה הנמצאת ב- $RAM \left[16384 + r \cdot 32 + \frac{c}{16} \right]$. כדי לקרוא או לכתוב פיקסל במסך הפיזי, צריך לקרוא או לכתוב את הביט התואם ב-RAM. כאשר הערך של הביט הוא 1 הפיקסל נצבע בשחור, וכאשר הערך הוא 0 הפיקסל נצבע בלבן.

דוגמה:

```
// Draw a single black dot at the screen's top left corner:
@SCREEN // Set the A register to point to the memory
        // word that is mapped to the 16 left-most
        // pixels of the top row of the screen.
M=1     // Blacken the left-most pixel.
```

מקלדת: הממשק של מחשב ה-Hack עם המקלדת הפיזית הוא דרך memory map של מילה אחת הנמצאת בכתובת 24576 ב-RAM. בכל פעם שלוחצים על מקש במקלדת בפיזית, קוד ה-ASCII שלו (של 16-ביט) מופיע ב- $RAM[24576]$. כאשר לא לוחצים על אף מקש, הקוד 0 מופיע במקום הזה.

בנוסף לערכי ה-ASCII הרגילים, המקלדת של ה-Hack מזהה גם את המקשים הבאים:

Key pressed	Code	Key pressed	Code
newline	128	end	135
backspace	129	page up	136
left arrow	130	page down	137
up arrow	131	insert	138
right arrow	132	delete	139
down arrow	133	esc	140
home	134	f1–f12	141–152

Syntax Conventions and File Format

Binary Code Files: קובץ קוד בינארי המורכב משורות טקסט. כל שורה היא רצף של 16 תווי ASCII של "0" ו-"1" שמקודדים הוראת שפת מכונה אחת. יחד, כל השורות בקובץ מייצגות תוכנית בשפת מכונה. כאשר תוכנית הכתובה בשפת מכונה עולה ל-instruction memory של המחשב, הקוד הבינארי שמיוצג ע"י השורה ה- n בקובץ יישמר בכתובת n של ה-instruction memory (הספירה של השורות בתוכנית ושל הכתובות בזיכרון מתחילה ב-0).

Assembly Language Files: קובץ הכתוב בשפת אסמבלי מורכב משורות טקסט, כאשר כל אחת מהן מייצגת הוראה, סמל או הצהרה:

- **הוראה** – A-instruction או C-instruction.
- **(Symbol)** – הפסאודו-פקודה הזו גורמת לאסמבלר להקצות לתויות *Symbol* את המקום בזיכרון שבו שמורה הפקודה הבאה של התוכנית. (נקראת "פסאודו-פקודה" כיוון שהיא לא מייצרת קוד מכונה).

קבועים וסמלים: קבועים חייבים להיות אי-שליליים ותמיד כתובים בסימון הדצימלי שלהם. סמל המוגדר ע"י המשתמש יכול להיות כל רצף של אותיות, ספרות, underscore (`_`), נקודה (`.`), דולר (`$`), ונקודותיים (`:`) שלא מתחיל בספרה.

Comments: טקסט שמתחיל בשני סלאשים (`//`) ומסתיים בסוף השורה הוא הערה ומתעלמים ממנו.

White Space: מתעלמים מתווי רווח ומשורות ריקות.

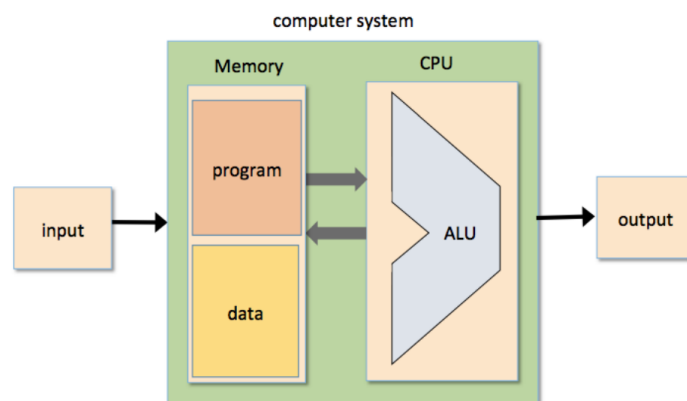
Case Conventions: כל ה-mnemonics של האסמבלי חייבים להיות כתובים באותיות גדולות. כל השאר (תוויות המוגדרות ע"י המשתמש ושמות משתנים) הם case sensitive. הקונבנציה היא להשתמש באותיות גדולות עבור תוויות ובאותיות קטנות עבור שמות משתנים.

ארכיטקטורת מחשב – Computer Architecture

בפרק זה, נשתמש בכל הציפיים שבנינו בפרקים 1-3 ובבנה מהם מחשב שיכול להריץ תוכניות הכתובות בשפת מכונה (שראינו בפרק 4), המחשב הזה נקרא Hack.

The Stored Program Concept: המחשב מבוסס על פלטפורמת חומרה קבועה המסוגלת לבצע מספר קבוע של הוראות פשוטות. ניתן לשלב את ההוראות הללו וליצור מהן תוכניות מתוחכמות. הלוגיקה של התוכניות הללו לא מוטבעת בחומרה. הקוד של התוכנית מאוחסן באופן זמני בזיכרון של המחשב ומשם מפעילים אותו.

ארכיטקטורת Von Neumann: מבוססת על (CPU) Central Processing Unit, המתקשר עם התקן זיכרון, שמקבל מידע מהתקן קלט כלשהו, ושולח מידע להתקן פלט כלשהו. בלב הארכיטקטורה הזו נמצא ה-Stored Program Concept – ההוראות שאומרות למחשב מה לעשות שמורות בזיכרון של המחשב, בנוסף למידע עליו המחשב פועל (ששמור גם הוא בזיכרון).



זיכרון – Memory

מנקודת מבט פיזית, הזיכרון הוא רצף לינארי של רגיסטרים שניתן לגשת אליהם, כאשר לכל אחד מהם יש כתובת ייחודית וערך שהוא מילה בגודל קבוע. מנקודת מבט לוגית, הזיכרון מחולק לשני אזורים. אזור אחד המיועד לאחסון מידע (למשל המערכים והאובייקטים של התוכניות הרצות באותו הרגע), ואזור אחר המיועד לאחסון הוראות של תוכניות. למרות שכל ה"data words" וה-"instruction words" האלו נראות אותו הדבר פיזית, הן משרתות מטרות שונות מאוד.

קיימות גרסאות של ארכיטקטורת Von Neumann בהן הזיכרון של המידע והזיכרון של ההוראות מנוהלים באותה יחידת הזיכרון הפיזית, וגרסאות בהן מחזיקים אותם בשתי יחידות זיכרון פיזיות נפרדות שיש להן מרחבי כתובות שונים ("ארכיטקטורת הרווארד"). נשתמש בגרסה השנייה במחשב ה-Hack שלנו.

לכל הרגיסטרים בזיכרון ניתן לגשת באותה הדרך: כדי לפעול על רגיסטר מסוים בזיכרון, צריך קודם כל לספק כתובת כדי לבחור אותו. הפעולה הזו נותנת גישה מיידית למידע של הרגיסטר.

Data Memory: תוכניות high-level פועלות על משתנים, מערכים ואובייקטים. לאחר שהתכונות מתורגמות לשפת מכונה, האבסטרקציה של המידע (למשתנים/מערכים/אובייקטים) הופכת לקוד בינארי שנשמר בזיכרון של המחשב. כאשר בוחרים רגיסטר מהזיכרון באמצעות הכתובת שלו, אפשר לקרוא את התוכן שלו או לכתוב תוכן לתוכו. כדי לקרוא אנחנו נשלוף את הערך של הרגיסטר הנבחר, וכדי לכתוב אנחנו נדרוס את הערך של הרגיסטר עם ערך חדש שנשמור בו.

Instruction Memory: לפני שהמחשב יוכל לבצע תוכניות high-level, צריך לתרגם אותן לשפת מכונה. לאחר התרגום, כל הצהרה של התוכנית תהפוך להיות סדרה של הוראה אחת או יותר בשפת מכונה. ההוראות הללו נשמרות ב-instruction memory של המחשב בתור קוד בינארי. בכל צעד של הרצת התוכנית, ה-CPU מביא (כלומר קורא) הוראת מכונה בינארית מרגיסטר נבחר ב-instruction memory, מפענח אותה, מבצע אותה, ומוצא מהי ההוראה הבאה שעליו להביא ולבצע.

Central Processing Unit

ה-CPU – החלק המרכזי של ארכיטקטורת המחשב – אחראי על ביצוע ההוראות של התכונות הנוכחיות. ההוראות הללו אומרות ל-CPU איזה חישוב הוא צריך לבצע, מאילו רגיסטרים הוא צריך לקרוא, לאילו רגיסטרים הוא צריך לכתוב, ומהי ההוראה הבאה שהוא צריך להביא ולבצע.

ה-CPU משתמש ב-3 רכיבי חומרה מרכזיים כדי לבצע את הפעולות הללו: ALU, קבוצה של רגיסטרים, ו-control unit.

Arithmetic Logic Unit (ALU): השבב של ה-ALU מבצע את כל הפעולות האריתמטיות והלוגיות שהן low-level. למשל – חיבור שני מספרים, חישוב פונקציית bit-wise And של שני מספרים, השוואת שני מספרים, ועוד.

רגיסטרים: כיוון שה-CPU הוא החלק המרכזי של המחשב, הביצועים שלו חייבים להיות יעילים ככל האפשר. כדי לשפר את הביצועים שלו, נרצה לשמור באופן מקומי תוצאות ביניים של תוכניות כך שיהיו קרובות ל-ALU, במקום לשמור אותן בשבב RAM נפרד ולשלוח אותן הלוך-חזור לשבב של ה-CPU. לכן, ל-CPU בדרך כלל יש בין 2 ל-32 רגיסטרים שכל אחד מהם יכול להחזיק מילה אחת.

Control Unit: הוראת מחשב מיוצגת ע"י קוד בינארי, ולפני שאפשר לבצע אותה צריך לפענח אותה. הפענוח של ההוראה נעשה ע"י control unit כלשהי, שהיא גם אחראית למצוא מה הפקודה הבאה שצריך לבצע ולהביא אותה.

אפשר לתאר את פעולת ה-CPU כמו לולאה החוזרת על עצמה: פענוח ההוראה הנוכחית, ביצוע שלה, מציאת ההוראה הבאה שצריך לבצע, הבאת ההוראה, פענוח שלה, וכך הלאה.

רגיסטרים

נניח שאין ל-CPU רגיסטרים משל עצמו. זה אומר שכל פעולה של ה-CPU שדורשת קלט או פלט תצטרך להסתמך על גישה לזיכרון. נרצה לדעת במה כרוכה כל גישה כזו לזיכרון. ראשית, ערך כתובת כלשהו מטייל מה-CPU לקלט ה-address של ה-RAM. לאחר מכן, ה-RAM משתמש בכתובת כדי לבחור רגיסטר ספציפי בזיכרון. לבסוף, התוכן של הרגיסטר מטייל בחזרה ל-CPU (פעולת קריאה) או מוחלף ע"י ערך אחר שמטייל מה-CPU (פעולת כתיבה).

נשים לב שהתהליך הזה מורכב משני שבבים נפרדים, bus של כתובת ו-bus של מידע, והתוצאה היא פעולה יקרה שלוקחת זמן. הפתרון לכך הוא הוספת רגיסטרים בתוך ה-CPU, ליד ה-ALU.

יש יתרון נוסף להוספת רגיסטרים בתוך ה-CPU. כדי לתת הוראה הכוללת רגיסטר של הזיכרון, כמו $Memory[addr] = value$, עלינו לספק כתובת בזיכרון שבדרך כלל דורשת הרבה ביטים. בפלטפורמת ה-Hack של 16-ביט, זה מאלץ אותנו להשתמש בשתי הוראות מכונה ובשני cycles של השעון, גם עבור משימות כמו $Memory[addr] = 0$ או $Memory[addr] = 1$.

כיוון שיש מעט רגיסטרים בתוך ה-CPU, הזיהוי של כל אחד מהם דורש מעט ביטים. לכן, פעולה כמו $someCPURegister = 1$ או $someCPURegister = 0$ דורשת רק הוראת מכונה אחת ו-cycle אחד של השעון.

Data Registers: הרגיסטרים הללו מספקים ל-CPU זיכרון לטווח הקצר.

לדוגמה, אם תוכנית רוצה לחשב $c \cdot (a - b)$, עלינו קודם לחשב את הערך של $(a - b)$ ולזכור אותו. אפשר לשמור את התוצאה הזמנית הזאת ברגיסטר בזיכרון, אבל פתרון הרבה יותר הגיוני יהיה לשמור אותו לוקאלית בתוך ה-CPU ע"י שימוש ב-data register.

ל-CPU בדרך יהיו בין 1 עד 32 data registers.

Address Registers: הרבה הוראות בשפת מכונה כוללות גישה לזיכרון – קריאת מידע, כתיבת מידע, והבאת הוראות. בכל אחת מהפעולות הללו, אנחנו חייבים לציין על איזה רגיסטר בזיכרון נרצה לפעול, ואפשר לעשות זאת ע"י מתן כתובת. במקרים מסוימים הכתובת היא חלק מההוראה, בעוד שבמקרים אחרים הכתובת מפורטת או מחושבת ע"י הוראה קודמת, ובמקרים האלו אנחנו חייבים לשמור את הכתובת איפשהו. ניתן לעשות זאת ע"י address register שנמצא ב-CPU.

בשונה מרגיסטרים רגילים, הפלט של address register בדרך כלל מחובר לקלט ה-address של רכיב זיכרון. לכן, אם נשים ערך ב-address register, נוכל לבחור רגיסטר מסוים בזיכרון שההוראות הבאות יוכלו לפעול עליו.

דוגמה: נניח שאנחנו רוצים לשנות את $Memory[17]$ ל-1. בשפת ה-Hack, נוכל לעשות זאת ע"י ההוראה @17 (שמגדירה $A = 17$ וגורמת לכך ש- M ייצג את $Memory[17]$) ואחריה ההוראה $M = 1$ (שמשנה את הרגיסטר הנבחר בזיכרון ל-1).

בנוסף, address register הוא רגיסטר, ובמידת הצורך יוכל לשמש בתור עוד data register.

דוגמה: נניח שאנחנו רוצים לשנות את הערך של הרגיסטר D ל-17. אפשר לעשות זאת ע"י ההוראה @17 ואחריה ההוראה $D = A$ (כאשר נתעלם מכך שההוראה @17 גם גורמת לכך ש- $Memory[17]$ נבחר).

Program Counter: כאשר ה-CPU מריץ תוכנית, הוא חייב תמיד לעקוב אחרי הכתובת של ההוראה הבאה שהוא צריך להביא ולבצע. הכתובת הזו שמורה בתוך רגיסטר מיוחד שנקרא Program Counter או PC. התוכן של PC יחושב ויעודכן כתוצאה מביצוע ההוראה הנוכחית, כפי שנראה בהמשך הפרק.

קלט ופלט

מחשבים מתקשרים עם הרבה גורמים חיצוניים (מסכים, מקלדות, דיסקים, מדפסות, סורקים, ממשקי רשת, כרטיסים, ועוד) באמצעות מערך מגוון של מכשירי קלט ופלט (I/O). מדעני מחשב המציאו סכמות כדי שכל המכשירים הללו ייראו למחשב אותו הדבר – memory-mapped I/O.

הרעיון הוא ליצור ייצוג בינארי לכל מכשיר כזה כדי שהוא ייראה ל-CPU כמו מקטע רגיל בזיכרון. בפרט, להקצות לכל מכשיר I/O אזור ייחודי בזיכרון, שהופך להיות ה"memory map" שלו.

במקרה של התקן קלט כמו מקלדת, ה-memory map ישקף באופן רציף את המצב של ההתקן: כאשר המשתמש לוחץ על מקש במקלדת, קוד בינארי המייצג את המקש הזה מופיע ב-memory map של המקלדת. במקרה של התקן פלט כמו מסך, המסך משקף באופן רציף את ה-memory map שלו: כאשר נכתוב ביט ב-memory map של המסך, פיקסל מסוים במסך יידלק/יכבה.

The Hack Hardware Platform: Specification

פלטפורמת ה-Hack היא מכונת Von Neumann של 16-ביט, שמטרתה להריץ תוכניות הכתובות בשפת המכונה Hack. הפלטפורמה מורכבת מ-CPU, שני מודולים נפרדים של זיכרון המשמשים בתור instruction memory ו-data memory, ושני התקני I/O שהם memory-mapped – מסך ומקלדת.

ה-CPU של ה-Hack מורכב מ-ALU ומ-3 רגיסטרים של 16-ביט – data register (D), address register (A), ו-program counter (PC). הרגיסטר D משמש לשמירת ערכי מידע, והרגיסטר A משמש עבור 3 מטרות – שמירת ערך מידע, הצבעה על כתובת ב-instruction memory, או הצבעה על כתובת ב-data memory.

ה-CPU של ה-Hack מבצע הוראות הכתובות בשפת המכונה Hack מהפורמט "*ixxacccccddjjj*" (הידוע גם בתור opcode) מתאר את סוג ההוראה – 0 עבור A-instruction או 1 עבור C-instruction. במקרה של A-instruction, נתייחס להוראה כאל ערך בינארי של 16-ביט שנטען לתוך הרגיסטר A . במקרה של C-instruction, נתייחס להוראה כאל רצף של control bits הקובעים איזו פונקציה ה-ALU צריך לחשב ובאיזה רגיסטרים צריך לשמור את הערכים שחושבו. במהלך ביצוע כל אחת מההוראות הללו, ה-CPU גם מוצא מהי ההוראה הבאה בתוכנית שצריך לבצע ולהביא אותה.

Central Processing Unit (CPU): ה-CPU של פלטפורמת ה-Hack צריך לבצע הוראות של 16-ביט בהתאם למפרט של שפת המכונה Hack. הוא צריך להיות מחובר לשני מודולים נפרדים של זיכרון – instruction memory שממנו הוא מביא הוראות לביצוע, ו-data memory שהוא יכול לקרוא ממנו או לכתוב לתוכו ערכי מידע. להלן המפרט:

/** The Hack Central Processing Unit consists of an ALU, two registers named A and D, and a program counter named PC (these internal chip-parts are not shown in the diagram). The inM input and outM output hold the values referred to as "M" in the Hack instruction syntax. The addressM output holds the memory address to which outM should be written.

The CPU is designed to fetch and execute instructions written in the Hack machine language. If instruction is an A-instruction, the CPU loads the 16-bit constant that the instruction represents into the A register. If instruction is a C-instruction, then (i) the CPU causes the ALU to perform the computation specified by the instruction, and (ii) the CPU causes this value to be stored in the subset of {A,D,M} registers specified by the instruction. If one of these registers is M, the CPU asserts the writeM control bit output (when writeM is 0, any value may appear in outM).

When the reset input is 0, the CPU uses the ALU output and the jump directive specified by the instruction to compute the address of the next instruction, and emits this address to the pc output. If the reset input is 1, the CPU sets pc to 0.

The outM and writeM outputs are *combinational*, and are affected instantaneously by the instruction's execution. The addressM and pc outputs are *clocked*: although they are affected by the instruction's execution, they commit to their new values only in the next time step. */

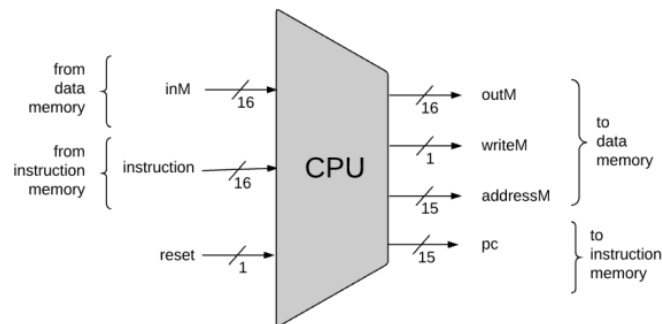
CHIP CPU

IN

```
instruction[16], // Instruction to execute.
inM[16],        // Value of Mem[A], the instruction's M input
reset;          // Signals whether to continue executing the current program
                // (reset==1) or restart the current program (reset==0).
```

OUT

```
outM[16],       // Value to write to Mem[addressM], the instruction's M output
addressM[15],   // In which address to write?
writeM,         // Write to the Memory?
pc[15];         // address of next instruction
```



Instruction Memory: ממומש ע"י התקן זיכרון לקריאה בלבד עם גישה ישירה שנקרא ROM. ה-ROM של ה-Hack מכיל 32K רגיסטרים של 16-ביט שניתן לגשת אליהם, כפי שניתן לראות כאן:

/** The instruction memory of the Hack computer, implemented as a read-only memory of 32K registers, each 16-bit wide.

Performs the operation $out = ROM32K[address]$.

In words: outputs the 16-bit value stored in the register selected by the address input. This value is taken to be the *current instruction*.

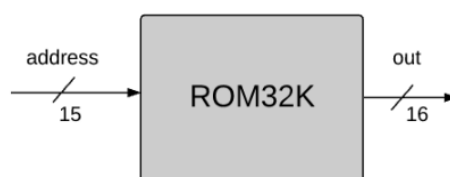
It is assumed that the chip is preloaded with a program written in the Hack machine language.

Software-based simulators of the Hack computer are expected to provide means for loading the chip with a Hack program, either interactively, or using a test script. */

CHIP ROM32K

IN address[15];

OUT out[16];



Input/Output: הגישה להתקני הקלט/פלט של מחשב ה-Hack מתאפשרת דרך ה-data memory, שהוא RAM לקריאה ולכתיבה המורכב מ-32K רגיסטרים של 16-ביט שניתן לגשת אליהם. אפשר לחבר לפלטפורמת ה-Hack שני התקנים חיצוניים – מסך ומקלדת. שניהם מתקשרים עם פלטפורמת המחשב דרך memory-mapped buffers באופן הבא: כאשר משתנה ב-memory map של המסך מופיע פיקסל שחור/לבן (בהתאם לביט) על המסך הפיזי, וכאשר לוחצים על מקש במקלדת הקוד שלו יופיע ב-memory map של המקלדת.

נפרט את השבבים של הממשק בין פלטפורמת החומרה ובין התקני ה-I/O:

- **Screen:** מחשב ה-Hack יכול לתקשר עם מסך פיזי המורכב מ-256 שורות של 512 פיקסלים בשחור-לבן בכל שורה. ה-memory map של המסך ממומשת ע"י שבב RAM שנקרא Screen ומתנהג כמו זיכרון רגיל (אפשר לקרוא ממנו ולכתוב לתוכו). בנוסף, כל ביט שנכתב משתקף בתור פיקסל על המסך הפיזי ($black = 1, white = 0$). המיפוי המדויק בין ה-memory map למסך הפיזי מתואר באופן הבא:

```
/** The Screen (memory map) functions exactly like a 16-bit, 8K RAM:

(1) out(t) = Screen[address(t)](t)
(2) if load(t) then Screen[address(t)](t+1) = in(t)

The chip implementation has the side effect of continuously refreshing a physical
screen. The physical screen consists of 256 rows and 512 columns of black and white
pixels (simulators of the Hack computer are expected to simulate this screen).

Each row in the physical screen, starting at the top left corner, is represented in the
Screen memory map by 32 consecutive 16-bit words. Thus the pixel at row r from the
top and column c from the left ( $0 \leq r \leq 255, 0 \leq c \leq 511$ ) is mapped on the c%16 bit
(counting from LSB to MSB) of the 16-bit word stored in Screen[r * 32 + c / 16]. */

CHIP Screen
IN
    in[16],      // what to write
    address[13]; // where to write (or read)
    load,        // write-enable bit

OUT
    out[16];     // Screen value at the given address
```

- **Keyboard:** הממשק של המחשב עם המקלדת הוא דרך שבב שנקרא Keyboard. כאשר לוחצים על מקש במקלדת הפיזית, קוד מיוחד של 16-ביט יהפוך להיות הפלט של השבב Keyboard, וכאשר לא לוחצים על אף מקש הפלט שלו יהיה 0.

```
/** The Keyboard (memory map) is connected to a standard, physical
keyboard. It is made to output the 16-bit scan-code associated with the
key which is presently pressed on the physical keyboard, or 0 if no key is
pressed.

The keyboard scan-codes are given in Appendix C of the book.
Simulators of the Hack computer are expected to implement the contract
described above. */

CHIP Keyboard
OUT out[16]; // The scan-code of the pressed key,
              // or 0 if no key is currently pressed.
```

Data Memory: מרחב הכתובות הידוע בתור ה-data memory של ה-Hack ממומש ע"י שבב שנקרא Memory, המורכב מ-3 רכיבי אחסון של 16-ביט – RAM (16K רגיסטרים לשמירת מידע רגיל), Screen (8K רגיסטרים שהם ה-memory map של המסך), ו-Keyboard (רגיסטר אחד שהוא ה-memory map של המקלדת). להלן המפרט:

/** The complete address space of the Hack computer's data memory, including RAM and memory-mapped I/O. Facilitates read and write operations, as follows:

Read: $\text{out}(t) = \text{Mem}[\text{address}(t)](t)$

Write: $\text{if load}(t) \text{ then Mem}[\text{address}(t)](t+1) = \text{in}(t)$

In words: the chip always outputs the value stored at the memory location specified by **address**.

If $\text{load}==1$, the **in** value is loaded into the register specified by **address**. This value becomes available through the **out** output from the next time step onward.

The memory access rules are as follows:

Only the top $16K+8K+1$ words of the address space are used.

$0x0000-0x5FFF$: accessing an address in this range results in accessing the RAM.

$0x4000-0x5FFF$: accessing an address in this range results in accessing the Screen.

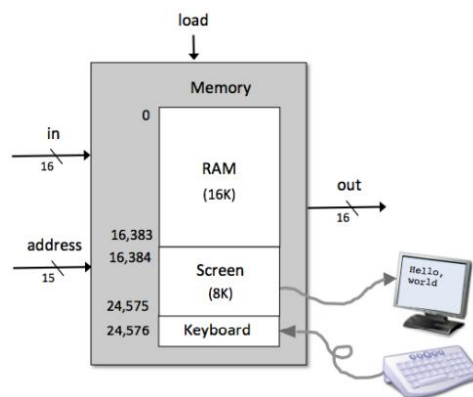
$0x6000$: accessing this address results in accessing the Keyboard.

$> 0x6000$: accessing an address in this range is invalid. */

CHIP Memory

IN $\text{in}[16]$, load , $\text{address}[15]$;

OUT $\text{out}[16]$;



Computer: מורכב מ-CPU, instruction memory, ו-data memory. המחשב יכול לתקשר עם מסך ועם מקלדת. להלן המפרט:

/** The HACK computer, consisting of CPU, ROM and Memory parts (these internal chip-parts are not shown in the diagram).

When $\text{reset}==0$, the program stored in the computer's ROM executes.

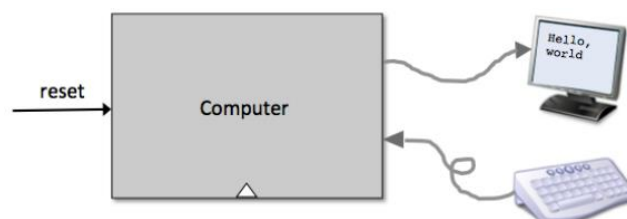
When $\text{reset}==1$, the execution of the program restarts.

Thus, to start a program's execution, the **reset** input must be pushed "up" (signaling 1) and "down" (signaling 0).

From this point onward, the user is at the mercy of the software. In particular, depending on the program's code, the screen may show some output, and the user may be able to interact with the computer via the keyboard. */

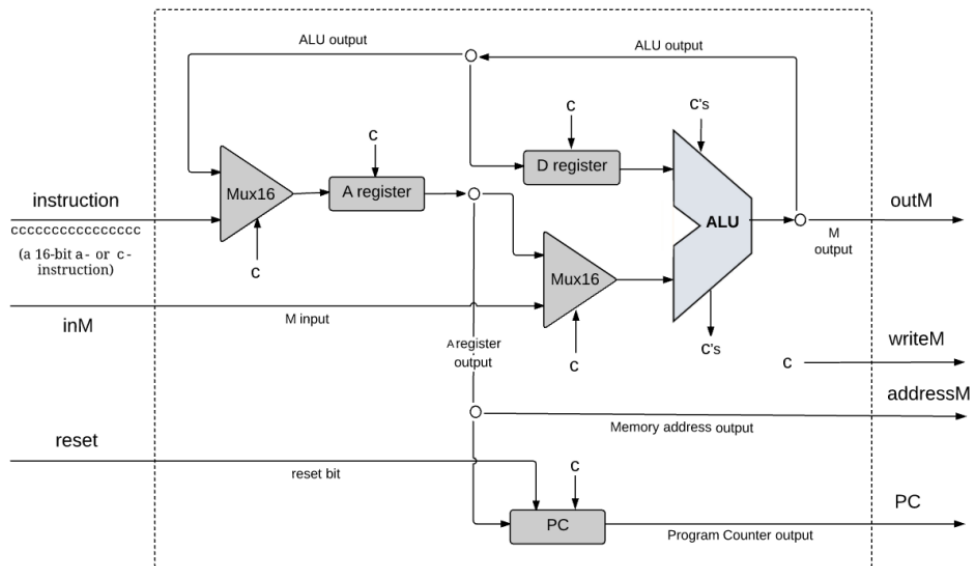
CHIP Computer

IN reset ;



מימוש

The Central Processing Unit: נרצה שער לוגי שיהיה מסוגל לבצע הוראת Hack נתונה ולקבוע איזו הוראה צריך להביא ולבצע אחריה. כדי לבצע זאת, המימוש של ה-CPU צריך לכלול שבב ALU המסוגל לחשב פונקציות אריתמטיות/לוגיות, קבוצה של רגיסטרים, program counter, ושערים נוספים שיעזרו לפענח, לבצע ולהביא הוראות. נתבונן בקונפיגורציה הבאה:



הביטים של ההוראה מסומנים ב-c, כיוון שבמקרה של C-instruction, ה-CPU מתייחס אליהם בתור control bits שהוא מחלץ מההוראה ומנתב אותם לשבבים שונים בתוכו. בפרט, כל סמל c בדיאגרמה שנכנס לשבב מייצג bit control כלשהו שחולץ מההוראה (במקרה של ה-ALU, הקלט "c's" מייצג את 6 ה-control bits המנחים את ה-ALU מה לחשב, והפלט "c's" מייצג את הפלטים ng-i zr).

הארכיטקטורה הזו משמשת עבור ביצוע 3 המשימות של ה-CPU: פענוח ההוראה הנוכחית, ביצוע ההוראה הנוכחית והחלטה איזו הוראה להביא ולבצע אחריה.

Instruction Decoding: הערך של הקלט instruction של ה-CPU מורכב מ-16 ביטים המייצגים C-instruction או A-instruction. כדי לפענח את ההוראה, ניתן לפרק את הקלט לשדות הבאים: "ixxaccddjjj". ה-i-bit מקודד את סוג ההוראה – 0 עבור A-instruction או 1 עבור C-instruction. במקרה של A-instruction, כל ההוראה מייצגת ערך של קבוע בגודל 16-ביט שצריך לטעון לתוך הרגיסטר A. במקרה של C-instruction, ה-a-bits וה-c-bits מקודדים את חלק ה-comp של ההוראה, ה-d-bits מקודדים את חלק ה-dest של ההוראה, וה-j-bits מקודדים את חלק ה-jump של ההוראה (ה-x-bits אינם בשימוש וניתן להתעלם מהם).

Instruction Execution: השדות של ההוראה שפוענחו מנותבים באותו הזמן לחלקים שונים של ארכיטקטורת ה-CPU. במקרה של C-instruction, ה-a-bit קובע האם ה-ALU יפעל על הקלט של הרגיסטר A או על הקלט של M, ה-c-bits קובעים איזו פונקציה ה-ALU יחשב, ה-d-bits קובעים אילו רגיסטרים צריכים "לקבל" את תוצאת הפלט של ה-ALU, וה-j-bits משמשים עבור branching control.

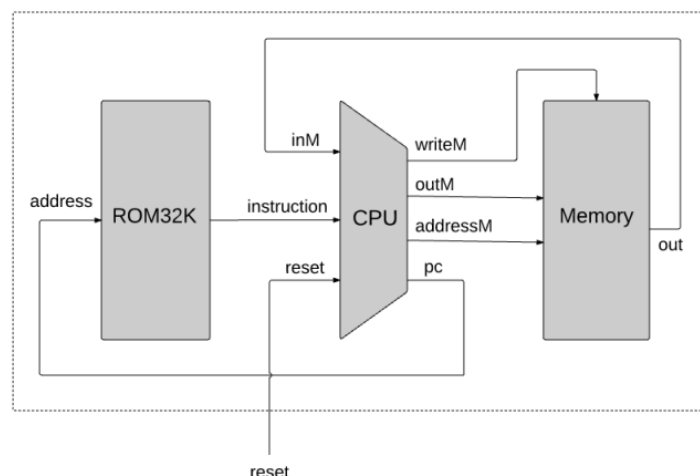
Instruction Fetching: כחלק מביצוע ההוראה הנוכחית, ה-CPU חייב לקבוע ולפלוט את הכתובת של ההוראה הבאה שצריך להביא ולבצע. הרכיב המרכזי בפעולה הזו הוא ה-Program Counter –

השבב ב-CPU שתפקידו לשמור את הכתובת של ההוראה הבאה. נתאר בהמשך כיצד לחבר את הפלט pc של ה-CPU לקלט $address$ של ה-instruction memory, כאשר החיבור הזה יגרום לכך שהפלט של ה-instruction memory תמיד יהיה ההוראה הבאה שצריך לבצע, והפלט הזה יהיה מחובר לקלט $instruction$ של ה-CPU.

לפי המפרט של מחשב ה-Hack, התוכנית הנוכחית שמורה ב-instruction memory, החל מהכתובת 0. לכן, אם נרצה להתחיל (או להתחיל מחדש) הרצה של תוכנית, צריך לאתחל את ה-Program Counter ל-0. לכן בדיאגרמה הקלט $reset$ של ה-CPU מגיע ישירות לקלט $reset$ של השבב PC. לכן, אם נטען את הביט הזה, נקבל $PC = 0$ מה שיגרום למחשב להביא את ההוראה הראשונה בתוכנית ולבצע אותה. לאחר מכן, לרוב, נרצה לבצע את ההוראה הבאה בתוכנית ולכן ברירת המחדל של ה-Program Counter היא הפעולה $PC++$. אם ההוראה דורשת את ביצוע הפעולה " $jump\ n$ ", כאשר n היא כתובת של הוראה הממוקמת איפשהו בתוכנית, לפי שפת ה-Hack עלינו לבצע רצף של שתי הוראות: ה- $A-instruction@n$ שמגדירה את הערך של הרגיסטר A להיות n , וה- $C-instruction$ הכוללת קפיצה ישירות לשם. לפי המפרט של השפה, ההרצה תמיד תגיע להוראה עליה מצביע הרגיסטר A . לכן, כאשר נממש את ה-CPU, נצטרך שער לוגי המממש את ההתנהגות הבאה: אם $jump = 1$ אז $PC = A$, אחרת $PC++$. הערך של הביטוי הבוליאני $jump$ תלוי ב- j -bits של ההוראה ובפלט של ה-ALU. נשים לב שבדיאגרמה הפלט של הרגיסטר A נכנס לתוך הקלט של הרגיסטר PC , וניזכר של- PC יש $load-bit$ שמאפשר לו לקבל ערך קלט חדש. לכן, אם נטען את ה- $load-bit$ הזה נגרום לביצוע הפעולה $PC = A$ במקום ברירת המחדל $PC++$, ועלינו לעשות זאת רק כאשר נצטרך לממש קפיצה. ה- j -bits של ההוראה הנוכחית מפרטים את תנאי הקפיצה והביטים zr ו- ng בפלט של ה-ALU קובעים האם התנאי מתקיים או לא, ולפיהם נקבע האם צריך לממש קפיצה או לא.

Memory: השבב Memory של ה-Hack הוא איחוד של 3 שבבים: $RAM16K$, $Screen$ ו- $Keyboard$. המשתמשים בשבב הזה רואים מרחב כתובות יחיד מהכתובת 0 עד הכתובת 24576. המימוש של השבב הזה צריך לתת אפקט של רציפות: למשל, אם הקלט $address$ של השבב הוא 16384, הלוגיקה של השבב צריכה לגשת לכתובת 0 בשבב $Screen$. ניתן לבצע זאת בדומה למה שעשינו בפרק 3 כדי לאחד יחידות RAM קטנות ליחידות גדולות.

Computer: אפשר לממש את השבב הזה באמצעות השבבים: CPU, data memory ו-instruction memory שנקרא $ROM32K$ באופן הבא:



אסמבלר – Assembler

בחצי הראשון של הקורס (פרקים 1-5) תיארנו ובנינו פלטפורמת חומרה של מחשב. החצי השני של הקורס (פרקים 6-12) נתמקד בהיררכיית התוכנה של המחשב, כאשר נפתח קומפיילר ומערכת הפעלה בסיסית עבור שפת תכנות object-based פשוטה. המודול הראשון והבסיסי ביותר בהיררכיית התוכנה הזה הוא האסמבלר. בפרק 4 ראינו שפות מכונה בייצוג האסמבלי ובייצוג הבינארי שלהן, ובפרק זה נתאר כיצד האסמבלר מתרגם תוכניות הכתובות בשפת מכונה לתוכניות הכתובות בייצוג בינארי.

כאמור, לשפות מכונה יש ייצוג סימבולי וייצוג בינארי. הקוד הבינארי מייצג הוראות מכונה אמיתיות שהחומרה מבינה.

דוגמה: 8 הביטים השמאליים ביותר של הוראה יכולים לייצג פעולה כמו *LOAD*, 8 הביטים הבאים יכולים לייצג רגיסטר כמו *R3*, ו-16 הביטים הנותרים יכולים לייצג כתובת כמו 7, ולפי העיצוב הלוגי של החומרה ושפת המכונה התבנית הזאת יכולה לגרום לחומרה לבצע את הפעולה "*load the contents of Memory[7] into register R3*".

השפה הסימבולית אותה נרצה לתרגם נקראת **אסמבלי (Assembly)** והתוכנית שמתרגמת נקראת **אסמבלר (Assembler)**. האסמבלר מפרסר כל פקודת אסמבלי לשדות שלה, מתרגם כל שדה לקוד הבינארי השקול לו, ומרכיב מכל הקודים שנוצרו הוראה בינארית שהחומרה יכולה לבצע.

Symbols: הוראות בינאריות מיוצגות בקוד בינארי. בהגדרה, הן מתייחסות לכתובות בזיכרון ע"י מספרים אמיתיים.

דוגמה: נניח שיש לנו תוכנית שמשתמשת במשתנה *weight* כדי לייצג את המשקל של כל מיני דברים, ונניח שהמשתנה הזה ממופה למקום 7 בזיכרון של המחשב. ברמת הקוד הבינארי, הוראות שפועלות על המשתנה *weight* חייבות לגשת אליו באמצעות שימוש בכתובת המפורשת 7. ברמת האסמבלי, נוכל לאפשר לכתוב פקודות כמו *LOAD R3, weight* במקום *LOAD R3, 7*. בשני המקרים הפקודה תגרום לאותה הפעולה: "*set R3 to the contents of Memory[7]*".

באופן דומה, במקום להשתמש בפקודות כמו *goto 250*, שפות אסמבלי מאפשרות פקודות כמו *goto loop* בהנחה שאיפשהו בתוכנית גרמנו לסימבול *loop* להצביע לכתובת 250.

באופן כללי, סימבולים מגיעים לתוכנית אסמבלי משני מקורות:

- משתנים (Variables): המתכנת יכול להשתמש בשמות משתנים סימבוליים, והמתרגם יקצה עבורם כתובות בזיכרון "אוטומטית". נשים לב שהערכים האמיתיים של הכתובות הללו אינם חשובים, כל עוד כל סימבול מתורגם לאותה הכתובת בכל במהלך התרגום של התוכנית.
- תוויות (Labels): המתכנת יכול לסמן מקומות בתוכנית עם סימבולים. למשל, יכול להצהיר על התווית *loop* שתצביע להתחלה של מקטע קוד מסוים. פקודות אחרות בתוכנית יכולות *goto loop*, בהתאם לתנאי או ללא תנאי.

Symbol Resolution: נתבון בתוכנית הבאה הכתובה בשפה low-level:

Code with symbols

```
00 // Computes sum=1+...+100
01   i=1
02   sum=0
03 loop:
04   if i=101 goto end
05   sum=sum+i
06   i=i+1
07   goto loop
08 end:
09 goto end
```

התוכנית מכילה 4 סימבולים המוגדרים ע"י המשתמש: שני משתנים – i ו- sum , ושתי תוויות – $loop$ ו- end . איך נוכל להמיר את התוכנית הזו לקוד ללא סימבולים?

ראשית נקבע שני כללי משחק שרירותיים: הקוד המתורגם יישמר בזיכרון של המחשב החל מהכתובת 0, והמקומות בזיכרון החל מהכתובת 1024 יוקצו עבור המשתנים. לאחר מכן, נבנה symbol table באופן הבא: עבור כל סימבול xxx בו ניתקל בקוד המקור, נוסיף שורה (xxx, n) ל- symbol table, כאשר n היא הכתובת בזיכרון המזוהה עם הסימבול (בהתאם לכללים). לאחר בניית ה- symbol table, נשתמש בה לתרגום התוכנית לגרסה ללא סימבולים:

Symbol table		Code with symbols resolved	
i	1024	00	M[1024]=1 // (M=memory)
sum	1025	01	M[1025]=0
loop	2	02	if M[1024]=101 goto 6
end	6	03	M[1025]=M[1025]+M[1024]
		04	M[1024]=M[1024]+1
		05	goto 2
		06	goto 6

(assuming that variables are allocated to Memory[1024] onward)

(assuming that each symbolic command is translated into one word in memory)

נשים לב שלפי הכללים, למשתנה i מוקצה הכתובת 1024 ולמשתנה sum מוקצה הכתובת 1025 (כל שני מספרים יתאימו כאן, כל עוד כל הרפרנסים ל- i ול- sum בתוכנית יהיו לאותן הכתובות).

הערות:

- נשים לב שמהנחה על הקצאת המקומות למשתנים נובע כי התוכנית הארוכה ביותר שנוכל להריץ היא באורך של 1024 הוראות. כיוון שתוכניות אמיתיות הרבה יותר גדולות, כתובת הבסיס לשמירת משתנים תהיה גבוהה יותר.
- ההנחה שכל פקודה בקוד המקור ממופה למקום אחד בזיכרון יכולה להיות נאיבית. בדרך כלל, פקודות אסמבלי מסוימות (כמו למשל $if\ i = 101\ goto\ end$) יכולות להיות מתורגמות למספר הוראות מכונה ולכן יתפסו מספר מקומות בזיכרון. ה- translator יכול להתמודד עם זה ע"י מעקב אחר מספר המילים שפקודה אחת מייצרת ולעדכן זאת ב"instruction memory counter" שלו בהתאם.
- ההנחה שכל משתנה מיוצג ע"י מקום אחד בזיכרון היא גם נאיבית. בשפות תכנות יש משתנים מסוגים שונים והם תופסים מקום שונה בזיכרון של מחשב היעד. (למשל, ב-C הסוג short מייצג 16-ביט והסוג double מייצג 64-ביט, אז אם תוכנית C רצה על מכונה של 16-ביט משתנים מסוג short יתפסו כתובת זיכרון אחת ומשתנים מסוג double יתפסו בלוק של 4 כתובות רצופות בזיכרון). לכן, כאשר מקצים מקום בזיכרון עבור משתנים, צריך לקחת בחשבון את הסוגים שלהם ואת רוחב המילה שיש בחומרת היעד.

The Assembler: לפי שניתן להריץ תוכנית אסמבלי על מחשב, חייבים לתרגם אותה לשפת המכונה הבינארית של המחשב. התרגום נעשה ע"י תוכנית שנקראת אסמבלר. האסמבלר מקבל בקלט stream של פקודות אסמבלי ומחזיר כפלט stream של הוראות בינאריות שקולות שהוא מייצר.

בהתאם ל-machine language specification, לא קשה לכתוב תוכנית שעבור כל פקודה סימבולית מבצעת את הפעולות הבאות (לא בהכרח לפי הסדר):

- לפרסר את הפקודה הסימבולית לשדות שלה.
- עבור כל שדה, לייצר את הביטים המתאימים בשפת המכונה.
- להחליף את כל הרפרנסים הסימבוליים (אם קיימים) עם כתובות מספריות של מקומות בזיכרון.
- להרכיב מהקודים הבינארים הוראת מכונה שלמה.

תרגום Hack Assembly לקוד בינארי

Syntax Convention and File Formats

שמות קבצים: תכונות בקוד מכונה ביארי ובקוד אסמבלי נשמרות בקבצי טקסט עם סיומות של "hack" ו-"asm" בהתאמה. לכן, קובץ Prog.asm מתורגם ע"י האסמבלר לקובץ Prog.hack.

Files (.hack) Binary Code: קוד בינארי מורכב משורות טקסט, כאשר כל שורה היא רצף של 16 תווי ASCII שהם "0" או "1" המקודדים הוראה אחת בשפת מכונה של 16-ביט. יחד, כל השורות בקובץ מייצגות תוכנית בשפת מכונה. כאשר טוענים תוכנית בשפת מכונה ל-instruction memory של המחשב, הקוד הבינארי בשורה ה-n של הקובץ נשמר בכתובת n של ה-instruction memory (הספירה של השורות התכנית ושל הכתובת מתחילה ב-0).

Files (.asm) Assembly Language: קובץ בשפת אסמבלי מורכב משורות טקסט, כאשר כל שורה מייצגת הוראה או הצהרה על סימבול:

- הוראה: A-instruction או C-instruction (מתוארות בהמשך).
- (symbol): פסאודו-פקודה הקושרת בין Symbol למקום בזיכרון בו שמורה הפקודה הבאה של התוכנית. זה נקרא "פסאודו-פקודה" כיוון שהיא לא מייצרת קוד מכונה.

קבועים וסמלים: קבועים חייבים להיות אי-שליליים ותמיד כתובים בסימון הדצימלי שלהם. סמל המוגדר ע"י המשתמש יכול להיות כל רצף של אותיות, ספרות, underscore (_), נקודה (.), דולר (\$), ונקודותיים (:). שלא מתחיל בספרה.

Comments: טקסט שמתחיל בשני סלאשים (//) ומסתיים בסוף השורה הוא הערה ומתעלמים ממנו.

White Space: מתעלמים מתווי רווח ומשורות ריקות.

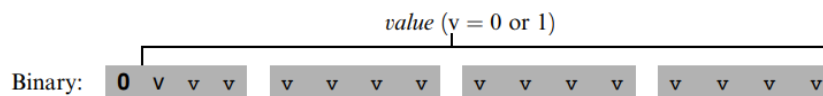
Case Conventions: כל ה-mnemonics של האסמבלי חייבים להיות כתובים באותיות גדולות. כל השאר (תוויות המוגדרות ע"י המשתמש ושמות משתנים) הם case sensitive. הקונבנציה היא להשתמש באותיות גדולות עבור תוויות ובאותיות קטנות עבור שמות משתנים.

הוראות – Instructions

שפת המכונה Hack מכילה שני סוגי הוראות הנקראים addressing instruction (A-instruction) ו-compute instruction (C-instruction).

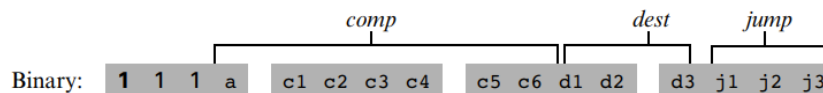
הפורמט של A-instruction:

A-instruction: *@value* // Where *value* is either a non-negative decimal number
// or a symbol referring to such number.



הפורמט של C-instruction:

C-instruction: *dest=comp;jump* // Either the *dest* or *jump* fields may be empty.
// If *dest* is empty, the "=" is omitted;
// If *jump* is empty, the ";" is omitted.



התרגום של כל אחד מ-3 השדות *jump*, *dest*, *comp* לצורה הבינארית שלו מפורט ב-3 הטבלאות הבאות:

תרגום comp:

<i>comp</i> (when a=0)	c1	c2	c3	c4	c5	c6	<i>comp</i> (when a=1)
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

תרגום *dest*:

<i>dest</i>	d1	d2	d3
null	0	0	0
M	0	0	1
D	0	1	0
MD	0	1	1
A	1	0	0
AM	1	0	1
AD	1	1	0
AMD	1	1	1

תרגום *jump*:

<i>jump</i>	j1	j2	j3
null	0	0	0
JGT	0	0	1
JEQ	0	1	0
JGE	0	1	1
JLT	1	0	0
JNE	1	0	1
JLE	1	1	0
JMP	1	1	1

סמלים – Symbols

פקודות של Hack Assembly יכולות להתייחס למקומות בזיכרון (כתובות) ע"י שימוש בקבועים או בסימבולים, כאשר הסימבולים נוצרים מ-3 מקורות:

1. **Predefined Symbols**: כל תוכנית Hack Assembly יכולה להשתמש בסימבולים

מוגדרים מראש:

<i>Label</i>	<i>RAM address</i>	<i>(hexa)</i>
SP	0	0x0000
LCL	1	0x0001
ARG	2	0x0002
THIS	3	0x0003
THAT	4	0x0004
R0-R15	0-15	0x0000-f
SCREEN	16384	0x4000
KBD	24576	0x6000

2. **Label Symbols**: הפסאודו-פקודה (*Xxx*) מגדירה את הסימבול *Xxx* להצביע למקום ב- instruction memory המחזיק את הפקודה הבאה בתוכנית. אפשר להגדיר תוויות פעם אחת בלבד, וניתן להשתמש בה בכל מקום בתוכנית האסמבלי, גם לפני השורה בה היא הוגדרה.

3. **Variable Symbols**: כל סימבול *Xxx* המופיע בתוכנית אסמבלי שאינו מוגדר מראש או מוגדר במקום אחר בתוכנית ע"י הפקודה (*Xxx*) הוא משתנה. משתנים ממופים למקומות רציפים בזיכרון כאשר נתקלים בהם לראשונה, החל מהכתובת 16 של ה-RAM.

דוגמה לתרגום

Assembly code (Prog.asm)

```
// Adds 1 + ... + 100
    @i
    M=1    // i=1
    @sum
    M=0    // sum=0
(LLOOP)
    @i
    D=M    // D=i
    @100
    D=D-A  // D=i-100
    @END
    D;JGT  // if (i-100)>0 goto END
    @i
    D=M    // D=i
    @sum
    M=D+M  // sum=sum+i
    @i
    M=M+1  // i=i+1
    @LLOOP
    0;JMP  // goto LOOP
(END)
    @END
    0;JMP  // infinite loop
```

Assembler

Binary code (Prog.hack)

```
(this line should be erased)
0000 0000 0001 0000
1110 1111 1100 1000
0000 0000 0001 0001
1110 1010 1000 1000
(this line should be erased)
0000 0000 0001 0000
1111 1100 0001 0000
0000 0000 0110 0100
1110 0100 1101 0000
0000 0000 0001 0010
1110 0011 0000 0001
0000 0000 0001 0000
1111 1100 0001 0000
0000 0000 0001 0001
1111 0000 1000 1000
0000 0000 0001 0000
1111 1101 1100 1000
0000 0000 0000 0100
1110 1010 1000 0111
(this line should be erased)
0000 0000 0001 0010
1110 1010 1000 0111
```

מימוש

התרגום של כל פקודת אסמבלי למבנה הבינארי השקול שלה הוא ישיר, כאשר כל פקודה מתורגמת בנפרד. בפרט, כל רכיב mnemonic (שדה) של פקודת האסמבלי מתורגם לקוד הבינארי המתאים (מהטבלאות שראינו) וכל סימבול בפקודה מתורגם לכתובת מספרית (כפי שראינו קודם).

המימוש מורכב מ-4 מודולים: **Parser** שמפרסר את הקלט, **Code** שמספק את הקוד הבינארי של כל ה-mnemonics באסמבלי, **SymbolTable** שמטפל בסימבולים, ו-**main** (תוכנית ראשית) שמריצה את כל תהליך התרגום.

The Parser Module: המטרה הראשית של ה-parser היא לפרק כל פקודת אסמבלי לרכיבים שלה (שדות וסימבולים). הוא קורא פקודה בשפת אסמבלי, מפרסר אותה, ומספק גישה נוחה לרכיבים של הפקודה, ובנוסף מסיר את כל ה-white spaces ואת ה-comments.

ה-API של ה-parser:

הפונקציה	ערך ההחזרה	ארגומנטים	Routine
פותחת את קובץ הקלט ומתכוננת לפרסר אותו	-	קובץ קלט	Constructor
האם יש עוד פקודות בקלט?	בוליאני (true או false)	-	hasMoreCommands
קוראת את הפקודה הבאה מהקלט והופך אותה להיות הפקודה הנוכחית. קוראים לפונקציה רק אם	-	-	advance

<i>hasMoreCommands()</i> מחזירה true. בתור התחלה אין פקודה נוכחית.			
מחזירה את סוג הפקודה הנוכחית: A_COMMAND עבור @Xxx כאשר Xxx הוא סימבול או מספר דצימלי, C_COMMAND עבור ,dest = comp; jump או L_COMMAND עבור (Xxx) כאשר Xxx הוא סימבול.	A_COMMAND ,C_COMMAND L_COMMAND	-	commandType
מחזירה את הסימבול או את המספר Xxx של הפקודה הנוכחית @Xxx או (Xxx). צריך לקרוא לפונקציה רק כאשר <i>commandType()</i> היא A_COMMAND או L_COMMAND.	מחרוזת	-	symbol
מחזירה את ה-mnemonic ב-C-command ב-dest הנוכחית (8 אפשרויות). צריך לקרוא לפונקציה רק כאשר <i>commandType()</i> היא C_COMMAND.	מחרוזת	-	dest
מחזירה את ה-mnemonic ב-C-command ב-comp הנוכחית (28 אפשרויות). צריך לקרוא לפונקציה רק כאשר <i>commandType()</i> היא C_COMMAND.	מחרוזת	-	comp
מחזירה את ה-mnemonic ב-C-command ב-jump הנוכחית (8 אפשרויות). צריך לקרוא לפונקציה רק כאשר <i>commandType()</i> היא C_COMMAND.	מחרוזת	-	jump

The Code Module: מתרגם mnemonics של Hack Assembly לקוד בינארי. ה-API:

הפונקציה	ערך ההחזרה	ארגומנטים	Routine
מחזירה את הקוד הבינארי של ה- dest mnemonic	3 ביטים	Mnemonic (מחרוזת)	dest
מחזירה את הקוד הבינארי של ה- comp mnemonic	7 ביטים	Mnemonic (מחרוזת)	comp
מחזירה את הקוד הבינארי של ה- jump mnemonic	3 ביטים	Mnemonic (מחרוזת)	jump

אסמבלר עבור תוכניות ללא סימבולים: נציע לבנות את האסמבלר בשני שלבים – בשלב הראשון כתיבת אסמבלר שמתרגם פקודות אסמבלי ללא סימבולים (אפשר לעשות זאת ע"י המודולים Parser ו-Code), ובשלב השני להרחיב את האסמבלר שיידע לטפל בסימבולים.

בשלב הראשון, הקלט הוא תוכנית Prog.asm שלא מכילה סימבולים, מה שאומר שבכל פקודות הכתובת מהצורה @Xxx ה-Xxx הם מספרים דצימליים, וקובץ הקלט לא כולל פקודות תווית – פקודות מהצורה (Xxx).

אפשר לממש את התוכניות הזו באופן הבא:

1. התוכנית תפתח קובץ פלט בשם Prog.hack.
 2. התוכנית תעבור על השורות (הוראות האסמבלי) בקובץ הקלט.
- עבור כל C-instruction: התוכנית תשרשר את הקודים הבינאריים של השדות בהוראה למילה אחת של 16-ביט, ולאחר מכן תכתוב את המילה לקובץ Prog.hack.
 - עבור כל A-instruction מהצורה @Xxx: התוכנית תתרגם את הקבוע הדצימלי שה-parser מחזיר לייצוג הבינארי שלו ותכתוב את התוצאה (מילה של 16-ביט) לקובץ Prog.hack.

The SymbolTable Module: כיוון שהוראות Hack יכולות להכיל סימבולים, צריך להמיר את הסימבולים לכתובות אמיתיות כחלק מתהליך התרגום. כדי לעשות זאת, האסמבלר יוצר ומתחזק symbol table שמכילה את ההתאמה בין סימבולים והמשמעות שלהם (במקרה של Hack – כתובות RAM ו-ROM). אפשר להשתמש ב-hash table בשביל זה.

כלומר, ה-SymbolTable מחזיק התאמה בין תוויות סימבוליות וכתובות מספריות. ה-API:

Routine	ארגומנטים	ערך החזרה	הפונקציה
Constructor	-	-	יוצר symbol table ריקה
addEntry	symbol (מחרוזת), address (int)	-	מוסיפה לטבלה את הזוג (symbol, address)
contains	symbol (מחרוזת)	בוליאני	האם ה-symbol table מכילה את ה-symbol הנתון כארגומנט?
getAddress	symbol (מחרוזת)	int	מחזירה את הכתובת המזוהה עם ה-symbol הנתון כארגומנט

אסמבלר עבור תוכניות עם סימבולים: תוכניות אסמבלי יכולות להשתמש בתוויות סימבוליות (יעדים של פקודות goto) לפני שהסימבולים מוגדרים. דרך נפוצה להתמודד עם זה היא לכתוב אסמבלר שמבצע שני מעברים, כלומר קורא פעמיים את כל הקוד מההתחלה לסוף. במעבר הראשון האסמבלר יבנה את ה-symbol table ולא ייצר קוד, ובמעבר השני בכל פעם שהוא ייתקל בתוויות סימבוליות הוא יחליף כל סימבול עם הכתובת המספרית המתאימה (כיוון שכבר נתקל בסימבול הזה במעבר הראשון) וייצר את הקוד הבינארי הסופי.

ניזכר שיש 3 סוגים של סימבולים בשפת ה-Hack: סימבולים מוגדרים מראש, תוויות ומשתנים. ה-symbol table תכיל את כל הסימבולים הללו ותטפל בהם.

אתחול: האסמבלר יאתחל את ה-symbol table עם כל הסימבולים המוגדרים מראש וכתובות ה-RAM שמוקצות להם, כפי שראינו.

מעבר ראשון: נעבור על כל תוכנית האסמבלי, שורה אחר שורה, ונבנה את ה-symbol table מבלי לייצר קוד. כאשר נעבור על השורות של התוכנית, נחזיק מונה עבור כתובת ה-ROM אליה נטען את הפקודה הנוכחית, כאשר נאתחל אותו ב-0 ונגדיל אותו ב-1 בכל פעם שניתקל ב-C-instruction או ב-A-instruction אבל לא נשנה אותו כאשר ניתקל בפקודת תווית או ב-comment. בכל פעם שניתקל בפקודת תווית (Xxx), נוסיף רשומה חדשה ל-symbol table המקשרת בין Xxx עם כתובת ה-ROM שבסוף תשמור את הפקודה הבאה בתוכנית. התוצאה של המעבר הזה היא הכנסה של כל התוויות בתוכנית יחד עם כתובות ה-ROM שלהן ל-symbol table כאשר נטפל במשתנים במעבר השני.

מעבר שני: נעבור שוב על כל התוכנית ונפרסר כל שורה. בכל פעם שניתקל ב-A-instruction סימבולית, כלומר $@Xxx$ כאשר Xxx הוא סימבול ולא מספר, נחפש את Xxx ב-symbol table. אם נמצא את הסימבול בטבלה, נחליף אותו עם הערך המספרי שלו ונשלים את תרגום הפקודה. אם לא נמצא אותו בטבלה, אז הוא בהכרח מייצג משתנה חדש. לכן נוסיף את הזוג (Xxx, n) ל-symbol table כאשר n היא כתובת ה-RAM הפנויה הבאה, ונשלים את תרגום הפקודה. כתובות ה-RAM שנקצה הן רצף מספרים שמתחיל בכתובת 16.

Virtual Machine (Arithmetic)

בפרקים 10-11 נתאר את הבנייה של **קומפיילר (Compiler)** שמטרתו לתרגם תוכניות high-level לקוד ביניים, ובפרקים 7-8 נתאר את הבנייה של **Translator** שמטרתו לתרגם את קוד הביניים לשפת מכונה של פלטפורמת החומרה.

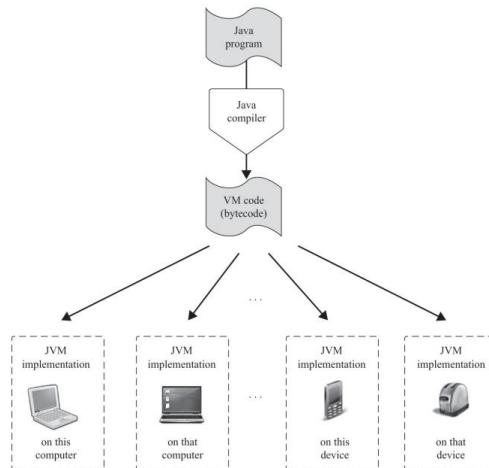
קוד הביניים רץ על מחשב אבסטרקטי שנקרא **מכונה וירטואלית – Virtual Machine (VM)**. יתרון אחד של מודל הקומפילציה הזה שהוא two-tier הוא שאותו קוד VM יכול לרוץ כפי שהוא על כל מכשיר עם מימוש VM כזה. אפשר לממש את ה-VM על מכשירי היעד ע"י שימוש ב-*interpreters*, או ע"י חומרה למטרות מיוחדות, או ע"י תרגום תוכניות VM לשפת המכונה של המכשיר.

נציג בפרק זה ארכיטקטורת VM ושפת VM טיפוסיות (שדומות בהתאמה ל-JVM ול-bytecode). המימוש של ה-VM מעל פלטפורמת ה-Hack כולל כתיבת תוכנית הנקראת VM translator שמתרגמת קוד VM לקוד אסמבלי של Hack. שפת ה-VM שנציג מכילה פקודות אריתמטיות-לוגיות, פקודות גישה לזיכרון שנקראות push ו-pop, פקודות branching, ופקודות call ו-return של פונקציות. בפרק הזה נבנה VM translator בסיסי שמממש פקודות אריתמטיות-לוגיות ופקודות push/pop.

The Virtual Machine Paradigm: לפני שתוכנית high-level יכולה לרוץ על מחשב יעד, צריך לתרגם אותה לשפת המכונה של המחשב. נפרק את תהליך הקומפילציה לשני שלבים – בשלב הראשון נפרסר ונתרגם את הקוד ה-high-level לקוד ביניים אבסטרקטי, ובשלב השני נתרגם את קוד הביניים לשפת מכונה low-level של חומרת היעד.

התרגום הראשון תלוי רק במפרט של שפת ה-high-level המקורית, והתרגום השני תלוי רק במפרט של שפת המכונה ה-low-level של היעד. כך ניתן לחלק את הקומפיילר לשתי תוכניות נפרדות והרבה יותר פשוטות: התוכנית הראשונה שעדיין תיקרא compiler תתרגם את הקוד ה-high-level לפקודות ביניים ב-VM, והתוכנית השנייה שנקראת VM translator תתרגם את פקודות ה-VM להוראות מכונה של פלטפורמת החומרה של היעד.

Two-Tiered Compilation Framework of Java Programs: תוכניות high-level מקומפלות לקוד ביניים ב-VM, אותו ניתן להריץ על כל פלטפורמת חומרה שיש לה מימוש JVM מתאים.



Stack Machine

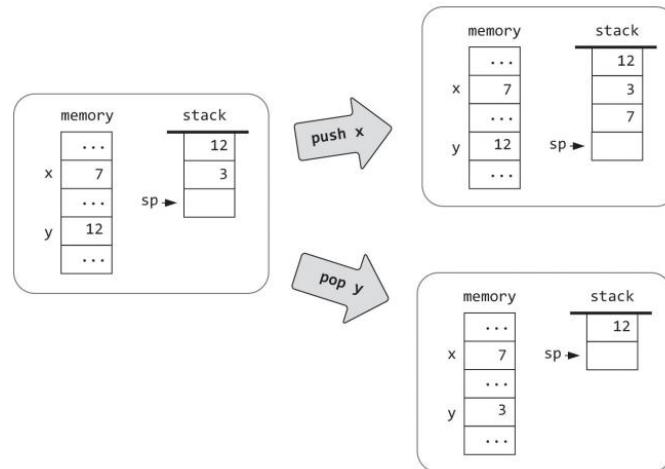
המטרה של שפת VM יעילה היא ליצור איזון נוח בין שפות תכנות high-level מצד אחד ושפות מכונה low-level מהצד השני. לכן על שפת ה-VM לספק דרישות המגיעות מלמעלה ומלמטה. נשיג זאת ע"י עיצוב שפת VM שיש לה פקודות אריתמטיות-לוגיות, פקודות push/pop, פקודות branching ופקודות של פונקציות, שהן מספיק "high" כדי שקוד ה-VM יהיה אלגנטי ובעל מבנה טוב ובו זמנית מספיק "low" כדי שקוד המכונה שנייצר ממנו יהיה יעיל. דרך לספק את הדרישות הללו היא לבסס את שפת הביניים הזו על ארכיטקטורה אבסטרקטית הנקראת **stack machine**.

Pop ו Push

המרכז של מודל ה-stack machine הוא מבנה נתונים אבסטרקטי שנקרא stack (מחסנית). מחסנית היא מרחב אחסון רציף שגדל וקטן לפי הצורך. המחסנית תומכת במספר פעולות, כאשר שתי פעולות המפתח הן push ו-pop.

פעולת ה-push מוסיפה ערך לראש המחסנית, ופעולת ה-pop מסירה את הערך שבראש המחסנית כאשר הערך שהיה לפניו הופך להיות בראש המחסנית. נשים לב שהלוגיקה של push/pop היא גישה לוגית של (LIFO) last-in-first-out – הערך שעושים לו pop הוא תמיד הערך האחרון שעשו לו push.

דוגמה:



אבסטרקציית ה-VM שלנו כוללת מחסנית וגם מקטע זיכרון רציף כמו RAM. נבחין שגישה למחסנית שונה מגישה קונבנציונלית לזיכרון. ראשית, ניתן לגשת למחסנית רק מהראש שלה, בעוד שזיכרון רגיל מאפשר גישה ישירה באמצעות אינדקס לכל ערך בזיכרון. שנית, ניתן לקרוא רק את הערך שנמצא בראש המחסנית והדרך היחידה לגשת אליו היא להסיר אותו מהמחסנית, בניגוד לקריאה מזיכרון רגיל שלא משנה את מצב הזיכרון. ודבר אחרון, כתיבה למחסנית היא הוספת ערך לראש המחסנית מבלי לשנות ערכים אחרים במחסנית, בניגוד לכתיבה למקום בזיכרון רגיל שדורסת את הערך הקודם שהיה בו.

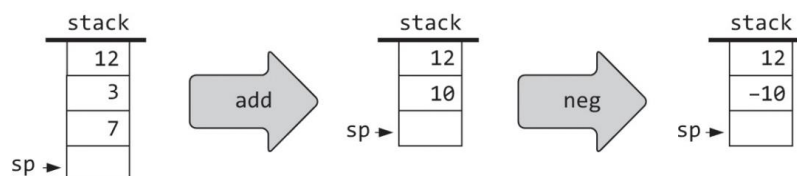
Stack Arithmetic

נתבונן בפעולה הגנרית $op\ x$ שבה האופרטור op מתייחס לאופרנדים x ו- y , למשל: $3 - 8, 7 + 5$, ועוד. ב-stack machine, כל פעולת $op\ x$ מתבצעת באופן הבא:

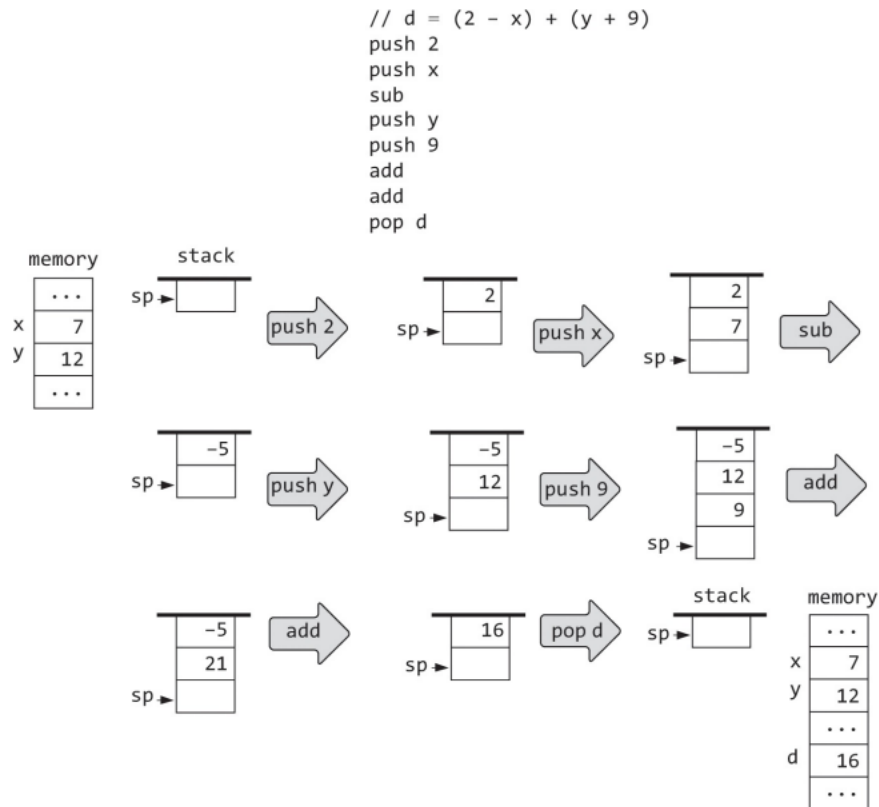
1. מוציאים את האופרנדים x ו- y מראש המחסנית (כלומר עושים להם pop).
2. מחשבים את הערך של $op\ x$.
3. מכניסים את הערך שחושב בשלב 2 לראש המחסנית (כלומר עושים לו push).

באופן דומה, אפשר לממש את הפעולה האונארית $op\ x$ ע"י הוצאת x מראש המחסנית, חישוב הערך $op\ x$ ולהכניס אותו לראש המחסנית.

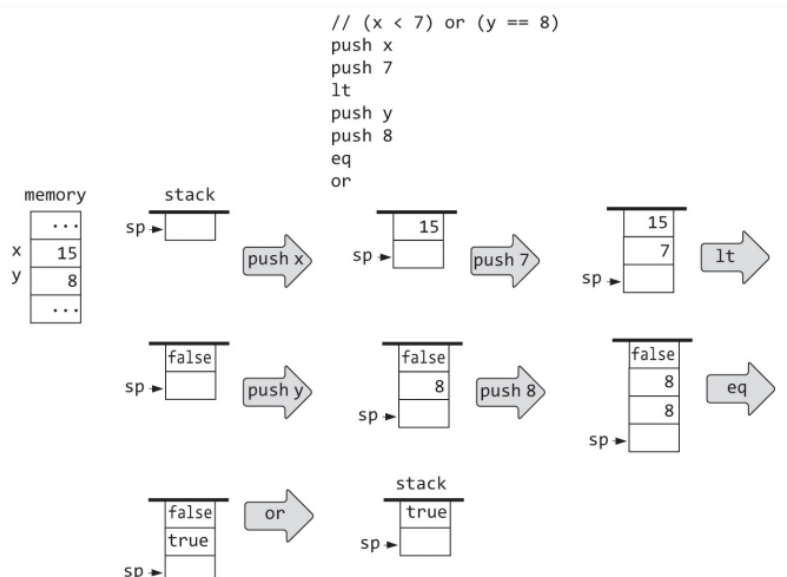
דוגמה – חיבור וחסור:



דוגמה – חישוב ביטויים אריתמטיים: נתבון בביטוי $d = (2 - x) + (y + 9)$ שנלקח מתוכנית high-level כלשהי. אז ה-stack-based evaluation שלו:



דוגמה – חישוב ביטויים לוגיים: נתבון בביטוי $(x < 7) \text{ or } (y == 8)$ שנלקח מתוכנית high-level כלשהי. אז ה-stack-based evaluation שלו:



נשים לב שמנקודת המבט של המחסנית, כל פעולה אריתמטית או לוגית מחליפה את האופרנדים של הפעולה עם תוצאת הפעולה, מבלי לשנות את שאר המחסנית.

דוגמה: כדי לחשב את הערך $6 - (11 + 7) \times 3$ נתחיל מ-`pop` של 11 ושל 7 והחישוב $11 + 7$, ואז להכניס את התוצאה בחזרה לביטוי, מה שמייצר את הביטוי $6 - 3 \times 18$. ההשפעה היא שהחלפנו את $(11 + 7)$ ב-18, שאר הביטוי נשאר כפי שהיה.

נקבל יתרון חשוב של `stack machine`: ניתן להמיר כל ביטוי אריתמטי ולוגי, לא משנה כמה הוא מסובך, לרצף של פעולות פשוטות על המחסנית. לכן, אפשר לכתוב קומפיילר שמתרגם ביטויים אריתמטיים ולוגיים הכתובים בשפה `high-level` לרצפים של פקודות מחסנית.

Virtual Memory Segments

נתאר פורמלית את פקודות ה-`push` וה-`pop`. לשפות `high-level` יש משתנים סימבוליים כמו x, y , `sum`, `count`, וכו'. אם השפה היא `object-based`, כל משתנה כזה יכול להיות משתנה `static` של המחלקה, `field` במופע של אובייקט, או משתנה `local` או `argument` של מתודה. במודל ה-`VM` שלנו אין משתנים סימבוליים. במקום, משתנים מיוצגים במקטעי זיכרון וירטואליים בעלי שמות כמו `static`, `this`, `local` ו-`argument`. בפרט, הקומפיילר ממפה את המשתנה הסטטי הראשון, השני, השלישי, ... שהוא מוצא בתוכנית `high-level` ל-`static 0`, `static 1`, `static 2`, וכך הלאה. שאר סוגי המשתנים ממופים באופן דומה למקטעים `local`, `this` ו-`argument`.

דוגמה: אם המשתנה הלוקאלי x ממופה ל-`local 1` והשדה y ממופה ל-`this 3`, הצהרה בתוכנית `high-level` כמו `let x = y` תתורגם ע"י הקומפיילר לפקודה `push this 3` שאחריה הפקודה `pop local 1`.

המקטעים בזיכרון: במודל ה-`VM` שלנו יש 8 מקטעי זיכרון:

מקטע (Segment)	תפקיד
<i>argument</i>	מייצג את הארגומנטים של הפונקציה
<i>local</i>	מייצג את המשתנים הלוקאליים של הפונקציה
<i>static</i>	מייצג את המשתנים הסטטיים שהפונקציה ראתה
<i>constant</i>	מייצג את הערכים הקבועים 0, 1, 2, 3, ..., 32767
<i>this</i>	בפרקים הבאים
<i>that</i>	בפרקים הבאים
<i>pointer</i>	בפרקים הבאים
<i>temp</i>	בפרקים הבאים

נשים לב שהגישה של פקודות VM לכל מקטעי הזיכרון הווירטואליים מתבצעת בדיוק באותה בדרך: ע"י שימוש בשם המקטע שאחריו אינדקס אי-שלילי.

VM Specification

מודל ה-VM שלנו הוא מבוסס מחסנית: כל פעולות ה-VM לוקחות את האופרנדים שלהן מהמחסנית ושומרות בה את התוצאות שלהן. יש רק data type אחד: signed integer של 16-ביט. תוכנית VM היא רצף של פקודות VM המחולקות ל 4 קטגוריות:

1. פקודות push/pop שמעבירות מידע בין המחסנית ובין מקטעי הזיכרון.
2. פקודות אריתמטיות-לוגיות המבצעות פעולות אריתמטיות ולוגיות.
3. פקודות branching המאפשרות פעולות branching מותנות או לא מותנות.
4. פקודות פונקציה המאפשרות פעולות קריאה לפונקציה וחזרה מפונקציה.

Comments and White Space: שורות שמתחילות ב-// נחשבות comments ונתעלם מהן. מותר שיהיו שורות ריקות ונתעלם מהן.

פקודות Push/Pop:

הפקודה *push segment index*: מכניסה את הערך של *segment[index]* למחסנית, כאשר *segment* הוא *argument*, *local*, *static*, *constant*, *this*, *that*, *pointer* או *temp* ו-*index* הוא מספר שלם ואי-שלילי.

הפקודה *pop segment index*: מוציאה את הערך הנמצא בראש המחסנית ושומרת אותו ב-*segment[index]*, כאשר *segment* הוא *argument*, *local*, *static*, *constant*, *this*, *that*, *pointer* או *temp* ו-*index* הוא מספר שלם ואי-שלילי.

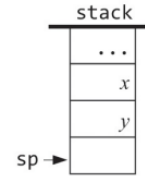
פקודות אריתמטיות-לוגיות:

- פקודות אריתמטיות: *neg*, *sub*, *add*
- פקודות השוואה: *lt*, *gt*, *eq*
- פקודות לוגיות: *not*, *or*, *and*

לפקודות *add*, *sub*, *eq*, *gt*, *lt*, *and* ו-*or* יש שני אופרנדים מפורשים. כדי לבצע אחת מהן, מימוש ה-VM מוציא את שני הערכים בראש המחסנית, מחשב את הפונקציה, ומכניס את התוצאה בחזרה למחסנית. לפקודות *neg* ו-*not* יש אופרנד אחד והן פועלות באופן דומה.

כל הפקודות האריתמטיות-לוגיות בשפת ה-VM:

Command	Computes	Comment
add	$x + y$	integer addition (two's complement)
sub	$x - y$	integer subtraction (two's complement)
neg	$-y$	arithmetic negation (two's complement)
eq	$x == y$	equality
gt	$x > y$	greater than
lt	$x < y$	less than
and	$x \text{ And } y$	bit-wise And
or	$x \text{ Or } y$	bit-wise Or
not	Not y	bit-wise Not



מימוש

נממש VM translator – תוכנית שמתרגמת פקודות VM להוראות בשפת מכונה. שתי המשימות העיקריות שלנו הן להחליט איך לייצג את המחסנית ואת הזיכרון הווירטואלי בפלטפורמת היעד, ולתרגם כל פקודת VM לרצף של הוראות low-level שפלטפורמת היעד תוכל להריץ.

לדוגמה, נניח שפלטפורמת היעד היא מכונת Von Neumann. במקרה הזה, נוכל לייצג את מחסנית ה-VM ע"י בלוק זיכרון ייעודי ב-RAM. הקצה התחתון של הבלוק הזה יהיה כתובת בסיס קבועה, והקצה העליון שלו ישתנה כאשר המחסנית גדלה וקטנה. לכן, בהינתן כתובת $stackBase$ קבועה, נוכל לנהל את המחסנית ע"י מעקב אחרי משתנה אחד – $Stack\ Pointer$ או SP , שמחזיק את הכתובת של הכניסה ב-RAM שנמצאת אחרי הערך שבראש במחסנית. כדי לאתחל את המחסנית, נגדיר את SP להיות $stackBase$. מהנקודה הזו והלאה, אפשר לממש כל פקודת $push\ x$ ע"י פעולות הפסאודו-קוד $RAM[SP] = x$ שאחריה $SP++$, ואפשר לממש כל פקודת $pop\ x$ ע"י SP שאחריה $x = RAM[SP]$.

נניח שהפלטפורמה היא מחשב Hack ושקבענו שבסיס המחסנית בכתובת 256 ב-RAM של ה-Hack. במקרה הזה, ה-VM translator יכול להתחיל בייצור קוד אסמבלי המממש את $SP = 256$:

@256

D = A

@SP

M = D

מנקודה זו והלאה, ה-VM translator יוכל לטפל בכל פקודת $push\ x$ ו- $pop\ x$ ע"י ייצור קוד אסמבלי שמממש את הפעולות $RAM[SP++] = x$ ו- $x = RAM[SP--]$ בהתאמה.

Standard VM Mapping on the Hack Platform

נרצה לממש את אבסטרקציית ה-VM על פלטפורמת חומרה ספציפית. בעיקרון, אנחנו יכולים לעשות זאת איך שאנחנו רוצים כל עוד המימוש אמין ויעיל. אבל יש הנחיות מימוש בסיסיות הידועות בתור

standard mappings עבור פלטפורמות חומרה שונות. בחלק זה נפרט את המיפוי הסטנדרטי של אבסטרקציית ה-VM שלנו על מחשב ה-Hack.

תוכנית VM: בינתיים, נסתכל על תוכנית VM בתור רצף של פקודות VM השמורות בקובץ טקסט שנקרא FileName.vm (האות הראשונה היא גדולה). ה-VM translator צריך לקרוא כל שורה בקובץ, להתייחס אליה כאל פקודת VM, ולתרגם אותה להוראה אחת או יותר הכתובה בשפת Hack. התוצאה היא רצף של הוראות באסמבלי של Hack שנשמור בקובץ טקסט שנקרא FileName.asm והוא יוחזר כפלט.

Data Type: לאבסטרקציית ה-VM יש רק data type אחד – signed integer. הטיפוס הזה ממומש בפלטפורמת ה-Hack כערך 16-ביט בשיטת המשלים ל-2. ערכי ה-VM הבוליאניים true ו-false מיוצגים ע"י 1- ו-0 בהתאמה.

שימוש ב-RAM: ה-RAM של ה-Hack מכיל 32K מילים של 16-ביט. מימושי VM צריכים להשתמש בחלק העליון של מרחב הכתובות הזה באופן הבא:

שימוש	כתובות ה-RAM
16 רגיסטרים וירטואליים (פירוט בהמשך)	0 – 15
משתנים סטטיים	16 – 255
מחסנית	256 – 2047

ניזכר כי ניתן לגשת לכתובות 0 עד 4 ב-RAM באמצעות הסימבולים *SP*, *LCL*, *ARG*, *THIS* ו-*THAT*. השימוש של הכתובות הללו במימוש ה-VM יהיה באופן הבא:

שם	מיקום	שימוש
SP	RAM[0]	<i>Stack Pointer</i> : הכתובת בזיכרון שאחרי כתובת הזיכרון שמכילה את הערך שבראש המחסנית
LCL	RAM[1]	כתובת הבסיס של המקטע <i>local</i>
ARG	RAM[2]	כתובת הבסיס של המקטע <i>argument</i>
THIS	RAM[3]	כתובת הבסיס של המקטע <i>this</i>
THAT	RAM[4]	כתובת הבסיס של המקטע <i>that</i>
TEMP	RAM[5 – 12]	המקטע <i>temp</i>
R13 R14 R15	RAM[13 – 15]	אם קוד האסמבלי שה-VM translator מייצר צריך משתנים, ניתן להשתמש ברגיסטרים הללו

כתובת הבסיס של מקטע היא כתובת פיזית ב-RAM. למשל, אם נרצה למפות את המקטע *local* למקטע הפיזי ב-RAM שמתחיל בכתובת 1017, נוכל לכתוב קוד Hack שקובע את הערך של *LCL* להיות 1017. נשים לב שההחלטה היכן למקם מקטעי זיכרון וירטואליים היא עניין עדין. לדוגמה, בכל

פעם שפונקציה מתחילה לרוץ, נצטרך להקצות מקום ב-RAM שיחזיק את מקטעי הזיכרון *local* ו-*argument* שלה. וכאשר פונקציה קוראת לפונקציה אחרת, נצטרך "להשהות" את המקטעים הללו ולהקצות מקום נוסף ב-RAM כדי לייצג את המקטעים של הפונקציה הנקראת. נדבר על זה בפרק הבא.

כעת, אנחנו יכולים להניח ש-*SP*, *ARG*, *LCL*, *THIS*, ו-*THAT* כבר מאותחלים לכתובות ב-RAM. נשים לב שבכל מקרה המימושים של ה-VM לא רואים את הכתובות הללו לעולם. הם יכולים לפעול עליהם בצורה סימבולית ע"י שימוש בשמות המצביעים.

דוגמה: נניח שאנחנו רוצים לעשות *push* לערך של הרגיסטר *D* למחסנית. אפשר לממש את הפעולה הזאת ע"י הלוגיקה $RAM[SP + +] = D$, או באסמבלי של Hack:

```
A = M
M = D
@SP
M = M + 1
```

Memory Segments Mapping

local, *argument*, *this*, *that*: נראה בפרק הבא איך המימוש של ה-VM ממפה את המקטעים הללו באופן דינאמי ב-RAM. כעת, כל שעלינו לדעת זה שכתובות הבסיס של המקטעים הללו שמורות ברגיסטרים *LCL*, *ARG*, *THIS*, *THAT* בהתאמה. לכן, כל גישה לרשומה ה-*i* של מקטע וירטואלי (בהקשר של פקודת "*push/pop segmentName i*") צריכה להיות מתורגמת לקוד אסמבלי שניגש לכתובת $(base + i)$ ב-RAM, כאשר *base* הוא אחד מהמצביעים *LCL*, *ARG*, *THIS* או *THAT*.

pointer: המקטע *pointer* מכיל בדיוק שני ערכים וממופה ישירות למקומות 3 ו-4 ב-RAM. ניזכר כי המקומות האלו ב-RAM נקראים גם *THIS* ו-*THAT* בהתאמה. לכן, כל גישה ל-*pointer* 0 אמורה להיות גישה למצביע *THIS*, וכל גישה ל-*pointer* 1 אמורה להיות גישה למצביע *THAT*.

דוגמה: הפקודה *pop pointer 0* צריכה לשנות את הערך של *THIS* להיות הערך שעשו לו *pop*, והפקודה *push pointer 1* צריכה לדחוף למחסנית את הערך הנוכחי של *THAT*.

temp: מקטע קבוע של 8 מילים שממופע ישירות למקומות 5 עד 12 ב-RAM. כל גישה ל-*temp i* כאשר $0 \leq i \leq 7$, צריכה להיות מתורגמת לקוד אסמבלי שניגש למקום $5 + i$ ב-RAM.

constant: מקטע הזיכרון הזה הוא באמת וירטואלי, שכן הוא לא תופס מקום פיזי ב-RAM. במימוש של ה-VM, כל גישה ל-*constant i* היא למעשה סיפוק של הקבוע *i*.

דוגמה: הפקודה *push constant 17* צריכה להיות מתורגמת לקוד אסמבלי שדוחף את הערך 17 למחסנית.

static: משתנים סטטיים ממופים לכתובות 16 עד 255 ב-RAM. ה-VM translator יכול לממש את המיפוי הזה באופן אוטומטי: כל רפרנס ל-*static i* בתוכנית VM שנמצא בקובץ *Foo.vm* יכו להיות מתורגם לסימבול אסמבלי *Foo.i*, והאסמבלר של ה-Hack ימפה את המשתנים הסימבוליים הללו ל-RAM החל מהכתובת 16. הקונבנציה הזו תגרום לכך שמשתנים סטטיים המופיעים בתוכנית VM ימופו לכתובות החל מהכתובת 16, לפי הסדר בו הם מופיעים בקוד ה-VM.

דוגמה: נתבונן בתוכנית VM שמתחילה בקוד הבא:

```
push constant 100
push constant 200
pop static 5
pop static 2
```

התרגום ימפה את *static 5* לכתובת 16 ב-RAM ואת *static 6* לכתובת 16 ב-RAM.

המימוש הזה של משתנים סטטיים גורם למשתנים סטטיים של קבצי VM שונים להתקיים במקביל, כיוון שלסימבולים *FileName.i* שלהם יש רישא ייחודית של שם הקובץ. נשים לב שכיוון שהמחסנית מתחילה בכתובת 256, המימוש מגביל את מספר המשתנים הסטטיים בתוכנית Jack ל- $255 - 16 + 1 = 240$ משתנים לכל היותר.

Assembly Language Symbols: נניח שתוכנית VM שאנחנו צריכים לתרגם שמורה בקובץ שנקרא *Foo.vm*. ה-VM translator שמבצע את מיפוי סטנדרטי של VM על פלטפורמת ה-Hack מייצר קוד אסמבלי שמשתמש בסימבולים הבאים: *SP, LCL, ARG, THIS, THAT*, ו-*Foo.i* כאשר *i* הוא מספר שלם ואי-שלילי. אם צריך לייצר קוד שמשתמש במשתנים שצריך לשמור זמנית, ה-VM translator יכול להשתמש בסימבולים *R13, R14* ו-*R15*.

Design Suggestions for the VM Implementation

שימוש: ה-VM translator מקבל ארגומנט אחד מה-command-line באופן הבא:

```
prompt > VMTranslator source
```

כאשר *source* הוא שם קובץ מהצורה *ProgName.vm*. האות הראשונה בשם הקובץ צריכה להיות אות גדולה, והסיומת *vm* היא הכרחית. הקובץ מכיל רצף של פקודות VM אחת או יותר. ה-VM translator יוצר קובץ פלט בשם *ProgName.asm* שמכיל הוראות אסמבלי שמממשות את פקודות ה-VM.

מבנה התוכנית: נציע לממש את ה-VM translator באמצעות 3 מודולים: תוכנית *main* הנקראת *VMTranslator*, *Parser* ו-*CodeWriter*. התפקיד של *Parser* היא להבין מה כל פקודת VM אמורה לעשות, התפקיד של *CodeWriter* הוא לתרגם את פקודת ה-VM המובנת להוראות אסמבלי שמממשות את הפעולה הרצויה על פלטפורמת ה-Hack, והתפקיד של *VMTranslator* הוא לנהל את תהליך התרגום.

The Parser: המודול הזה מפרסר קובץ vm. יחיד. הוא מספק שירותים של קריאת פקודת VM, פירוק הפקודה לרכיבים שלה, וסיפוק גישה נוחה לרכיבים הללו. בנוסף, ה-parser מתעלם ממכל ה-white spaces ומ-comments. המטרה של ה-parser היא לטפל בכל פקודות ה-VM, כולל פקודות branching ופקודות פונקציה אותן נממש בפרק הבא. ה-API:

פונקציה	ערך החזרה	ארגומנטים	Routine
לפתוח את קובץ הקלט ולהתכונן לפרסר אותו	-	קובץ קלט	Constructor
האם יש עוד שורות בקלט?	boolean	-	hasMoreLines
קוראת את הפקודה הבאה מהקלט והופכת אותה לפקודה הנוכחית. צריך לקרוא לה רק אם <i>hasMoreLine</i> היא true. באתחול אין פקודה נוכחית	-	-	advance
מחזירה קבוע המייצג את הסוג של הפקודה הנוכחית. אם הפקודה הנוכחית היא פקודה אריתמטית-לוגית, תחזיר C_ARITHMETIC	C_ARITHMETIC, C_PUSH, C_POP, C_LABEL, C_GOTO, C_IF, C_FUNCTION, C_RETURN, C_CALL	-	commandType
מחזירה את הארגומנט הראשון של הפקודה הנוכחית. במקרה של C_ARITHMETIC, מחזירה את הפקודה עצמה. לא צריך לקרוא לה אם הפקודה הנוכחית היא C_RETURN	string	-	arg1
מחזירה את הארגומנט השני של הפקודה הנוכחית. צריך לקרוא לה רק אם הפקודה הנוכחית היא C_PUSH, C_POP, C_FUNCTION או C_CALL	int	-	arg2

דוגמה: אם הפקודה הנוכחית היא *push local 2*, אז קריאה ל-*arg1()* תחזיר "*local*" וקריאה ל-*arg2()* תחזיר 2. אם הפקודה הנוכחית היא *add*, אז קריאה ל-*arg1()* תחזיר "*add*" ו-*arg2()* לא תיקרא.

The CodeWriter: מתרגם פקודת VM אחרי שפרסרו אותה לקוד באסמבלי של Hack. ה-API:

פונקציה	ארגומנטים	Routine
לפתוח את קובץ הפלט ולהתכונן לכתוב אליו	קובץ פלט	Constructor
כותבת לקובץ הפלט את קוד האסמבלי שמממש את הפקודה הלוגית-אריתמטית הנתונה command	command (string)	writeArithmetic
כותבת לקובץ הפלט את קוד האסמבלי שמממש את פקודה ה-push או pop הנתונה command	command (C_PUSH/C_POP), segment (string), index (int)	writePushPop

דוגמה: הקריאה `writePushPop(C_PUSH, "local", 2)` תייצר הוראות אסמבלי שמממשות את פקודת ה-`push local 2` VM. הקריאה `writeArithmetic("add")` תייצר הוראות אסמבלי שעושות pop לשני האיברים שנמצאים בראש המחסנית, מחברות אותם ודוחפות את התוצאה למחסנית.

The VM Translator: התוכנית הראשית שמנהלת את תהליך התרגום, באמצעות שימוש ב-Parser וב-CodeWriter. היא מקבלת את השם של קובץ הקלט `Prog.vm` מה-command-line. היא בונה Parser כדי לפרסר את קובץ הקלט, ויוצאת קובץ פלט `Prog.asm` אליו. היא תכתוב את הוראות האסמבלי המתורגמות. עבור כל פקודה, התוכנית תשתמש ב-Parser וב-CodeWriter כדי לפרסר את הפקודה לשדות שלה ולייצר מהם רצף של הוראות אסמבלי שהיא תכתוב לקובץ הפלט.

Virtual Machine (Control)

בפרק הקודם למדנו כיצד להשתמש בפקודות אריתמטיות-לוגיות ובפקודות push/pop ב-VM וכיצד לממש אותם. בפרק זה נלמד כיצד להשתמש וכיצד לממש פקודות branching ופקודות פונקציה ב-VM, כך שנקבל VM translator שלם מעל פלטפורמת ה-Hack.

נראה כיצד אפשר להשתמש במחסנית כדי לבצע משימות מסובכות כמו קריאות מקוננות לפונקציות, העברת פרמטרים, רקורסיה, ומשימות הקצאה ומחזור של זיכרון כדי לתמוך בהרצת התוכנית תוך כדי זמן ריצה.

Run-Time System: לכל מערכת מחשב חייב להיות מודל run-time. המודל הזה עונה על שאלות חשובות שבלעדיהן תוכניות לא יכולות לרוץ – כיצד להתחיל הרצה של תוכנית, מה המחשב צריך לעשות כאשר תוכנית מסתיימת, כיצד להעביר ארגומנטים מפונקציה אחת לאחרת, כיצד להקצות משאבי זיכרון לפונקציות רצות, כיצד לשחרר משאבי זיכרון כאשר אין בהם צורך, ועוד.

High-Level Magic

שפות high-level מאפשרות כתיבת תוכניות במונחים שהם high-level.

דוגמה: ביטוי כמו $x = -b + \sqrt{b^2 - 4 \cdot a \cdot c}$ ניתן לכתוב בצורה המפורשת הבאה:
$$x = -b + \text{sqrt}(\text{power}(b, 2) - 4 * a * c)$$

נשים לב להבדל בין פעולות פרימיטיביות כמו + או - שמובנות בסינטקס של השפה ה-high-level ובין פונקציות כמו sqrt או power שהן הרחבה של השפה הבסיסית.

פקודות branching מעניקות לשפה כוח ביטוי נוסף, ומאפשרות לכתוב קוד מותנה כמו
$$\text{if } (a == 0) \{ x = (-b + \text{sqrt}(\text{power}(b, 2) / 4 * a * c)) - (2 * a) \} \text{ else } \{ x = -c/b \}$$

שוב, ניתן לראות שקוד high-level מאפשר ביטוי של לוגיקה שהיא high-level.

אבל לא משנה כמה "גבוהה" תהיה השפה ה-high-level, לבסוף היא חייבת להיות ממומשת על פלטפורמת חומרה כלשהי שיכולה להריץ רק הוראות מכונה פרימיטיביות. לכן, צריך למצוא פתרונות low-level כדי לממש פקודות branching ופקודה קריאה וחזרה מפונקציה.

פונקציות: יחידות תכנות עצמאיות שיכולות לקרוא אחת לשנייה. למשל – הפונקציה solve יכולה לקרוא ל-sqrt, ו-sqrt יכולה לקרוא ל-power. רצף הקריאות יכול להיות עמוק בכל שנרצה, וגם רקורסיבי. הפונקציה הקוראת (ה-caller) מעבירה ארגומנטים לפונקציה הנקראת (ה-callee) ומשהה את הריצה שלה עד שהפונקציה הנקראת תשלים את הריצה שלה. ה-callee משתמשת בארגומנטים שהועברו לה כדי להריץ או לחשב משהו, ואז היא מחזירה ערך (שיכול להיות void) ל-caller. אחרי זה ה-caller חוזרת לפעולה וממשיכה את הריצה שלה.

באופן כללי, בכל פעם שפונקציה אחת (ה-caller) קוראת לפונקציה (ה-callee), צריך לטפל בדברים הבאים:

- שמירת כתובת החזרה, שהיא הכתובת בקוד של ה-caller אליה הריצה של התוכנית צריכה לחזור לאחר שה-callee מסיימת את הריצה שלה.
- שמירת משאבי הזיכרון של ה-caller.
- הקצאת משאבי הזיכרון הנדרשים עבור ה-callee.
- הארגומנטים שמועברים ע"י ה-caller צריכים להיות זמינים לקוד של ה-callee.
- התחלת ההרצה של הקוד של ה-callee.

כאשר ה-callee מסיימת את ריצתה ומחזירה ערך, צריך לטפל בדברים הבאים:

- ערך ההחזרה של ה-callee צריך להיות זמין לקוד של ה-caller.
- מחזור משאבי הזיכרון שהיו בשימוש ע"י ה-callee.
- החזרת משאבי הזיכרון של ה-caller שנשמרו קודם לכן.
- שליפת כתובת החזרה שנשמרה קודם לכן.
- המשך הרצת הקוד של ה-caller, החל מכתובת החזרה.

Branching

ברירת המחדל של תוכניות מחשב היא הרצה רציפה, כלומר ביצוע פקודה אחת אחרי השנייה. ממספר סיבות, כמו כניסה לאיטרציה חדשה בלולאה, אפשר לנתב מחדש את הרצף הזה בעזרת פקודות branching. בתכנות low-level, אפשר לעשות branching באמצעות פקודות *goto destination*. אפשר לפרט מהו היעד במספר דרכים, הפרימיטיבית ביותר היא הכתובת בזיכרון של ההוראה הבאה שצריך לבצע. פירוט אבסטרקטי יותר הוא תווית סימבולית (שמקושרת לכתובת פיזית בזיכרון). הגרסה הזו דורשת שיהיו בשפה הוראות "תיוג" שמטרתן להקצות תוויות סימבוליות למקומות נבחרים בקוד. בשפת ה-VM שלנו, נעשה זאת ע"י שימוש בפקודת *labeling* שהסינטקס שלה הוא *label symbol*.

שפת ה-VM תומכת בשתי צורות של branching:

1. Unconditional Branching: הפקודה *goto symbol* שמשמעותה לקפוץ לבצע את הפקודה שנמצאת אחרי פקודת ה-*label symbol* בקוד.
2. Conditional Branching: הפקודה *if – goto symbol* שמשמעותה היא: הוצאת הערך שנמצא בראש המחסנית, אם הוא לא *false* לקפוץ לבצע את הפקודה שנמצאת אחרי פקודת ה-*label symbol* בקוד, אחרת לבצע את הפקודה הבאה בקוד.

משתמע מכך שלפני פקודת *conditional goto* צריך ראשית לפרט תנאי. בשפת ה-VM שלנו, זה מתבצע ע"י דחיפת ביטוי בוליאני למחסנית.

דוגמה: הקומפיילר שנפתח בפרקים 10-11 יתרגם את הפקודה *goto LOOP if (n < 100)* ל:

```
push n
push 100
lt
if – goto LOOP
```

דוגמה: נתבונן בפונקציה שמקבלת שני ארגומנטים, x ו- y , ומחזירה את המכפלה $x \cdot y$. ניתן לעשות זאת ע"י הוספה חוזרת של x למשתנה מקומי, נניח sum , במשך y פעמים, ואז להחזיר את הערך של sum . פונקציה שמממשת אלגוריתם כפל נאיבי והתרגום שלה לשפת VM ייראו כך:

High-level code	VM code
<code>// Returns x * y</code>	<code>// Returns x * y</code>
<code>int mult(int x, int y) {</code>	<code>function mult(x,y)</code>
<code> int sum = 0;</code>	<code> push 0</code>
<code> int i = 0;</code>	<code> pop sum</code>
<code> while (i < y) {</code>	<code> push 0</code>
<code> sum += x;</code>	<code> pop i</code>
<code> i++;</code>	<code> label WHILE_LOOP</code>
<code> }</code>	<code> push i</code>
<code> return sum;</code>	<code> push y</code>
<code>}</code>	<code> lt</code>
	<code> neg</code>
	<code> if-goto WHILE_END</code>
	<code> push sum</code>
	<code> push x</code>
	<code> add</code>
	<code> pop sum</code>
	<code> push i</code>
	<code> push 1</code>
	<code> add</code>
	<code> pop i</code>
	<code> goto WHILE_LOOP</code>
	<code> label WHILE_END</code>
	<code> push sum</code>
	<code> return</code>

הדוגמה הזו מראה כיצד ניתן לבטא לוגיקה של לולאה באמצעות פקודות ה-branching ב-VM – *goto, if – goto, label*.

נשים לב שהתנאי הבוליאני $(i < y)$! הממומש ע"י הפקודות

push i
push y
lt
ng

נדחף לתוך המחסנית לפני הפקודה *if – goto WHILE_END*. ראינו בפרק הקודם שאפשר להשתמש בפקודות VM כדי לבטא ולהעריך ביטויים בוליאניים. כפי שניתן לראות בדוגמה, מבנים של control בשפה high-level כמו *if* ו-*while* ניתנים למימוש באמצעות פקודות *goto* ו-*if – goto* בלבד. באופן כללי, כל מבנה של flow of control בשפות תכנות high-level ניתן למימוש ע"י פקודות לוגיות ופקודות branching ב-VM.

מימוש: רוב שפות המכונה ה-low-level, כולל Hack, מאפשרות הצהרה על תוויות סימבוליות ופעולות "*goto label*" מותנות או לא מותנות. לכן, אם נבסס את מימוש ה-VM שלנו על תוכנית שמתרגמת פקודות VM להוראות אסמבלי, מימוש פקודות ה-branching של ה-VM די פשוט.

מערכת הפעלה – Operating System: מערכת ההפעלה שלנו כתובת בשפת Jack ומתורגמת ע"י Jack compiler לשפת VM. התוצאה תהיה ספריה של 8 קבצים הנקראים *Math. vm*, *Memory. vm*, *String. vm*, *Array. vm*, *Output. vm*, *Screen. vm*, *Keyboard. vm*, ו-*Sys. vm*. כל קובץ OS מציע אוסף של פונקציות שימושיות שכל פונקציית VM יכולה לקרוא להן.

דוגמה: בכל פעם שפונקציית VM צריכה לבצע כפל או חילוק היא יכולה לקרוא לפונקציה *Math.multiply* או לפונקציה *Math.divide*.

פונקציות

בכל שפת תכנות יש מספר קבוע של פעולות שהן built-in. בנוסף, בשפות high-level (ובחלק משפות low-level) ניתן להרחיב זאת לאוסף פתוח של פעולות המוגדרות ע"י המתכנת. בדרך כלל, הפעולות הללו ייקראו subroutines, procedures, methods, או functions. בשפת ה-VM נתייחס לכל יחידות התכנות הללו בתור פונקציות. בשפות שהן well-designed, פקודות שהן built-in ופונקציות שהוגדרו ע"י המתכנת נראות ומרגישות אותו הדבר.

דוגמה 1: כדי לחשב את $x + y$ על ה-stack machine, נעשה $push x, push y$ ואז add . כאשר אנחנו עושים זאת, אנחנו מצפים שהמימוש של add יוציא את שני הערכים שבראש המחסנית, יחבר אותם, וידחוף את התוצאה למחסנית. נניח כעת שמישהו כתב פונקציה $power$ שמחשבת את x^y . כדי להשתמש בפונקציה הזאת, נבצע את הפעולות הבאות:

```
push x
push y
call power
```

דוגמה 2: אפשר להעריך את הביטוי $(x + y)^3$ באמצעות הפקודות הבאות:

```
push x
push y
add
push 3
call power
```

נשים לב שההבדל היחיד בין הפעלה של פעולה פרימיטיבית ובין קריאה לפונקציה היא מילת המפתח $call$ שנמצאת לפני הפונקציה. כל השאר בדיוק אותו הדבר: שתי הפעולות דורשות שה-caller יבין את הקרקע ע"י דחיפת ארגומנטים למחסנית, מצופה משתי הפעולות לצרוך את הארגומנטים שלהן, ומצופה משתי הפעולות לדחוף את ערכי החזרה שלהם למחסנית.

דוגמה: נתבונן בתוכנית VM שמחשבת את הפונקציה $\sqrt{x^2 + y^2}$, שנקראת גם $hypot$.

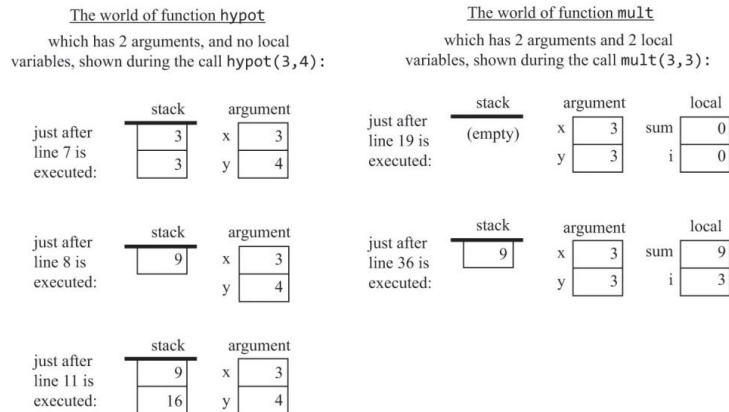
```
0 function main()
  // Computes hypot(3,4)
  1   push 3
  2   push 4
  3   call hypot
  4   return

  5 function hypot(x,y)
    // Computes sqrt(x*x + y*y)
    6   push x
    7   push x
    8   call mult
    9   push y
   10  push y
   11  call mult
   12  add
   13  call sqrt
   14  return

  15 function mult(x,y)
    // Computes x * y (same as in figure 8.1)
    16  push 0
    17  pop sum
    18  push 0
    19  pop i
    ...
   36  push sum
   37  return
```

התוכנית מכילה 3 פונקציות עם ההתנהגות הבאה בזמן הריצה: *main* קוראת ל-*hypot*, ואז *hypot* קוראת ל-*mult* פעמיים (קיימת גם קריאה לפונקציה *sqrt* שאחריה לא נעקוב).

נשים לב שבמהלך זמן הריצה, כל פונקציה רואה עולם פרטי, המכיל את מחסנית העבודה שלה ואת מקטעי הזיכרון שלה:



העולמות הנפרדים הללו מחוברים דרך 2 "wormholes": כאשר פונקציה מכילה את הפקודה *call mult*, הארגומנטים שהיא דוחפת למחסנית לפני הקריאה מועברים איכשהו למקטע ה-*argument* של ה-*callee*. באופן דומה, כאשר פונקציה מכילה את הפקודה *return*, הערך האחרון שנדחף למחסנית לפני הפקודה מועתק איכשהו למחסנית של ה-*caller* ומחליף את הארגומנטים שנדחפו קודם לכן.

מימוש: נשתמש במונח *calling chain* כדי להתייחס לכל הפונקציות שמעורבות בהרצת התוכנית. כאשר תוכנית VM מתחילה לרוץ, ה-*calling chain* מכילה רק פונקציה אחת, נניח *main*. בנקודה מסוימת, ייתכן כי *main* תקרא לפונקציה אחרת, נניח *foo*, וייתכן כי הפונקציה הזו תקרא לפונקציה אחרת, נניח *bar*. בנקודה זו, שרשרת הקריאות היא $main \rightarrow foo \rightarrow bar$. כל פונקציה בשרשרת הקריאות מחכה שהפונקציה שהיא קראה לה תחזיר. לכן, הפונקציה היחידה שבאמת פעילה בשרשרת הקריאות היא האחרונה, שנקרא לה הפונקציה הנוכחית, כלומר הפונקציה הנוכחית שרצה.

פונקציות בדרך כלל משתמשות במשתנים מקומיים (*local*) וארגומנטים (*argument*). המשתנים הללו הם זמניים – מקטעי הזיכרון שמייצגים אותם צריכים להיות מוקצים כאשר הפונקציה מתחילה לרוץ, וניתן למחזר אותם כאשר הפונקציה מחזירה. במהלך זמן הריצה, כל פונקציה צריכה לרוץ באופן בלתי תלוי בכל הקריאות האחרות ולתחזק מחסנית משל עצמה, משתנים מקומיים וארגומנטים. זה עלול להיות בעייתי במקרים של קריאות מקוננות או רקורסיה.

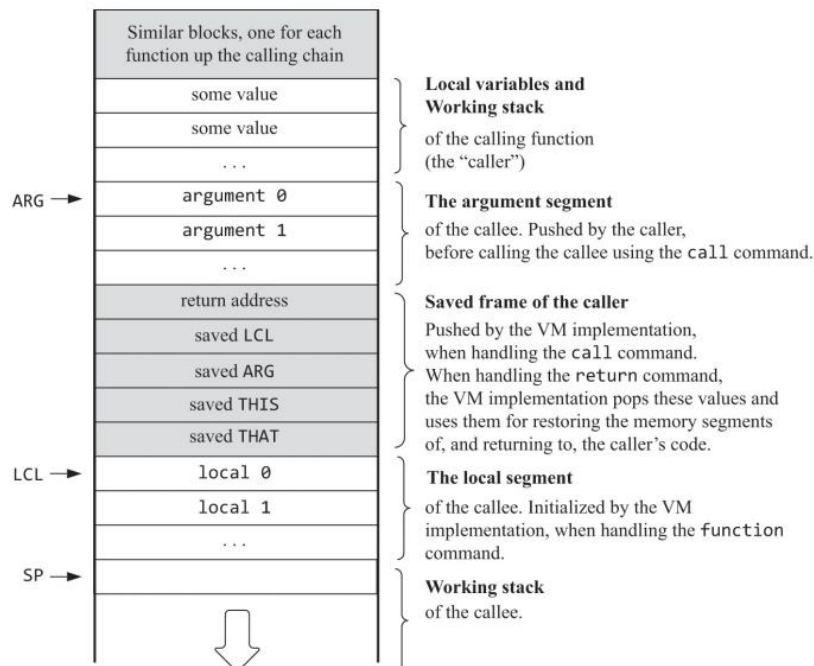
בכל נקודת זמן נתונה רק פונקציה אחת רצה (הפונקציה שבסוף שרשרת הקריאות), בעוד שכל שאר הפונקציות בשרשרת מחכות שהיא תחזיר. המודל הזה של Last-In-First-Out (LIFO) מתאים למבנה הנתונים של המחסנית.

נניח שהפונקציה הנוכחית היא *foo*. נניח ש-*foo* כבר דחפה ערכים כלשהם ל-*working stack* שלה ושינתה רשומות כלשהן במקטעי הזיכרון שלה. נניח שבשלב מסוים *foo* רוצה לקרוא לפונקציה אחרת *bar*. בשלב זה נצטרך להשהות את ההרצה של *foo* עד ש-*bar* תסיים את הריצה שלה. כיוון שהמחסנית גדלה רק בכיוון אחד, מחסנית העבודה של *bar* לעולם לא תדרוס ערכים שנדחפו לפני כן. לכן, שמירת מחסנית העבודה של ה-*caller* מתקבלת "בחינם" ממבנה

המחסנית. ניזכר שהשתמשנו במצביעים *LCL*, *ARG*, *THIS* ו-*THAT* כדי להצביע על כתובות הבסיס ב-RAM של המקטעים *local*, *argument*, *this* ו-*that* של הפונקציה הנוכחית. אם נרצה להשהות את המקטעים הללו, אנחנו יכולים לדחוף את המצביעים שלהם למחסנית ולהוציא אותם בהמשך, כאשר נרצה להחזיר את *foo* לחיים. נשתמש במילה *frame* כדי להתייחס לערכי המצביעים שצריך בשביל לשמור ולהחזיר את מצב הפונקציה.

כעת, אנחנו משתמשים באותו מבנה נתונים כדי להחזיק גם את מחסניות העבודה וגם את ה-frames של כל הפונקציות במעלה שרשרת הקריאות. מעכשיו נקרא לזה ה-*global stack*.

ה-*global stack* כאשר ה-*callee* רצה:



כאשר מטפלים בפקודה *call functionName*, מימוש ה-VM דוחף את ה-*frame* של ה-*caller* למחסנית. לאחר מכן, נוכל לקפוץ לביצוע הקוד של ה-*callee*. כפי שנראה בהמשך, כאשר נטפל בפקודה *function functionName*, נשתמש בשם הפונקציה כדי ליצור תווית סימבולית ייחודית שמסמנת את תחילת הפונקציה, ונוכל לייצר קוד אסמבלי שגורם לפעולה *"goto functionName"*.

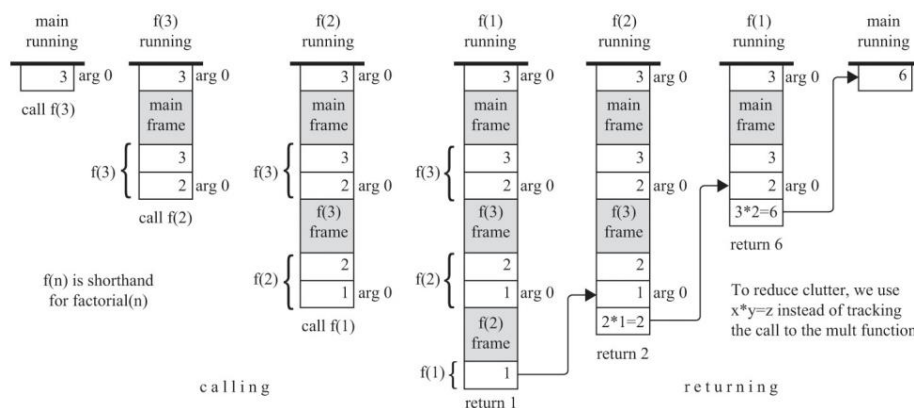
כאשר חוזרים מה-*callee* ל-*caller*, הפקודה *return* לא מציינת כתובת חזרה, והיא מפורשת באופן הבא: לנתב את הרצת התוכנית למקום בזיכרון שמחזיק את הפקודה שלאחר פקודת הקריאה שהופעלה ע"י הפונקציה הנוכחית. מימוש ה-VM יכול לעשות זאת ע"י שמירת כתובת החזרה לפני שעוברים להרצת ה-*callee*, ושליפת כתובת החזרה וקפיצה אליה לאחר שה-*callee* מחזירה. אבל איפה נשמור את כתובת החזרה? כאשר אנחנו נתקלים בפקודה *call foo* בקוד ה-VM, אנחנו יודעים מהי הפקודה שנרצה לבצע לאחר ש-*foo* תסיים לרוץ – פקודת האסמבלי שנמצאת לאחר פקודות האסמבלי המממשות את הפקודה *call foo*. לכן, ה-*VM translator* יכול לשתול תווית בדיוק שם, בקוד האסמבלי שהוא מייצר, ולדחוף את התווית הזו למחסנית. כאשר ניתקל בפקודת *return* בקוד VM, נוכל להוציא מהמחסנית את כתובת החזרה שנשמרה קודם לכן, נקראה לה *returnAddress*, ולגרום לפעולה *goto returnAddress* באסמבלי.

מימוש ה-VM בפועל: נראה אילוסטרציה צעד אחר צעד של איך מימוש ה-VM תומך בפעולת הקריאה והחזרה של פונקציה. נדגים זאת עבור הפונקציה *factorial* שמטרתה לחשב את $n!$ רקורסיבית.

הקוד ה-high-level והקוד השקול ב-VM:

High-level code	VM code
<pre>// Tests the factorial function int main() { return factorial(3); }</pre>	<pre>// Tests the factorial function function main push 3 call factorial return // Computes n! function factorial(n) push n push 1 eq if-goto BASE_CASE push n push n push 1 sub call factorial call mult return label BASE_CASE push 1 return</pre>

מצבים נבחרים של ה-global stack במהלך הריצה של *factorial(3)*:



מנקודת המבט של *main*: כדי להבין את הקרקע, היא דוחפת את הקבוע 3 למחסנית, ואז קוראת ל-*factorial* ועוברת להשהיה. בנקודה מסוימת אחרי זה, נגמרת ההשהיה ובעת המחשנית שלה מכילה 6.

מאחורי הקלעים, כל פעולת *call* ממומשת ע"י שמירת ה-frame של ה-caller במחסנית וקפיצה להרצת ה-callee. כל פעולת *return* ממומשת ע"י:

- שימוש ב-frame האחרון שנשמר כדי לקבל את כתובת החזרה בתוך הקוד של ה-caller ולשחרר את מקטעי הזיכרון שלה.
- העתקת הערך שבראש המחסנית (ערך החזרה) למקום במחסנית המזוהה עם *argument 0*.
- קפיצה להרצת הקוד של ה-caller החל מכתובת החזרה.

VM Specification

Branching Commands

- **label label**: מוסיפה תווית למיקום הנוכחי בקוד של הפונקציה. ניתן לקפוץ רק למקומות שיש להם תווית. ה-scope של התווית הוא הפונקציה שבה היא מוגדרת. התווית היא מחרוזת המורכבת מכל רצף של אותיות, ספרות, underscore (_), נקודה (.), ונקודותיים (:). שלא מתחיל בספרה. פקודת ה-label יכולה להופיע בכל מקום בפונקציה, לפני או אחרי פקודות ה-goto שמתייחסות אליה.
- **goto label**: גורמת לפעולת goto ללא תנאי, הגורמת להרצת התוכנית להמשיך מהמקום המסומן ע"י התווית label. פקודת ה-goto והיעד אליו קופצים חייבים להיות ממוקמים באותה הפונקציה.
- **if – goto label**: גורמת לפעולת goto מותנית. הערך בראש המחסנית נשלף. אם הערך שלו שונה מ-0, ההרצה תמשיך מהמקום המסומן ע"י התווית. אחרת, ההרצה תמשיך מהפקודה הבאה בתוכנית. פקודת ה-if – goto והיעד אליו קופצים חייבים להיות ממוקמים באותה הפונקציה.

Function Commands

- **function functionName nVars**: מסמנת את ההתחלה של פונקציה שנקראת functionName. הפקודה מודיעה שלפונקציה יש nVars משתנים לוקאליים.
- **call functionName nArgs**: קוראת לפונקציה functionName. הפקודה מודיעה שנדחפו nArgs ארגומנטים למחסנית לפני הקריאה.
- **return**: מעבירה את ההרצה לפקודה שנמצאת לאחר פקודת ה-call בקוד של הפונקציה שקראה לפונקציה הנוכחית.

VM Program

תוכניות VM מיוצרות מתוכניות high-level הכתובות בשפות כמו Jack. ה-Jack compiler מתרגם כל קובץ מחלקה FileName.jack לקובץ מתאים FileName.vm שמכיל פקודות VM. אחרי הקומפילציה, כל constructor, function, ו-method שנקראת bar בקובץ FileName.jack מתורגמת לפונקציית VM שנקראת FileName.bar. ה-scope של השמות של פונקציות VM הוא גלובאלי – כל פונקציות ה-VM בכל קבצי ה-vm. בתיקייה של התוכנית יכולות לראות אחת את השנייה ולקרוא אחת לשנייה ע"י שם הפונקציה המלא והייחודי FileName.functionName.

Program Entry Point: בכל תוכנית Jack חייב להיות קובץ אחד הנקרא Main.jack, וחייבת להיות בו פונקציה אחת שנקראת *main*. לכן, לאחר הקומפילציה, בכל תוכנית VM אמור להיות קובץ אחד בשם Main.vm שיש בו פונקציה אחת שנקראת *Main.main*, וזו ה-entry point של האפליקציה. נממש את זה באופן הבא: כאשר נתחיל ריצה של תוכנית VM, הפונקציה הראשונה שתרוץ תמיד תהיה פונקציית VM ללא ארגומנטים שנקראת *Sys.init* (היא חלק ממערכת ההפעלה). הפונקציה הזו קוראת לפונקציית ה-entry point בתוכנית של המשתמש.

Implementation

Function Call and Return

ניתן להתבונן בקריאה לפונקציה ולחזרה מפונקציה משתי נקודות מבט – נקודת המבט של הפונקציה הקוראת (ה-caller), ונקודת המבט של הפונקציה הנקראת (ה-callee).

נקודת המבט של ה-callee	נקודת המבט של ה-caller
לפני תחילת הריצה, מקטע ה- <i>argument</i> כבר אותחל עם ערכי הארגומנטים שהועברו ע"י ה-caller, ומקטע המשתנים הלוקאליים הוקצה ואותחל עם 0-ים	לפני קריאה לפונקציה, צריך לדחוף למחסנית את מספר הארגומנטים (<i>nArgs</i>) שה-callee מצפה לקבל
לפני החזרה, צריכה לדחוף את ערך החזרה למחסנית	לאחר מכן, לקרוא ל-callee ע"י שימוש בפקודת הקריאה <i>FileName.functionName nArgs</i>
	לאחר שה-callee מחזירה, ערכי הארגומנטים שנדחפו לפני הקריאה נעלמים מהמחסנית, וערך החזרה מופיע בראש המחסנית. חוץ מהשינוי הזה, ה- <i>working stack</i> נראית בדיוק כמו שנראתה לפני הקריאה
	לאחר שה-callee מחזירה, כל מקטעי הזיכרון נראים בדיוק כמו שנראו לפני הקריאה, חוץ מזה שהתוכן של מקטע ה- <i>static</i> אולי השתנה, ומקטע ה- <i>temp</i> לא מוגדר

כל פקודת *call* מייצרת קוד אסמבלי ששומר את ה-frame של ה-caller במחסנית וקופץ להרצת ה-callee. כל פקודת *function* מייצרת קוד אסמבלי שמאתחל את המשתנים הלוקאליים של ה-callee. כל פקודת *return* מייצרת קוד אסמבלי שמעתיק את ערך החזרה לראש ה-*working stack* של ה-caller, משחרר את המצביעים למקטעים של ה-caller, וקופץ להרצה החל מכתובת החזרה והלאה.

מימוש פקודות ה-function של שפת ה-VM ע"י הוראות Hack Assembly:

פקודת ה-VM	פסאודו-קוד באסמבלי
<i>call f nArgs</i> (קריאה לפונקציה <i>f</i> המודיעה על כך שנדחפו <i>nArgs</i> ארגומנטים למחסנית לפני הקריאה)	<i>push returnAddress</i> <i>push LCL</i> <i>push ARG</i>

$push\ THIS$ $push\ THAT$ $ARG = SP - 5 - nArgs$ $LCL = SP$ $goto\ f$ $(returnAddress)$	
(f) $repest\ nVars\ times:$ $push\ 0$	$function\ f\ nVars$ (הצהרה על פונקציה f המודיעה שלפונקציה יש $nVars$ משתנים לוקאליים)
$frame = LCL$ $retAddr = * (frame - 5)$ $* ARG = pop()$ $SP = ARG + 1$ $THAT = * (frame - 1)$ $THIS = * (frame - 2)$ $ARG = * (frame - 3)$ $LCL = * (frame - 4)$ $goto\ retAddr$	$return$ (סיום הרצת הפונקציה הנוכחית והחזרת ה-control ל-caller)

Standard VM Mapping on the Hack Platform

המחסנית: בפלטפורמת ה-Hack, מקומות 0 עד 15 ב-RAM שמורים עבור מצביעים ורגיסטרים וירטואליים, ומקומות 16 עד 255 ב-RAM שמורים עבור משתנים סטטיים. המחסנית ממופה החל מכתובת 256 והלאה. החל מנקודה זו, כאשר ה-VM translator נתקל בפקודות כמו $push$, pop , add וכו' בקוד ה-VM המקורי, הוא מייצר קוד אסמבלי שמבצע את הפעולות הללו בכך שפועל על הכתובת ש- SP מצביע אליה ומשנה את הערך של SP במידת הצורך.

סימבולים מיוחדים: כאשר מתרגמים פקודות VM ל-Hack Assembly, ה-VM translator מטפל בשני סוגים של סימבולים – סימבולים מוגדרים מראש כמו SP , LCL ו- ARG , וייצור תוויות סימבוליות עבור סימון של כתובות חזרה ונקודות התחלה של פונקציות.

עבור כל פקודת $function$, ה-VM translator מייצר תווית באסמבלי עבור תחילת הפונקציה. עבור כל פקודת $call$, ה-VM translator מייצר הוראת $goto$ באסמבלי, יוצרת תווית לכתובת החזרה ודוחף אותה למחסנית, ומכניס את התווית לקוד שהוא מייצר. עבור כל פקודת $return$, ה-VM translator מוציא את כתובת החזרה מהמחסנית ומייצר הוראת $goto$.

דוגמה:

VM code	Generated assembly code
function Main.main	(Main.main)
...	...
call Point.new	goto Point.new
// Next VM command	(Main.main\$ret0)
...	// Next VM command (in assembly)
...	...
function Point.new	(Point.new)
...	...
return	goto Main.main\$ret0

:Naming Conventions

שימוש	Symbol
סימבול מוגדר מראש שמצביע לכתובת הזיכרון ב-RAM הנמצאת אחרי הכתובת שמכילה את הערך שבראש המחשנית	<i>SP</i>
סימבולים מוגדרים מראש שמצביעים על כתובות הבסיס ב-RAM של המקטעים הווירטואליים <i>this</i> , <i>argument</i> , <i>local</i> , <i>that</i> ו- <i>that</i> של פונקציית ה-VM הנוכחית שרצה	<i>LCL, ARG, THIS, THAT</i>
כל רפרנס ל- <i>static i</i> המופיע בקובץ <i>vm Xxx</i> מתורגם לסימבול <i>Xxx.i</i> באסמבלי. האסמבלר יקצה למשתנים הסימבוליים הללו כתובות RAM החל מהכתובת 16	<i>Xxx.i</i> (מייצג משתנים סטטיים)
תהי <i>foo</i> פונקציה בקובץ <i>vm Xxx</i> . עבור כל פקודת <i>label bar</i> ב- <i>foo</i> , נייצר את הסימבול <i>Xxx.foo\$bar</i> ונכניס אותו לקוד האסמבלי. כאשר מתרגמים פקודות <i>goto bar</i> ו- <i>if – goto bar</i> (בתוך <i>foo</i>) לאסמבלי, צריך להשתמש בתווית <i>Xxx.foo\$bar</i> במקום בתווית <i>bar bar</i>	<i>functionName\$label</i> (יעדים של פקודות goto)
עבור כל פקודת <i>function foo</i> בקובץ <i>vm Xxx</i> נייצר את הסימבול <i>Xxx.foo</i> ונכניס אותו לקוד האסמבלי כדי לסמן את נקודת ההתחלה של פונקציה. האסמבלר יתרגם את הסימבול הזה לכתובת הפיזית שבה הקוד של הפונקציה מתחיל.	<i>functionName</i> (נקודת התחלה של פונקציה)
תהי <i>foo</i> פונקציה בקובץ <i>vm Xxx</i> . עבור כל פקודת <i>call</i> בתוך <i>foo</i> , נייצר את הסימבול <i>Xxx.foo\$ret.i</i> ונכניס אותו לקוד האסמבלי, כאשר <i>i</i> הוא אינדקס שרץ (סימבול אחד כזה מיוצר עבור כל פקודת <i>call</i> ב- <i>foo</i>). הסימבול הזה מסמן את כתובת החזרה בקוד של ה- <i>caller</i> . האסמבלר מתרגם את הסימבול הזה לכתובת הפיזית בזיכרון של הפקודה שנמצאת לאחר פקודת ה- <i>call</i>	<i>functionName\$ret.i</i> (כתובת חזרה)
סימבולים מוגדרים מראש היכולים לשמש עבור כל מטרה. למשל, אם ה- <i>VM translator</i> מייצר קוד אסמבלי שצריך להשתמש במשתנים <i>low-level</i> לשמירה זמנית, ניתן להשתמש בהם	<i>R13 – R15</i>

Bootstrap Code: המיפוי הסטנדרטי של VM על פלטפורמת ה-Hack קובע שהמחשנית תמופה החל מהכתובת 256 ב-RAM, ושפונקציית ה-VM הראשונה שצריכה לרוץ היא פונקציית ה-*OS Sys.init*. אם נרצה שהמחשב יריץ מקטע קוד קבוע מראש כאשר הוא *boots up*, נוכל לשים את הקוד הזה ב-*instruction memory* של מחשב ה-Hack החל מהכתובת 0. להלן הקוד:

SP = 256
call Sys.init

Design Suggestions for the VM Implementation

נרחיב את המודולים VMTranslator, Parser ו-CodeWriter מהפרויקט הקודם.

The VMTranslator: אם הקלט הוא קובץ יחיד Prog.vm, ה-VMTranslator בונה Parser כדי לפרסר את הקובץ ו-CodeWriter שמתחיל ביצירת קובץ פלט Prog.asm. לאחר מכן, ה-VMTranslator נכנס ללולאה שמשתמשת ב-Parser כדי לעבור על כל קובץ הקלט ולפרסר כל שורה בתור פקודת VM. עבור כל פקודה שפרסר, ה-VMTranslator משתמש ב-CodeWriter כדי לייצר קוד אסמבלי ולכתוב אותו לקובץ הפלט.

The Parser: זהה ל-Parser מהפרויקט הקודם.

The CodeWriter: להלן API ל-CodeWriter שמטפל בכל הפקודות בשפת ה-VM:

הפונקציה	ארגומנטים	Routine
פותחת קובץ פלט ומתכוון לכתוב אליו. כותבת הוראות אסמבלי שהן ה-bootstrap code שגורם לתחילת הרצת התוכנית בתחילת קובץ הפלט	קובץ פלט	Constructor
מודיעה שהתחיל התרגום של קובץ VM חדש (נקראת ע"י ה-VMTranslator)	filename (string)	setFileName
כותבת לקובץ הפלט את קוד האסמבלי שמממש את הפקודה האריתמטית-לוגית הנתונה command	command (string)	writeArithmetic (פרויקט 7)
כותבת לקובץ הפלט את קוד האסמבלי שמממש את פקודת ה-push או ה-pop הנתונה command	command (C_PUSH/C_POP), segment (string), index (int)	writePushPop (פרויקט 7)
כותבת קוד אסמבלי שגורם לפקודת ה-label	label (string)	writeLabel
כותבת קוד אסמבלי שגורם לפקודת ה-goto	label (string)	writeGoto
כותבת קוד אסמבלי שגורם לפקודת ה-if-goto	label (string)	writeIf
כותבת קוד אסמבלי שגורם לפקודת ה-function	functionName (string), nVars (int)	writeFunction
כותבת קוד אסמבלי שגורם לפקודת ה-call	functionName (string), nArgs (int)	writeCall
כותבת קוד אסמבלי שגורם לפקודת ה-return	-	writeReturn

High-Level Language

נציג בפרק זה שפת תכנות high-level שנקראת Jack. בפרקים 10-11 נכתוב קומפיילר שמתרגם תוכניות Jack לקוד VM, ובפרק 12 נפתח מערכת הפעלה פשוטה עבור פלטפורמת ה-Hack.

נתחיל עם דוגמאות של תוכניות Jack. הדוגמה הראשונה היא תוכנית Hello World, הדוגמה השנייה תדגים תכנות ושימוש במערכים, הדוגמה השלישית תדגים כיצד ניתן לממש טיפוסים אבסטרקטיים בשפת ה-Jack, והדוגמה הרביעית תדגים מימוש של רשימה מקושרת.

דוגמה 1 – Hello World: כאשר אנחנו מריצים תוכנית Jack מקומפלת, הריצה תמיד תתחיל בפונקציה *Main.main*. לכן, כל תוכנית Jack חייבת לכלול לפחות מחלקה אחת בשם *Main*, והמחלקה הזו חייבת לכלול לפחות פונקציה אחת בשם *Main.main*.

```
/** Prints "Hello World". File name: Main.jack */
class Main {
    function void main() {
        do Output.printString("Hello World");
        do Output.println(); // New line
        return;              // The return statement is mandatory
    }
}
```

ב-Jack יש ספרייה שנקראת ה-Jack OS, והיא מרחיבה את השפה הבסיסית עם מגוון אבסטרקציות ושירותים כמו פונקציות מתמטיות, עיבוד מחרוזות, ניהול זיכרון, גרפיקה, ופונקציות קלט/פלט. שתי פונקציות OS כאלו נקראות ע"י תוכנית ה-Hello World ומשפיעות על פלט התוכנית. התוכנית גם מדגימה פורמטים של comments ש-Jack תומכת בהם.

דוגמה 2 – Procedural Programming and Array Handling: ב-Jack יש הצהרות שמטפלות בהשמות ובאיטרציות. התוכנית הבאה מדגימה את היכולות הללו בהקשר של עיבוד מערכים:

```
/** Inputs a sequence of integers, and computes their average. */
class Main {
    function void main() {
        var Array a; // Jack arrays are not typed
        var int length;
        var int i, sum;
        let i = 0;
        let sum = 0;
        let length = Keyboard.readInt("How many numbers? ");
        let a = Array.new(length); // Constructs the array
        while (i < length) {
            let a[i] = Keyboard.readInt("Enter a number: ");
            let sum = sum + a[i];
            let i = i + 1;
        }
        do Output.printString("The average is: ");
        do Output.printInt(sum / length);
        do Output.println();
        return;
    }
}
```

רוב שפות התכנות ה-high-level מציעות הצהרה על מערך כחלק מהסינטקס הבסיסי של השפה. ב-Jack, בחרנו להתייחס למערכים כמופעים של מחלקה Array שהיא חלק מה-OS שמרחיבה את השפה הבסיסית.

דוגמה 3 – Abstract Data Types: לכל שפת תכנות יש קבוצה קבועה של data types פרימיטיביים, ב-Jack יש 3 כאלה: *int*, *char* ו-*boolean*. בשפות object-based, מתכנתים יכולים ליצור טיפוסים חדשים ע"י יצירת מחלקות שמייצגות data types אבסטרקטיים במידת הצורך.

נניח שאנחנו רוצים לתת ל-Jack את היכולת לטפל במספרים רציונליים כמו $\frac{2}{3}$ או $\frac{314159}{100000}$ בלי לאבד את הדיוק. ניתן לעשות זאת ע"י מחלקת Jack שיוצרת אובייקטים של שברים מהצורה $\frac{x}{y}$ כאשר x ו- y הם integers. המחלקה הזו תוכל לספק אבסטרקציה של שברים לכל תוכנית Jack שצריכה לייצג מספרים רציונליים ולפעול עליהם.

שימוש במחלקות: נתבונן בשלד של מחלקה (אוסף של חתימות של מתודות) שמפרט שירותים מסוימים שנצפה לקבל מאבסטרקציה של שברים. מפרט כזה נקרא API. נראה את ה-API של המחלקה Fraction ומחלקת Jack שמשתמשת בה כדי ליצור ולפעול על אובייקטים של Fraction:

```
/** Represents the Fraction type and related operations (class skeleton) */
class Fraction {

    /** Constructs a (reduced) fraction from x and y */
    constructor Fraction new(int x, int y)

    /** Returns the numerator of this fraction */
    method int getNumerator()

    /** Returns the denominator of this fraction */
    method int getDenominator()

    /** Returns the sum of this fraction and the other one */
    method Fraction plus(Fraction other)

    /** Prints this fraction in the format x/y */
    method void print()

    /** Disposes this fraction */
    method void dispose() {

        // More fraction-related methods:
        // minus, times, div, invert, etc.
    }

}

// Computes and prints the sum of 2/3 and 1/5
class Main {
    function void main() {
        // Creates 3 fraction variables (pointers to Fraction objects)
        var Fraction a, b, c;
        let a = Fraction.new(4,6); // a = 2/3
        let b = Fraction.new(1,5); // b = 1/5

        // Adds up the two fractions, and prints the result
        let c = a.plus(b); // c = a + b
        do c.print(); // Should print "13/15"
        return;
    }
}
```

משתמשים של אבסטרקציה לא צריכים לדעת הכל על המימוש. כל מה שהם צריכים זה את ה-API של המחלקה. ה-API מודיע מה הפונקציונליות שהמחלקה מציעה וכיצד להשתמש בה.

מימוש מחלקות: כעת נראה מימוש אפשרי לאבסטרקציה שתיארנו קודם:

```

/** Represents the Fraction type and related operations. */
class Fraction {
    // Each Fraction object has a numerator and a denominator
    field int numerator, denominator;
    /** Constructs a (reduced) fraction from x and y */
    constructor Fraction new(int x, int y) {
        let numerator = x;
        let denominator = y;
        do reduce(); // Reduces this fraction
        return this; // Returns a reference to the new object
    }
    // Reduces this fraction
    method void reduce() {
        var int g;
        let g = Fraction.gcd(numerator, denominator);
        if (g > 1) {
            let numerator = numerator / g;
            let denominator = denominator / g;
        }
        return;
    }
    // Computes the greatest common divisor of two given integers
    function int gcd(int a, int b) {
        // Applies Euclid's algorithm
        var int r;
        while (~(b = 0)) {
            let r = a - (b * (a / b)); // r = remainder
            let a = b;
            let b = r;
        }
        return a;
    }
}
// The Fraction class declaration continues on the top right

```

```

/** Accessors */
method int getNumerator() {
    return numerator;
}
method int getDenominator() {
    return denominator;
}
/** Returns the sum of this fraction and the other one */
method Fraction plus(Fraction other) {
    var int sum;
    let sum = (numerator * other.getDenominator() +
              (other.getNumerator() * denominator));
    return Fraction.new(sum, denominator *
                       other.getDenominator());
}
/** Prints this fraction in the format x/y */
method void print() {
    do Output.printInt(numerator);
    do Output.printString("/");
    do Output.printInt(denominator);
    return;
}
/** Disposes this fraction */
method void dispose() {
    // Frees the memory held by this object
    do Memory.deAlloc(this);
    return;
}
// More fraction-related methods can come here:
// minus, times, div, invert, etc.
} // End of the Fraction class declaration.

```

המחלקה Fraction מדגימה כמה תכונות מפתח של תכנות object-based ב-Jack. Fields (שדות) מפרטים תכונות של אובייקטים (נקראים גם member variables). Constructors (בנאים) הם subroutines שיוצרות אובייקטים חדשים, ו-methods (מתודות/שיטות) הן subroutines שפועלות על האובייקט הנוכחי (מתייחסות אליו ע"י שימוש במילת המפתח *this*). Functions הן subroutines ברמת המחלקה (נקראות גם static methods) שלא פועלות על אובייקט מסוים. המחלקה Fraction גם מדגימה את כל סוגי ההצהרות הזמינות בשפת ה-Jack: *return*, *while*, *if*, *do*, *let*.

דוגמה 4 – מימוש Linked List: מבנה הנתונים *list* מוגדר רקורסיבית בתור ערך שמלווה אותו רשימה. הערך *null* (מקרה הבסיס) גם מוגדר כרשימה. נראה מימוש אפשרי ב-Jack לרשימה של integers:

```

/** Represents a list of integers. */
class List {
    field int data; // A list consists of an int value,
    field List next; // followed by a List

    /** Creates a list whose head is car and whose tail is cdr */
    // These identifiers are used in memory of the Lisp programming language
    constructor List new(int car, List cdr) {
        let data = car;
        let next = cdr;
        return this;
    }

    /** Accessors */
    method int getData() { return data; }
    method List getNext() { return next; }

    /** Prints the elements of this list */
    method void print() {
        // Initializes a pointer to the first element of this list
        var List current;
        let current = this;
        // Iterates through the list
        while (~(current = null)) {
            do Output.printInt(current.getData());
            do Output.printChar(32); // Prints a space
            let current = current.getNext();
        }
        return;
    }
}
// The List class declaration continues on the top right

```

```

/** Disposes this List */
method void dispose() {
    // Disposes the tail of this list, recursively
    if (~(next = null)) {
        do next.dispose();
    }
    // Uses an OS routine to free the memory
    // held by this object.
    do Memory.deAlloc(this);
    return;
}
// More list-related methods can come here
} // End of the List class declaration.

```

```

// Client code example:
// Creates, prints, and disposes the list (2,3,5),
// which is shorthand for the list (2,(3,(5,null))).
// (This code can appear in any Jack class):
...
var List v;
let v = List.new(5,null);
let v = List.new(2,List.new(3,v));
do v.print(); // Prints 2 3 5
do v.dispose(); // Disposes the list
...

```

מערכת ההפעלה: תוכניות Jack עושות שימוש נרחב במערכת ההפעלה של Jack. ניתן להשתמש בשירותי מערכת ההפעלה באופן ישיר ואין צורך לכלול או לייבא קוד חיצוני כלשהו. מערכת ההפעלה מכילה 8 מחלקות:

מחלקת ה-OS	השירותים
Math	פעולות מתמטיות נפוצות: <code>max(int, int), sqrt(int), ...</code>
String	ייצוג מחרוזות ופעולות הקשורות למחרוזות: <code>length(), charAt(int), ...</code>
Array	ייצוג מערכים ופעולות הקשורות למערכים: <code>new(int), dispose()</code>
Output	לאפשר פלט טקסט למסך: <code>printString(String), printInt(int), println(), ...</code>
Screen	לאפשר פלט גרפי למסך: <code>setColor(boolean), drawPixel(int, int), drawLine(int, int, int, int), ...</code>
Keyboard	לאפשר קלט מהמקלדת: <code>readLine(String), readInt(String), ...</code>
Memory	לאפשר גישה ל-RAM: <code>peek(int), poke(int, int), alloc(int), deAlloc(Array)</code>
Sys	לאפשר שירותים הקשורים להרצת התוכנית: <code>halt(), wait(int), ...</code>

The Jack Language Specification

Syntactic Elements

תוכנית Jack היא רצף של tokens המופרדים ע"י מספר שירותי של white space ו-comments. ה-tokens יכולים להיות סימבולים, מילים שמורות, קבועים, ו-identifiers, המוגדרים באופן הבא:

White space comments-ו	מתעלמים מתווים של רווח ושל שורה חדשה, ומתעלמים מ-comments. תומכים בפורמטים הבאים של comments: <code>// Comment to end of line</code> <code>/* Comment until closing */</code> <code>/** API documentation */</code>
סימבולים	() - משמש עבור grouping של ביטויים אריתמטיים וכדי לעטוף רשימות ארגומנטים (בקריאות ל-subroutines) ורשימות פרמטרים (בהצהרות על subroutines) [] - משמש עבור אינדקסים של מערכים { } - משמש עבור grouping של יחידות תוכנה והצהרות , - מפריד בין רשימת משתנים ; - terminator של הצהרה = - אופרטור להשמה ולהשוואה . - חברות במחלקה (class membership) + * / & ~ < > - אופרטורים

	מילים שמורות <i>function, method, constructor, class</i>: רכיבי תוכנה: <i>void, char, boolean, int</i>: טיפוסים פרימיטיביים: <i>field, static, var</i>: הצהרות על משתנים: <i>return, while, else, if, do, let</i>: הצהרות: <i>null, false, true</i>: ערכים קבועים: <i>this</i>: רפרנס לאובייקט:
קבועים	• Integer constants: ערכים בטווח 0 ל-32767. מספרים שליליים אינם קבועים, אלא ביטויים המכילים את האופרטור האונארי מינוס שמופעל על <i>integer constant</i>. כתוצאה מכך הטווח של הערכים החוקיים הוא -32767 עד 32767 • String constants: עטופים ב-double quote (") ויכולים להכיל כל תו חוץ מ-newline או double quote. התווים הללו מסופקים ע"י פונקציות ה-OS <i>String.newLine()</i> ו-<i>String.doubleQuote()</i> • Boolean constants: הם <i>true</i> ו-<i>false</i> • הקבוע <i>null</i> מסמן רפרנס שהוא <i>null</i>
Identifiers	מורכבים מרצף שרירותי וארוך של אותיות של $(A - Z, a - z)$, ספרות $(0 - 9)$, ו-"_". התו הראשון חייב להיות אות או "_". השפה היא case sensitive: <i>x</i> ו-<i>X</i> הם שני identifiers שונים

Program Structure

תוכנית Jack היא אוסף של מחלקה אחת או יותר השמורות באותה התיקיה. חייבת להיות מחלקה אחת הנקראת *Main*, ובמחלקה הזאת חייבת להיות פונקציה הנקראת *main*. ההרצה של תוכנית Jack מקומפלט תמיד תתחיל בפונקציה *Main.main*.

יחידת התוכנה הבסיסית ב-Jack היא מחלקה (class). כל מחלקה *Xxx* נמצאת בקובץ נפרד שנקרא *xxx.jack* ומקומפל בנפרד. הקובציה היא ששמות מחלקה מתחילים באות גדולה. שם הקובץ חייב להיות זהה לשם המחלקה, כולל אותיות גדולות.

מבנה הצהרה על מחלקה:

```
class name {
    field variable declarations // Must precede the subroutine declarations
    static variable declarations // Must precede the subroutine declarations
    subroutine declarations // Constructor, method and function declarations, in any order
}
```

כל הצהרה על מחלקה מפרטת שם שדרכו ניתן לגשת גלובלית לשירותי המחלקה. לאחר מכן, מופיע רצף של 0 או יותר הצהרות של שדות, ו-0 או יותר הצהרות על משתנים סטטיים. לאחר מכן, מופיע רצף של הצהרה אחת או יותר על subroutine, כאשר כל אחת מהן מגדירה מתודה (method), פונקציה (function) או בנאי (constructor). המטרה של מתודות היא לפעול על האובייקט הנוכחי, פונקציות הן מתודות סטטיות ברמת המחלקה שלא מקושרות לאובייקט מסוים, ובנאים יוצרים ומחזירים אובייקטים חדשים מטיפוס המחלקה.

מבנה הצהרה על subroutine:

```
subroutine type name (parameter-list) {  
    local variable declarations  
    statements  
}
```

כאשר subroutine היא constructor, method או function. לכל subroutine יש שם שדרכו ניתן לגשת אליה, ו-type שמציין את הטיפוס של הערך שהיא מחזירה. אם ה-subroutine לא מחזירה ערך, הטיפוס המוצהר יהיה void. רשימת הפרמטרים היא רשימה של זוגות מהצורה (type identifier) המופרדים בפסיקים.

דוגמה לרשימת פרמטרים: (int x, boolean sign, Fraction g)

אם ה-subroutine היא מתודה או פונקציה, הטיפוס שהיא מחזירה יכול להיות כל טיפוס פרימיטיבי שהשפה תומכת בו (int, char, boolean), כל אחד מטיפוסי המחלקה שספריית המחלקה הסטנדרטית מספקת (String או Array), או כל אחד מהטיפוסים שמחלקות אחרות בתוכנית ממשות (למשל כמו Fraction או List שראינו). אם ה-subroutine היא בנאי, ייתכן שיש לה שם שרירותי, אבל הטיפוס שהיא מחזירה חייב להיות שם המחלקה אליה היא שייכת. למחלקה יכולים להיות 0, 1 או יותר בנאים. הקונבנציה היא שאחד מהבנאים נקרא new. אחרי החתימה, ההצהרה על subroutine מכילה רצף של 0 או יותר הצהרות של משתנים לוקאליים (הצהרות var) ואז רצף של 0 או יותר statements. כל subroutine חייבת להסתיים עם ההצהרה reutrn expression. במקרה של void, כאשר ה-subroutine לא מחזירה כלום, היא חייבת להסתיים עם ההצהרה return. בנאים חייבים להסתיים עם ההצהרה return this. הפעולה הזו מחזירה את כתובת הזיכרון של האובייקט החדש שנבנה, המסומנת ע"י this.

Data Types

הטיפוס של משתנה יכול להיות פרימיטיבי (int, char או boolean), או ClassName כאשר ClassName הוא String, Array או שם של מחלקה שנמצאת בתיקייה של התוכנית.

טיפוסים פרימיטיביים: ב-Jack יש 3 סוגי טיפוסים פרימיטיביים:

1. int: integer בגודל 16-ביט בשיטת המשלים ל-2.
2. char: מספר שלם אי-שלילי בגודל 16-ביט.
3. boolean: true או false.

כל אחד מהטיפוסים הפרימיטיביים הללו מיוצג ע"י ערך בגודל 16-ביט. ניתן לעשות השמה של ערך מכל טיפוס למשתנה מכל טיפוס בלי casting.

מערכים: הצהרה על מערכים היא ע"י מחלקת ה-Array OS. ניתן לגשת לאיברים של מערך באמצעות הנוטציה arr[i], כאשר האינדקס של האיבר הראשון הוא 0. ניתן ליצור מערך עם מספר מימדים ע"י יצירת מערך של מערכים. אין טיפוס מוגדר לאיברים של מערך, וייתכן כי לאיברים

שונים באותו המערך יהיו טיפוסים שונים. ההצהרה על מערך יוצרת רפרנס, כאשר בניית המערך מתבצעת ע"י קריאה לבנאי `.Array.new(arrayLength)`.

Object Types: מחלקת Jack שמכילה לפחות מתודה אחת מגדירה object type. יצירת אובייקט היא עניין של שני צעדים.

דוגמה:

```
// This client code example uses the Car and Employee classes, whose code is not shown here.
// The Car class has two fields: model (a String) and licensePlate (a String).
// The Employee class has two fields: name (a String) and car (a Car).
...
// Declares a Car object and two Employee objects (three pointer variables):
var Car c;
var Employee emp1, emp2;
...
// Constructs a new car:
let c = Car.new("Aston Martin", "007"); // Sets c to the base address of a memory block
                                         // containing the new car's data.
// Constructs a new employee, and assigns a car to it:
let emp1 = Employee.new("Bond", c);
...
// Creates an alias of Bond:
let emp2 = emp1; // Only the reference (address) is copied, no new object is constructed.
// We now have two Employee pointers referring to the same object.
```

Strings: מחרוזות הן מופעים של מחלקת ה-String, שמממשת מחרוזות בתור מערכים של ערכי char. ה-Jack compiler מזהה את הסינטקס "foo" ומתייחס אליו בתור אובייקט String. ניתן לגשת לתוכן של אובייקט String באמצעות `charAt(index)` ומתודות נוספות המתועדות ב-API של המחלקה String.

דוגמה:

```
var String s; // An object variable
var char c;   // A primitive variable
...
let s = "Hello World"; // Sets s to the String object "Hello World"
let c = s.charAt(6);    // Sets c to 87, the integer character code of 'W'
```

ההצהרה `let s = "Hello World"` שקולה להצהרה `let s = String.new(11)` שאחריה יש 11 קריאות למתודה: `do s.appendChar(72), ..., do s.appendChar(100)` כאשר הארגומנט של `appendChar` הוא הקוד integer של התו. כך הקומפיילר למעשה מטפל בתרגום של ההצהרה הזו. נשים לב ששפת ה-Jack לא תומכת בביטויים של תו אחד, כמו למשל 'H'. הדרך היחידה לייצג תו היא להשתמש בקוד ה-integer שלו או בקריאה למתודה `charAt`.

Type Conversions: המפרט של שפת ה-Jack לא מגדיר מה קורה כאשר משימים ערך מטיפוס אחד למשתנה מטיפוס אחר. מצופה מכל ה-Jack compilers לתמוך ולבצע את ההשמות הבאות:

- אפשר לעשות השמה של ערך *char* למשתנה *int* ולהפך, בהתאם למפרט של ה-Jack. לדוגמה:

```
var char c;
let c = 33; // 'A'

// Equivalently:
var String s;
let s = "A";
let c = s.charAt(0);
```

- אפשר לעשות השמה של *int* למשתנה שהוא רפרנס (מכל טיפוס אובייקט), ובמקרה כזה הוא יפורש בתור כתובת בזיכרון. לדוגמה:

```
var Array arr; // Creates a pointer variable
let arr = 5000; // Sets arr to 5000
let arr[100] = 17; // Sets the contents of memory address 5100 to 17
```

- אפשר לעשות השמה של משתנה מטיפוס אובייקט למשתנה מטיפוס *Array* ולהפך. זה מאפשר לגשת לשדות של האובייקט בתור איברים במערך, ולהפך. לדוגמה:

```
// Creates the array [2,5]:
var Array arr;
let arr = Array.new(2);
let arr[0] = 2;
let arr[1] = 5;

// Creates the Fraction 2/5:
var Fraction x;
let x = arr; // sets x to the base address of the memory block
              // representing the array [2,5]

do Output.printInt(x.getNumerator()) // prints "2"
do x.print()                        // prints "2/5"
```

משתנים – Variables

קיימים 4 סוגים של משתנים ב-Jack. **משתנים סטטיים** מוגדרים ברמת המחלקה וכל ה-subroutines של המחלקה יכולות לגשת אליהם. **משתני שדה (Field)** מוגדרים ברמת המחלקה ומשמשים לייצוג התכונות של אובייקטים יחידים, וכל הבנאים והמתודות של המחלקה יכולים לגשת אליהם. **משתנים לוקאליים** משמשים את ה-subroutines עבור חישובים מקומיים. **משתני פרמטר** מייצגים את הארגומנטים שהועברו ל-subroutine ע"י ה-caller. ערכים לוקאליים ופרמטרים נוצרים לפני תחילת ההרצה של ה-subroutine והם ממוחזרים כאשר ה-subroutine מחזירה. ה-scope של משתנה הוא האזור בתוכנית שהוא ניתן לזהות את המשתנה.

סוגי משתנים ב-Jack:

סוג	תיאור	הצהרה ב-	Scope
Class Variables	<i>static type varName1, varName2, ...;</i> קיים עותק אחד של כל משתנה סטטי, והעותק הזה משותף לכל ה-subroutines של המחלקה	הצהרה על מחלקה	המחלקה שבה הצהירו עליהם

המחלקה שבה הצהירו עליהם	הצהרה על מחלקה	<i>field type varName1, varName2, ... ;</i> לכל אובייקט (מופע של המחלקה) יש עותק פרטי של משתני השדה	Field Variables
ה-subroutine שבה הצהירו עליהם	הצהרה על subroutine	<i>var type varName1, varName2, ... ;</i> נוצרים כאשר subroutine מתחילה לרוץ ונפטרים מהם כאשר ה-subroutine מחזירה	Local Variables
ה-subroutine שבה הצהירו עליהם	הצהרה על subroutine	מייצגים את הארגומנטים שמועברים ל- subroutine, ומתייחסים אליהם כמו למשתנים לוקאליים שהערכים שלהם אותם ע"י ה-caller של ה-subroutine	Parameter Variables

אתחול משתנים: משתנים סטטיים לא מאותחלים, והמתכנת צריך לאתחל אותם לפני שהוא משתמש בהם. משתני שדה לא מאותחלים, מצופה שהם יאותחלו ע"י הבנאי של המחלקה. משתנים לוקאליים לא מאותחלים, וזאת אחריות המתכנת לאתחל אותם. משתני פרמטר מאותחלים לערכים של הארגומנטים שהועברו ע"י ה-caller.

Variable Visibility: לא ניתן לגשת למשתנים סטטיים ולמשתני שדה ישירות מחוץ למחלקה שבה הם הוגדרו. ניתן לגשת אליהם רק דרך מתודות accessor ו-mutator אם מעצב המחלקה מאפשר כאלו.

Statements

בשפת ה-Jack יש 5 סוגי הצהרות כפי שניתן לראות כאן:

תיאור	סינטקס	Statement
פעולת השמה. סוג המשתנה יכול להיות סטטי, לוקאלי, שדה, או פרמטר	<i>let varName = expression;</i> או <i>let varName[expression1] = expression2;</i>	<i>let</i>
הצהרת <i>if</i> טיפוסית, עם פסקת <i>else</i> אופציונלית. הסוגריים המסולסלים הם הכרחיים, גם אם <i>statements</i> הם הצהרה אחת	<i>if(expression) {</i> <i>statements1;</i> <i>} else {</i> <i>statements2;</i> <i>}</i>	<i>if</i>
הצהרת <i>while</i> טיפוסית. הסוגריים המסולסלים הם הכרחיים, גם אם <i>statements</i> הם הצהרה אחת	<i>while(expression) {</i> <i>statements;</i> <i>}</i>	<i>while</i>
משמשת כדי לקרוא לפונקציה או למתודה, ומתעלמת מערך ההחזרה אם יש כזה	<i>do function or method call;</i>	<i>do</i>
משמשת כדי להחזיר ערך מ-subroutine. הצורה השנייה בשימוש ע"י subroutines שהן <i>void</i> . בנאים חייבים להחזיר את הערך <i>this</i>	<i>return expression;</i> או <i>return;</i>	<i>return</i>

ביטויים – Expressions

ביטוי ב-Jack הוא אחד מהבאים:

- קבוע (constant).
- שם משתנה ב-scope. המשתנה יכול להיות סטטי, שדה, משתנה לוקאלי, או פרמטר.
- המילה *this*, המציינת את האובייקט הנוכחי (לא ניתן להשתמש בה בפונקציות).
- איבר במערך, ע"י שימוש בסינטקס $arr[expression]$, כאשר *arr* הוא שם משתנה מטיפוס *Array* ב-scope.
- קריאה ל-subroutine שמחזירה טיפוס שאינו *void*.
- ביטוי שלפניו יש אחת מהאופרטורים האנאריים – או $\sim$:
 - $expression$ – זו שלילה אריתמטית.
 - $\sim expression$ זו שלילה בוליאנית (bit-wise עבור integers).
- ביטוי מהצורה $expression\ op\ expression$ כאשר *op* הוא אחד מהאופרטורים הבינאריים הבאים:
 - $+$ $-$ $*$ $/$ שהם אופרטורים אריתמטיים של integers.
 - $\&$ $|$ שהם אופרטורים ל-And בוליאני ו-Or בוליאני (bit-wise עבור integers).
 - $<$ $>$ $=$ שהם אופרטורים של השוואה.
- $(expression)$ כלומר ביטוי בתור סוגריים.

Order Priority and Order of Evaluation: סדר פעולות לא מוגדר ע"י השפה, מלבד שצריך לחשב קודם כל את הביטויים שבסוגריים. לכן, הערך של הביטוי $2 + 3 * 4$ הוא לא צפוי, בעוד שמובטח כי הערך של הביטוי $2 + (3 * 4)$ יהיה 14. ה-Jack compiler מחשב ביטויים משמאל לימין, אז הערך של הביטוי $2 + 3 * 4$ יהיה 20.

Subroutine Calls

קריאה ל-subroutine קוראת לפונקציה, בנאי או מתודה, באמצעות שימוש בסינטקס $subroutineName(exp_1, exp_2, \dots, exp_n)$ כאשר כל ארגומנט *exp* הוא ביטוי. מספר הארגומנטים והטיפוסים שלהם חייבים להתאים לפרמטרים של ה-subroutine כפי שהם מצוינים בהצהרה של ה-subroutine. חייבים להופיע סוגריים גם אם רשימת הארגומנטים ריקה.

ניתן לקרוא ל-subroutines מהמחלקה בה הן הוגדרו או ממחלקות אחרות, בהתאם לכללי הסינטקס שנפרט כעת.

קריאות לפונקציות/קריאות לבנאי:

- $className.functionName(exp_1, exp_2, \dots, exp_n)$
 - $className.constructorName(exp_1, exp_2, \dots, exp_n)$
- תמיד צריך לציון את ה-*className*, גם אם הפונקציה/הבנאי באותה המחלקה של ה-caller.

קריאות למתודה:

- כדי להפעיל את המתודה על האובייקט שנתייחס אליו בתור *varName*:
varName.methodName(exp₁, exp₂, ..., exp_n)
- כדי להפעיל את המתודה על האובייקט הנוכחי:
methodName(exp₁, exp₂, ..., exp_n)
שקול ל-*.this.methodName(exp₁, exp₂, ..., exp_n)*

דוגמאות לקריאות ל-subroutines:

```
class Foo {  
    ...  
    method void f() {  
        var Bar b;           // Declares a local variable of class type Bar  
        var int i;           // Declares a local variable of primitive type int  
        ...  
        do Foo.g()           // Calls function g of the current class  
        do Bar.h()           // Calls function h of class Bar  
        do m()               // Calls method m of the current class, on the this object  
        do b.q()             // Calls method q of class Bar, on object b  
        let i = w(b.s(), Foo.t()) // Calls method w on the this object,  
                                // Calls method s of class Bar on object b,  
                                // Calls function or constructor t of class Foo.  
    }  
}
```

Object Construction and Disposal

כדי לבנות אובייקט צריך ראשית להצהיר על משתנה רפרנס (מצביע לאובייקט), וכדי להשלים את הבנייה (אם נרצה) צריך לקרוא לבנאי מהמחלקה של האובייקט. לכן, מחלקה שממשת טיפוס (כמו Fraction) חייבת להכיל לפחות בנאי אחד. הקונבנציה היא שלפחות לבנאי אחד קוראים *.new*.

ניתן לבנות אובייקטים ולעשות להם השמה למשתנים ע"י שימוש בביטוי מהצורה:

let varName = className.constructorName(exp₁, exp₂, ..., exp_n)
בנאים בדרך כלל מכילים קוד שמאתחל את השדות של האובייקט החדש לארגומנטים שהועברו ע"י ה-caller.

דוגמה: *let c = Circle.new(x, y, 50)*

כאשר אין יותר צורך באובייקט מסוים, אפשר להיפטר ממנו, כדי לפנות את הזיכרון שהוא תופס. כדאי למתכנתי Jack להיפטר מאובייקטים שאין בהם יותר צורך כדי להימנע מדליפות זיכרון.

דוגמה: נניח שאין יותר באובייקט ש-c מצביע עליו. אפשר לעשות לו deallocate מהזיכרון ע"י קריאה לפונקציית ה-OS *Memory.deAlloc(c)*.

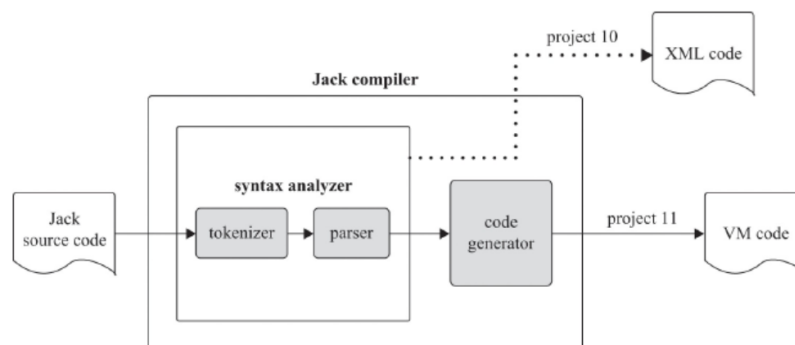
Compiler – Syntax Analysis

בפרק זה נתחיל לבנות קומפיילר לשפת ה-Jack. קומפיילר הוא תוכנית שמתרגמת תוכניות משפת מקור לשפת יעד. תהליך הקומפילציה מבוסס על שתי משימות שונות: הראשונה היא הבנת הסינטקס של תוכנית המקור וגילוי הסמנטיקה שלה, והשנייה היא ביטוי מחדש של הסמנטיקה באמצעות הסינטקס של שפת היעד. המשימה הראשונה נקראת Syntax Analysis ונתאר אותה בפרק זה, והמשימה השנייה נקראת Code Generation ונתאר אותה בפרק הבא.

הפלט של ה-syntax analyzer יהיה קובץ XML שהתוכן שלו ישקף את המבנה התחבירי של קוד המקור. נתחיל ברקע שמתאר את הקונספטים הדרושים שלנו כדי לבנות syntax analyzer: ניתוח לקסיקלי, דקדוקים חסרי-הקשר, parse trees ואלגוריתמים רקורסיביים לפרסר. זה מכין את הקרקע עבור הדקדוק של שפת ה-Jack והפלט שמצופה שה-Jack Analyzer ייצר.

רקע

ה-Syntax Analysis מחולק ל-2 תתי-שלבים: tokenizing – grouping של תווי קלט ליחידות שפה שנקראות tokens, ו-parsing – ה-grouping של ה-tokens להצהרות ממבנה מסוים בעלות משמעות.



בפרק זה נתמקד ב-syntax analyzer שמטרתו היא להבין את המבנה של התוכנית. בני אדם יכולים להבחין מהי בנייה תקינה של תוכנית ומה לא. בני אדם יכולים לזהות היכן מתחילות ומסתיימות מחלקות ומתודות, מהן declarations, מהן statements, מהם ביטויים וכיצד הם בנויים, ועוד. כדי לזהות את בניות השפה הללו, בני אדם ממפים אותן לטווח של תבניות טקסט המקובלות ע"י הדקדוק של השפה.

ניתן לפתח syntax analyzer הפועל באופן דומה ע"י בנייה המתאימה לדקדוק נתון – קבוצה של כללים שמגדירים את הסינטקס של שפת תכנות. להבין, כלומר לפרסר, תוכנית נתונה זה לקבוע את ההתאמה המדויקת בין הטקסט של התוכנית וחוקי הדקדוק. כדי לעשות זאת, נצטרך קודם כל להפוך את הטקסט של התוכנית לרשימה של tokens, כפי שנתאר בעת.

Lexical Analysis

כל מפרט של שפת תכנות מכיל את הטיפוסים של tokens, או מילים, שהשפה מזהה. בשפת ה-Jack יש 5 קטגוריות של tokens: keywords (כמו class ו-while), סימבולים (כמו + ו-<), integer constants (כמו 17 ו-314), string constants (כמו "FAQ"), ו-identifiers שהם תוויות טקסטואליות המשמשות עבור שמות של משתנים, מחלקות ו-subroutines.

בצורה הכי פשוטה שלה, תוכנית מחשב היא stream של תווים השמורים בקובץ טקסט. הצעד הראשון בניתוח הסינטקס של התוכנית היא לקבץ את התווים הללו ל-tokens, כאשר מתעלמים מ-white spaces ומ-comments. המשימה הזו נקראת lexical analysis.

הלקסיקון של שפת ה-Jack:

keyword: 'class' | 'constructor' | 'function' |
'method' | 'field' | 'static' | 'var' |
'int' | 'char' | 'boolean' | 'void' |
'true' | 'false' | 'null' | 'this' | 'let' |
'do' | 'if' | 'else' | 'while' | 'return'

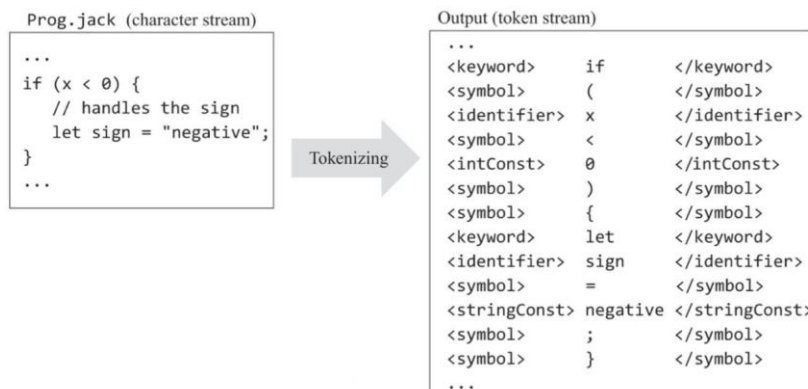
symbol: '{' | '}' | '(' | ')' | '[' | ']' | ':' | ';' | ',' |
'+' | '-' | '*' | '/' | '%' | '&' | '|' | '<' | '>' | '=' | '~'

integerConstant: a decimal integer in the range 0 ... 32767

stringConstant: "" a sequence of characters, not including double quote or newline ""

identifier: a sequence of letters, digits, and underscore ('_'), not starting with a digit.

דוגמה ל-lexical analysis:



דקדוקים – Grammars

לאחר שנפתח את היכולת לגשת לטקסט נתון בתור stream של tokens (או מילים), נוכל להמשיך את הניסיון לקבץ את המילים הללו למשפטים תקינים.

דוגמה – חלק מהדקדוק של שפת ה-Jack ומקטעי קוד שהדקדוק מקבל או דוחה:

Jack grammar (subset)

```

statements: statement*

statement: letStatement |
          ifStatement |
          whileStatement

letStatement: 'let' varName '=' expression ';'

ifStatement: 'if' '(' expression ')'
            '{' statements '}'

whileStatement: 'while' '(' expression ')'
               '{' statements '}'

expression: term (op term)?

term: varName | constant

varName: a string not beginning with a digit

constant: a non-negative integer

op: '+' | '-' | '=' | '>' | '<'
        
```

Input examples

let x = 100; ✓

let x = x + 1; ✓

if (x = 1)
 {
 let x = 100;
 let x = x + 1;
 } ✗

while (lim < 100) {
 if (x = 1) {
 let z = 100;
 while (z > 0) {
 let z = z - 1;
 }
 }
 let lim = lim + 10;
 } ✓

דקדוק הוא קבוצה של חוקים, כאשר כל חוק מכיל צד ימני וצד שמאלי. הצד השמאלי מציין את שם החוק שאינו חלק מהשפה והוא לא חשוב במיוחד. הצד השמאלי מתאר את התבנית הלשונית שהחוק מפרט. התבנית הזו היא רצף (משמאל לימין) שמכיל 3 בלוקים: terminals (tokens), non-terminals (שמות של חוקים אחרים), ו-quantifiers (מיוצגים ע"י 5 הסמלים: |, *, ?, (,), -). טרמינלים, כמו **if**, מצוינים ב-bold ועטופים ב-'!'. לא-טרמינלים, כמו *expression*, מצוינים ב-italic. כמתים מצוינים בפונט רגיל.

דוגמה: החוק $ifStatement: 'if' '(' expression ')' '{' statements '}'$ קובע כי כל מופע תקין של *ifStatement* חייב להתחיל עם ה-token **if**, אחריו ה-token {, אחריו מופע תקין של *expression* (מוגדר במקום אחר בדקדוק), אחריו ה-token {, אחריו מופע תקין של *statements* (מוגדר במקום אחר בדקדוק), אחריו ה-token }.

כאשר יש יותר מדרך אחת לפרסר תבנית, נשתמש בכמת | כדי לציין את האלטרנטיבות.

דוגמה: החוק $statement: letStatement \mid ifStatement \mid whileStatement$ קובע כי *statement* יכולה להיות להיות *letStatement*, *ifStatement*, או *whileStatement*.

הכמת * משמש כדי לסמן "0, 1, או יותר פעמים".

דוגמה: החוק $statements: statement *$ קובע כי *statements* מייצג 0, 1, או יותר מופעים של *statement*.

באופן דומה, הכמת ? משמש כדי לסמן "0 או 1 פעמים".

דוגמה: החוק $expression: term (op term)?$ קובע כי *expression* הוא *term* שייתכן כי אחריו מגיע הרצף *op term* וייתכן שלא. נובע מכך ש-*x* הוא *expression*, וכך גם $x + 17$, $5 * 7$, $x < y$.

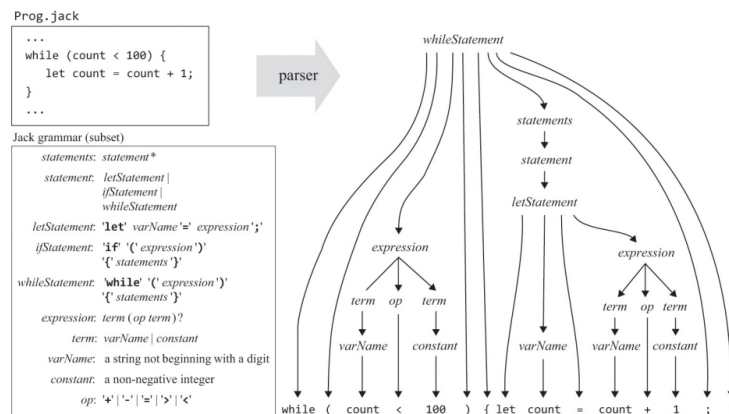
הכמתים (ו-) משמשים לקבץ איברים של דקדוק.

דוגמה: (op term) קובע כי בהקשר של החוק הזה, *op* שאחריו *term* צריך להיות איבר דקדוקי אחד.

Parsing

דקדוקים הם רקורסיביים. נתבונן ב-statement הבאה: $\{if(x < 0) \{f(y > 0) \{...\}\}\}$. כיצד נוכל לדעת האם הקלט הזה מתקבל ע"י הדקדוק? לאחר שנקבל את ה-token הראשון ובבין שיש לנו תבנית if, נתמקד בחוק $ifStatement: 'if'('expression')\{'statements'\}$. החוק אומר שאחרי ה-token **if** חייב להיות ה-token $($, שאחרי $expression$, ואחרי ה-token $)$. ואכן, הדרישות הללו מספקות את איבר הקלט $(x < 0)$. אם נחזור לחוק, נצפה לראות את ה-token $\{$, שאחרי $statements$, ואחרי זה ה-token $\}$. כעת, $statements$ מוגדר בתור 0 או יותר מופעים של $statement$, שיכולה להיות $ifStatement$, $letStatement$ או $whileStatement$, מה שמתאים לאיבר הקלט $\{...\}$ $if(y > 0)$ שהוא $ifStatement$.

נשים לב שדקדוק של שפת תכנות יכול לשמש כדי לוודא האם קלטים נתונים מתקבלים או נדחים. ה-parser מייצר התאמה מדויקת בין קלט נתון מצד אחד ובין תבניות סינטקס של הדקדוק. אפשר לייצג את ההתאמה הזו ע"י מבנה נתונים שנקרא parse tree או derivation tree. כך זה נראה:



אם ניתן לבנות עץ כזה, ה-parser יגיד שהקלט תקין. אחרת, הוא יוכל לדווח שהקלט מכיל שגיאות סינטקס. כדי לייצג parse tree באופן טקסטואלי, ה-parser יחזיר כפלט קובץ XML שהפורמט שלו משקף את מבנה העץ. ע"י בחינת קובץ ה-XML שמתקבל כפלט, נוכל לוודא שה-parser מפרסר את הקלט בצורה נכונה. קובץ XML שמייצג את ה-parse tree מהדוגמה הקודמת:

```

Prog.xml
...
<whileStatement>
  <keyword> while </keyword>
  <symbol> ( </symbol>
  <expression>
    <term> <varName> count </varName> </term>
    <op> <symbol> < </symbol> </op>
    <term> <constant> 100 </constant> </term>
  </expression>
  <symbol> ) </symbol>
  <symbol> { </symbol>
  <statements>
    <statement> <letStatement>
      <keyword> let </keyword>
      <varName> count </varName>
      <symbol> = </symbol>
      <expression>
        <term> <varName> count </varName> </term>
        <op> <symbol> + </symbol> </op>
        <term> <constant> 1 </constant> </term>
      </expression>
      <symbol> ; </symbol>
    </letStatement> </statement>
  </statements>
  <symbol> } </symbol>
</whileStatement>
...

```

Parser

Parser הוא "סוכן" שפועל בהתאם לדקדוק נתון. ה-parser מקבל כקלט stream של tokens ומנסה לייצר כפלט את ה-parse tree המזוהה עם הקלט. במקרה שלנו, הקלט אמור להיות במבנה המתאים לדקדוק של ה-Jack, והפלט כתוב ב-XML.

קיימים מספר אלגוריתמים לבניית parse trees. גישת ה-top-down, הידועה גם בתור recursive descent parsing, מנסה לפרסר את הקלט ה-tokenized רקורסיבית, ע"י שימוש במבנים מקוננים שקיימים בדקדוק של השפה. אפשר ליישם אלגוריתם כזה באופן הבא: עבור כל כלל לא טריוויאלי בדקדוק, נוסיף לקוד של ה-parser רוטינה שמטרתה לפרסר את הקלט בהתאם לכלל.

דוגמה: ניזכר בדקדוק שראינו קודם:

Jack grammar (subset)

```
statements: statement*

statement: letStatement |
          ifStatement |
          whileStatement

letStatement: 'let' varName '=' expression ';'

ifStatement: 'if' '(' expression ')'
            '{' statements '}'

whileStatement: 'while' '(' expression ')'
               '{' statements '}'

expression: term (op term)?

term: varName | constant

varName: a string not beginning with a digit

constant: a non-negative integer

op: '+' | '-' | '=' | '>' | '<'
```

אפשר לממש אותו ע"י שימוש בקבוצת רוטינות שנקראות *compileStatement*, *compileExpressions*, *compileIf*, *compileLet*, *compileStatements*, וכך הלאה.

הלוגיקה של לפרסר כל רוטינת *compileXXX* צריכה לעקוב אחרי תבנית הסינטקס המצוינת בצד ימין של הכלל *XXX*.

דוגמה: נתמקד בכלל *whileStatement: 'while' '(' expression ')' '{' statements '}'*. לפי הסכמה שלנו, נממש את הכלל הזה ע"י רוטינה שנקראת *compileWhile*. ניתן לממש זאת באמצעות הפסאודו-קוד הבא:

```
// This routine implements the rule whileStatement:
// 'while' '(' expression ')' '{' statements '}'
// Should be called if the current token is 'while'.
compileWhile():
    print("<whileStatement>")
    process("while")
    process("(")
    compileExpression()
    process(")")
    process("{")
    compileStatements()
    process("}")
    print("</whileStatement>")

// A helper routine that handles
// the current token, and advances
// to get the next token.
process(str):
    if (currentToken == str)
        printXMLToken(str)
    else
        print("syntax error")
    // Gets the next token
    currentToken =
        tokenizer.advance()
```

תהליך הפרסור ימשיך עד שנפרסר באופן מלא את חלקי ה-*expression* וה-*statements*. ייתכן כי חלק ה-*statement* יכול גם *whileStatement*, ובמקרה זה נמשיך לפרסר רקורסיבית.

דוגמה: נתבונן בחוק הבא שמציין את ההגדרה של משתנה מחלקה סטטי ומשתנה שדה של מופע בשפת ה-Jack:

```
classVarDec: ('static' | 'field') type varName (',' varName)* ';'
              (where type and varName are defined elsewhere in the full Jack grammar)

Examples:  static int count;
           static char a, b, c;
           field boolean sign;
           field int up, down, left, right;
```

הכלל הזה מייצג שני אתגרים – ה-*token* הראשון שלו הוא אחת מהמילים *static* או *field*, והוא מכיל מספר הצהרות על משתנים. המימוש של רוטינת *compileClassVarDec* המתאימה יכול לטפל בעיבוד של ה-*token* הראשון באופן ישיר מבלי לקרוא לפונקציית עזר, ולהשתמש בלולאה כדי לטפל בכל ההצהרות על משתנים שיש בקלט.

יכולים להיות מימושים מעט שונים כדי לפרסר כללי דקדוק שונים, אבל כולם מקיימים את אותו ה"חוזה": כל רוטינת *compileXXX* צריכה לקבל קלט ולטפל בכל ה-*tokens* שיוצרים את *XXX*, לקדם את ה-*tokenizer* בדיוק כמספר ה-*tokens* הללו, ולהחזיר כפלט את ה-*parse tree* של *XXX*.

מספיק לנו להתבונן ב-*token* אחד קדימה כדי לדעת לאיזה כלל צריך לקרוא. למשל, אם ה-*token* הנוכחי הוא *let* אנחנו יודעים שיש לנו *letStatement*, אם ה-*token* הנוכחי הוא *while* אנחנו יודעים שיש לנו *whileStatement*, וכך הלאה. בדקדוק כמו שראינו בהתחלה, מספיק לנו *token* אחד כדי לדעת מה הכלל הבא שצריך להשתמש בו. לדקדוקים המקיימים את התכונה הזאת קוראים (1) *LL*, ואפשר לטפל בהם עם אלגוריתמים רקורסיביים פשוטים ללא *backtracking*.

Specification

The Jack Language Grammar: נתאר מפרט פורמלי של שפת ה-Jack המיועד עבור מפתחים של Jack compiler. המפרט של השפה, או הדקדוק, משתמש בסימונים הבאים:

מייצג <i>tokens</i> שמופיעים מילה במילה	'xxx'
מייצג שמות של איברים שהם טרמינלים ולא טרמינלים	xxx
משמש ל-grouping	()
x או y	x y
x שאחריו יש y	x y
x מופיע 0 או 1 פעמים	x?
x מופיע 0 או יותר פעמים	x*

הדקדוק המלא של Jack:

Lexical elements:	The Jack language includes five types of terminal elements (tokens):
<i>keyword:</i>	'class' 'constructor' 'function' 'method' 'field' 'static' 'var' 'int' 'char' 'boolean' 'void' 'true' 'false' 'null' 'this' 'let' 'do' 'if' 'else' 'while' 'return'
<i>symbol:</i>	'{' '}' '(' ')' '[' ']' '.' ':' ';' '+' '-' '*' '/' '%' ' ' '<' '>' '=' '~'
<i>integerConstant:</i>	A decimal integer in the range 0...32767
<i>StringConstant:</i>	"" A sequence of characters not including double quote or newline ""
<i>identifier:</i>	A sequence of letters, digits, and underscore ('_'), not starting with a digit
Program structure:	A Jack program is a collection of classes, each appearing in a separate file. The compilation unit is a class. A <i>class</i> is a sequence of tokens, as follows:
<i>class:</i>	'class' className '{' classVarDec* subroutineDec* '}'
<i>classVarDec:</i>	('static' 'field') type varName (';' varName)* ';'
<i>type:</i>	'int' 'char' 'boolean' className
<i>subroutineDec:</i>	('constructor' 'function' 'method') ('void' type) subroutineName '(' parameterList ')' subroutineBody
<i>parameterList:</i>	((type varName) (';' type varName)*)?
<i>subroutineBody:</i>	'{' varDec* statements '}'
<i>varDec:</i>	'var' type varName (';' varName)* ';'
<i>className:</i>	identifier
<i>subroutineName:</i>	identifier
<i>varName:</i>	identifier
Statements:	
<i>statements:</i>	statement*
<i>statement:</i>	letStatement ifStatement whileStatement doStatement returnStatement
<i>letStatement:</i>	'let' varName '(' '[' expression ']')? '=' expression ';'
<i>ifStatement:</i>	'if' '(' expression ')' '{' statements '}' ('else' '{' statements '}')?
<i>whileStatement:</i>	'while' '(' expression ')' '{' statements '}'
<i>doStatement:</i>	'do' subroutineCall ';'
<i>returnStatement</i>	'return' expression? ';'
Expressions:	
<i>expression:</i>	term (op term)*
<i>term:</i>	integerConstant stringConstant keywordConstant varName varName '[' expression ']' '(' expression ')' (unaryOp term) subroutineCall
<i>subroutineCall:</i>	subroutineName '(' expressionList ')' (className varName) '.' subroutineName '(' expressionList ')'
<i>expressionList:</i>	(expression (';' expression) *)?
<i>op:</i>	'+' '-' '*' '/' '%' ' ' '<' '>' '='
<i>unaryOp:</i>	'-' '~'
<i>keywordConstant:</i>	'true' 'false' 'null' 'this'

Implementation

המימוש המוצע מבוסס על 3 מודולים: JackAnalyzer – תוכנית ראשית שיוצרת את שאר המודולים וקוראת להם, JackTokenizer – ה-tokenizer, CompilationEngine – ה-parser, הרקורסיבי.

The Jack Tokenizer

המודול הזה מתעלם מכל ה-comment וה-white spaces בגשת לקלט באמצעות token אחד בכל פעם. בנוסף, הוא מפרסר ומספק את הסוג של כל token כפי שהוגדר ע"י הדקדוק של ה-Jack.

הפונקציה	ערך החזרה	ארגומנטים	Routine
פותח את קובץ הקלט ומתכוון לעשות לו tokenize	-	קובץ קלט	<i>Constructor</i>

האם יש עוד tokens בקלט?	Boolean	-	<i>hasMoreTokens</i>
מביאה את ה-token הבא מהקלט והופכת אותו ל-token הנוכחי. צריך לקרוא לה רק אם <i>hasMoreTokens</i> הוא true. באתחול אין token נוכחי	-	-	<i>advance</i>
מחזירה את הסוג של ה-token הנוכחי, בתור קבוע	KEYWORD, SYMBOL, IDENTIFIER, INT_CONST, STRING_CONST	-	<i>tokenType</i>
מחזירה את ה-keyword שהיא ה-token הנוכחי בתור קבוע. צריך לקרוא לה רק אם <i>tokenType</i> הוא KEYWORD	CLASS, METHOD, FUNCTION, CONSTRUCTOR, INT, BOOLEAN, CHAR, VOID, VAR, STATIC, FIELD, LET, DO, IF, ELSE, WHILE, RETURN, TRUE, FALSE, NULL, THIS	-	<i>keyword</i>
מחזירה את התו שהוא ה-token הנוכחי. צריך לקרוא לה רק אם <i>tokenType</i> הוא SYMBOL	char	-	<i>symbol</i>
מחזירה את המחרוזת שהיא ה-token הנוכחי. צריך לקרוא לה רק אם <i>tokenType</i> הוא IDENTIFIER	string	-	<i>identifier</i>
מחזירה את הערך המספרי של ה-token הנוכחי. צריך לקרוא לה רק אם <i>tokenType</i> הוא INT_CONST	int	-	<i>intVal</i>
מחזירה את ערך המחרוזת של ה-token הנוכחי, מבלי "פותרות או סוגרות. צריך לקרוא לה רק אם <i>tokenType</i> הוא STRING_CONST	string	-	<i>stringVal</i>

The CompilationEngine

מחזיר כפלט מבנה המייצג את קוד המקור שהתקבל בקלט עטוף בתגיות XML. מקבל את הקלט שלו מ-JackTokenizer וכותב את הפלט שלו לקובץ פלט. הפלט מיוצר ע"י סדרת רוטיות *compileXXX*, כאשר המטרה של כל אחת מהן היא לקמפל מבנה ספציפי XXX של שפת ה-Jack. כל רוטינה כזו נקראת רק אם ה-token הנוכחי הוא XXX.

כללי דקדוק שאין להם רוטינות *compileXXX* מתאימות: *className, type, subroutineName, subroutineCall, statement, varName*. הצגנו את הכללים הללו כדי להפוך את הדקדוק של שפת ה-Jack ליותר "מבני". עדיף לטפל בלוגיקת הפרסור של כל אחד מהכללים הללו ברוטינות שמממשות את הכללים שמתייחסים אליהם. למשל, במקום לכתוב רוטינת *compileType* בכל פעם ש-*type* מופיע בכלל *XXX* כלשהו, הפרסור של הטיפוסים האפשריים יבוצע באופן ישיר ע"י הרוטינה *compileXXX*.

Token Lookahead: ה-token הנוכחי הוא מספיק כדי לקבוע לאיזה רוטינה של *CompileEngine* צריך לקרוא. המקרה היחיד היוצא מן הכלל הוא כאשר מפרסרים *term*, שמתרחש כאשר מפרסרים *expression*.

כדי להדגים זאת, נתבונן ב-*expression* הבא:

$$y + arr[5] - p.get(row) * count() - Math.sqrt(dist)/2$$

הוא מורכב מ-6 *terms*: המשתנה *y*, האיבר במערך *arr[5]*, קריאה למתודה של האובייקט *p* *p.get(row)*, קריאה למתודה *count()* של האובייקט *this*, קריאה לפונקציה *Math.sqrt(dist)*, והקבוע 2.

נניח שאנחנו מפרסים את הביטוי הזה וה-token הנוכחי הוא אחד מה-*identifiers: p, arr, y*, *count*, או *Math*. בכל אחד מהמקרים הללו, אנחנו יודעים שיש לנו *term* שמתחיל עם *identifier*, אבל אנחנו לא יודעים איזו אפשרות פרסור לבצע אחרי זה. החדשות הטובות הן שהתבוננות ב-token הבא תפתור לנו את הדילמה.

הצורך בפעולת ה-lookahead הלא שגרתית הזו מתרחש פעמיים ב-*CompilationEngine*: כאשר מפרסרים *term* (מה שקורה רק כאשר מפרסרים *expression*), וכאשר מפרסרים *subroutineCall*. נשים לב ש-*subroutineCall* מופיעה בשני מקומות בלבד: ב-*statement* של *do subroutineCall* או ב-*term*.

לכן, נציע לפרסר *statements* של *do subroutineCall* כאילו הסינטקס שלהם הוא *do expression*. זה מונע מאיתנו לכתוב את הקוד עם ה-lookahead פעמיים. נובע מכך שניתן לפרסר *subroutineCall* באופן ישיר ברוטינת *compileTerm*.

הפונקציה	Routine
מקבלת כארגומנטים קובץ קלט וקובץ פלט, ויוצרת compilation engine חדש עם הקלט והפלט הנתונים. הרוטינה הבאה שתיקרא (ע"י ה-JackAnalyzer) חייבת להיות <i>compileClass</i>	<i>Constructor</i>
מקמפלת מחלקה שלמה	<i>compileClass</i>
מקמפלת הצהרה על משתנה סטטי, או הצהרה על שדה	<i>compileClassVarDec</i>
מקמפלת מתודה שלמה, פונקציה שלמה, או בנאי שלם	<i>compileSubroutine</i>
מקמפלת רשימת פרמטרים (יכולה להיות ריקה), ולא מטפלת בסוגריים העוטפים, כלומר ב-tokens (ו-)	<i>compileParameterList</i>
מקמפלת את גוף ה-subroutine	<i>compileSubroutineBody</i>
מקמפלת הצהרות על var	<i>compileVarDec</i>
מקמפלת רצף של statements. לא מטפלת בסוגריים המסולסלים העוטפים, כלומר ב-tokens { ו- }.	<i>compileStatements</i>

let statement מקמפלת	<i>compileLet</i>
if statement, עם בלוק else אופציונלי אחרי מקמפלת	<i>compileIf</i>
while statement מקמפלת	<i>compileWhile</i>
do statement מקמפלת	<i>compileDo</i>
return statement מקמפלת	<i>compileReturn</i>
expression מקמפלת	<i>compileExpression</i>
term. אם ה-token הנוכחי הוא identifier, הרוטינה חייבת להחליט אם הוא משתנה, איבר במערך או קריאה ל-subroutine. מספיק להתבונן ב-token הבא, שיכול להיות [, (, או . כדי להבדיל בין האפשרויות. כל token אחר אינו חלק מה-term הזה ואי אפשר לדלג עליו	<i>compileTerm</i>
מקמפלת רשימה (ייתכן ריקה) של expressions המופרדים בפסיק. מחזירה int שהוא מספר ה-expressions ברשימה	<i>compileExpressionList</i>

The compileExpressionList Routine: מחזירה את מספר ה-expressions ברשימה. ערך ההחזרה נחוץ עבור ייצור קוד VM שישלים את פיתוח הקומפילר בפרויקט הבא. בפרויקט הזה אנחנו לא מייצרים קוד VM ולכן לא נשתמש בערך ההחזרה וניתן להתעלם ממנו ברוטינות שקוראות ל-*compileExpressionList*.

The Jack Analyzer

התוכנית הראשית שמניעה את כל תהליך ה-syntax analysis, ע"י שימוש בשירותים של JackTokenizer ושל CompilationEngine. עבור כל קובץ מקור *xxx.jack*, ה-analyzer:

1. מייצר JackTokenizer מקובץ הקלט *xxx.jack*.
2. מייצר קובץ פלט בשם *xxx.xml*.
3. משתמש ב-JackTokenizer וב-CompilationEngine כדי לפרסר את קובץ הקלט ולכתוב את הקוד המפורסר לקובץ הפלט.

Compiler – Code Generation

חילקנו את בניית הקומפיילר לשני מודולים: `syntax analyzer` (שפיתחנו בפרק הקודם) ו-`code generator` (נושא הפרק הזה). בנינו את ה-`syntax analyzer` כדי לפתח וכדי להראות את היכולת לפרסר תוכניות high-level ל-`syntax elements`. בפרק הזה נרחיב את ה-`analyzer` לקומפיילר שלם: תוכנית שממירה את האיברים שפרסרנו לפקודות VM שמטרתן לרוץ על המכונה הווירטואלית שתיארנו בפרקים 7-8. נשתמש בסינטקס של שפת המקור (Jack) כדי לנתח את טקסט המקור ולהבין את הסמנטיקה שלו, ולאחר מכן נשתמש בזה כדי לבטא מחדש את הסמנטיקה שפרסרנו באמצעות שפת היעד (VM).

ראשית, נתאר כיצד קומפיילרים משתמשים ב-`symbol tables` כדי למפות משתנים סימבוליים למקטעי זיכרון וירטואליים. לאחר מכן, נציג אלגוריתמים כדי לקמפל `expressions` ומחרוזות של תווים. לאחר מכן נציג טכניקות כדי לקמפל `statements` כמו `let`, `if`, `while`, `do`, ו-`return`. יחדיו, היכולת לקמפל `expressions` ו-`statements` יוצרת את הבסיס לבניית קומפיילרים. זה מכין את השטח עבור קומפילציה של אובייקטים ומערכים שגם על זה נדבר.

Code Generation

מתכנתים של שפות high-level עובדים עם `abstract building blocks` כמו משתנים, `expressions`, `statements`, `subroutines`, אובייקטים ומערכים. מתכנתים משתמשים בהן כדי לתאר מה הם רוצים שהתוכנית תעשה. התפקיד של הקומפיילר הוא לתרגם את הסמנטיקה הזו לשפה שמחשב היעד מבין. במקרה שלנו, מחשב היעד הוא מכונה וירטואלית.

במהלך הפרק, נציג דוגמאות קומפילציה של מספר חלקים של המחלקה `Point`:

```
/** Represents a two-dimensional point.
    File name: Point.jack. */
class Point {

    // The coordinates of this point:
    field int x, y

    // The number of Point objects constructed so far:
    static int pointCount;

    /** Constructs a two-dimensional point and
        initializes it with the given coordinates. */
    constructor Point new(int ax, int ay) {
        let x = ax;
        let y = ay;
        let pointCount = pointCount + 1;
        return this;
    }

    /** Returns the x coordinate of this point. */
    method int getX() { return x; }

    /** Returns the y coordinate of this point. */
    method int getY() { return y; }

    /** Returns the number of Point constructed so far. */
    function int getPointCount() {
        return pointCount;
    }

    // Class declaration continues on top right.
```

```
/** Returns a point which is this point plus
    the other point. */
    method Point plus(Point other) {
        return Point.new(x + other.getX(),
                        y + other.getY());
    }

    /** Returns the Euclidean distance between
    this and the other point. */
    method int distance(Point other) {
        var int dx, dy;
        let dx = x - other.getX();
        let dy = y - other.getY();
        return Math.sqrt((dx*dx) + (dy*dy));
    }

    /** Prints this point, as "(x,y)" */
    method void print() {
        do Output.printString("(");
        do Output.printInt(x);
        do Output.printString(",");
        do Output.printInt(y);
        do Output.printString(")");
        return;
    }

} // End of Point class declaration.
```

במחלקה הזו קיימים כל הסוגים האפשריים של משתנים (`field`, `static`, `local`, ו-`argument`) וכל הסוגים האפשריים של `subroutines` (בנאי, מתודה, ופונקציה) וגם `subroutines` שמחזירות

טיפוסים פרימיטיביים, טיפוס אובייקט, ו-void. המחלקה הזו גם מדגימה קריאות לפונקציה, קריאות לבנאי, וקריאות למתודה על האובייקט הנוכחי (this) ועל אובייקטים אחרים.

Handling Variables

אחת מהמטרות הבסיסיות של קומפיילרים היא מיפוי של כל המשתנים שהוצהרו בתוכנית המקור ה-high-level ל-RAM של פלטפורמת היעד. כל הטיפוסים הפרימיטיביים ב-Jack (char, int, ו-boolean) הם בגודל 16-ביט, וכך גם הכתובות והמילים ב-RAM של ה-Hack. לכן, ניתן למפות כל משתנה ב-Jack (כולל מצביעים שמחזיקים כתובות של 16-ביט) בדיוק למילה אחת בזיכרון.

אתגר נוסף איתו קומפיילרים מתמודדים הוא שלמשתנים מסוגים שונים יש "life cycles" שונים. משתנים סטטיים ברמת המחלקה משותפים באופן גלובלי לכל ה-subroutines במחלקה. לכן, צריך לשמור עותק יחיד של כל משתנה סטטי במהלך כל הריצה של התוכנית. במשתני שדה ברמת המופע (instance-level) נטפל באופן שונה – לכל אובייקט (מופע של המחלקה) חייבת להיות קבוצה פרטית של משתני השדה שלו, וכאשר אין יותר צורך באובייקט צריך לשחרר את הזיכרון הזה. משתנים לוקאליים וארגומנטים ברמת ה-subroutine נוצרים בכל פעם ש-subroutine מתחילה את הריצה שלה וצריך לשחרר אותם כאשר ה-subroutine מסיימת את ריצתה.

כל מה שעלינו לעשות: למפות את המשתנים הסטטיים ל-`static 0, static 1, ...`, למפות משתני שדה ל-`this 0, this 1, ...`, למפות משתנים לוקאליים ל-`local 0, local 1, ...`, ולמפות משתני ארגומנטים ל-`argument 0, argument 1, ...`.

ניזכר כי עבדנו קשה כדי לייצר קוד אסמבלי שממפה באופן דינאמי את מקטעי הזיכרון הווירטואליים על ה-RAM כחלק מההשפעה של פרוטוקול הקריאה והחזרה מפונקציה. לכן, הדבר היחיד שנדרש מהקומפיילר הוא למפות את המשתנים ה-high-level למקטעי זיכרון וירטואליים. ניתן לבצע זאת באמצעות symbol table.

Symbol Table: בכל פעם שהקומפיילר נתקל במשתנים ב-statement, כמו למשל $let\ y = foo(x)$, הוא צריך לדעת מה המשתנים הללו מייצגים. האם x הוא משתנה סטטי, שדה של אובייקט, משתנה לוקאלי, או ארגומנט של subroutine? האם הוא מייצג Boolean, integer, char, או טיפוס של מחלקה? צריך לענות על כל השאלות הללו בכל פעם שהמשתנה x מופיע בקוד המקור. כמובן, צריך לטפל באופן דומה במשתנה y .

ניתן לנהל את התכונות הללו של משתנים באמצעות symbol table. כאשר יש הצהרה בקוד המקור על משתנה סטטי, שדה, לוקאלי, או ארגומנט, הקומפיילר מקצה עבורו את המקום הפנוי הבא במקטע ה-VM המתאים – static, this, local, או argument – ומתעד את המיפוי ב-symbol table. בכל פעם שיינתקל במשתנה במקום כלשהו בקוד, הקומפיילר יחפש את השם שלו ב-symbol table, ישלוף את התכונות שלו, וישתמש בו כפי שצריך עבור ה-code generation.

תכונה חשובה של שפות high-level היא separate namespaces: אותו identifier יכול לייצר דברים שונים באזורים שונים בתוכנית. כדי לאפשר מרחבי שמות נפרדים, כל identifier מזהה עם scope שהוא האזור בתוכנית שבו הוא מוכר. ב-Jack, ה-scope של משתנים סטטיים ומשתני שדה הוא המחלקה שבה הצהירו עליהם, וה-scope של משתנים לוקאליים וארגומנטים הוא ה-

subroutine שבה הצהירו עליהם. הקומפיילרים של Jack יכולים לממש את אבסטרקציות ה-scope ע"י ניהול שתי symbol tables נפרדות.

דוגמה ל-symbol tables:

High-level (Jack) code

```
class Point {
  field int x, y;
  static int pointCount;
  ...
  method int distance(Point other) {
    var int dx, dy;
    let dx = x - other.getx();
    let dy = y - other.gety();
    return Math.sqrt((dx*dx) + (dy*dy));
  }
  ...
}
```

name	type	kind	#
x	int	field	0
y	int	field	1
pointCount	int	static	0

Class-level
symbol table

name	type	kind	#
this	Point	arg	0
other	Point	arg	1
dx	int	var	0
dy	int	var	1

Subroutine-level
symbol table

ה-scopes הם מקוננים, כאשר scopes פנימיים מסתירים את החיצוניים. לדוגמה, כאשר הקומפיילר של ה-Jack נתקל ב-expression $x + 17$, הוא קודם כל בודק האם x הוא משתנה ברמת ה-subroutine (משתנה לוקאלי או ארגומנט). אם הבדיקה הזו נכשלת, הקומפיילר בודק האם x הוא משתנה סטטי או שדה. בשפת ה-Jack יש רק שתי רמות של scoping: ה-subroutine הנוכחית שמקמפלים, והמחלקה שבה הצהירו על ה-subroutine. לכן, הקומפיילר צריך רק שתי symbol tables (בניגוד לשפות בהן יש עומק לא מוגבל של scopes מקוננים, ששם אפשר לשמור את ה-symbol tables בתור רשימה מקושרת המתחילה ב-scope הפנימי ביותר עד החיצוני ביותר).

Handling Variable Declarations:

כאשר הקומפיילר של ה-Jack מתחיל לקמפל הצהרה על מחלקה, הוא יוצר את ה-symbol table של רמת המחלקה ואת ה-symbol table של רמת ה-subroutine. כאשר הקומפיילר מפרסר הצהרה על משתנה סטטי או שדה, הוא מוסיף שורה חדשה ל-class-level symbol table. השורה מתעדת את השם, את הטיפוס (Boolean, integer, char, או שם מחלקה), את הסוג (static או field), ואת האינדקס בסוג של המשתנה.

כאשר הקומפיילר של ה-Jack מתחיל לקמפל הצהרה על subroutine (בנאי, מתודה, או פונקציה), הוא מאפס את ה-subroutine-level symbol table. אם ה-subroutine היא מתודה, הקומפיילר מוסיף את השורה $\langle this, className, arg, 0 \rangle$ ל-subroutine-level symbol table. כאשר הקומפיילר מפרסר הצהרה על משתנה לוקאלי או על ארגומנט, הוא מוסיף שורה חדשה ל-subroutine-level symbol table שמתעדת את השם, את הטיפוס (Boolean, integer, char, או שם מחלקה) את הסוג (var או arg), ואת האינדקס בסוג של המשתנה. האינדקס של כל סוג (var או arg) מתחיל ב-0 וגדל ב-1 אחרי כל פעם שמוסיפים משתנה מהסוג הזה לטבלה.

Handling Variables in Statements:

כאשר הקומפיילר נתקל במשתנה ב-statement, הוא מחפש את שם המשתנה ב-subroutine-level symbol table. אם הוא לא מוצא את המשתנה, הוא מחפש אותו ב-class-level symbol table. ברגע שהמשתנה נמצא, הקומפיילר יכול להשלים את התרגום של ה-statement.

דוגמה: בהינתן ה-symbol tables מהדוגמה הקודמת, נניח שאנחנו מקמפלים את ה-statement-
ה-high-level $let\ y = y + dy$. הקומפילר יתרגם את ה-statement הזה לפקודות ה-VM:

```
push this 1
push local 1
add
pop this 1
```

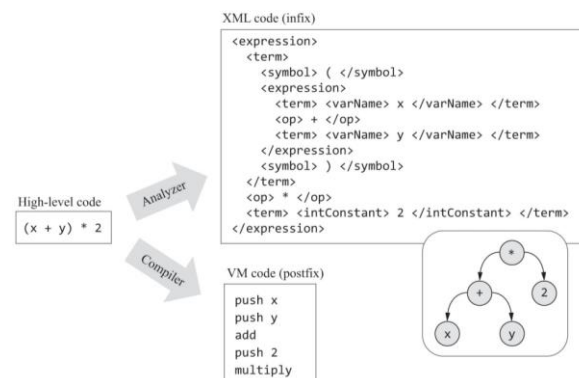
Compiling Expressions

נתבונן בקומפילציה של expressions פשוטים כמו $x + y - 7$, כאשר הכוונה ב"פשוטים" היא רצף של $term\ operator\ term\ operator\ term\ \dots$, כאשר כל $term$ הוא משתנה או קבוע, וכל $operator$ הוא $+$, $-$, $*$, או $/$.

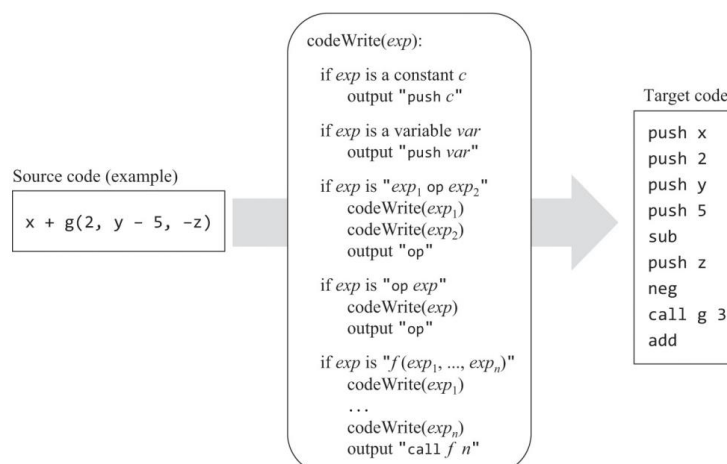
ב-Jack, ה-expressions כתובים ע"י infix notation: כדי לחבר את x ו- y נכתוב $x + y$. אבל המטרה של הקומפילציה שלנו היא postfix notation: מבטאים את אותו החיבור בקוד ה-VM ע"י הפקודות

```
push x
push y
add
```

בפרק הקודם ראינו אלגוריתם שהפלט שלו הוא קוד המקור המפורסר ב-infix ע"י שימוש בתגיות XML. צריך לשנות את הפלט של האלגוריתם הזה כדי לייצר פקודות postfix:



לסיכום, נצטרך אלגוריתם שיודע לפרסר infix expression ולייצר ממנו כפלט קוד postfix שמממש את אותה הסמנטיקה על stack machine. נציג אלגוריתם כזה:



האלגוריתם מעבד את ה-expression שקיבל כקלט משמאל לימין ומייצר קוד VM ככל שהוא ממשיך. האלגוריתם הזה גם מטפל באופרטורים אונאריים ובקריאות לפונקציה. האלגוריתם מניח שה-expression שקיבל כקלט הוא חוקי. המימוש הסופי של האלגוריתם צריך להחליף את המשתנים הסימבוליים בפלט עם המיפוי התואם מה-symbol table.

אם נריץ את קוד ה-VM שאלגוריתם ה-codeWrite ייצר, הריצה תסתיים בכך שערך ה-expression יהיה בראש המחסנית.

נתבונן בהגדרה הדקדוקית של expressions ב-Jack, ובמספר דוגמאות ל-expressions שמתאימים להגדרה הזו:

Definition (from the Jack grammar):

expression: *term* (*op term*) *

term: *integerConstant* | *stringConstant* | *keywordConstant* | *varName* |
varName '[' *expression* ']' | '(' *expression* ')' | (*unaryOp term*) | *subroutineCall*

subroutineCall: *subroutineName* '(' *expressionList* ')' |
(*className* | *varName*) '.' *subroutineName* '(' *expressionList* ')'

expressionList: (*expression* ',' *expression*) *) ?

op: '+' | '-' | '*' | '/' | '%' | '^' | '<' | '>' | '='

unaryOp: '-' | '~'

keywordConstant: 'true' | 'false' | 'null' | 'this'

The definitions of *integerConstant*, *stringConstant*, *keywordConstant*, and other elements are given in the complete Jack grammar (figure 10.6), and are self-explanatory.

Examples: 5

x

x + 5

(-b + Math.sqrt(b*b - (4 * a * c))) / (2 * a)

arr[i] + foo(x)

foo(Math.abs(arr[x + foo(5)]))

הקומפילציה של expressions ב-Jack תבצע ברוטינה הנקראת *compileExpression*. המפתח של הרוטינה הזו יצטרך להתחיל עם האלגוריתם codeWrite שהוצג, ולהרחיב אותו כדי שיטפל בכל האפשרויות שמתוארות בדקדוק של expression.

Compiling Strings

שפות שהן object-oriented בדרך כלל מתייחסות למחרוזות בתור מופעים של מחלקה שנקראת String. בכל פעם שמחרוזת מופיעה ב-statement או ב-expression בתוכנית high-level, הקומפיילר מייצר קוד שקורא לבנאי של המחלקה String, שיוצר ומחזיר אובייקט String חדש. לאחר מכן, הקומפיילר מאתחל את האובייקט החדש עם התווים של המחרוזת. זה מתבצע ע"י רצף של קריאות למתודה appendChar של String, כל קריאה עבור כל תו ב-string constant.

המימוש הזה של string constants יכול להיות מאוד בזבזני, ויכול להוביל לדליפות זיכרון פוטנציאליות. לדוגמה, נתבונן ב-statement הבאה:

Output.println("Loading ... please wait")

נוצר אובייקט String חדש, והאובייקט הזה יישאר ברקע מבלי לעשות כלום. ב-Jack יש רק מחלקת String אחת ואין אופטימיזציות הקשורות למחרוזות (כמו garbage collector בשפות אחרות).

Operating System Services: כאשר מטפלים במחרוזות, הקומפיילר יכול להשתמש בשירותים של מערכת ההפעלה במידת הצורך. מפתחים של הקומפיילר של ה-Jack יכולים להניח שכל בנאי, מתודה, ופונקציה הנמצאת ב-API של ה-OS זמינה בתור פונקציית VM מקומפלט. כלומר, אפשר לקרוא לכל אחת מפונקציות ה-VM הללו בקוד שהקומפיילר מייצר.

Compiling Statements

שפת ה-Jack מכילה 5 סוגי statements: `let`, `do`, `return`, `if`, ו-`while`. נראה כעת כיצד הקומפיילר של ה-Jack מייצר קוד VM שמטפל בסמנטיקה של ה-statements הללו.

Compiling Return Statements: כעת, כשאנחנו יודעים כיצד לקמפל expressions, הקומפילציה של `return expression` היא פשוטה: ראשית, נקרא לרוטינה `compileExpression` שמייצרת קוד VM שמחשב את `expression` ושם את הערך שלו בראש המחסנית, ולאחר מכן נייצר את פקודת ה-`return VM`.

Compiling Let Statements: נטפל ב-statements מהצורה `let varName = expression`. כיוון שפרסור הוא תהליך משמאל לימין, ראשית נזכור את ה-`varName`. לאחר מכן, נקרא ל-`compileExpression` שתשים את הערך של `expression` בראש המחסנית. לבסוף, נייצר את פקודת ה-`pop varName VM`, כאשר `varName` הוא המיפוי ב-symbol table של `varName` (לדוגמה: `static 1, local 3`).

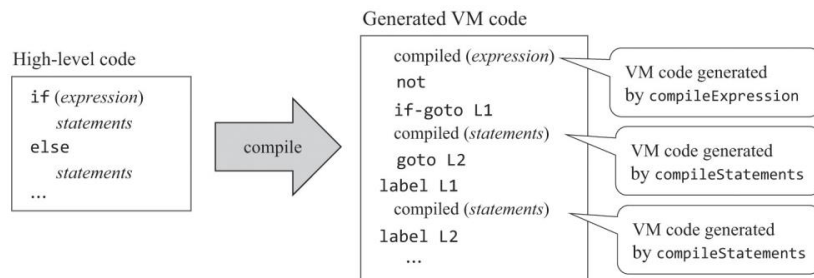
נטפל בקומפילציה של statements מהצורה `let varName[expression1] = expression2` בהמשך הפרק, בחלק המוקדש לטיפול במערכים.

Compiling Do Statements: נדבר כרגע על קומפילציה של קריאות לפונקציה מהצורה `do className.functionName(exp1, exp2, ..., expn)`. המטרה של האבסטרקציה של `do` היא לקרוא ל-subroutine עבור ה-effect שלה ולהתעלם מערך ההחזרה שלה. בפרק 10 המלצנו לקמפל statements כאלו כאילו הסינטקס שלהן הוא `do expression`. נחזור על ההמלצה הזו גם כאן: כדי לקמפל statement מהצורה `do className.functionName(...)`, נקרא ל-`compileExpression` וניפטר מהאיבר שבראש המחסנית (הערך של ה-expression) ע"י יצירת פקודה כמו `pop temp 0`.

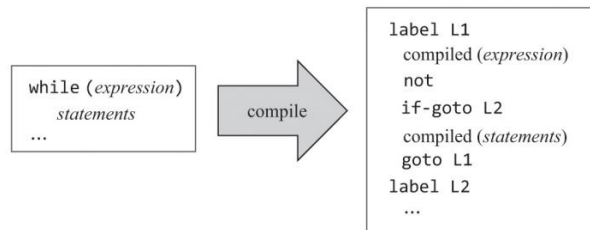
נדבר על קומפילציה של קריאות למתודה מהצורה `do varName.methodName(...)` ומהצורה `do methodName(...)` בהמשך הפרק, בחלק המוקדש לקומפילציה של קריאות למתודה.

Compiling If and While Statements: בשפות low-level כמו אסמבלי ו-VM, הביצוע של control flow statements הוא באמצעות `conditional goto` ו-`unconditional goto`.

קומפילציה של if statements:



קומפילציה של while statements:



ה- $L1$ labels ו- $L2$ מיוצרות ע"י הקומפיילר.

כאשר הקומפיילר מזהה את ה- if keyword, הוא יודע שהוא צריך לפרסר תבנית מהצורה $if (expression) \{statements\} else \{statements\}$ לכן, הקומפיילר מתחיל בקריאה ל- $compileExpression$, שמייצר פקודות VM שמטרתן לחשב את הערך של $expression$ ולדחוף אותו למחסנית. לאחר מכן, הקומפיילר מייצר את פקודת ה- not VM שמטרתה לשלול את הערך של $expression$. לאחר מכן, הקומפיילר יוצר $label$, $L1$, ומשתמש בה כדי לייצר את פקודת ה- $VM: goto L1$. לאחר מכן, הקומפיילר קורא ל- $compileStatements$. המטרה של הרוטינה הזו היא לקמפל רצף מהצורה $statement; statement; ... statement;$, כאשר כל $statement$ היא $let, do, return, if$, או $while$. קוד ה- VM שמתקבל מסומן בדוגמאות למעלה בתור " $compiled (statements)$ ".

בדרך כלל, תוכנית high-level מכילה מספר מופעים של if ושל $while$. לכן הקומפיילר יכול לייצר $labels$ שהן ייחודיות באופן גלובאלי, למשל $labels$ שהסיומת שלהן היא $counter$ רץ. בנוסף, $control flow statements$ לעיתים קרובות מקוננות – לדוגמה, if בתוך $while$ בתוך $while$ אחר וכך הלאה. ניתן לטפל ב-nesting הזה כיוון שרוטינת $compileStatements$ היא רקורסיבית.

Handling Objects

שפות שהן object-oriented מאפשרות להצהיר ולפעול על אבסטרקציות הידועות בתור אובייקטים. כל אובייקט ממומש פיזית בבלוק בזיכרון, שניתן להתייחס אליו בתור משתנה סטטי, שדה, משתנה לוקאלי, או ארגומנט. משתנה הרפרנס, הידוע גם בתור משתנה אובייקט או מצביע, מכיל את כתובת הבסיס של הבלוק בזיכרון. מערכת ההפעלה מממשת את המודל הזה ע"י ניהול אזור לוגי ב-RAM שנקרא $heap$. ה- $heap$ משמשת בתור $memory pool$ שממנה אפשר להוציא בלוקים של זיכרון, במידת הצורך, כדי לייצג אובייקטים חדשים. כאשר אין צורך יותר באובייקט, אפשר לשחרר את בלוק הזיכרון שלו ולמחזר אותו בחזרה אל ה- $heap$. הקומפיילר מבצע את פעולות ניהול הזיכרון הללו ע"י קריאות לפונקציות OS כפי שנראה בהמשך.

בכל נקודה נתונה במהלך ריצה של תוכנית, ה-heap יכולה להכיל אובייקטים רבים. נניח שאנחנו רוצים להפעיל מתודה, נניח *foo*, על אחד מהאובייקטים הללו, נניח *p*. בשפה object-oriented ניתן לעשות זאת ע"י קריאה למתודה (*p.foo*). נשים לב שהמטרה של *foo* היא לפעול על placeholder הידוע בתור האובייקט הנוכחי או *this*. בפרט, כאשר פקודות VM בקוד של *foo* מתייחסות ל-*this 0*, *this 1*, *this 2*, והלאה, הן צריכות להשפיע על השדות של *p*, האובייקט עליו *foo* נקראה. מה שמוביל לשאלה – כיצד המקטע *this* מיישר קו עם *p*?

Compiling Constructors

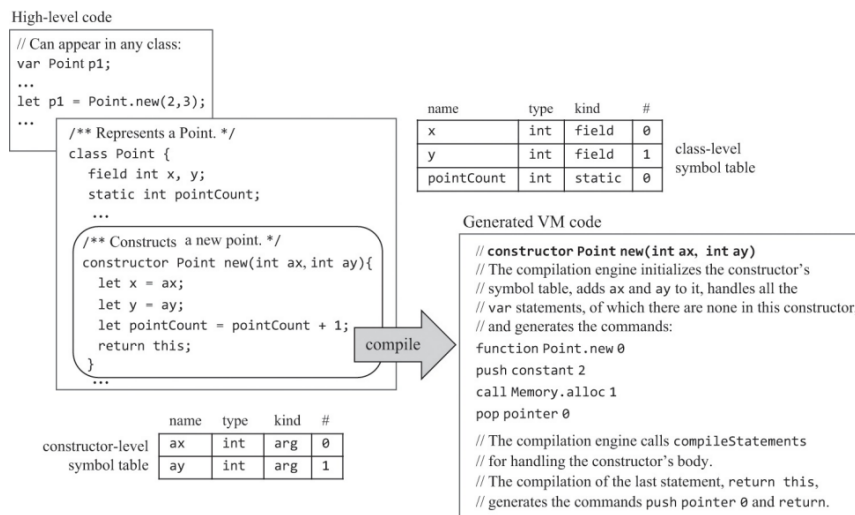
Compiling Constructor Calls: בניית אובייקט היא עניין של שני שלבים: הראשון הוא הצהרה של משתנה מטיפוס של מחלקה כלשהי, למשל $var\ Point\ p$, והשני הוא יצירת מופע של האובייקט ע"י קריאה לבנאי של המחלקה, למשל $let\ p = Point.new(2,3)$.

בניית אובייקט מנקודת המבט של ה-caller:

ראשית, נשים לב שקומפילציה של statements כמו `let p = Point.new(2,3)` ו-`let p = Point.new(5,7)` היא קומפילציה של `let statements` וקריאות ל-`subroutine`. הבנאי יוצר את שני האובייקטים שניתן לראות בציור של ה-RAM. שנית, נשים לב שהכתובות הפיזיות 6012 ו-5943 הן לא רלוונטיות – הקוד ה-high-level וקוד ה-VM המקומפל לא יודעים היכן האובייקטים מאוחסנים בזיכרון. הרפרנסים לאובייקטים הללו הם סימבוליים, דרך `p1` ו-`p2` בקוד ה-high-level, ודרך `local 0` ו-`local 1` בקוד המקומפל. שלישית, במהלך זמן הקומפילציה ה-symbol table מתעדכנת, נוצר קוד low-level, וזהו. האובייקטים ייבנו ויקושרו למשתנים רק במהלך זמן הריצה, וזה אם וכאשר הקוד המקומפל ירוץ.

Compiling Constructors: ראשית, נשים לב שבנאי הוא `subroutine`. הוא יכול שיהיו לו ארגומנטים, משתנים לוקאליים, וגוף של `statements`, ולכן הקומפיילר מתייחס אליו כאל `subroutine`. בנוסף לכך, הקומפיילר צריך לייצר גם קוד שיוצר אובייקט חדש והופך את האובייקט החדש להיות האובייקט הנוכחי (הידוע גם בתור `this`) – האובייקט שעליו הקוד של הבנאי פועל.

בניית אובייקט מנקודת המבט של הבנאי:



היצירה של אובייקט חדש דורשת למצוא בלוק RAM פנוי שגודל מספיק כדי לאחסן את המידע של האובייקט וכדי לסמן את הבלוק בתור "בשימוש". לפי ה-API של ה-OS, פונקציית ה-OS `Memory.alloc(size)` יודעת כיצד למצוא בלוק RAM זמין בגודל `size` (מספר שהוא מילה בגודל 16-ביט) ומחזירה את כתובת הבסיס של הבלוק.

הפונקציות `Memory.alloc` ו-`Memory.deAlloc` משתמשות באלגוריתמים חכמים כדי להקצות ולשחרר משאבי RAM ביעילות. קומפיילרים מייצרים קוד low-level שמשתמש ב-`alloc` וב-`deAlloc`.

לפני הקריאה ל-`Memory.alloc`, הקומפיילר קובע את גודל הבלוק הדרוש, שניתן לחשב אותו באמצעות ה-symbol table. לדוגמה, ה-symbol table של המחלקה `Point` מציינת שכל אובייקט `Point` מאופיין ע"י שני ערכי `int` (קואורדינטות ה-x וה-y של הנקודה). לכן, הקומפיילר מייצר את הפקודות `push constant 2` ו-`call Memory.alloc 1` שגורמת לקריאה לפונקציה `Memory.alloc(2)`. הפונקציה `alloc` מוצאת בלוק RAM זמין בגודל 2, ודוחפת את כתובת הבסיס שלו למחסנית (המקבילה ב-VM להחזרת ערך). הצהרת ה-VM הבאה היא `pop pointer 0` שקובעת את הערך של `THIS` להיות כתובת הבסיס ש-`alloc` החזירה. החל

מנקודה זו, מקטע ה-*this* של הבנאי מיישר קו עם בלוק ה-RAM שהוקצה עבור ייצוג האובייקט שנבנה.

כאשר המקטע *this* מיישר קו כמו שצריך, נוכל להמשיך לייצר קוד בקלות. לדוגמה, כאשר נקרא לרוטינה *compileLet* כדי לטפל ב-statement $let\ x = ax$, היא מחפשת ב-symbol tables וקובעת ש-*x* הוא *this 0* ו-*ax* הוא *argument 0*. לכן, *compileLet* מייצרת את הפקודות:

```
push argument 0
pop this 0
```

הפקודה האחרונה משתמשת בהנחה שמקטע ה-*this* מיישר קו כמו שצריך עם כתובת הבסיס של האובייקט החדש, וזה התבצע כאשר קבענו את *pointer 0* (למעשה *THIS*) לכתובת הבסיס שהוחזרה ע"י *alloc*. האתחול הזה מבטיח שכל פקודות ה-*push / pop this i* יגיעו ליעדים הנכונים ב-RAM (או יותר מדויק ב-heap).

לפי המפרט של שפת ה-Jack, כל בנאי חייב להסתיים עם ה-statement *return this*. הקונבנציה הזו מחייבת את הקומפיילר לסיים את הגרסה המקומפלת של הבנאי עם *push pointer 0* ו-*return*. הפקודות הללו דוחות למחסנית את הערך של *THIS*, כתובת הבסיס של האובייקט שנבנה.

ב-statements מהצורה $let\ varName = className.constructorName(...)$ לבסוף ב-*varName* תישמר כתובת הבסיס של האובייקט החדש.

Compiling Methods

Compiling Method Calls: נניח שאנחנו רוצים לחשב את המרחק האוקלידי בין שתי נקודות במישור $p1$ ו- $p2$. בתכנות object-oriented, נממש את $p1$ ואת $p2$ בתור מופעים של מחלקה *Point* כלשהי, וחישוב המרחק יתבצע ע"י קריאה למתודה כמו $p1.distance(p2)$. בניגוד לפונקציות, מתודות הן subroutines שתמיד פועלות על אובייקט נתון, וזאת האחריות של ה-caller לציין את האובייקט הזה.

נשים לב שנוכל לתאר את *distance* בתור פרוצדורה לחישוב המרחק מנקודה נתונה לאחרת, ונוכל לתאר את $p1$ בתור ה-data עליו הפרוצדורה פועלת.

כיוון שאין בשפת ה-VM את הקונספט של אובייקטים או מתודות, הקומפיילר מטפל בקריאות למתודה כמו $p1.distance(p2)$ כאילו הן קריאות פרוצדורליות כמו $distance(p1, p2)$. בפרט, הוא מתרגם את $p1.distance(p2)$ לפקודות הבאות:

```
push p1
push p2
call distance
```

באופן כללי, ב-Jack יש שני סוגים של קריאות למתודה:

// Applies a method to the object referred to by *varName*:

```
varName.methodName(exp1, exp2, ..., expn)
```

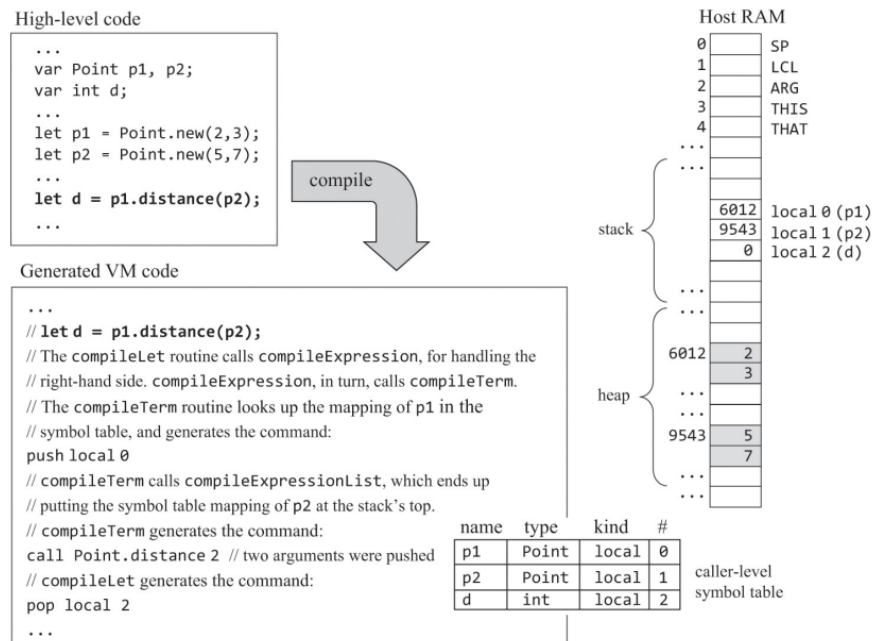
// Applies a method to the *current object*:

```
methodName(exp1, exp2, ..., expn) // Same as this.methodName(exp1, exp2, ..., expn)
```

כדי לקמפל את הקריאה למתודה $varName.methodName(exp_1, exp_2, \dots, exp_n)$, נתחיל בלייצר את הפקודה *push varName* כאשר *varName* הוא המיפוי של *varName* לפי ה-

symbol table. אם בקריאה למתודה אין *varName*, נדחוף את המיפוי של *this* לפי ה-symbol table. לאחר מכן, נקרא ל-*compileExpressionList*. הרטינה הזו מבצעת *n* קריאות ל-*compileExpression*, פעם אחת עבור כל expression בסוגריים. לבסוף, נייצר את הפקודה *call className.methodName n + 1*, המודיעה שנדחפו *n + 1* ארגומנטים למחסנית. המקרה של קריאה למתודה ללא ארגומנטים יתורגם ל: *call className.methodName 1*. נשים לב ש-*className* הוא ה-type של ה-*varName* identifier לפי ה-symbol table.

דוגמה לקומפילציה של קריאה למתודה מנקודת המבט של ה-caller:



Compiling Methods: נתבונן במימוש של המתודה distance ב-Java:

```

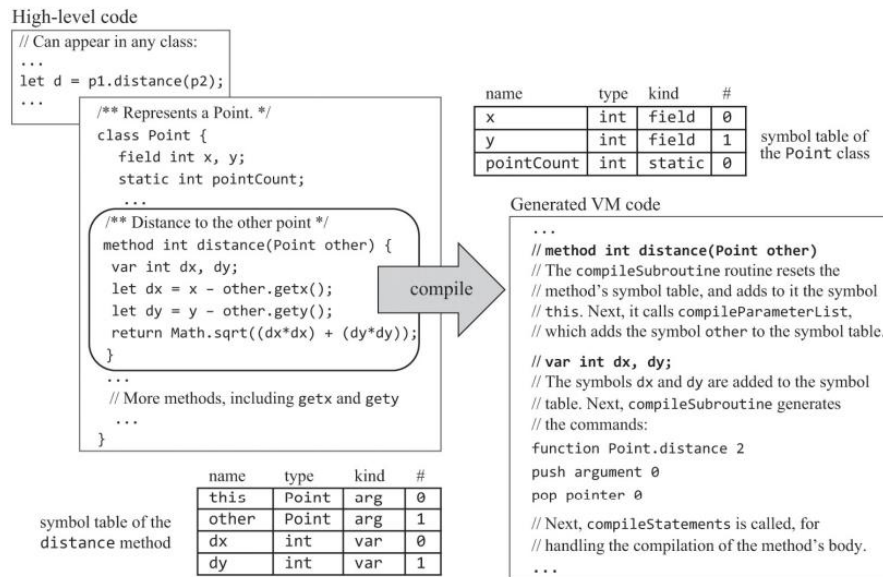
/** A Point class method: returns the distance between this Point and the other one. */
int distance(Point other) {
    int dx, dy;
    dx = x - other.x;
    dy = y - other.x;
    return Math.sqrt((dx*dx) + (dy*dy));
}

```

המטרה של *distance* היא לפעול על האובייקט הנוכחי שמיוצג ב-Java וב-Jack ע"י ה-identifier *this*. נשים לב ששפת ה-Jack לא תומכת בביטויים מהצורה *object.field*. לכן, כדי לפעול על שדות של אובייקטים שהם לא האובייקט הנוכחי נוכל להשתמש רק במתודות. לדוגמה, ביטויים כמו *x - other.x* ימומשו ב-Jack בתור *other.getX()*, כאשר *getX* היא מתודת accessor במחלקה *Point*.

כאשר הקומפיילר נתקל ב-expressions כמו *other.getX()*, הוא מחפש את *x* ב-symbol tables ומוצא שהוא מייצג את השדה הראשון באובייקט הנוכחי. האובייקט הנוכחי הוא בהכרח הארגומנט הראשון שהועבר ע"י ה-caller של המתודה. לכן, מנקודת המבט של ה-callee, האובייקט הנוכחי הוא בהכרח האובייקט שכתובת הבסיס שלו היא הערך של *argument 0*.

קומפילציה של מתודה מנקודת המבט של ה-callee:



הדוגמה מתחילה בקוד של ה-caller שמבצע את הקריאה למתודה $p1.distance(p2)$. אם נתבונן בגרסה המקומפלת של ה-callee נשים לב שהקוד מתחיל עם $push\ argument\ 0$ ואחרי זה $pop\ pointer\ 0$. הפקודות הללו קובעות את ערך המצביע $THIS$ של המתודה לערך של $argument\ 0$, שמכיל את כתובת הבסיס של האובייקט עליו המתודה נקראה ואמורה לפעול. לכן, החל מנקודה זו והלאה, מקטע ה- $this$ של המתודה מיישר קו עם כתובת הבסיס של אובייקט היעד, מה שגורם לכל פקודת $push / pop\ this\ i$ ליישר קו כמו שצריך גם כן. לדוגמה, ה-expression $other.getx()$ יקומפל לפקודות הבאות:

```
push this 0
push argument 1
call Point.getx 1
sub
```

כיוון שהתחלנו את המתודה המקומפלת ע"י קביעת $THIS$ לכתובת הבסיס של האובייקט הנקרא, מובטח כי $this\ 0$ (וכל רפרנס $this\ i$ אחר) יגיע לשדה הנכון של האובייקט הנכון.

Compiling Arrays

ב-Jack מערכים ממומשים בתור מופעים של המחלקה $Array$, שהיא חלק ממערכת ההפעלה. לכן, מערכים ואובייקטים מוצהרים, ממומשים, ונשמרים בדיוק באותה הדרך. למעשה, מערכים הם אובייקטים, עם ההבדל שהאבסטרקציה של המערך מאפשרת לגשת לאיברים של המערך באמצעות אינדקס, לדוגמה $let\ arr[3] = 17$.

ע"י שימוש בנוטציה של מצביע, נבחין כי אפשר לכתוב את $arr[i]$ בתור $(arr + i) *$ כלומר הכתובת $arr + i$ בזיכרון. כדי לחשב את הכתובת הפיזית של $arr[i]$, נבצע את הפקודות:

```
push arr
push i
add
```

שהתוצאה שלהן היא דחיפה של כתובת היעד למחסנית. לאחר מכן, נבצע את הפקודה:

```
pop pointer 1
```

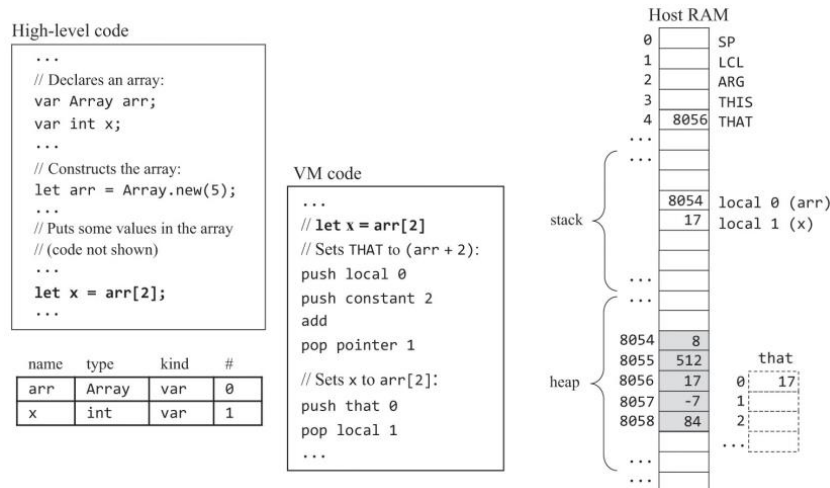
לפי מפרט ה-VM, הפעולה הזו מאחסנת את כתובת היעד במצביע $THAT[4]$ (RAM) של

המתודה, שגורמת לכך שכתובת הבסיס של המקטע הווירטואלי *that* תישר קו עם כתובת היעד. לכן, נוכל לבצע את הפקודות:

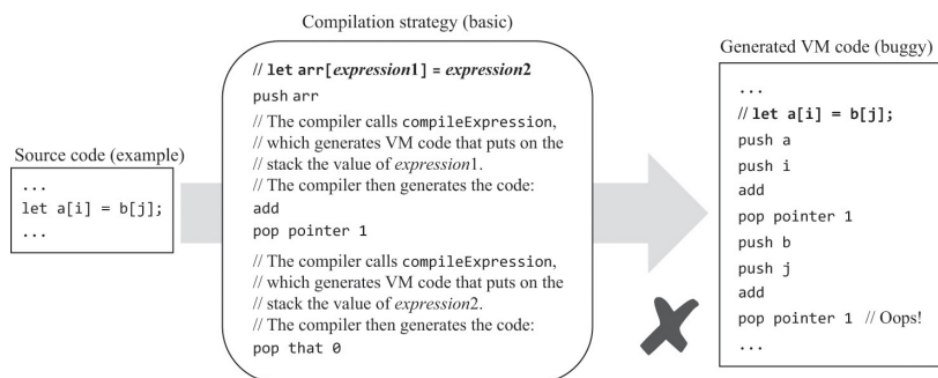
push that 0

pop x

שמשלימות את התרגום ה-low-level של $x = arr[i]$. נתבונן בדוגמה המפורטת:



אסטרטגיית הקומפילציה הזאת עובדת עבור statements כמו $let a = b[j]$, אבל נכשלת ב-statements בהן הצד השמאלי של ההשמה הוא עם אינדקס כמו $let a[i] = b[j]$. נתבונן בדוגמה:



נשים לב שהערך שנשמר ב-*pointer 1* נדרס והכתובת של $a[i]$ הולכת לאיבוד.

נרצה להיות מסוגלים לקמפל כל מופע של $let arr[expression1] = expression2$. נתחיל בלייצר את הפקודה *push arr* כפי שעשינו קודם, נקרא ל-*compileExpression*, ונייצר את הפקודה *add*. רצף הפקודות הללו שם את כתובת היעד ($arr + expression1$) בראש המחסנית. לאחר מכן, נקרא ל-*compileExpression* שתסתיים בכך שיהיה בראש המחסנית את הערך של $expression2$. בנקודה זו, נשמור את הערך הזה – נוכל לעשות זאת ע"י *pop temp 0*. הפעולה הזו הופכת את ($arr + expression1$) להיות האיבר בראש המחסנית. לכן, אנחנו יכולים עכשיו לבצע את הפקודות הבאות:

pop pointer 1

push temp 0

pop that 0

התיקון הזה, יחד עם האופי הרקורסיבי של הרוטינה *compileExpression*, גורם לאסטרטגיית

הקומפילציה הזו לטפל ב-statements מהצורה $let\ arr[expression1] = expression2$ מכל סיבוכיות רקורסיבית, כמו נניח $let\ a\left[b[i] + a\left[j + b[a[3]]\right]\right] = b[b[j] + 2]$.

יש כמה דברים שהופכים את הקומפילציה של מערכים ב-Jack לפשוטה יחסית. ראשית, למערכים ב-Jack אין טיפוס, המטרה שלהם היא לאחסן ערכים של 16-ביט בלי מגבלות. שנית, כל הטיפוסים הפרימיטיביים ב-Jack הם ברוחב 16-ביט, כל הכתובות הן ברוחב 16-ביט, וכך גם רוחב של מילה של ה-RAM.

Implementation

Standard Mapping Over the Virtual Machine

:Naming Files and Functions

- קובץ *Xxx.jack* של מחלקת Jack מקומפל לקובץ של מחלקת VM בשם *Xxx.vm*.
- Subroutine של Jack בשם *עץ* בקובץ *Xxx.jack* מקופלת לפונקציית VM בשם *Xxx.yyy*.

מיפוי משתנים:

- המשתנה הסטטי הראשון, השני, השלישי, ... שמוצהרים בהצהרה של מחלקה ממופים לרשומות *static 0, static 1, static 2, ...* במקטע הווירטואלי.
- משתנה השדה הראשון, השני, השלישי, ... המוצהרים בהצהרה של מחלקה ממופים ל-*this 0, this 1, this 2, ...*
- המשתנה הלוקאלי הראשון, השני, השלישי, ... שמוצהרים ב-statements var של subroutine ממופים ל-*local 0, local 1, local 2, ...*
- משתנה הארגומנט הראשון, השני, השלישי, ... המוצהרים ברשימת הפרמטרים של פונקציה או של בנאי (אבל לא מתודה) ממופים ל-*argument 0, argument 1, ... argument 2*
- משתנה הארגומנט הראשון, השני, השלישי, ... המוצהרים ברשימת הפרמטרים של מתודה ממופים ל-*argument 0, argument 1, argument 2, argument 3, ...*

מיפוי שדות של אובייקט (Object Fields): כדי שהמקטע הווירטואלי *this* יישר קו עם האובייקט

המועבר ע"י ה-caller של מתודה, נשתמש בפקודות ה-VM:

push argument 0

pop pointer 0

מיפוי איברים של מערך: הרפרנס $arr[expression]$ מקומפל ע"י קביעת *pointer 1* ל- $(arr + expression)$ וגישה ל-*that 0*.

מיפוי קבועים:

- פרנסים לקבועים *null* ו-*false* ב-Jack מקומפלים ל-*push constant 0*.
- פרנסים לקבוע *true* ב-Jack מקומפלים ל-*push constant 1* ואחריו *neg*. הרצף הזה דוחף את הערך 1- למחסנית.
- פרנסים לקבוע *this* ב-Jack מקומפל ל-*push pointer 0*. הפקודה הזו דוחפת את כתובת הבסיס של האובייקט הנוכחי למחסנית.

Implementation Guidelines

Handling Identifiers: אפשר לטפל ב-*identifiers* המשמשים עבור שמות של משתנים ע"י *symbol tables*. במהלך הקומפילציה של קוד Jack חוקי, נוכל להניח שכל *identifier* שלא נמצא ב-*symbol tables* הוא או שם של subroutine או שם של מחלקה. אין צורך לשמור *identifiers* כאלה ב-*symbol table*.

Compiling Expressions: הרוטינה *compileExpression* צריכה לעבד את הקלט בתור הרצף *term op term op term ...*. כדי לעשות זאת, היא צריכה לממש את אלגוריתם ה-*codeWrite* שראינו, ולהרחיב אותו כדי שהוא יטפל בכל ה-*terms* האפשריים המופיעים בדקדוק של ה-Jack. נשים לב שרוב הפעולה בקומפילציה של *expressions* מתרחשת בקומפילציה של ה-*terms* המרכיבים אותן.

הדקדוק של *expression* והרוטינה *compileExpression* המתאימה הם רקורסיביים. לדוגמה, כאשר *compileExpression* מזהה סוגר שמאלי, היא צריכה לבצע קריאה רקורסיבית ל-*compileExpression* כדי לטפל ב-*expression* הפנימי. זה מבטיח שהערך של ה-*expression* הפנימי יחושב קודם. מלבד הקדימות הזו, שפת ה-Jack לא תומכת בסדר פעולות של אופרטורים.

הביטוי $x * y$ יתקמפל לפקודות:

```
push x
push y
call Math.multiply 2
```

הביטוי x/y יתקמפל לפקודות:

```
push x
push y
call Math.divide 2
```

המחלקה *Math* היא חלק ממערכת ההפעלה שנפתח בפרק הבא.

Compiling Strings: נטפל בכל *string constant* מהצורה "*ccc ... c*" באופן הבא:

1. נדחוף את אורך המחרוזת למחסנית ונקרא לבנאי *String.new*.
2. נדחוף את ה-*character code* של *c* למחסנית ונקרא למתודה *appendChar* של המחלקה *String*, פעם אחת עבור כל תו *c* במחרוזת.

הבנאי *new* והמתודה *appendChar* מחזירים את המחרוזת כערך ההחזרה שלהם (כלומר הם דוחפים את אובייקט המחרוזת למחסנית).

Compiling Function Calls and Constructor Calls: הגרסה המקומפלת של קריאה לפונקציה

או קריאה לבנאי שיש להם n ארגומנטים חייבת:

1. לקרוא ל-*compileExpressionList*, שתקרא n פעמים ל-*compileExpression*.
2. לגרום לקריאה להודיע שנדחפו n ארגומנטים למחסנית לפני הקריאה.

Compiling Method Calls: הגרסה המקומפלת של קריאה למתודה שיש לה n ארגומנטים

חייבת:

1. לדחוף למחסנית רפרנס לאובייקט שעליו המתודה נקראה לפעול.
2. לקרוא ל-*compileExpressionList*, שתקרא n פעמים ל-*compileExpression*.
3. לגרום לקריאה להודיע שנדחפו $n + 1$ ארגומנטים למחסנית לפני הקריאה.

Compiling Do Statements: ממליצים לקמפל statements מהצורה *do sunroutineCall* כאילו הן statements מהצורה *do expression*, ואז לשלוף את הערך שנמצא בראש המחסנית ע"י הפקודה *pop temp 0*.

Compiling Classes: כשמתחילים לקמפל מחלקה, הקומפיילר יוצר class-level symbol table ומוסיף אליה את כל משתני השדה והמשתנים הסטטיים שמצהירים עליהם בהצהרה על המחלקה. הקומפיילר גם יוצר subroutine-level symbol table ריקה. לא מיוצר כאן קוד.

Compiling Subroutines

- כאשר מתחילים לקמפל subroutine (בנאי, פונקציה, או מתודה), הקומפיילר מאתחל את ה-symbol table של ה-subroutine. אם ה-subroutine היא מתודה, הקומפיילר מוסיף ל-symbol table את המיפוי $\langle this, className, arg, 0 \rangle$.
- לאחר מכן, הקומפיילר מוסיף ל-symbol table את כל הפרמטרים (אם קיימים) שהוצהרו ברשימת הפרמטרים של ה-subroutine. אחרי זה, הקומפיילר מטפל בכל ההצהרות על var (אם קיימות) ע"י הוספת כל המשתנים הלוקאליים של ה-subroutine ל-symbol table.
- בשלב זה, הקומפיילר מתחיל לייצר קוד, ומתחיל עם הפקודה:
function className.sunroutineName nVars
כאשר $nVars$ זה מספר המשתנים הלוקאליים ב-subroutine.
- אם ה-subroutine היא מתודה, הקומפיילר מייצר את הקוד הבא:
push argument 0
pop pointer 0
הרצף הזה גורם ליישור קו של מקטע הזיכרון הווירטואלי *this* עם כתובת הבסיס של האובייקט שעליו המתודה נקראה.

:Compiling Constructors

- ראשית, הקומפיילר מבצע את כל הפעולות המתוארות בחלק הקודם המסתיים בייצור הפקודה `function className.constructorName nVars`.
- לאחר מכן, הקומפיילר מייצר את הקוד הבא:

```
push constant nFields  
call Memory.alloc 1  
pop pointer 0
```

כאשר `nFields` זה מספר השדות במחלקה המקומפלת. התוצאה של זה היא הקצאת בלוק זיכרון של `nFields` מילים של 16-ביט, ויישור קו של מקטע הזיכרון הווירטואלי `this` עם כתובת הבסיס של הבלוק החדש שהוקצה.
- הבנאי המקומפל חייב להסתיים עם הפקודות:

```
push pointer 0  
return
```

הרצף הזה מחזיר ל-caller את כתובת הבסיס של האובייקט החדש שנבנה ע"י הבנאי.

:Compiling Void Methods and Void Functions

מיופנה מכל פונקציית VM לדחוף ערך למחסנית לפני שהיא מחזירה. כאשר מקמפלים מתודה או פונקציה שהיא `void` ב-Jack, הקונבנציה היא לסיים את הקוד שמייצרים ב-

```
push constant 0  
return
```

:Compiling Arrays statements מהצורה `let arr[expression1] = expression2` מקומפלות באמצעות הטכניקה שתיארנו קודם לכן. טיפ למימוש: כאשר מטפלים במערכים, לעולם אין צורך להשתמש ברשומות של `that` שהאינדקס שלהם גדול מ-0.

מערכת ההפעלה: נתבונן ב-expression הבא: `Math.sqrt((dx * dx) + (dy * dy))` הקומפיילר מקמפל אותו לפקודות ה-VM הבאות:

```
push dx  
push dx  
call Math.multiply 2  
push dy  
push dy  
call Math.multiply 2  
add  
call Math.sqrt 1
```

כאשר `dx` ו-`dy` הם המיפויים ב-symbol table של `dx` ושל `dy`. הדוגמה הזו מראה את שתי הדרכים בהן מערכת ההפעלה באה לידי ביטוי במהלך הקומפילציה. ראשית, אבסטרקציות מסוימות שהן high-level כמו `x * y` מקומפלות ע"י ייצור קוד שקורא ל-subroutines של מערכת ההפעלה כמו `Math.multiply`. שנית, כאשר expression ב-Jack כולל קריאה high-level לרוטינה של מערכת ההפעלה, לדוגמה `Math.sqrt(x)`, הקומפיילר מייצר קוד VM שעושה בדיוק את אותה הקריאה ע"י שימוש ב-VM postfix syntax.

Software Architecture

ארכיטקטורת הקומפיילר המוצעת נבנית מה-syntax analyzer שתיארנו בפרק 10. ההצעה היא להשתמש במודולים הבאים:

- JackCompiler: תוכנית ראשית, יוצרת את כל שאר המודולים וקוראת להם.
- JackTokenizer: ה-tokenizer עבור שפת ה-Jack.
- SymbolTable: עוקבת אחרי כל המשתנים שנמצאים בקוד ה-Jack.
- VMWriter: כותב קוד VM.
- CompilationEngine: מנוע קומפילציה רקורסיבי.

The JackCompiler: המודול הזה מניע את תהליך הקומפילציה. הוא פועל על שם קובץ מהצורה *Xxx.jack* או על שם של תיקייה שמכילה קובץ אחד כזה או יותר. עבור כל קובץ מקור *Xxx.jack* התוכנית:

1. יוצרת JackTokenizer מקובץ הקלט *Xxx.jack*.
 2. יוצרת קובץ פלט בשם *Xxx.vm*.
 3. משתמשת ב-CompilationEngine, ב-SymbolTable, וב-VMWriter כדי לפרסר את קובץ הקלט ולכתוב את קוד ה-VM המתורגם לתוך קובץ הפלט.
- ניזכר שהרוטינה הראשונה שצריך לקרוא לה כאשר מקמפלים קובץ *jack*. היא *compileClass*.

The JackTokenizer: המודול הזה זהה ל-tokenizer שנבנה בפרויקט 10.

The SymbolTable: המודול הזה מספק שירותים עבור בניית symbol tables, מילוי שלהן, ושימוש בהן כדי לעקוב אחרי התכונות של symbol: שם, טיפוס, סוג, ואינדקס רץ עבור על סוג.

Routine	ארגומנטים	ערך החזרה	הפונקציה
Constructor	-	-	יוצר symbol table חדשה
reset	-	-	מרוקן את ה-symbol table ומאפס את כל 4 האינדקסים ל-0. צריך לקרוא לה כאשר מתחילים לקמפל הצהרה על subroutine
define	name (string), type (string), kind (STATIC, FIELD, ARG, or VAR)	-	מגדירה (מוסיפה לטבלה) משתנה חדש עם השם, הטיפוס והסוג הנתונים. מקצה לו את הערך של האינדקס של ה-kind הנתון, ומוסיפה 1 לאינדקס
varCount	kind (STATIC, FIELD, ARG, or VAR)	int	מחזירה את מספר המשתנים מאותו ה-kind הנתון שמוגדרים בטבלה
kindOf	name (string)	(STATIC, FIELD, ARG, VAR, NONE)	מחזירה את ה-kind של ה-identifier. אם ה-identifier לא נמצא, מחזירה NONE

מחזירה את ה-type של המשתנה הנתון	string	name (string)	<i>typeOf</i>
מחזירה את האינדקס של המשתנה הנתון	int	name (string)	<i>indexOf</i>

* הערה לגבי המימוש: במהלך הקומפילציה של קובץ מחלקה של Jack, הקומפיילר של ה-Jack משתמש בשני מופעים של SymbolTable.

The VMWriter: המודול הזה מכיל רוטינות פשוטות לכתיבת פקודות VM לקובץ הפלט.

הפונקציה	ארגומנטים	Routine
יוצר קובץ vm. עבור הפלט ומכין אותו לכתיבה	קובץ פלט	<i>Constructor</i>
כותבת את פקודת ה-push ב-VM	segment (CONSTANT, ARGUMENT, LOCAL, STATIC, THIS, THAT, POINTER, TEMP), index (int)	<i>writePush</i>
כותבת את פקודת ה-pop ב-VM	segment (CONSTANT, ARGUMENT, LOCAL, STATIC, THIS, THAT, POINTER, TEMP), index (int)	<i>writePop</i>
כותבת פקודה אריתמטית-לוגית ב-VM	command (ADD, SUB, NEG, EQ, GT, LT, AND, OR, NOT)	<i>writeArithmetic</i>
כותבת את פקודת ה-label ב-VM	label (string)	<i>writeLabel</i>
כותבת את פקודת ה-goto ב-VM	label (string)	<i>writeGoto</i>
כותבת את פקודת ה-if-goto ב-VM	label (string)	<i>writeIf</i>
כותבת את פקודת ה-call ב-VM	name (string), nArgs (int)	<i>writeCall</i>
כותבת את פקודת ה-function ב-VM	name (string), nVars (int)	<i>writeFunction</i>
כותבת את פקודת ה-return ב-VM	-	<i>writeReturn</i>

The CompilationEngine: המודול הזה מנהל את תהליך הקומפילציה. למרות שה-API שלו זהה ל-API שהוצג בפרויקט 10, נחזור עליו.

ה-CompilationEngine מקבל את הקלט שלו מ-JackTokenizer ומשתמש ב-VMWriter כדי לכתוב את קוד ה-VM של הפלט. הפלט מיוצר ע"י סדרה של רוטינות *compileXXX*, שהמטרה של כל אחת מהן היא לטפל בקומפילציה של מבנה ספציפי XXX בשפת ה-Jack (לדוגמה, *compileWhile* מייצרת קוד VM שמממש while statements). כל רוטינת *compileXXX* מקבלת מהקלט ומטפלת בכל ה-tokens שיוצרים את XXX, מקדמת את ה-tokenizer בדיוק כמספר ה-tokens הללו, וכותבת לקובץ הפלט את קוד ה-VM שמתאים לסמנטיקה של XXX. אם

XXX היא חלק מ-expression, ולכן יש לה ערך, קוד ה-VM של הפלט צריך לחשב את הערך הזה ולהשאיר אותו בראש המחסנית. כל רוטינת *compileXXX* נקראת רק אם ה-token הנוכחי הוא XXX. כיוון שה-token הראשון בקובץ jack. חוקי הוא ה-keyword *class*, תהליך הקומפילציה מתחיל בקריאה לרוטינה *compileClass*.

הפונקציה	Routine
מקבלת כארגומנטים קובץ קלט וקובץ פלט, ויוצרת compilation engine חדש עם הקלט והפלט הנתונים. הרוטינה הבאה שתיקרא חייבת להיות <i>compileClass</i>	<i>Constructor</i>
מקמפלת מחלקה שלמה	<i>compileClass</i>
מקמפלת הצהרה על משתנה סטטי, או הצהרה על שדה	<i>compileClassVarDec</i>
מקמפלת מתודה שלמה, פונקציה שלמה, או בנאי שלם	<i>compileSubroutine</i>
מקמפלת רשימת פרמטרים (יכולה להיות ריקה), ולא מטפלת בסוגריים העוטפים, כלומר ב-tokens (ו-)	<i>compileParameterList</i>
מקמפלת את גוף ה-subroutine	<i>compileSubroutineBody</i>
מקמפלת הצהרות על <i>var</i>	<i>compileVarDec</i>
מקמפלת רצף של statements. לא מטפלת בסוגריים המסולסלים העוטפים, כלומר ב-tokens { }-.	<i>compileStatements</i>
מקמפלת let statement	<i>compileLet</i>
מקמפלת if statement, עם בלוק else אופציונלי אחרי	<i>compileIf</i>
מקמפלת while statement	<i>compileWhile</i>
מקמפלת do statement	<i>compileDo</i>
מקמפלת return statement	<i>compileReturn</i>
מקמפלת expression	<i>compileExpression</i>
מקמפלת term. אם ה-token הנוכחי הוא identifier, הרוטינה חייבת להחליט אם הוא משתנה, איבר במערך או קריאה ל-subroutine. מספיק להתבונן ב-token הבא, שיכול להיות [, (, או . כדי להבדיל בין האפשרויות. כל token אחר אינו חלק מה-term הזה ואי אפשר לדלג עליו	<i>compileTerm</i>
מקמפלת רשימה (ייתכן ריקה) של expressions המופרדים בפסיק. מחזירה int שהוא מספר ה-expressions ברשימה	<i>compileExpressionList</i>

* הערה: לכללי הדקדוק הבאים ב-Jack אין רוטינת *compileXXX* מתאימה ב-
CompilationEngine :type, className, subroutineName, varName, subroutineCall, statement

מערכת ההפעלה

המטרה של מערכת ההפעלה היא לסגור את הפערים שבין החומרה והתוכנה של המחשב, מה שגורם למערכת המחשב להיות יותר נגישה למתכנתים, לקומפילרים, ולמשתמשים.

מערכת ההפעלה שלנו היא מינימלית, והמטרה שלה היא לעטוף שירותי חומרה שהם low-level בשירותי תוכנה high-level הידידותיים למשתמש, ולהרחיב שפות שהן high-level עם פונקציות נפוצות ו-data types אבסטרקטיים.

מערכת ההפעלה של ה-Jack היא אוסף של מחלקות תומכות, כאשר כל מחלקה מספקת קבוצה של שירותים הקשורים אליה דרך קריאות ל-subroutines של Jack.

הקוד של מערכת ההפעלה חייב לפעול קרוב לחומרה, לפעול על הזיכרון, קלט/פלט, ו-processing devices באופן כמעט ישיר. כיוון שמערכת ההפעלה תומכת בהרצה של כמעט כל תוכנית שרצה על המחשב, היא חייבת להיות יעילה מאוד.

נתחיל בהצגה של אלגוריתמי מפתח שלרוב נמצאים בשימוש במימושים של מערכת ההפעלה. זה כולל פעולות מתמטיות, פעולות על מחרוזות, ניהול זיכרון, פלט טקסטואלי וגרפי, וקלט מהמקלדת.

פעולות מתמטיות

4 הפעולות המתמטיות חיבור, חיסור, כפל וחילוק נמצאות בליבה של כמעט כל תוכנית מחשב. אם לולאה שרצה מיליון פעמים מכילה expressions שמשתמשים באחת מהפעולות הללו, הן חייבות להיות ממומשות באופן יעיל.

חיבור ממומש בחומרה, ברמת ה-ALU, וניתן להשיג חיסור "בחינם" משיטת המשלים ל-2. נציג כעת אלגוריתמים יעילים לחישוב כפל, חילוק, ושורש ריבועי.

היעילות במקום הראשון: אלגוריתמים מתמטיים פועלים על ערכים של n ביטים, כאשר n בדרך כלל מייצג 16/32/64 ביטים. ככלל, נחפש אלגוריתמים שזמן הריצה שלהם הוא פונקציה פולינומיאלית בגודל המילה n . אלגוריתמים שזמן הריצה שלהם תלוי בערכים של המספרים הללו הם לא מקובלים, כיוון שהערכים הללו הם אקספוננציאליים ב- n . לדוגמה, נניח שאנחנו רוצים לממש את פעולת הכפל $x \times y$ באופן נאיבי, ע"י שימוש באלגוריתם של חיבור שחוזר על עצמו: $for\ i = 1 \dots y\ \{sum = sum + x\}$. אם y הוא ברוחב של 64 ביטים, הערך שלו יכול להיות גדול מ-9,000,000,000,000,000,000.

זמן הריצה של האלגוריתמים שנציג עבור כפל, חילוק ושורש ריבועי לא יהיה תלוי בערכים ש- n הביטים מייצגים, שיכולים להיות בגודל של 2^n , אלא ב- n מספר הביטים שלהם. נשתמש בנוסחה $O(n)$ כדי לתאר זמן ריצה שהוא בסדר הגודל של n . זמן הריצה של כל האלגוריתמים האריתמטיים שנציג הוא $O(n)$, כאשר n הוא הרוחב בביטים של הקלט.

כפל – Multiplication: נתבונן במתודת הכפל הסטנדרטית שלמנו בבית הספר היסודי. כדי לחשב 356 פעמים 73, אנחנו מציבים את שני המספרים אחד מעל השני (עם הצמדה לצד הימני). לאחר מכן, נכפול את 356 ב-3. אחרי זה, נזיז את 356 מקום אחד שמאלה, ונכפול את 3560 ב-7 (שזה

אותו הדבר כמו לכפול את 356 ב-73). לבסוף, נסכום את העמודות ונקבל את התוצאה. הפרוצדורה הזו מתבססת על התובנה ש- $356 \times 73 = 356 \times 70 + 356 \times 3$.

אלגוריתם הכפל בגרסה הבינארית:

$$\begin{array}{r}
 x = 27 = \dots 000011011 \\
 y = 9 = \dots 000001001 \quad i\text{-th bit of } y \\
 \hline
 \dots 000011011 \quad 1 \\
 \dots 000110110 \quad 0 \\
 \dots 001101100 \quad 0 \\
 \dots 011011000 \quad 1 \\
 \hline
 x * y = 243 = \dots 011110011 \quad \text{sum}
 \end{array}$$

```

// Returns  $x * y$ , where  $x, y \geq 0$ .
multiply(x, y):
    sum = 0
    shiftedx = x
    for i = 0 ... n - 1 do
        if ((i-th bit of y) == 1)
            sum = sum + shiftedx
            shiftedx = 2 * shiftedx
    return sum

```

עבור הביט ה- i של y , לכל i , נעשה i פעמים shift-left ל- x (אותו הדבר כמו לכפול את x ב- 2^i). לאחר מכן, נתבונן בביט ה- i של y : אם הוא 1, נוסיף את ה- $shifted\ x$ לסכום. אחרת, לא נעשה כלום. נשים לב שניתן לחשב את $2 * shiftedx$ באופן יעיל ע"י left-shifting של הייצוג ה-bitwise של $shiftedx$ או ע"י הוספת $shiftedx$ לעצמו.

זמן ריצה: אלגוריתם הכפל מבצע n איטרציות, כאשר n הוא הרוחב בביטים של הקלט y . בכל איטרציה, האלגוריתם מבצע מספר פעולות חיסור והשוואה. נובע כי זמן הריצה הכולל של האלגוריתם הוא $a + b \cdot n$, כאשר a הוא הזמן שלוקח לאתחל מספר משתנים ו- b הוא הזמן שלוקח לבצע מספר פעולות חיסור והשוואה. פורמלית, זמן הריצה של האלגוריתם הוא $O(n)$.

חילוק – Division: הדרך הנאיבית לחשב את החלוקה של שני מספרים בגודל n ביטים, x/y , היא לספור כמה פעמים אפשר לחסר את y מ- x עד שהשארית נהיית קטנה מ- y . זמן הריצה של האלגוריתם הזה הוא פרופורציונלי לערך של x ולכן אקספוננציאלי במספר הביטים n .

נוכל לנסות לחסר צ'אנקים גדולים של y מ- x בכל איטרציה. לדוגמה, נניח שאנחנו צריך לחלק את 175 ב-3. נתחיל בשאלה: מה המספר הגדול ביותר $(10, 20, \dots, 80, 90) = x$ כך ש- $3 \cdot x \leq 175$? התשובה היא 50. במילים אחרות, הצלחנו לחסר 50 3-ים מ-175, ולהוריד 50 איטרציות מהגישה הנאיבית. זה מותיר אותנו עם השארית $175 - 3 \cdot 50 = 25$. נוכל לשאול: מהו המספר הגדול ביותר $(1, 2, \dots, 7, 8, 9) = x$ כך ש- $3 \cdot x \leq 25$? התשובה היא 8. בינתיים התשובה היא $50 + 8 = 58$. השארית היא $25 - 3 \cdot 8 = 1$, כלומר קטנה מ-3, אז נעצור את התהליך ונודיע ש- $175/3 = 58$ עם שארית של 1.

הטכניקה הזו היא הרציונל שמאחורי שיטת החילוק הארוך שלמדנו בבית הספר. הגרסה הבינארית היא זהה, פרט לכך שבמקום להשתמש בחזקות של 10, נשתמש בחזקות של 2. האלגוריתם מבצע n איטרציות, כאשר n הוא מספר הספרות ב-dividend, ובכל איטרציה יש מספר פעולות כפל (למעשה shifting), השוואה וחיסור. שוב, יש לנו אלגוריתם שמחשב את x/y שזמן הריצה שלו לא תלוי בערכים של x ושל y . זמן הריצה הוא $O(n)$, כאשר n הוא רוחב הביטים של הקלטים.

אלגוריתם חילוק יותר אלגנטי וקל למימוש (אותה היעילות):

```
// Returns the integer division  $x / y$ ,  
// where  $x \geq 0$  and  $y > 0$ .  
divide( $x, y$ ):  
    if ( $y > x$ ) return 0  
     $q = \text{divide}(x, 2 * y)$   
    if ( $(x - 2 * q * y) < y$ )  
        return  $2 * q$   
    else  
        return  $2 * q + 1$ 
```

נניח שאנחנו רוצים לחלק את 480 ב-17. האלגוריתם הנ"ל מבוסס על התובנה הבאה:

$$\frac{480}{17} = 2 \cdot \left(\frac{240}{17}\right) = 2 \cdot \left(2 \cdot \left(\frac{120}{17}\right)\right) = 2 \cdot \left(2 \cdot \left(2 \cdot \left(\frac{60}{17}\right)\right)\right) = \dots$$

העומק של הרקורסיה הזו חסום ע"י מספר הפעמים שאפשר לכפול את y ב-2 לפני שמגיעים ל- x . זה גם לכל היותר מספר הביטים הנדרשים כדי לייצג את x . לכן זמן הריצה של האלגוריתם הוא $O(n)$, כאשר n הוא רוחב הביטים של הקלטים.

סכנה אחת באלגוריתם היא שפעולת הכפל גם דורשת $O(n)$ פעולות. אבל אם נבחן את הלוגיקה של האלגוריתם נגלה שניתן לחשב את הערך של הביטוי $(2 * q * y)$ בלי כפל – ע"י הערך שלו ברמה הקודמת של הרקורסיה תוך שימוש בחיבור.

שורש ריבועי – Square Root: לפונקציית השורש $y = \sqrt{x}$ יש שתי תכונות אטרקטיביות: היא פונקציה מונוטונית עולה, והפונקציה ההופכית שלה היא $y = x^2$ שאותה אנחנו יודעים לחשב באופן יעיל. זה כל מה שאנחנו צריכים כדי לחשב שורש ריבועי באופן יעיל, באמצעות שימוש בסוג של חיפוש בינארי.

אלגוריתם לחישוב שורש ריבועי:

```
// Computes the integer part of  $y = \sqrt{x}$   
// Strategy: finds an integer  $y$  such that  $y^2 \leq x < (y+1)^2$  (for  $0 \leq x < 2^n$ )  
// by performing binary search in the range  $0 \dots 2^{n/2} - 1$   
sqrt( $x$ ):  
     $y = 0$   
    for  $j = (n/2 - 1) \dots 0$  do  
        if  $(y + 2^j)^2 \leq x$  then  $y = y + 2^j$   
    return  $y$ 
```

כיוון שמספר האיטרציות בחיפוש הבינארי שהאלגוריתם מבצע חסום ע"י $\frac{n}{2}$ כאשר n הוא מספר הביטים ב- x , זמן הריצה של האלגוריתם הוא $O(n)$.

מחרוזות – Strings

בנוסף ל-data types הפרימיטיביים, רוב שפות התכנות מציעות טיפוס string המיועד לייצוג רצף של תווים. כל ה-string constants המופיעים בתוכניות Jack ממומשים בתור אובייקטים של *String*. המחלקה *String* כוללת מגוון מתודות לעיבוד מחרוזות, כמו הוספת תו למחרוזת, מחיקת התו האחרון, וכך הלאה. המתודות המאתגרות במחלקה *String* הן אלו שממירות ערכים מספריים למחרוזות, ומחרוזות של תווים ספרתיים לערכים מספריים.

String Representation of Numbers

כאשר בני אדם צריכים לקרוא מספרי קלט, ורק במקרה הזה, צריכה להתבצע המרה לסימון הדצימלי. כאשר מספרים כאלו מגיעים מהתקן קלט כמו מקלדת, או מרונדרים להתקן פלט כמו מסך, הם מסווגים בתור מחרוזות של תווים, כאשר כל תו מייצג אחת מהספרות בין 0 ל-9. תת-הקבוצה של התווים הרלוונטיים היא:

Character:	'0'	'1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'
Character code:	48	49	50	51	52	53	54	55	56	57

ניתן לראות שניתן להמיר בקלות תווים ספרתיים ל-integers שהם מייצגים, ולהפך. הערך המספרי של התו c , כאשר $48 \leq c \leq 57$, הוא $c - 48$. קוד התו של המספר x , כאשר $0 \leq x \leq 9$, הוא $x + 48$.

ברגע שאנחנו יודעים כיצד לטפל בתווים של ספרה אחת, אנחנו יכולים לפתח אלגוריתמים להמרה של כל מספר למחרוזת ולהמרה של כל מחרוזת של תווים ספרתיים למספר המתאים. אלגוריתמי ההמרה הללו יכולים להיות איטרטיביים או רקורסיביים.

String-Integer להמרת

int to string:

```
// Returns the string representation of
// a nonnegative integer.
int2String(val):
    lastDigit = val % 10
    c = character representing lastDigit
    if (val < 10)
        return c (as a string)
    else
        return int2String(val / 10).appendChar(c)
```

string to int:

```
// Returns the integer value of a string
// of digit characters, assuming that str[0]
// represents the most significant digit.
string2Int(str):
    val = 0
    for (i = 0 ... str.length()) do
        d = integer value of str.charAt(i)
        val = val * 10 + d
    return val
```

קל לראות שזמן הריצה של כל אחד מהאלגוריתמים הללו הוא $O(n)$, כאשר n הוא מספר התווים הספרתיים בקלט.

Memory Management

בכל פעם שתוכנית יוצרת מערך חדש או אובייקט חדש, חייבים להקצות בלוק זיכרון מגודל מסוים כדי לייצג את המערך או את האובייקט החדש. וכאשר אין יותר צורך במערך או באובייקט, צריך למחזר את המקום שלו ב-RAM.

הבלוקים בזיכרון המייצגים מערכים ואובייקטים נלקחים וממוחזרים מתוך אזור ייעודי ב-RAM שנקרא *heap*. כאשר ה-OS מתחילה לרוץ, היא מאחלת מצביע הנקרא *heapBase*, המכיל את כתובת הבסיס של ה-*heap* ב-RAM (ב-Jack ה-*heap* מתחילה בדיוק אחרי שהמחסנית מסתיימת, כלומר $heapBase = 2048$).

Memory Allocation Algorithm (Basic): מבנה הנתונים שהאלגוריתם הזה מנהל הוא מצביע יחיד, שנקרא *free*, שמצביע להתחלה של המקטע ב-*heap* שעדיין לא הוקצה.

האלגוריתם:

```
init():
    free = heapBase

// Allocates a memory block of size words.
alloc(size):
    block = free
    free = free + size
    return block

// Frees the memory space of the given object.
deAlloc(object):
    do nothing
```

זה ניהול *heap* בזבזני, שכן הוא מעולם לא דורש בחזרה מקום בזיכרון. אבל, אם התוכניות שלנו משתמשות רק בכמה אובייקטים ומערכים קטנים, ולא ביותר מדי מחרוזות, אפשר להסתדר עם זה.

Memory Allocation Algorithm (Improved): האלגוריתם הזה מנהל רשימה מקושרת של מקטעי זיכרון זמינים, הנקראת *freeList*. כל מקטע ברשימה מתחיל עם שני שדות של *housekeeping*: האורך של המקטע, ומצביע למקטע הבא ברשימה.

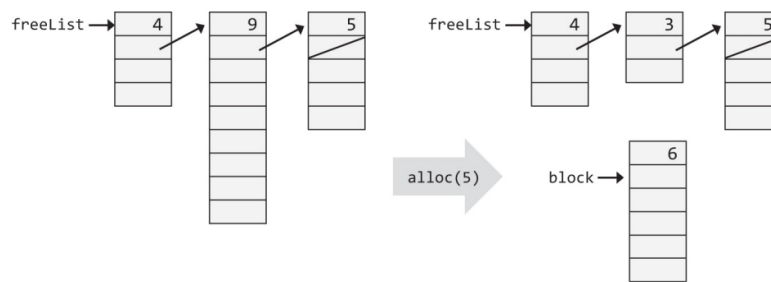
האלגוריתם:

```
init():
    freeList = heapBase
    freeList.size = heapSize
    freeList.next = 0

// Allocates a memory block of size words.
alloc(size):
    search freeList using best-fit or first-fit heuristics
        to obtain a segment with segment.size ≥ size + 2
    if no such segment is found, return failure
        (or attempt defragmentation)
    block = base address of the found space
    update the freeList and the fields of block
        to account for the allocation
    return block

// Frees the memory space of the given object.
deAlloc(object):
    append object to the end of the freeList
```

המחשה:



כאשר אנחנו מתבקשים להקצות בלוק זיכרון מגודל נתון, האלגוריתם צריך לחפש ב-*freeList* מקטע מתאים. קיימות שתי היוריסטיקות לחיפוש הזה: *best – fit* מוצאת את המקטע הקצר ביותר שהוא מספיק ארוך כדי לייצג את הגודל הדרוש, בעוד ש-*first – fit* מוצאת את המקטע הראשון שהוא ארוך מספיק. כאשר נמצא מקטע מתאים, בלוק הזיכרון הנדרש נלקח ממנו (במקום לפני תחילת הבלוק שמוחזר, $block[-1]$, שמור כדי להחזיק את האורך שלו, שיהיה בשימוש במהלך ה-*deallocation*).

לאחר מכן, נעדכן את האורך של המקטע הזה ב-*freeList*, כך שהוא ישקף את האורך של החלק שנשאר ממנו אחרי ההקצאה. אם לא נשאר זיכרון במקטע הזה, או אם החלק שנותר הוא קטן מדי, כל המקטע נמחק מה-*freeList*.

כאשר אנחנו מתבקשים להחזיר בלוק זיכרון של אובייקט שאינו בשימוש, האלגוריתם מוסיף את הבלוק ש-*deallocated* לסוף של ה-*freeList*.

Peek and Poke: שתי פונקציות OS פשוטות שלא קשורות להקצאת משאבים. הפונקציה *Memory.peek(addr)* מחזירה את הערך שנמצא בכתובת *addr* ב-RAM, והפונקציה *Memory.poke(addr, value)* קובעת את המילה בכתובת *addr* ב-RAM ל-*value*.

Graphical Output

אנחנו מניחים שהמחשב מחובר למסך שחור-לבן פיזי המסודר בתור *grid* של שורות ועמודות, כאשר כל חיתוך של שורה ועמודה הוא פיקסל. הקונבנציה היא שהעמודות ממוספרות משמאל לימין והשורות ממוספרות מלמעלה למטה. לכן, הפיקסל (0, 0) ממוקם בפינה השמאלית העליונה של המסך. אנחנו מניחים שהמסך מחובר למערכת המחשב דרך *memory map* – אזור ייעודי ב-RAM שבו כל פיקסל מיוצג ע"י ביט אחד.

הפעולה הבסיסית ביותר שניתן לבצע על המסך היא לצייר פיקסל יחיד המצוין ע"י הקואורדינטות (x, y) . זה מתבצע ע"י הדלקה/כביובי של הביט המתאים ב-*memory map*. פעולות אחרות כמו ציור קו וציור עיגול נבנים על הפעולה הבסיסית הזו. חבילת הגרפיקה מתחזקת צבע נוכחי שניתן לקבוע אותו לשחור או לבן. כל פעולות הציור משתמשות בצבע הנוכחי.

ציור פיקסל (*drawPixel*): ניתן לצבוע פיקסל נבחר במקום (x, y) במסך ע"י מציאת הביט התואם ב-*memory map* ולקבוע אותו לצבע הנוכחי. כיוון שה-RAM הוא התקן של *n*-ביט, הפעולה הזו דורשת קריאה וכתביבה של ערך שהוא *n*-ביט.

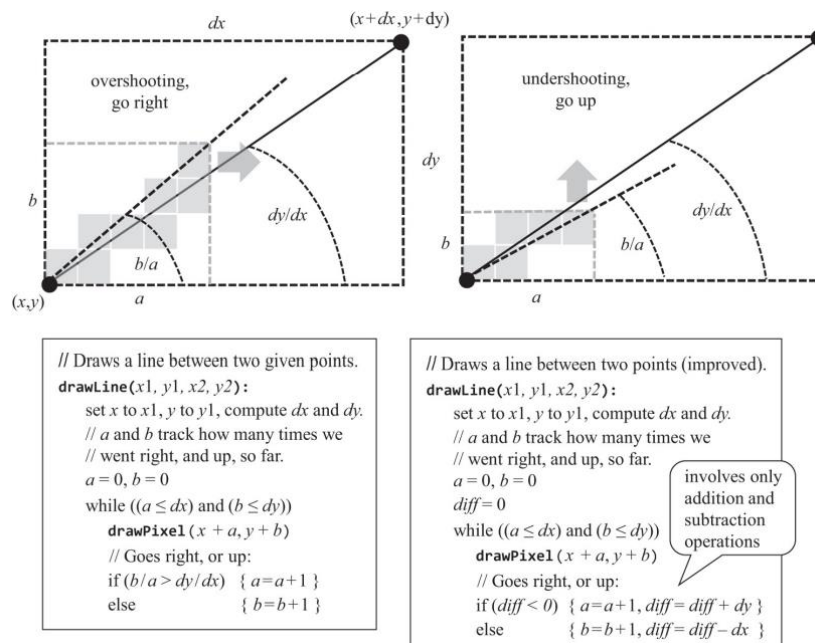
```
// Sets pixel (x,y) to the current color.
drawPixel(x,y):
    Using x and y, compute the RAM address where
    the pixel is represented;
    Using Memory.peek, get the 16-bit value of
    this address;
    Using some bitwise operation, set (only) the bit
    that corresponds to the pixel to the current color;
    Using Memory.poke, write the modified 16-bit
    value "back" to the RAM address.
```

בפרק 5 ראינו את המפרט של ה-memory map של מסך ה-Hack. צריך להשתמש במיפוי הזה כדי לממש את האלגוריתם *drawPixel*.

ציור קו (*drawLine*): כאשר אנחנו מתבקשים לרנדור "קו" רציף בין שתי "נקודות" על ה-grid שעשוי מפיקסלים בדידים, הכי טוב שאנחנו יכולים לעשות זה אומדן של הקו באמצעות ציור סדרה של פיקסלים לאורך קו דמיוני המחבר את שתי הנקודות. ה"עט" שנשתמש בו כדי לצייר את הקו יכול לזוז ל-4 כיוונים בלבד: למעלה, למטה, שמאלה, וימינה. הדרך היחידה לגרום לקו להיראות טוב היא להשתמש ברזולוציית מסך גבוהה עם הפיקסלים הקטנים ביותר שיכולים להיות.

הפרוצדורה של ציור קו מ- (x_1, y_1) ל- (x_2, y_2) מתחילה בצביעת הפיקסל (x_1, y_1) ואז לעשות זיג-זג בכיוון של (x_2, y_2) עד שמגיעים לפיקסל הזה.

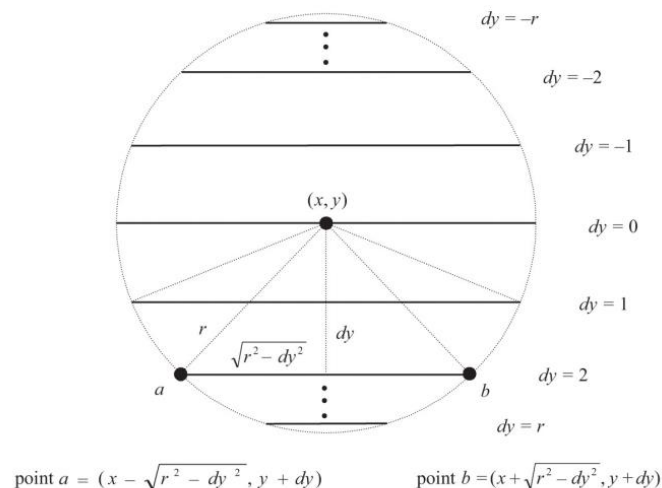
אלגוריתם לציור קו – הגרסה הבסיסית (שמאל) והגרסה המשופרת (ימין):



השימוש בשתי פעולות חילוק בכל איטרציה של הלולאה גורם לאלגוריתם להיות לא נכון ולא מדויק. השיפור הראשון הוא להחליף את התנאי $\frac{b}{a} > \frac{dy}{dx}$ בתנאי השקול $a \cdot dy < b \cdot dx$ שדורש רק כפל

של integers. נשים לב שיכול להיות שהתנאי החדש ייבדק בלי כפל, אז ניתן לעשות זאת באופן יעיל ע"י תחזוק משתנה שמעדכן את הערך של $(a \cdot dy - b \cdot dx)$ בכל פעם ש- a או b גדלים. זמן הריצה של אלגוריתם ציור הקו הזה הוא $O(n)$, כאשר n הוא מספר הפיקסלים בקו המצויר. האלגוריתם משתמש רק בפעולות חיבור וחסור, וניתן לממש אותו באופן יעיל בתוכנה או בחומרה.

ציור עיגול (`drawCircle`): נראה אלגוריתם שמשתמש ב-3 רוטינות שכבר מימשנו – כפל, שורש ריבועי, וציור קו:



```
// Draws a filled circle of radius r, centered at (x,y).
drawCircle(x, y, r):
    for each dy = -r to r do:
        drawLine((x - sqrt(r^2 - dy^2), y + dy), (x + sqrt(r^2 - dy^2), y + dy))
```

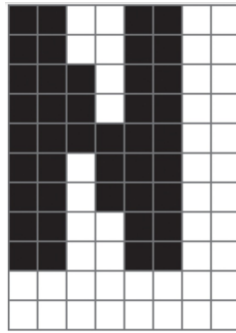
האלגוריתם מבוסס על ציור רצף של קווים אופקיים (כמו הקו ab בציור), אחד עבור כל שורה בטווח שבין $r - y$ ו- $y + r$. כיוון ש- r מצוין בפיקסלים, האלגוריתם מסתיים בציור קו בכל שורה לאורך הקוטר הצפוני-דרומי של העיגול, שהתוצאה היא עיגול שלם ומלא.

Character Output

כדי לפתח יכולת להציג תווים, ראשית נהפוך את המסך הפיזי שלנו למסך לוגי -character-oriented שמתרים לרנדר bitmapped images קבועות המייצגות תווים. לדוגמה, נתבונן במסך פיזי שמכיל 256 שורות של 512 פיקסל כל אחת. אם נרצה grid של 11 שורות ו-8 עמודות לציור תו אחד, המסך שלנו יוכל להציג 23 שורות של 64 תווים כל אחת, עם 3 שורות אקסטרה של פיקסלים שלא בשימוש.

Fonts: ניתן לחלק את קבוצת התווים שמחשבים משתמשים בהם לתתי-קבוצות של printable ו-non-printable. לכל תו שהוא printable בקבוצת התווים של ה-Hack, קיימת bitmap image. ייעודית בגודל 11 על 8. יחד, התמונות הללו נקראות font.

דוגמה – ה-bitmap של האות N:



כדי לטפל ברווח בין התווים, כל תמונה של תו כוללת לפחות פיקסל אחד של רווח לפני התו הבא בשורה ולפחות רווח של פיקסל אחד בין שורות שכנות. ה-font של ה-Hack מורכב מ-95 bitmap images, אחת עבור כל תו printable בקבוצת התווים של ה-Hack.

Cursor: תווים בדרך כלל מוצגים אחד אחרי השני, משמאל לימין, עד שמגיעים לסוף השורה. לדוגמה, נתבונן בתוכנית שבה מופיעה ה-statement `print("a")` ואחריה (אולי לא מיד) מופיעה ה-statement `print("b")`. זה אומר שהתוכנית רוצה להציג `ab` על המסך. כדי לגרום להמשכיות הזאת, החבילה של כתיבת תווים מתחזקת cursor גלובאלי שעוקב אחרי המקום במסך שמו צריך לצייר את התו הבא. המידע שה-cursor מחזיק הוא מונה של העמודות והשורות, נניח `cursor.col` ו-`cursor.row`. אחרי שמציגים תו, נעדכן `cursor.col + 1`. בסוף השורה נעדכן `cursor.col = 0` ו-`cursor.row + 1`. כאשר מגיעים לתחתית המסך, אפשר לגלול (כלומר לעשות scrolling operation) או לנקות את המסך ולהתחיל מחדש ע"י קביעת ה-cursor ל-`(0,0)`.

כתיבת טיפוס מידע שהם לא תווים נובעת מהיכולת הבסיסית הזו: מחרוזות נכתבות תו אחרי תו, ומספרים ממירים קודם למחרוזת ואז כותבים את המחרוזת.

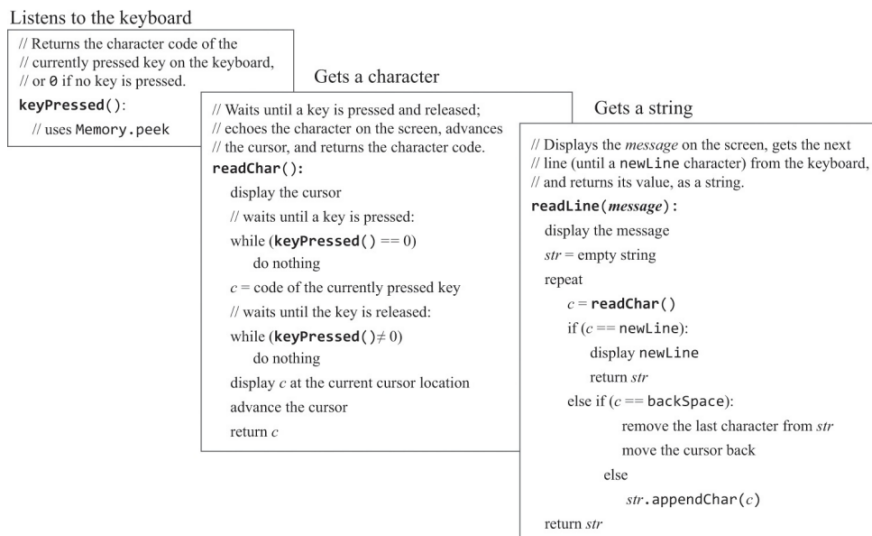
Keyboard Input

נתבונן ב-statement הבאה: `let name = Keyboard.readLine("Enter your name")`. הפונקציה לא תסיים לרוץ עד שהמשתמש ילחץ על מקשים כלשהם במקלדת ובסוף על ENTER. המימוש של הפונקציה `readLine` צריך להתמודד עם זה שבני אדם לוחצים על מקשים ומשחררים אותם למשך זמן לא צפוי ויכולים לצאת להפסקת קפה באמצע, ועם זה שבני אדם מחבבים backspacing, מחיקה וכתיבה מחדש של תווים.

נתאר כיצד מנוהל הקלט מהמקלדת, בשלושה שלבים של אבסטרקציה: זיהוי המקש הנוכחי שנלחץ במקלדת, "לכידת" קלט של תו בודד, ו"לכידת" קלט של מספר תווים.

זיהוי קלט מהמקלדת (`keyPressed`): כדי לזהות על איזה מקש לוחצים במחשב ה-Hack, המקלדת מרעננת ברצף רגיסטר של 16-ביט בזיכרון שהכתובת שלו שמורה במצביע שנקרא `KBD`. אם לוחצים על מקש כלשהו במקלדת, הכתובת הזו מכילה את ה-character code של המקש, אחרת היא מכילה 0.

מימוש הפונקציה `keyPressed`:



קריאת תו בודד (`readChar`): משך הזמן שעובר בין הלחיצה על המקש ועד השחרור של המקש הוא לא צפוי. לכן, אנחנו צריכים לכתוב קוד שאדיש לאי-הוודאות הזו. בנוסף, כאשר משתמשים לוחצים על מקשים במקלדת, נרצה לתת להם פידבק של איזה מקשים נלחצו. נרצה להציג cursor גרפי במקום במסך אליו יגיע הקלט הבא, ואחרי שכמה מקשים יילחצו נרצה להדהד את תווי הקלט ע"י הצגת ה-bitmap שלהם על המסך במקום של ה-cursor. כל הפעולות הללו ימומשו ע"י פונקציית ה-`readChar`.

קריאת מחרוזת (`readLine`): קלט עם מספר תווים שהוקלד ע"י המשתמש נחשב סופי אחרי שנלחץ מקש ה-ENTER, מה שמפיק את התו `newLine`. עד שמקש ה-ENTER נלחץ, המשתמש צריך להיות יכול לעשות `backspace`, למחוק, ולכתוב מחדש תווים שנכתבו קודם לכן. כל הפעולות הללו מסופקות ע"י פונקציית ה-`readLine`.

תוכנית high-level מסתמכת על אבסטרקציית ה-`readLine`, שמסתמכת על אבסטרקציית ה-`readChar`, שמסתמכת על אבסטרקציית ה-`keyPressed`, שמסתמכת על אבסטרקציית ה-`Memory.peek`, שמסתמכת על החומרה.

The Jack OS Specification

מערכת ההפעלה של ה-Jack מאוגדת ב-8 מחלקות:

- **Math**: מספקת פעולות מתמטיות
- **String**: מממשת את הטיפוס String
- **Array**: מממשת את הטיפוס Array

- **Memory**: מטפלת בפעולות על הזיכרון
- **Screen**: מטפלת בפלט גרפי למסך
- **Output**: מטפלת בפלט של תווים למסך
- **Keyboard**: מטפלת בקלט מהמקלדת
- **Sys**: מספקת שירותים הקשורים להרצה

Implementation

כל מחלקת OS היא אוסף של subroutines (בנאים, פונקציות ומתודות). חלק מהן ראינו כיצד לממש במהלך הפרק הזה, וחלק קלות למימוש.

פונקציות *init*: חלק ממחלקות ה-OS משתמשות במבני נתונים שתומכים במימוש של חלק מה-subroutines שלהן. עבור כל *OSClass*, כזו, אפשר להצהיר על מבני הנתונים הללו באופן סטטי ברמת המחלקה ולא תחיל אותם בפונקציה שנקרא לה *OSClass.init*. פונקציות ה-*init* הן למטרות פנימיות ולא מתועדות ב-API של ה-OS.

Math

multiply: בכל איטרציה *i* של אלגוריתם הכפל, מחלצים את הביט ה-*i* של הכופל השני. נציע לבצע את הפעולה הזו בפונקציית עזר *bit(x, i)* שמחזירה *true* אם הביט ה-*i* של המספר *x* הוא 1, ו-*false* אחרת. אפשר לממש את הפונקציה *bit(x, i)* בקלות באמצעות פעולות shifting. אבל Jack לא תומכת ב-shifting, אז במקום יהיה נוח להגדיר מערך סטטי קבוע באורך 16, נניח *twoToThe*, ולקבוע כל איבר *i* של המערך להיות 2^i . המערך הזה יכול לשמש כדי לתמוך במימוש של *bit(x, i)* ואפשר לבנות אותו בפונקציה *Math.init*.

divide: אלגוריתמי הכפל והחילוק שהצגנו מיועדים לפעול על מספרים אי-שליליים. אפשר לטפל במספרים שהם signed ע"י הפעלת האלגוריתמים על ערכים מוחלטים וקביעת הסימן של ערך ההחזרה בהתאם. עבור אלגוריתם הכפל, אין צורך בכך: מכפלה של מספרים הנתונים בשיטת המשלים ל-2 תחזיר את התוצאה הנכונה.

באלגוריתם החילוק, כופלים את *y* בחזקה של 2 עד ש- $y > x$ ולכן *y* יכול לגרום ל-overflow. ניתן לזהות את ה-overflow ע"י בדיקה מתי *y* הופך להיות שלילי.

sqrt: באלגוריתם השורש, החישוב של $(y + 2^j)^2$ יכול לגרום ל-overflow. כדי לפתור את הבעיה הזו ניתן לשנות את הלוגיקה של האלגוריתם ל:

$$\text{if } (y + 2^j)^2 \leq x \text{ and } (y + 2^j)^2 > 0 \text{ then } y = y + 2^j$$

String

כל ה-string constants שמופיעים בתוכנית Jack ממומשים בתור אובייקטים של המחלקה *String*. בפרט, כל מחרוזת ממומשת בתור מערך של ערכי *char*, תכונת *maxLength* שמחזיקה את האורך המקסימלי של המחרוזת, ותכונת *length* שמחזירה את האורך האמיתי של המחרוזת.

דוגמה: נתבונן ב-statement הבאה: *let str = "scooby"*. כאשר הקומפיילר מטפל ב-statement הזה, הוא קורא לבנאי של *String*, שיוצר מערך של *char* עם *maxLength = 6* ו-*length = 6*. אם נקרא יותר מאוחר למתודה *str.eraseLastChar()*, ה-*length* של המערך יתפח ל-5, והמחרוזת תהפוך להיות "scoob".

באופן כללי, איברים במערך שמעבר ל-*length* לא נחשבים לחלק מהמחרוזת.

setInt ו-**intValue**: ניתן לממש את ה-subroutines האלה ע"י האלגוריתמים שהצגנו. נשים לב שאף אחד מהאלגוריתמים לא מתמודד עם מספרים שליליים – פרט שצריך להתמודד איתו במימוש.

doubleQuote ו-**backSpace**, **newLine**: הקוד של התווים האלה הוא 128, 129 ו-34.

את שאר המתודות של *String* אפשר לממש באופן ישיר ע"י פעולה על מערך ה-*char* ועל שדה ה-*length* שמאפיינים כל אובייקט *String*.

Array

ה-subroutine ה-**new** הזאת היא לא בנאי, אלא פונקציה. לכן, המימוש של הפונקציה זאת צריך להקצות מקום בזיכרון למערך החדש ע"י קריאה מפורשת לפונקציית ה-OS *Memory.alloc*.

dispose: מתודת ה-void הזאת נקראת ב-statements כמו *do arr.dispose()*. המימוש של *dispose* עושה למערך deallocation ע"י קריאה לפונקציית ה-OS *Memory.dealloc*.

Memory

peek ו-**poke**: הפונקציות הללו מספקות גישה ישירה לזיכרון של ה-host. כדי להשיג גישה כזו בשפה high-level ולממש את הפונקציות הללו, ננצל trapdoor שמאפשרת שליטה בזיכרון של המחשב. הטריק מתבסס על שימוש חריג של משתנה רפרנס (מצביע). Jack לא מונעת מהמתכנת להקצות קבוע למשתנה רפרנס. ניתן להתייחס לקבוע הזה בתור כתובת בזיכרון. כאשר משתנה רפרנס הוא מערך, הטריק הזה מספק גישה לכל מילה ב-RAM של ה-host.


```
// Creates a Jack-level "proxy" of the RAM:
var Array memory;
let memory = 0; // No problem . . .
...
// Gets the value of the RAM at address i:
let x = memory[i];
...
// Sets the value of the RAM at address i:
let memory[i] = 17;
...
```

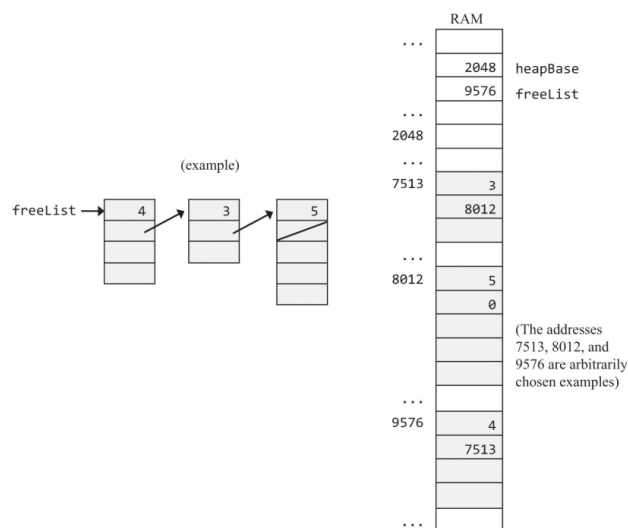
לפי 2 השורות הראשונות של הקוד, כתובת הבסיס של המערך *memory* מצביע לכתובת הראשונה ב-RAM של המחשב (הכתובת 0). כדי לקבל או לקבוע את הערך של המקום ב-RAM שהכתובת שלו היא *i*, כל שעלינו לעשות היא לפעול על האיבר *memory[i]* במערך. זה יגרום לקומפיילר לפעול על המקום ב-RAM שהכתובת שלו היא $0 + i$, שזה מה שאנחנו רוצים.

לא מוקצה למערכים ב-Jack מקום ב-heap בזמן הקומפילציה, אלא בזמן הריצה, אם וכאשר קוראים לפונקציית ה-*new* של המערך. נשים לב כי אם *new* היה בנאי ולא פונקציה, הקומפיילר ומערכת ההפעלה היו מקצים למערך כתובת סתמית ב-RAM שאנחנו לא יכולים לשלוט בה. הטריק הזה עובד כיוון שאנחנו משתמשים במשתנה המערך בלי לאתחל אותו כמו שצריך.

אפשר להצהיר על המערך *memory* ברמת המחלקה ולאתחל אותו בפונקציה *Memory.init*. כך המימוש של *Memory.peek* ו-*Memory.poke* נעשה טריוויאלי.

***alloc* ו-*deAlloc*:** הפונקציות הללו יכולות להיות ממומשות ע"י אחד מהאלגוריתמים שראינו – *best – fit* או *first – fit*. המיפוי הסטנדרטי של VM על פלטפורמת ה-Hack מציין שהמחסנית ממופה לכתובות 256 עד 2047 ב-RAM. לכן, ה-heap יכולה להתחיל בכתובת 2048.

כדי לממש את הרשימה המקושרת *freeList*, המחלקה *Memory* יכולה להצהיר על משתנה סטטי, *freeList* ולתחזק אותו. למרות ש-*freeList* מאותחל לערך של *heapBase* (2048), ייתכן כי אחרי מספר פעולות *alloc* ו-*deAlloc*, *freeList* יהפוך להיות כתובת אחרת בזיכרון, כפי שניתן לראות באיור:



עבור יעילות, מומלץ לכתוב קוד Jack שמנהל את הרשימה המקושרת *freeList* ישירות ב-RAM, כפי שניתן לראות באיור. אפשר לאתחל את הרשימה המקושרת בפונקציה *Memory.init*.

Screen

המחלקה *Screen* מתחזקת *current color* שהוא בשימוש ע"י כל הפונקציות המציירות של המחלקה. הצבע הנוכחי יכול להיות מיוצג ע"י משתנה סטטי בוליאני.

drawPixel: אפשר לצייר פיקסל על המסך באמצעות *Memory.peek* ו-*Memory.poke*.
ה-map memory של המסך בפלטפורמת ה-Hack מציינת שהפיקסל בעמודה *col* ובשורה *row* (עבור $0 \leq col \leq 511$ ו- $0 \leq row \leq 255$) ממופה לביט ה- $col \% 16$ של המקום בזיכרון $16384 + row \cdot 32 + \frac{col}{16}$. ציור של פיקסל אחד דורש שינוי ביט אחד במילה שניגשים אליה (ורק את הביט הזה).

drawLine: צריך להכליל אספקטים מסוימים של האלגוריתם כדי לצייר קווים שמורחבים ל-4 כיוונים. ניזכר שהבסיס של המסך (הקואורדינטות (0,0)) הוא הפינה השמאלית העליונה. לכן, צריך לשנות חלק מהכיוונים ומפעולות הפלוס/מינוס של האלגוריתם במימוש ה-*drawLine* שלנו. האלגוריתם לא צריך לטפל במקרים המיוחדים של ציור קו ישר, כאשר $dx = 0$ או $dy = 0$.

drawCircle: האלגוריתם שתיארנו יכול לגרום ל-overflow. כדי לפתור את זה אפשר להגביל את הרדיוס של המעגל להיות לכל היותר 181.

Output

המחלקה *Output* היא ספריה של פונקציות להצגת תווים. המחלקה מניחה שהמסך הוא character-oriented ומורכב מ-23 שורות (הממוספרות 0 ... 22 מלמעלה למטה) של 64 תווים כל אחת (הממוספרים 0 ... 63 משמאל לימין). האינדקס של המיקום של התו השמאלי העליון הוא (0,0). cursor ויזואלי, הממומש בתור ריבוע קטן מלא, מסמן היכן יוצג התו הבא. כל תו מוצג ע"י רינדור של תמונה מלבנית על המסך, בגובה 11 פיקסלים וברוחב 8 פיקסלים (שכוללים את השוליים עבור רווח בין תווים ובין שורות). האוסף של כל התמונות של התווים נקרא font.

Font Implementation: ה-font הוא אוסף של 59 bitmap images מלבניות שכל אחת מהן מייצגת תו שהוא printable. ה-font נמצא במחלקת ה-*Output* של ה-OS. עבור כל תו שהוא printable, נגדיר מערך שמחזיק את ה-bitmap של התו ומורכב מ-11 איברים שכל אחד מהם מתאים לשורה של 8 פיקסלים. בפרט, נקבע את הערך של הרשומה ה-*j* במערך לערך integer שהייצוג הבינארי שלו מקודד את 8 הפיקסלים שמופיעים בשורה ה-*j* של ה-bitmap של התו. נגדיר גם מערך סטטי בגודל 127 שערכי האינדקסים שלו יהיו 126 ... 32 ויתאימו לקודים של

התווים ה-printable בקבוצת התווים של ה-Hack (רשומות 0 ... 31 לא יהיו בשימוש). נגדיר כל רשומה i במערך הזה להיות מערך בעל 11 רשומות שמייצג את ה-bitmap image של התו שהקוד שלו הוא i .

ניתן להפעיל את הקוד של המימוש הזה בפונקציה `Output.init` שגם מאתחלת את ה-cursor.

printChar: מציגה את התו במקום של ה-cursor ומקדמת את ה-cursor עמודה אחת קדימה. כדי להציג תו במקום (row, col) כאשר $0 \leq row \leq 22$ ו- $0 \leq col \leq 63$, נכתוב את ה-bitmap של התו לקופסה של פיקסלים בטווחים $[11 \cdot row, 11 \cdot row + 10]$ ו- $[8 \cdot col, 8 \cdot col + 7]$.

printString: ניתן לממש באמצעות רצף של קריאות ל-`printChar`.

printInt: ניתן לממש באמצעות המרה של המספר למחרוזת ואז להדפיס את המחרוזת.

Keyboard

ארגון הזיכרון של מחשב ה-Hack מפרט שה-memory map של המקלדת הוא רגיסטר בזיכרון של 16-ביטי שממוקם בכתובת 24576.

keyPressed: ניתן לממש בקלות באמצעות `Memory.peek()`.

readChar ו-**readString**: ניתן לממש באמצעות האלגוריתמים שראינו.

readInt: ניתן לממש באמצעות קריאת מחרוזת ולהמיר אותה לערך int ע"י שימוש במתודה של `String`.

Sys

wait: הפונקציה הזו צריכה לחכות מספר נתון של מילי-שניות ואז להחזיר. ניתן לממש אותה ע"י כתיבת לולאה שרצה בערך `duration` מילי-שניות לפני שהיא עוצרת.

halt: ניתן לממש באמצעות כניסה ללולאה אינסופית.

init: לפי המפרט של שפת ה-Jack, תוכנית Jack היא קבוצה של מחלקה אחת או יותר. מחלקה אחת חייבת להיקרא *Main*, והמחלקה הזו חייבת להכיל פונקציה שנקראת *main*. כדי להתחיל להריץ תוכנית, צריך לקרוא לפונקציה *Main.main*.

מערכת ההפעלה היא גם אוסף של מחלקות Jack מקומפלות. כאשר המחשב boots up, נרצה להתחיל להריץ את מערכת ההפעלה ושהיא תתחיל להריץ את התוכנית הראשית. שרשרת הפקודות הללו ממומשת באופן הבא: לפי המיפוי הסטנדרטי של VM על פלטפורמת ה-Hack, ה-VM translator כותב קוד bootstrap (בשפת מכונה) שקורא לפונקציית ה-*OS.Sys.init*. קוד ה-bootstrap הזה שמור ב-ROM החל מהכתובת 0. כאשר עושים reset למחשב, ה-program counter נקבע ל-0, קוד ה-bootstrap מתחיל לרוץ, והפונקציה *OS.Sys.init* נקראת.

הפונקציה *OS.Sys.init* צריכה לעשות שני דברים: לקרוא לכל פונקציות ה-*init* של כל מחלקות ה-OS האחרות, ואז לקרוא ל-*Main.main*.

עוד נאנד

עמית בן ארצי

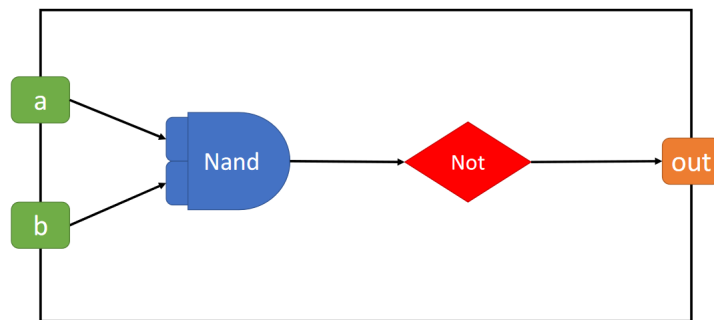
25 בינואר 2021

להערות - Amit.Benartzy@mail.huji.ac.il, [Whatsapp](#)

הערה: יש כאן הרבה צילומי מסך מהספר וכל מיני דברים שמצאתי בדרייב, ומדברים שאנשים שלחו בווצאפ ואני לא רוצה לקחת לאף אחד זכויות יוצרים ☺

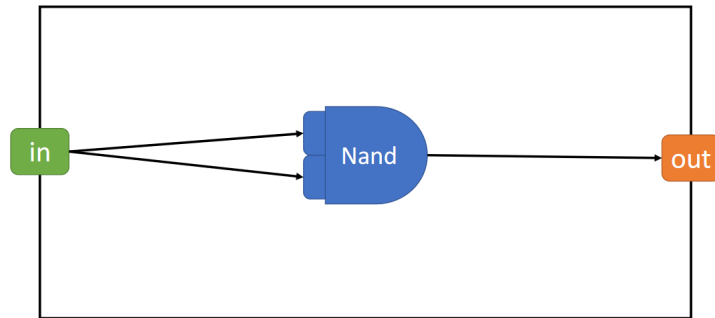
שאלה איך ממשים בצורה הכי יעילה את השבב AND?

And (2)



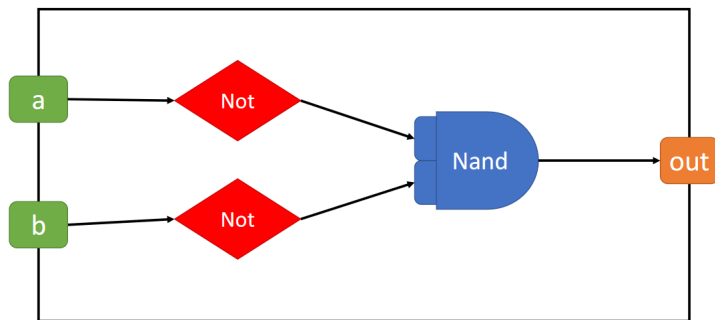
שאלה איך ממשים בצורה הכי יעילה את השבב NOT?

Not (1)



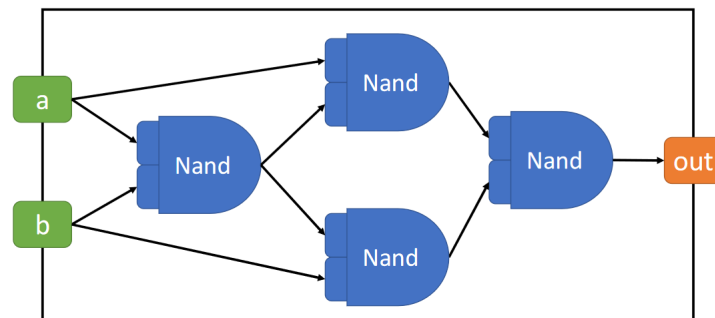
שאלה איך ממשים בצורה הכי יעילה את השבב OR?

Or (3)



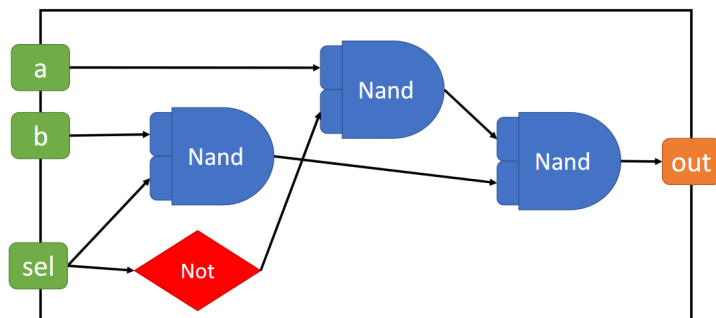
שאלה איך ממשים בצורה הכי יעילה את השבב XOR?

Xor (4)



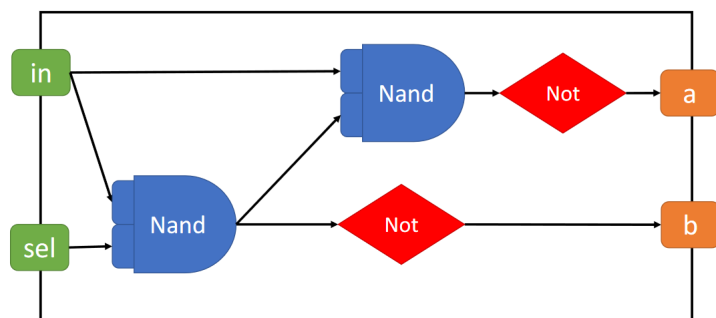
שאלה איך ממשים בצורה הכי יעילה את השבב MUX?

Mux (4)



שאלה איך ממשים בצורה הכי יעילה את השבב DMUX?

DMux (4)



שאלה באלו, מהי המשמעות של f ?

תשובה אם f אמת אז הוא חיובי, ולכן עושים חיבור. אם f שקר על עושים $x \& y$ כי שקרנים משקרים ביחד

שאלה מה יש בכתובות האלו בראם?

Register	Name	Usage
RAM[0]	SP	Stack pointer: points to the next topmost location in the stack;
RAM[1]	LCL	Points to the base of the current VM function's local segment;
RAM[2]	ARG	Points to the base of the current VM function's argument segment;
RAM[3]	THIS	Points to the base of the current this segment (within the heap);
RAM[4]	THAT	Points to the base of the current that segment (within the heap);
RAM[5–12]		Holds the contents of the temp segment;
RAM[13–15]		Can be used by the VM implementation as general-purpose registers.

שאלה מה הגודל של טיפוס פרימיטיבי?

תשובה 16 ביט

שאלה מה אנחנו שומרים בHeap?

תשובה אובייקטים ומערכים

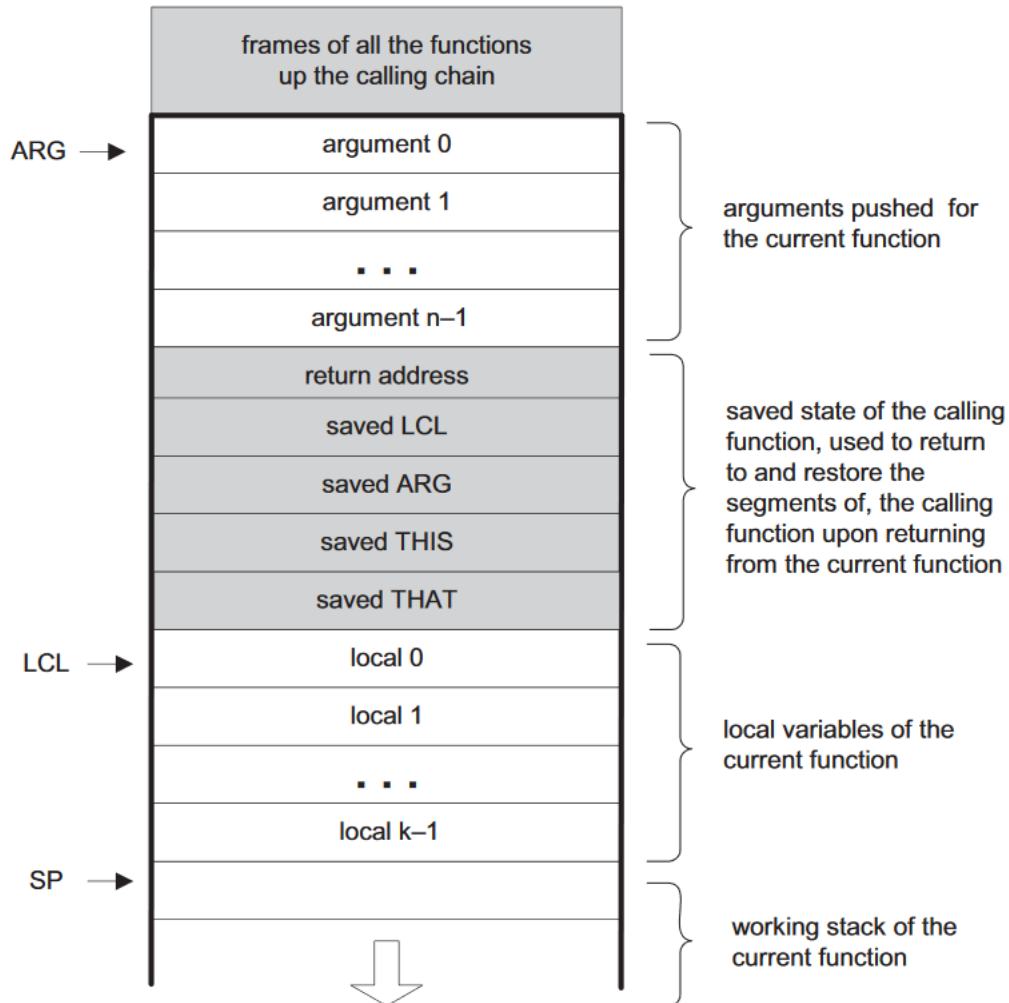
שאלה ביצעתי את השורה `call f n`. מה קורה?

```

push return-address // (Using the label declared below)
push LCL             // Save LCL of the calling function
push ARG             // Save ARG of the calling function
push THIS            // Save THIS of the calling function
push THAT            // Save THAT of the calling function
ARG = SP - n - 5     // Reposition ARG (n = number of args.)
LCL = SP             // Reposition LCL
goto f               // Transfer control
(return-address)     // Declare a label for the return-address

```

שאלה כיצד נראה הזיכרון אחרי `call f n`?



שאלה מה קורה אחרי ביצוע הפקודה `function f k`?

function f k (declaring a function f that has k local variables)	(f) repeat k times: PUSH 0	// Declare a label for the function entry // k = number of local variables // Initialize all of them to 0
--	----------------------------------	---

שאלה מה קורה אחרי ביצוע return? למה נרצה לעשות את זה אחרי ריטורן? מה המשמעות של זה?

return (from a function)	FRAME = LCL RET = *(FRAME-5) *ARG = pop() SP = ARG+1 THAT = *(FRAME-1) THIS = *(FRAME-2) ARG = *(FRAME-3) LCL = *(FRAME-4) goto RET
------------------------------------	---

שאלה מה עושים בbootstrap?

```
SP=256          // Initialize the stack pointer to 0x0100
call Sys.init    // Start executing (the translated code of) Sys.init
```

שאלה מה הכתובות RAM של הבאים?

שימו לב! ששמתי לב שבספר שלי יש אי תאימות בין מקומות שונים עם הכתובות אז עדיף לדלג על זה (צילום מסך של אי התאימות בעמוד האחרון).

RAM addresses	Usage
0–15	Sixteen virtual registers, usage described below
16–255	Static variables (of all the VM functions in the VM program)
256–2047	Stack
2048–16483	Heap (used to store objects and arrays)
16384–24575	Memory mapped I/O

שאלה מהו סדר הפעולות כשרוצים לעשות למשל arr[2]=15?

תשובה תחילה, לנוחות, נניח ש-arr שמור במקום 0 this במחלקה כלשהי. נפעל באופן הבא:

1. push constant 2 //we want to put thing in arr[2]
2. push this 0 //this is the base address of arr
3. add //address of arr[2]
4. push constant 15 //this is what we want to put in arr[2]
5. pop temp 0 //temp 0 = 15
6. pop pointer 1 //that = address of arr[2]
7. push temp 0 // push temp0 = 15
8. pop that 0 // put 15 inside that 0 = pointer 1 = arr[2]

שאלה מה ההבדל בין הפקודה 1 pop pointer לבין הפקודה 0 pop that?

תשובה הפקודה הראשונה תחזיר לנו את הערך של that ואילו השנייה תחזיר לנו את הערך של Memory[that].

שאלה האם Jack היא Sensitive Case?

תשובה כן

שאלה איך עוברים בין תו ASCII שלו?

תשובה הוספת הקבוע 48.

שאלה מה נוכל לעשות כדי שהאלגוריתם לציור מעגל לא יגרום לגלישה?

תשובה הגבלת הרדיוס ל-181.

שאלה מה ההבדל בין Type ל-Kind?

תשובה Type יכול להיות int String וכו'. Kind מדבר על באיזה סגמנט אנחנו שומרים דברים - field local, argument, static.

שאלה מתי אנחנו הופכים את הפונקציות * , - לפונקציות מהמחלקת Math?

תשובה בקומפיילר (ג'ק ← VM)

שאלה בקוד ג'ק יש לי מחרוזת "string", מה יעשה הקומפיילר?

תשובה ייקרא ל-String.new(length) ואח"כ יוסיף תו תו למחרוזת.

שאלה מה עושה הפונקציה Sys.init()?

תשובה קוראת ל-init של כל מחלקה, קוראת ל-Main.main() ולבסוף עושה Sys.halt().

שאלה מה המחלקה Array.jack יכולה לעשות?

תשובה לבנות מערך חדש (new) ולעשות dispose.

שאלה נבצע את הפקודות הבאות:

1. `let x = 0;`
2. `let x[index] = 15;`

מה יקרה? האם נקבל שגיאת קומפילציה?

תשובה לא נקבל שגיאת קומפילציה, ובפרט בדיוק כך נגדיר את RAM במחלקה Memory.jack! בפרט מה שיקרה זה שנגרום למקום ה-inedx בראם (כלומר ב-heap) להיות שווה ל-15.

שאלה בתרגיל 12 Memory.jack, מהו ההבדל בין first-fit לבין best-fit?

תשובה first-fit ימצא את ההתאמה הראשונה, כלומר את הסגמנט הראשון ברשימה המקושרת של הסגמנטים הפנויים שמספיק גדולה כדי להכיל את מה שצריך. לעומת זאת best-fit תחזיר לנו את הסגמנט הקטן ביותר ברשימה המקושרת, שגודל מספיק כדי להכיל את הנדרש.

שאלה ביצענו את הפקודה `let x = 3; Memory.alloc(3);` מה יימצא ב-RAM[x-1]?

תשובה ב-1 - x נמצא את גודל הבלוק.

שאלה איזה שערים אוניברסליים אנחנו מכירים?

תשובה Nand,Nor

הערה אני משתמשת כאן ב- Δ בתור NAND בתור סימן למרות שזהו אינו סימון מקובל כי לדעתי זה יותר נוח ללמידה.

שאלה מהם חוקי דה מורגן?

$$x \text{NOR} y = (\neg x) \wedge (\neg y) \quad x \Delta y = (\neg x) \vee (\neg y)$$

שאלה איך נביע את הבאים עם נאנד?

תשובה $\Delta = \text{NAND}$

$$x \wedge y = (x \Delta y) \Delta (x \Delta y)$$

$$\neg x = x \Delta x$$

$$x \vee y = (x \Delta x) \Delta (y \Delta y)$$

שאלה כיצד נבנה את השער NAND באמצעות שערי NOR?

1. `Nor(a=x, b=x, out=notX)`
2. `Nor(a=y, b=y, out=notY)`
3. `Nor(a=notX, b=notY, out=XandY)`
4. `Nor(a=XandY, b=XandY, out=out)`

שאלה כיצד נבנה את השער NAND באמצעות Mux?

1. `XandY=Mux(a=x, b=y, sel=x, out=out)`
2. `Not(x)=Mux(a=true, b=false, sel=x, out=out)`

שאלה כיצד נבנה את השער NAND באמצעות HalfAdder?

1. `XandY=HalfAdder(a=x, b=y, sum=dont_care, out=out)`
2. `Not(x)=(a=x, b=True, sum=out, carry=dont_care)`

שאלה האם נוכל להגיע לכל טבלת אמת באמצעות Not And, Or?

תשובה כן

שאלה מהו האלגוריתם היעיל לכפל?

```

function int multiply(int x, int y) {
    var int product, shiftedX, digit, andProduct;
    let shiftedX = x;
    while(digit < 16) {
        let andProduct = y & shiftLeftValues[digit];
        if (~(andProduct = 0)) {
            let product = product + shiftedX;
        }
        let shiftedX = shiftedX + shiftedX;
        let digit = digit + 1;
    }
    return product;
}

```

כאשר ערכי ShiftedLeftValues מאותחלים באופן הבא :

1. ShiftedLeftValues[0] = 0000000000000001=1
2. ShiftedLeftValues[1] = 0000000000000010=2
3. ⋮
4. ShiftedLeftValues[14] = 0100000000000000=16384
5. ShiftedLeftValues[15] = 1000000000000000=16384 + 16384 \\important because 32768 won't compile

שאלה מהו האלגוריתם היעיל לחילוק?

תשובה הפונקציה הבאה שהיא helper מניחה ששיש פונקציה עוטפת, כך ש :

1. נותר לנו לחלק את $|x|$ ב- $|y|$ והפונקציה העוטפת מתעסקת בסימנים

2. יש משתנה גלובאלי outputY2 שמאותחל לאפס לפני ההרצה הראשונה

```

function int divide_helper(int x, int y) {
    var int output;
    if ((y > x) | (y < 0)) {
        return 0;
    }
    let output = Math.divide_helper(x, y + y);
    let output = output + output;
    if ((x - outputY2) < y) {
        return output;
    }
    let outputY2 = outputY2 + y;
    return output + 1;
}

```

שאלה מהו האלגוריתם היעיל לשורש?

```
sqrt(x):  
  // Compute the integer part of  $y = \sqrt{x}$ . Strategy:  
  // Find an integer  $y$  such that  $y^2 \leq x < (y+1)^2$  (for  $0 \leq x < 2^n$ )  
  // By performing a binary search in the range  $0 \dots 2^{n/2} - 1$ .  
   $y = 0$   
  for  $j = n/2 - 1 \dots 0$  do  
    if  $(y + 2^j)^2 \leq x$  then  $y = y + 2^j$   
  return  $y$ 
```

```
function int sqrt(int x) {  
  var int y, counter, possible_sqrt, squared_possible_sqrt;  
  let counter = 7;  
  while( ~(counter < 0) ) {  
    let possible_sqrt = y + shiftLeftValues[counter];  
    let squared_possible_sqrt = possible_sqrt * possible_sqrt;  
    if( ~(squared_possible_sqrt > x) ) {  
      let y = possible_sqrt;  
    }  
    let counter = counter - 1;  
  }  
  return y;  
}
```

שאלה מהו האלגוריתם היעיל לקו ישר בסיסי (כלומר בהינתן $x_1 < x_2, y_1 < y_2$)

(זו הייתה שאלה במבחן 2018 מועד א')

```
1. var int a, b, diff;
2. let a = 0;
3. let b = 0;
4. let dx = x2 - x1;
5. let dy = y2-y1;
6. let diff = 0;
7. while (~(x2 < (x1 + a)) & (y2 < (y1 + b)))) {
    (a) do Screen.drawPixel(x1 + a, y1 + b);
    (b) if (diff < 0) {
        i. let a = a + 1;
        ii. let diff = diff + dy; }

    (c) else {
        i. let b = b + 1;
        ii. let diff = diff - dx; }

    (d) }
```

שאלה מהו האלגוריתם לציור מעגל?

```
drawCircle(x, y, r):
  for each  $dy \in -r \dots r$  do
    drawLine from  $(x - \sqrt{r^2 - dy^2}, y + dy)$  to  $(x + \sqrt{r^2 - dy^2}, y + dy)$ 
```

```

/** Draws a filled circle of radius r<=181 around (x,y), using the current color. */
function void drawCircle(int x, int y, int r) {
    var int x1, y1, x2, y2;
    var int dy;
    let dy = -r;
    while(~(dy > r))
    {
        let x1 = x - Math.sqrt((r*r) - (dy*dy));
        let y1 = y + dy;

        let x2 = x + Math.sqrt((r*r) - (dy*dy));
        let y2 = y + dy;

        do Screen.drawLine(x1, y1, x2, y2);

        let dy = dy + 1;
    }
    return;
}

```

שאלה מהו האלגוריתם למעבר מString אל int?

```

// Convert a string to a non-negative number
string2Int(s):
    v = 0
    for i = 1 ... length of s do
        d = integer value of the digit s[i]
        v = v * 10 + d
    return v
// (Assuming that s[1] is the most
// significant digit character of s.)

```

שאלה מהו האלגוריתם למעבר מint אל String?

```

// Convert a non-negative number to a string
int2String(n):
    lastDigit = n % 10
    c = character representing lastDigit
    if n < 10
        return c (as a string)
    else
        return int2String(n/10).append(c)

```

שאלה מהו האלגוריתם הבסיסי ל-alloc?

alloc(size):

```

Search freeList using best-fit or first-fit heuristics
  to obtain a segment with segment.length > size
If no such segment is found, return failure
  (or attempt defragmentation)
block = needed part of the found segment
  (or all of it, if the segment remainder is too small)
Update freeList to reflect the allocation
block[-1] = size + 1 // Remember block size, for de-allocation
Return block

```

שאלה מהו האלגוריתם הבסיסי ל-dealloc?

```

// Deallocate a decommissioned object.
deAlloc(object):
  segment = object - 1
  segment.length = object[-1]
  Insert segment into the freeList

```

שאלה מהו האלגוריתם ל-readChar?

```

function char readChar() {
  var char c;
  let c = Keyboard.keyPressed();
  while(c = 0)
  {
    let c = Keyboard.keyPressed();
  }
  // Wait until key is no longer pressed
  while(-(Keyboard.keyPressed() = 0)){ }
  do Output.printChar(c);
  return c;
}

```

תודה רבה לחנה ששלחה את המימושים היעילים בוצאפ לכולם!

שאלה מהם המימושים הכי יעילים לתרגום ← VMASM?

Push commands

<u>Push constant i</u>	<u>Push segment i</u> (for segment = argument, local, this, that)	<u>Push temp i</u>	<u>Push static i</u>	<u>Push pointer</u> <u>0/1</u>
6	8	8	6	6
@i D=A @SP M=M+1 A=M-1 M=D	@i D=A @segment AD=D+M @SP M=M+1 A=M-1 M=D	@i D=A @5 AD=D+M @SP M=M+1 A=M-1 M=D	@file_name.i D=M @SP M=M+1 A=M-1 M=D	@0/1 D=M @SP M=M+1 A=M-1 M=D

Pop commands



<u>Pop segment i</u> (for segment = argument, local, this, that)	<u>Pop temp i</u>	<u>Pop static i</u>	<u>Pop pointer 0\1</u>
9	12	5	5
@segment D=M @i D=D+A @SP AM=M-1 D=D+M A=D-M M=D-A	@5 D=A @i D=D+A @SP AM=M-1 D=D+M A=D-M M=D-A	@SP AM=M-1 D=M @file_name.i M=D	@SP AM=M-1 D=M @THIS\THAT M=D



Arithmetic commands

ADD\SUB\AND\OR	NOT\NEG	LT\GT\EQ (without dealing with overflow)
5	3	12
@SP AM=M-1 D=M A=A-1 M=D+M \ M-D \ D&M \ D M	@SP A=M-1 M= -M \ !M	@SP AM=M-1 D=M A=A-1 D=M-D M=-1 @END D;JLT\JGT\JEQ @SP A=M-1 M=0 (END)

Flow control

label	goto label	if-goto label
(func_name\$label)	@func_name\$label 0;JMP	@SP A=M-1 D=M @func_name\$label D;JNE

שאלה מהו ההבדל בין פקודות `pop segment i` לבין `pop temp i`?

תשובה במקום `@segment` שמים `@5`, ואותו דבר ב`push`.

שאלה באילו מקרים נוכל להגיע לתרגומים קצרים אפילו יותר?

תשובה נשים לב שאם $n = 0$ נוכל לדלג ברוב המקרים על השורה שמתחילה ב-`@n` וזו שאחריה

שאלה מתי מוקצה הזיכרון למשתנה לוקאלי?

תשובה כשמגיעים לפונקציה - כי כשקוראים לפונקציה עם k משתנים לוקאליים עושים k פעמים את הפקודה `push local 0`

שאלה מי מקצה זיכרון למשתנה סטטי?

תשובה אסמבלר (מבחן 2012 מועד א' חורף)

שאלה יש לי תוכנה שמכילה n קבצי ג'ק, וקימפלתי אותה. כמה קבצי VM אקבל? כמה קבצי ASM אקבל? כמה קבצי Hack אקבל?

תשובה אקבל n קבצי VM, קובץ ASM אחד וקובץ Hack אחד.

שאלה אילו מהתווים הבאים יכולים להיות בפקודה מסוג *(symbol)* בשפת אסמבלי? אילו הגבלות חלות עליי? תווים: אותיות, ספרות, underscore, נקודה, דולר, נקודותיים, אחוז, חזקה ^, האשטאג, אמפרסנט, סוגריים, כוכביות

תשובה אסור לנו להתחיל בספרה והסמל יכול להכיל רק אותיות, ספרות, underscore, נקודה, דולר, ונקודותיים.

שאלה האם אסמבלי היא שפת case-sensitive? כלומר האם *(symbol)* ו-*(Symbol)* הם אותו דבר?

תשובה היא כן case-sensitive, ושני הסימנים לעיל שונים לגמרי!

שאלה נתונים לנו הביטים של d1,d2,d3 של פקודת C כלשהי. מהי המשמעות של כל אחד מהם?

תשובה d1 מייצג את A, d2 מייצג את D, ו-d3 מייצג את M. סה"כ נוכל לזכור $d1-d2-d3=ADM$.

<i>dest</i>	d1	d2	d3
null	0	0	0
M	0	0	1
D	0	1	0
MD	0	1	1
A	1	0	0
AM	1	0	1
AD	1	1	0
AMD	1	1	1

שאלה מה מייצגים הביטים j1, j2, j3 של פקודת C?

תשובה

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

שאלה יש לנו ביט באורך 16. היכן נמצא ה-MSB? היכן נמצא ה-LSB?

תשובה ה-MSB נמצא במקום ה-15 וה-LSB במקום האפס.

שאלה כיצד נתכנן שבב Mux4Way?

```
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
        Mux16(a=a, b=b, sel=sel[0], out=out1);
        Mux16(a=c, b=d, sel=sel[0], out=out2);
        Mux16(a=out1, b=out2, sel=sel[1], out=out);
}
```

שאלה כיצד נתכנן בב Dmux4Way?

```

CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
        //dmux in by second bit to get ab and cd
        //than dmux ab and cd by first bit to get a,b,c,d
        DMux(in=in, sel=sel[1], a=DMuxab, b=DMuxcd);
        DMux(in=DMuxab, sel=sel[0], a=a, b=b);
        DMux(in=DMuxcd, sel=sel[0], a=c, b=d);
}

```

שאלה כיצד נתכנן שבב Dmux8Way?

```

CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
        //Dmux in by the third bit to get abcd and efgh
        //than DMux4Way to get a,b,c,d and e,f,g,h by first and second bits
        DMux(in=in, sel=sel[2], a=DMuxabcd, b=DMuxefgh);
        DMux4Way(in=DMuxabcd, sel=sel[0..1], a=a, b=b, c=c, d=d);
        DMux4Way(in=DMuxefgh, sel=sel[0..1], a=e, b=f, c=g, d=h);
}

```

שאלה יש לי ביט בשיטת המשלים ל-2 באורך n . מהו המספר הכי גדול שהוא יכול להיות?

תשובה $2^{n-1} - 1$

שאלה יש לי ביט בשיטת המשלים ל-2 באורך n . מהו המספר הכי קטן שהוא יכול להיות?

תשובה -2^{n-1}

שאלה כיצד נייצג מספרים שליליים בבינארית?

תשובה בהינתן מספר x , נחשב את $2^n - x$ ונכתוב אותו בייצוג בינארי.

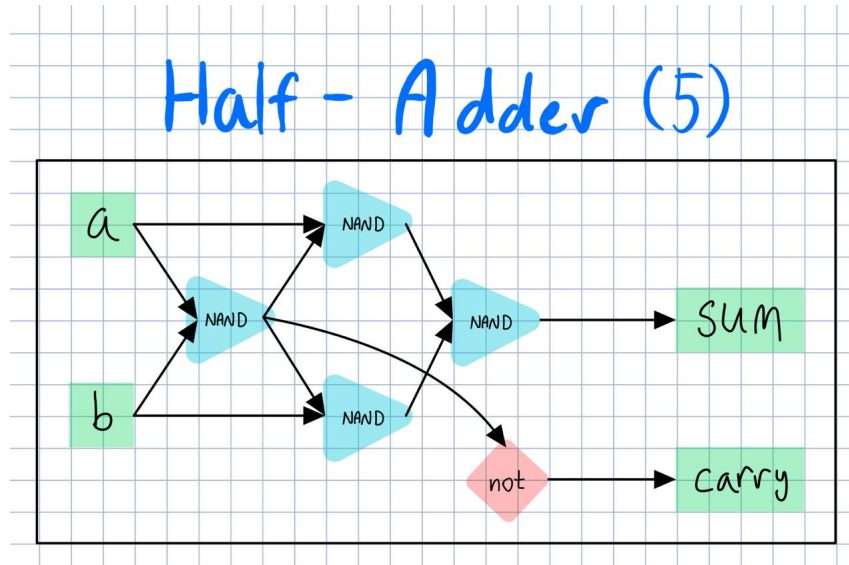
שאלה כיצד נייצג מספרים חיוביים בבינארית?

תשובה פשוט נמיר אותם למספר בינארי כרגיל

שאלה בהינתן מספר בינארי x , כיצד נהפוך אותו ל- $-x$?

תשובה נהפוך את כל הביטים של x ואז נוסיף 1. למשל 0001 מייצג את המספר הדצימאלי 1. נהפוך את כל הביטים ונוסיף 1 ונקבל 1111, שזה הייצוג של -1.

שאלה כיצד נממש HalfAdder בצורה הכי יעילה שיש?



שאלה מה ה-FullAdder אמור לעשות? כיצד תיראה טבלת האמת שלו?

תשובה הוא אמור לקבל 3 מספרים, לסכום אותם, ולהחזיר את הסכום שלהם ואת ה-carry שלהם. למעשה הסכום הוא ה-LSB של $a + b + c$ וה-carry הוא ה-MSB של $a + b + c$. לכן כצפוי, נקבל שזוהי טבלת האמת שלו:

a	b	c	carry	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

שאלה כיצד נממש FullAdder?

תשובה למעשה מספיק לנו לסכום את התוצאות של שני HalfAdder-ים:

```
CHIP FullAdder {
    IN a, b, c; // 1-bit inputs
    OUT sum,    // Right bit of a + b + c
        carry; // Left bit of a + b + c

    PARTS:
    HalfAdder(a=a, b=b, sum=sum1, carry=carry1);
    HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);
    Or(a=carry1, b=carry2, out=carry);
}
```

שאלה כיצד נממש Adder?

```
CHIP Add16 {
  IN a[16], b[16];
  OUT out[16];

  PARTS:
    HalfAdder(a=a[0],b=b[0],sum=out[0], carry=c1);
    FullAdder(a=a[1],b=b[1],c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2],b=b[2],c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3],b=b[3],c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4],b=b[4],c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5],b=b[5],c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6],b=b[6],c=c6, sum=out[6], carry=c7);
    FullAdder(a=a[7],b=b[7],c=c7, sum=out[7], carry=c8);
    FullAdder(a=a[8],b=b[8],c=c8, sum=out[8], carry=c9);
    FullAdder(a=a[9],b=b[9],c=c9, sum=out[9], carry=c10);
    FullAdder(a=a[10],b=b[10],c=c10, sum=out[10], carry=c11);
    FullAdder(a=a[11],b=b[11],c=c11, sum=out[11], carry=c12);
    FullAdder(a=a[12],b=b[12],c=c12, sum=out[12], carry=c13);
    FullAdder(a=a[13],b=b[13],c=c13, sum=out[13], carry=c14);
    FullAdder(a=a[14],b=b[14],c=c14, sum=out[14], carry=c15);
    FullAdder(a=a[15],b=b[15],c=c15, sum=out[15], carry=c16);
}
```

שאלה מתי נתעלם מ-Carry ב-Adder?

תשובה למעשה אנחנו לעולם לא משתמשים ב-carry האחרון של הביט ה-15.

שאלה בשפת ה-HDL, מהו 1 ומהו 0?

תשובה נוכל להציב $a = \text{true}$ ו- $b = \text{false}$ כדי לקבל אמת ושקר בהתאמה.

שאלה מהו צ'יפ קומבינטורי?

תשובה צ'יפ שמסתמך על קומבינציה כלשהי של ה-input וה-output.

שאלה מהי הפונקציה של DFF?

תשובה



$$\text{out}(t) = \text{in}(t-1)$$

שאלה מהי הפונקציה של ביט? מה המשמעות שלה?

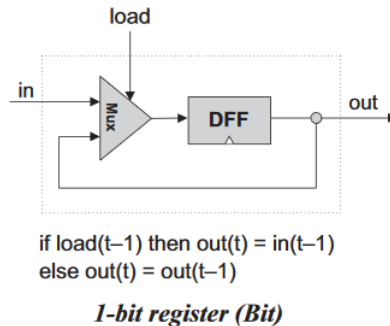
תשובה הפונקציה היא

$$\text{out}(t) = \begin{cases} \text{in}(t-1) & \text{load}(t-1) = \text{True} \\ (t-1) \text{ out} & \text{Else} \end{cases}$$

כלומר במילים: אם אני עכשיו בזמן t , אני מוציאה משהו שמסתמך על אינפוט שכבר קיבלתי בזמן $t - 1$. אם $\text{load}(t - 1) = \text{True}$, אז אני אבחר את האינפוט החדש שלי $\text{in}(t - 1)$; אחרת אני אמשיך להוציא את מה שכבר הוצאתי במחזור הקודם, $\text{out}(t - 1)$.

שאלה כיצד נממש את הצ'יפ של ביט?

תשובה



שאלה בהינתן זיכרון RAM, אילו אינפוטים נצטרך לתת כדי לשנות את הביט ה- x לערך z ?

תשובה $\text{in} = z, \text{address} = x, \text{load} = 1$

שאלה בהינתן זיכרון RAM, אילו אינפוטים נצטרך לתת כדי לקרוא את הביט ה- y ?

תשובה $\text{in} = (\text{doesn't matter}), \text{address} = x, \text{load} = 0$

שאלה באופן כללי שלא קשור למימוש בקורס - מהו צ'יפ counter? מה הפונקציה שהוא מבצע?

תשובה צ'יפ שמבצע את הפונקציה $\text{out}(t) = \text{out}(t - 1) + c$, לרוב עבור $c = 1$. בד"כ יהיו לא תכונות נוספות - כמו לעשות reset וכו'.

שאלה מדוע לא נרצה לתכנן צ'יפים קומבינטוריים שיש בהם feedback loop? מדוע בצ'יפים עם DFF אין את הבעיה הזו?

תשובה בצ'יפים קומבינטוריים אנחנו נקבל מצב שבו הפלט תלוי בקלט שתלוי בפלט, וזו בעיה. אבל בצ'יפים עם שעון הקלט יהיה תלוי בפלט של המחזור הקודם, וזה כבר בסדר.

שאלה כיצד נקבל את הזמן של מחזור אחד (טיק-טוק)? איזו בעיה יכולה לצוץ אם נקבע זמן מחזור קטן מדי?

תשובה בעצם אם נשלח ל-ALU את הפקודה $x + y$, סביר להניח שפיזית הם לא יגיעו שניהם באותו זמן ביחד. עד אז - אולי נקבל למשל את ה- x ועדיין לא את ה- y ונקבל ערכים שהם חסרי משמעות. לכן אנחנו נקבע זמן טיק-טוק שהוא מעט ארוך יותר מהזמן שלוקח פיזית לאינפוט הכי רחוק להגיע ליעד. נבחין בכך שבתחילת מחזור תמיד יש משמעות לפלט של למשל ה-ALU, אך באמצע מחזור יכול להיות מאוד שהוא יפלוט ערכים חסרי משמעות.

שאלה האם נוכל לבנות שבבי DFF מ-NAND?

תשובה כן! אבל לא למדנו לעשות את זה בקורס.

שאלה מהי הפונקציה של שבב ה-PC שלנו? מהו תפקידו במערכת?

תשובה ה-PC הוא ה-counter שלו, והפונקציה שלו נראית כך:

$$\text{out}(t) = \begin{cases} 0 & \text{reset}(t - 1) = \text{True} \\ \text{in}(t - 1) & \text{load}(t - 1) = \text{True} \\ \text{out}(t - 1) + 1 & \text{inc}(t - 1) = \text{True} \\ \text{out}(t - 1) & \text{Else} \end{cases}$$

מה המשמעות של זה? בעקרון זה אומר שאנחנו תומכים בכמה פעולות שונות, שיש להן סדר עדיפויות. אם עושים reset נחזור לפקודה האפס; אם עושים load נבדוק לאן אמרו לאיזה פקודה אנחנו צריכים להצביע עכשיו; אם עושים inc נוסיף אחד ואחרת לא נעשה שום דבר מעניין.

שאלה מה חשוב שנזכור בבואנו לתכנן שבב שיש בו יחסית הרבה nested-ifs, כמו ה-PC?

תשובה אם בד"כ בכתיבת קוד אנחנו נתחיל מהמקרה הראשון ונתקדם למקרה הכי nested, כאן אנחנו נתחיל עם שבב Mux בין המקרה הכי nested למקרה השני הכי nested, ולאט לאט נתקדם.

שאלה כיצד ייראה מימוש של שבב ה-PC?

תשובה

```
// noInc = out[t]
// yesInc = out[t] + 1
Inc16(in=noInc, out=yesInc);

// if (inc = 1): out = out + 1
// else out[t+1] = out[t]
Mux16(a=noInc, b=yesInc, sel=inc, out=out1);

// if(load = 1): out = in
// else: return out1 (handeld)
Mux16(a=out1, b=in, sel=load, out=out2);

// if(reset = 1): out = 0
// else: reurn out2
Mux16(a=out2, b=false, sel=reset, out=out3);

Register(in=out3, load=true, out=noInc, out=out);
```

שאלה מהו צ'יפ ה-Register?

תשובה הוא כמו צ'יפ Bit16, מכיל 16 ביטים והפונקציה שלו זהה לזאת של bit.

שאלה כיצד נבנה צ'יפ של RAM?

תשובה נביט למשל על הצ'יפ הבא של RAM8:

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address, a=load1, b=load2, c=load3, d=load4, e=load5, f=load6, g=load7, h=load8);

    Register(in=in, load=load1, out=out1);
    Register(in=in, load=load2, out=out2);
    Register(in=in, load=load3, out=out3);
    Register(in=in, load=load4, out=out4);
    Register(in=in, load=load5, out=out5);
    Register(in=in, load=load6, out=out6);
    Register(in=in, load=load7, out=out7);
    Register(in=in, load=load8, out=out8);

    Mux8Way16(a=out1, b=out2, c=out3, d=out4, e=out5, f=out6, g=out7, h=out8, sel=address, out=out);
}
```

מה בעצם עשינו כאן? התחלנו מ-Dmux, שלמעשה בוחר לנו כתובת: הוא שם את load בצ'יפ עם הכתובת שבחרנו ובכל השאר שם אפסים. אח"כ בכל רג'יסטר אנחנו שמים את ה-in הנתון לנו ואת הלואדים, ואז מחזירים שוב לפי הסלקטור של הכתובת. יש הרבה גדלים של RAM אבל למעשה כולם אותו דבר.

שאלה מהו ה-RAM של מחשב ה-Hack?

תשובה הזיכרון הכללי (data memory)

שאלה מהו ה-ROM של המחשב ה-Hack?

תשובה הזיכרון לפקודות (instructions memory)

שאלה איזה כתובת תמיד פולט ה-ROM?

תשובה ROM[PC] (כאשר PC הוא ה-counter שלנו).

שאלה מהו הגודל של מסך המחשב?

תשובה 512×256 פיקסלים, או 32×16 מילים באורך 16-ביט.

שאלה איזה צ'יפ מקביל לזה של המסך?

תשובה צ'יפ $8K$ ראם.

שאלה כיצד עובד הצ'יפ של המקלדת?

תשובה כשלוחצים על לחצן במקלדת, הפלט של קוד ה-ASCII של הלחצן הוא הפלט של צ'יפ המקלדת. אחרת הפלט הוא 0.

שאלה מהי הכתובת של המקלדת?

תשובה 24576

שאלה מהי הכתובת של המסך?

תשובה 16384-24575

שאלה באיזה גודל ה-ROM? מה המשמעות של זה לגבי התוכנה שלנו?

תשובה ה-ROM הוא בגודל $2^{15} = 32,768$. זה בעצם אומר לנו שתוכנה באורך של מעל 32,768 שורות תהיה ארוכה מדי למחשב שלנו.

שאלה איזה שורות סופר האסמלר?

תשובה רק פקודות A, C בלי לייבלים

שאלה איזה ביט מבדיל בין פקודות A לבין פקודות C ?

תשובה הביט הראשון, אם הוא 0 הוא A ואם הוא 1 הוא C

שאלה איזה ביט מחליט האם אנחנו משנים את הערך של הרג'יסטר A או M ?

תשובה הערך של הביט a : אם a הוא שקר נשנה את A , אם a הוא אמת נשנה את M

שאלה על מה מחליט הביט a ?

תשובה אם a הוא שקר נשנה את A , אם a הוא אמת נשנה את M

שאלה מהו המבנה של פקודת C ?

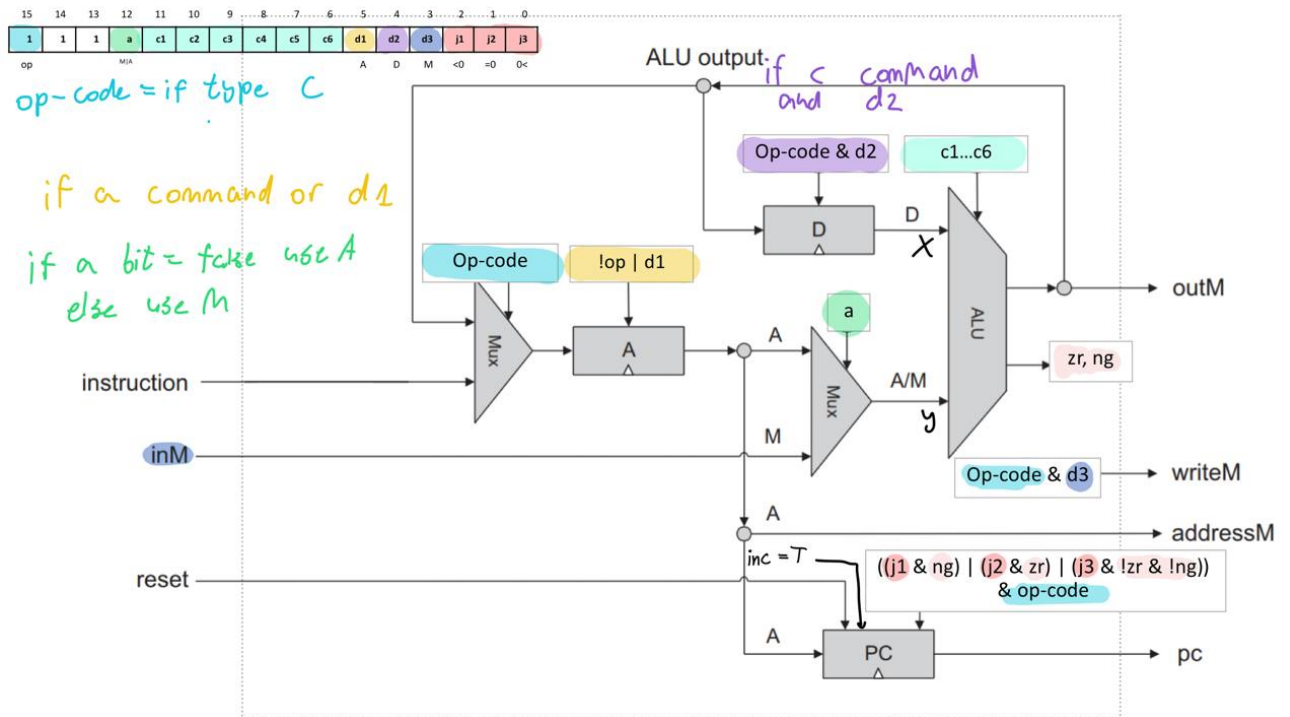
תשובה $111 - a - c_1 \dots c_6 - d_1 \dots d_3 - j_1 \dots j_3$

שאלה אילו לייבלים שמורים בשפת Hack?

תשובה

Label	RAM address	(hexa)
SP	0	0x0000
LCL	1	0x0001
ARG	2	0x0002
THIS	3	0x0003
THAT	4	0x0004
R0-R15	0-15	0x0000-f
SCREEN	16384	0x4000
KBD	24576	0x6000

שאלה כיצד נראה ה-ALU?



שאלה מהם השלבים של האסמבלר?

תשובה נפעל כך:

1. תחילה, נאתחל את טבלת הסמלים עם 23 הסמלים המוכרים לנו.
 2. אחר כך, נעבור שורה שורה ונחפש רק לייבלים. נסיף אותם לטבלת הסמלים והערכים שלהם יהיו מספר השורה שבהם הם מופיעים (אנחנו כמובן לא סופרים שורות ריקות, שורות של הערות, ושורות של לייבלים)
 3. אחר כך, נעבור מחדש על הקוד.
- (א) אם ניתקל בסמלים שמוכרים לנו - נחליף אותם בערך שמופיע בטבלה,

(ב) אחרת אם ניתקל במשתנה ששמו אינו בטבלת הסמלים - נוסיף אותו לטבלה, כאשר המשתנה הראשון יקבל את הערך 16 וכן הלאה.

שאלה מה יש ב-symbol table של האסמבלר?

תשובה לייבלים והשורות של הלייבלים, סמלי variable והמקומות בזיכרון שמתאימים להם.

שאלה אילו סוגי סמלים יש באסמבלי?

תשובה יש שלושה סוגי סמלים: לייבלים (label), סמלים מוגדרים מראש כמו LCL, וסמלים שהמשתמש מגדיר כמו @sum (שהם בעצם משתנים שאפשר לשמור).

שאלה כמה משנים מוגדרים מראש יש בשפת האק?

תשובה 23

שאלה מהו משתנה variable symbol באסמבלי?

תשובה כל משתנה @var שלא מופיע בלייבל כלשהו (כלומר לא קיים בקוד (var)).

שאלה כיצד יפעל האסמבלר עם משתני variable symbol?

תשובה ייתן להם כתובת כלשהי, כאשר הראשון יקבל את הכתובת 16 וכן הלאה (כמו משתנים סטטיים).

שאלה איך נממש בצורה יעילה ציור של פיקסל בודד על המסך?

```
/** Draws the (x,y) pixel, using the current color. */
function void drawPixel(int x, int y) {
    var int pixelBit;
    var int mask;
    var int temp;

    let temp = Screen.efficientMulitplication(y, 5); //y*32 = y*(2^5)

    let pixelBit = temp + (x/16);
    let mask = masks[x & 15]; // = masks[x % 16]

    if(current_color)
    {
        let screen_address[pixelBit] = screen_address[pixelBit] | mask;
    }
    else
    {
        let screen_address[pixelBit] = screen_address[pixelBit] & (-mask);
    }

    return;
}
```

כאשר masks הוא מערך בגודל 16 שמכיל בתא ה- x אם הערך 2^x .

שאלה איזה מהבאים נכון:

1. var bool x;
2. var boolean y;

תשובה רק השני!!!

שאלה בקוד ג'ק תקין יש את התנאי הלוגי $(y = 8) \mid (x < 7)$. כיצד נכתוב אותו ב-VM, בהנחה שהסטאק ריק, x הסטטי היחיד ב-main, ו- y הוא ה-field היחיד במחלקה גם כן?

1. `push static 0`
2. `push constant 7`
3. `lt`
4. `push this 0`
5. `push constant 8`
6. `eq`
7. `or`

שתי הנקודות הרגישות שהיו לי כאן הן:

1. אני לא צריכה להתחכם יותר מדי בשביל ה"או" - מספיק לי לדחוף את שניהם פשוט ואז לעשות או

2. דברים שמוגדרים כ-field שמורים ב-this!!!

שאלה איזה מהבאים הוא ייצוג קאנוני?

1. `(a and b) or (b and Not(c))`
2. `(a or b) and (Not(b)) and (c or d)`

תשובה רק הראשון! חשוב חשוב לזכור שזה "או" של וגמים ושרק זה נחשב לייצוג הקאנוני!

שאלה תהי `void_func` פונקציה מסוג `void`. האם הביטוי הבא חוקי?

```
let x = Main.void_func();
```

תשובה לא! כי זה אסור לפי מבחן 2018 מועד ב', סיבה - כי פונקציית `void` אינה `expression` ואי אפשר לקרוא לה כך.

שימו לב! אני די בטוחה שהם מורידים נקודות לאנשים שסתם כותבים `Class Decleration` במקום `classVarDec` ולכן השאלות הדביליות הללו:

שאלה יש לי מספר, איזה אלמנט מילוני הוא?

תשובה `integerConstant`

שאלה יש לי סטרינג, איזה אלמנט מילוני זה?

תשובה `StringConstant`

שאלה יש לי הכרזה על מחלקה, איזה אלמנט מילוני זה?

תשובה `classVarDec`

שאלה אני מכריזה על `subroutine`, איזה אלמנט מילוני זה?

תשובה `subroutineDec`

שאלה איזה מהבאים אינו נכון לפי הספר? (כן זה קטנוני, אני יודעת)

1. `letStatement`
2. `ifStatement`
3. `whileStatement`
4. `doStatement`
5. `returnStatement`

תשובה `returnStatement` אינו נכון - כי כוונתם מתחילים באותיות קטנות חוץ מ-`return` שאוהב להרגיש מיוחד ומתחיל באות גדולה, ולכן הוא `ReturnStatement`.

שאלה מהו `KeywordConstant`?

תשובה אחד הבאים:

1. `true`
2. `false`
3. `null`
4. `this`

שאלה איך מחשבים את $D - B$ ב-ALU?

תשובה

$$zx = 0, nx = 0, zy = 1, ny = 1, f = 1, no = 1$$

שאלה כיצד נגדיר קונסטרוקטור של מחלקה לדוגמה?

```
constructor Fraction new(int a, int b) {  
    let numerator = a; let denominator = b;  
    do reduce(); // If a/b is not reduced, reduce it  
    return this;  
}
```

שאלה איזה תגובות תקניות יש בג'ק?

```
// Comment to end of line  
/* Comment until closing */  
/** API documentation comment */
```

שאלה איזה טיפים פרימיטיביים יש בג'ק?

`int, boolean, char, void`

Primitive types

שאלה מבחינת Jack - האם הביטוי 1 – הוא Integer או Expression?

תשובה Expression הוא Expression! 1 הוא מספר ומינוס אחד הוא expression.

שאלה איזה תווים אי אפשר לשים בביטוי "..."? כיצד נפתור את הבעיה?

תשובה התווים double-quotes ו-newline, ולכן נוכל להוסיף את התווים הללו ע"י הפונקציה String.newLine() ו-String.doubleQuote().

שאלה האם יש סוג אובייקט שקונסטקטור חייב להחזיר?

תשובה כן! הם חייבים להחזיר את סוג האובייקט של המחלקה.

שאלה מתי מוקצים משתנים פרימטיביים? היכן הם מוקצים?

תשובה הם מוקצים כאשר הם מוכרזים - והקומפיילר עושה להם אלוקציה:

Variables of primitive types are allocated to memory when they are declared. For example, the declarations `var int age;` `var boolean gender;` cause the compiler to create the variables `age` and `gender` and to allocate memory space to them.

שאלה מתי מוקצה הזיכרון של אובייקטים?

תשובה כשקוראים לקונסטקטור - עד אז נוצר לנו רק פויינטר לאובייקט. מכיוון שמערך הוא סוג של אובייקט - זה אותו דבר עם מערכים.

שאלה האם אפשר לשים במערך מספר ואז סטרינג?

תשובה כן - למערכים אין סוג.

שאלה מה ההבדל בין `char` ל-`int`?

תשובה מבחינתנו אין ממש הבדל כזה, כמו בדוגמה כאן:

```
var char c; var String s;
let c = 33; // 'A'
// Equivalently:
let s = "A"; let c = s.charAt(0);
```

שאלה האם נוכל להתייחס לאובייקט בתור מערך?

תשובה כן! וכל קומפיילר חייב לתמוך בפעולה זו! הנה דוגמה מהספר

- An object variable (whose type is a class name) may be converted into an Array variable, and vice versa. The conversion allows accessing the object fields as array entries, and vice versa. Example:

```
// Assume that class Complex has two int fields: re and im.
var Complex c; var Array a;
let a = Array.new(2);
let a[0] = 7; let a[1] = 8;
let c = a; // c==Complex(7,8)
```

שאלה האם פונקציה יכולה לגשת למשתנה שהוא Field?

תשובה לא!

שאלה מבחינת סינטקס - מה יכול לסיים את השורה הבאה?

```
let variable = syntax_type;
```

תשובה רק Expression!

```
let variable = expression;
```

שאלה מבחינת סינטקס - מה יכול לסיים את השורה הבאה?

```
let variable[syntax_type_1] = syntax_type_2;
```

תשובה רק Expression בשניהם!

```
let variable[expression] = expression;
```

שאלה מבחינת סינטקס - מה יכול להשלים את השורות הבאות?

```
1. if (syntax){  
    (a) syntax  
2. }  
3. else{  
    (a) syntax  
4. }
```

תשובה

```
1. if (expression){  
    (a) statements  
2. }  
3. else{  
    (a) statements  
4. }
```

שאלה מבחינת סינטקס - מה יכול להשלים את השורות הבאות?

```
1. while (syntax){  
    (a) syntax  
2. }
```

תשובה

```
1. while (expression){
```


(a) statements

2. }

שאלה מבחינת סינטקס - מה יכול לסיים את השורה הבאה?

do syntax;

תשובה

do function-or-method-call;

שאלה איך נראית פקודת החזרה תקינה?

Return *expression*;
or
return;

הערה לא תואם את פרק 10 ועד כמה שידוע לי **return** תמיד צריך להתחיל באות קטנה.

שאלה מבחינת ג'ק - מהו *expression*? (נשים לב שבערך בכל סוג *statement* יש *expression* והם בודקים במבחנים נקודות מאוד מאוד עדינות כאן)

תשובה הוא צריך להיות אחד מהבאים:

1. קבוע
2. שם משתנה
3. *this*
4. *array[expression]*
5. קריאה ל-subroutine שלא *void*
6. ביטוי שמתחיל במינוס או ב-*~*, כלומר *expression*-ו-*expression* *~*
7. ביטוי מהצורה *expression operator expression* עם אחד האופרטורים הבאים: *= < > | & * / - +* (כאשר השווה הוא להשוואה ולא להשמה)
8. ביטוי בתוך סוגריים, כלומר (*expression*)

שאלה יש לי פונקציה או מתודה ללא פרמטרים, האם אוכל לקרוא לה בלי הסוגריים שלה?

תשובה לא!

שאלה איזה סוג סינטקס הוא ארגומנט?

תשובה *expression*

שאלה האם הפקודה הבאה חוקית?

let *x* = **Math.sqrt**(*a****Math.sqrt**(*c*-17)+3);

תשובה כן! בהנחה כמובן שכל המשתנים מוגדרים. אפשר להכניס לפונקציה בתור ארגומנט ביטויים מאוד מורכבים אם רוצים.

שאלה אני בתוך מחלקה עם המתודה *my_method*. האם מותר לי לבצע קריאה *my_method(arg)*?

תשובה כן! כלומר אין צורך בתוך מחלקה לבצע *my_class.my_method(arg)*.

שאלה אני בתוך מחלקה עם המתודה my_func. האם מותר לי לבצע קריאה my_func(arg)?

תשובה לא! אני חייבת לכתוב my_class.my_func(arg).

שאלה כיצד נקרא למתודה מחוץ למחלקה?

תשובה object_name.method(arg), כאשר object_name הוא אובייקט מהסוג של המחלקה.

שאלה כיצד נקרא לפונקציה מחוץ למחלקה?

תשובה class_name.function(arg)

שאלה כיצד נקרא לקונסטרקטור מחוץ למחלקה?

תשובה class_name.constructor(arg)

שאלה כיצד נקרא למתודה בתוך המחלקה?

תשובה method(arg)

שאלה כיצד נקרא לפונקציה בתוך המחלקה?

תשובה .class_name.function(arg)

שאלה איזה מחלקות חייבות קונסטרקטור?

תשובה מחלקות שמגדירות לנו type חדש.

שאלה האם dispose זו מתודה או פונקציה?

תשובה מתודה! הרי פונקציה לא מתעסקת עם instance-ים של אובייקטים!

שאלה מה קורה כשקוראים לקונסטרקטור?

תשובה הדברים הבאים:

1. קומפילר מבקש ממערכת המהפעלה אלו קציה לאובייקט חדש

2. מערכת ההפעלה מחזיקה את כתובת הבסיס של הסגמנט

3. הקומפילר שם את כתובת הבסיס ב-this

4. לרוב קונסטרקטור שעושה את העבודה שלו מאתחל את הערכים

שאלה האם הפקודה הבאה חוקית?

```
return void_func();
```

תשובה לא! כי returnStatement צריך להיות בצורה return expression; אך פונקציית void אינה expression!

שאלה מה נוכל לעשות כדי שפונקציה תוכל לגשת למשתנים הפנימיים של אובייקט כלשהו?

תשובה נוכל לשלוח לה את this בתור ארגומנט, ואחר כך נוכל להתייחס ל-this בתור בסיס של מערך ולהשפיע על המשתנים הפנימיים.

שאלה מה תעשה שורת הקוד הבאה?

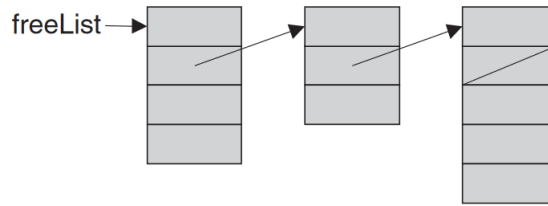
```
return Output.printInt((a=a));
```

תשובה תדפיס -1, כי $a = a$ זה אמת ואמת זה -1.

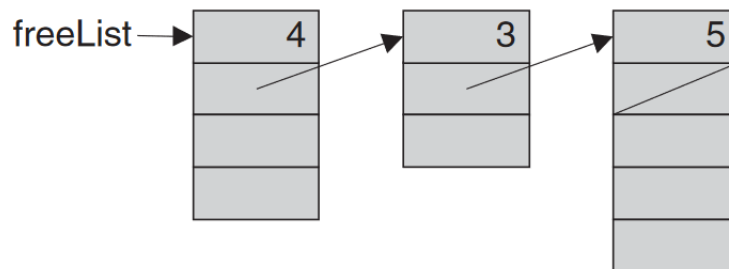
שאלה מהו הערך של true? מהו הערך של false?

תשובה 1 – ואפס בהתאמה

שאלה מהו הגודל של הסגמנטים הבאים?



תשובה חשוב לשים לב שגודל הסגמנט כולל את המקום שבו כתוב גודל הסגמנט!



שאלה האם הביטוי הבא חוקי:

`let c = "A";`

תשובה לא יודעת - לבדוק!!

שאלה נניח שנשנה את המיקום של הסטאק, היכן נצטרך לבצע שינויים בקוד?

תשובה ב-VMTranslator - כי הוא מאתחל את SP אל 256!!

שאלה האם הפקודה הבאה חוקית? `do x[j].func();`

תשובה לא, מכמה סיבות:

1. קודם כל מבחינת סינטקס - מתודות לא יכולות להיקרא על תאים במערך

2. נניח שזה היה נכון מבחינת סינטקס - אז בטבלת הסמלים func הייתה פונקציה של המחלקה Array, אז בעצם היינו קוראים ל-Array.func() שלא קיימת ושוב היינו מגיעים לשגיאה.

שאלה מהי הכתובת של המקלדת? היכן מוגדרת הכתובת?

תשובה 24576, היא מוגדרת כסמל KBD באסמבלר

שאלה מהי הכתובת של המסך? היכן מוגדרת הכתובת?

תשובה 16384-24575, היא מוגדרת כסמל SCREEN באסמבלר

שאלה מהי הכתובת של ה-Heap? היכן מוגדרת הכתובת?

תשובה 16383-2048, מוגדר במחלקה Memory בתרגיל 12

שאלה אילו דברים בזיכרון מוגדרים מראש במחלקה Memory.jack?

תשובה ה-heapBase=2048 וגם אורך ה-RAM

שאלה מהי הכתובת של ה-Stack? היכן מוגדרת הכתובת?

תשובה 256-2047, מוגדר כbootstrap ב-VMTranslator עושים $SP = 256$

שאלה מהי הכתובת של המשתנים הסטטיים? היכן מוגדרת הכתובת?

תשובה 16-255, מוגדר באסמבלר - שם כל המשתנים הסטטיים מקבלים כתובת הכל מ-16 ואילך.

נקודות ממבחינים

1. אם שמתי בג'ק מערך - לעשות לו dispose!
2. לזכור שצריך בדיקה לאוברפלואו בחלק מהאלגוריתמים!
3. כשאומרים ייצוג קאנוני - הכוונה היא ל"או" של "וגמים"
4. בשאלה האחרונה של האלגוריתם - תמיד לשאול את עצמי מה אני יכולה לעשות כדי לשפר יעילות.

קצת טריקים של טבלת האמת של ב-ALU

(when a=0) comp mnemonic	c1	c2	c3	c4	c5	c6	(when a=1) comp mnemonic
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

רוב הדברים כאן הם פעולות של חיבור, and, והצבה של אפסים ואחדים שזה די בנאלי. כדי להגיע לאופרטור or פשוט צריך לזכור ש- $x \vee y = (\neg x) \wedge (\neg y)$. האופרטור היחיד שהוא קצת טריקי זה המינוס, אבל בעצם יש כאן רק טריק אחד. נניח שאנחנו מחשבים את $-D$, נקבל שצריך את האופרטורים:

$$zx = 0, nx = 0, zy = 1, ny = 1, f = 1, no = 1$$

ואם אנחנו זוכרים את הטריק היחיד הזה, נוכל גם לדעת מהם הביטים של $A - D$ - פשוט במקום לאפס את A (שזה y מבחינת ה-ALU), לא נאפס אותו:

$$zx = 0, nx = 0, zy = 0, ny = 1, f = 1, no = 1$$

ואם נרצה לחשב דווקא את $D - A$ פשוט "נהפוך" בין ה- x לבין ה- y ונקבל:

$$zx = 0, nx = 1, zy = 0, ny = 0, f = 1, no = 1$$

הסבר על ההחזרה של פונקציה

return (from a function)	<pre>FRAME = LCL RET = *(FRAME-5) *ARG = pop() SP = ARG+1 THAT = *(FRAME-1) THIS = *(FRAME-2) ARG = *(FRAME-3) LCL = *(FRAME-4) goto RET</pre>
------------------------------------	--

קודם נשאל את עצמנו - מה המטרות שלנו? מה אנחנו רוצים לעשות כאן בכלל? אז ככה:

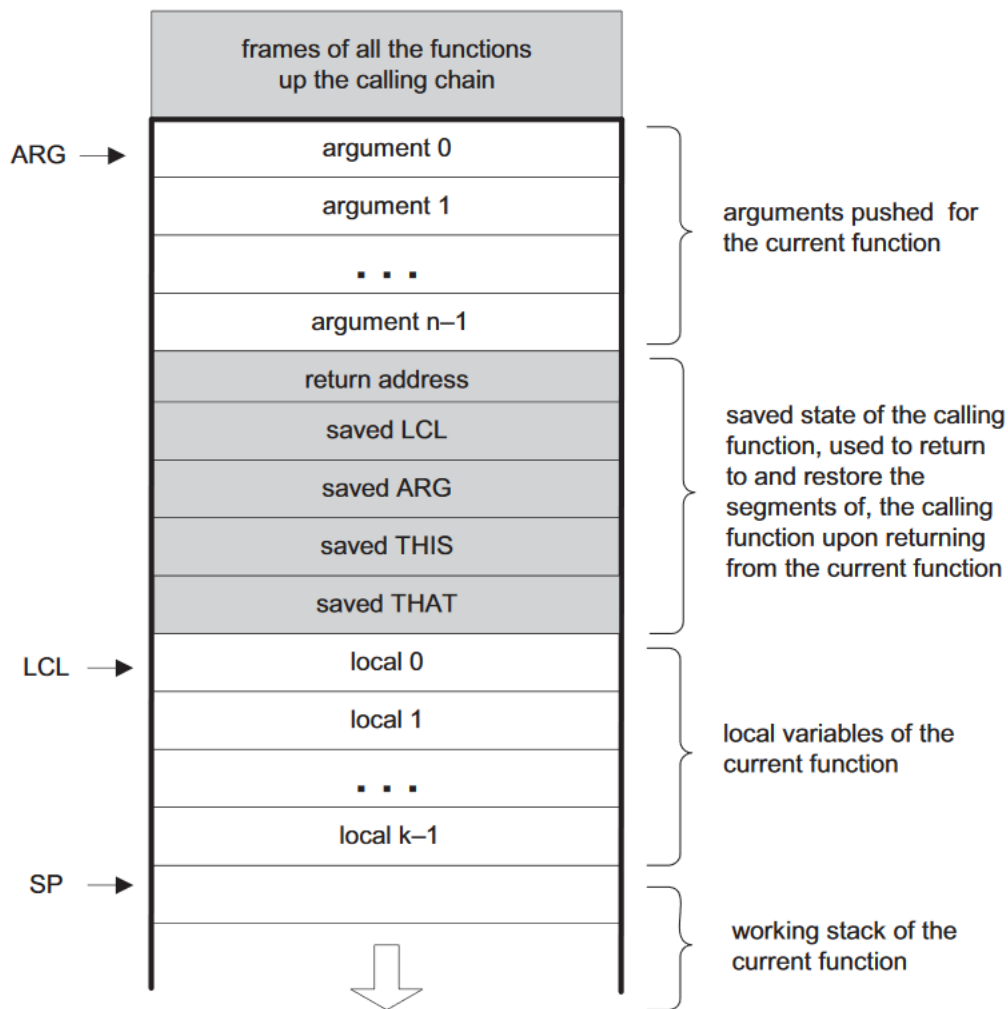
1. אנחנו רוצים לחזור למקום הקודם שהיינו בו לפני הקריאה לפונקציה
2. אנחנו רוצים שכל הפוינטרים שיש לנו יחזרו למקום הקודם שלהם
3. אנחנו רוצים שבמקום אחד מתחת ל-SP יהיה הערך המוחזר של הפונקציה
4. אנחנו רוצים "לסמן" למחשב שכבר לא אכפת לנו מכל הערכים הלוקאליים והארגומנטים של הפונקציה שסיימנו איתה

מה בעצם קורה כאן?

1. קודם אנחנו עושים $LCL = FRAME$, כי אם נראה בצירוף LCL מצביע על בדיוק אחרי הערכים ששמרנו לעצמנו להמשך
2. אחר כך אנחנו לוקחים קודם כל את המקום שאליו אנחנו חוזרים - RET
3. אנחנו עושים $*(ARG) = pop()$, כלומר עכשיו ARG מצביע על הכתובת שיש ב- $SP-1$, שהיא הכתובת של ערך ההחזרה
4. עכשיו נעשה $SP = ARG + 1$, כלומר עכשיו SP מצביע על מקום אחד אחרי ARG . זאת אומרת שמבחינתנו, ברגע שנסיים את רצף הפעולות, נוכל לשים ערכים חדשים החל מהמקום SP ואילך.
5. לפני שנשים ערכי זבל - אנחנו נשחזר את כל הפוינטרים שהיינו בהם לפני הקריאה
6. נחזור ל- RET

נשים לב לחשיבות של הסדר:

1. למה אנחנו קודם כל שומרים את RET ? כי במקרה שבו אין ארגומנטים, ARG מצביע בדיוק על הכתובת החזרה, ואז בשורה הבאה אנחנו עלולים להרוס את הכל.
2. מה היה קורה אם היינו משנים את SP רק בסוף? הקטע הוא כזה - אם היינו משנים את הפוינטר של הארגומנט אחרי שכבר החזרנו אותו להיות הפוינטר של הארגומנט לפני הקריאה לפונקציה אז היינו עושים כאן בלאגן ומחרבשים את הכל



הטו דו ליסט שלי

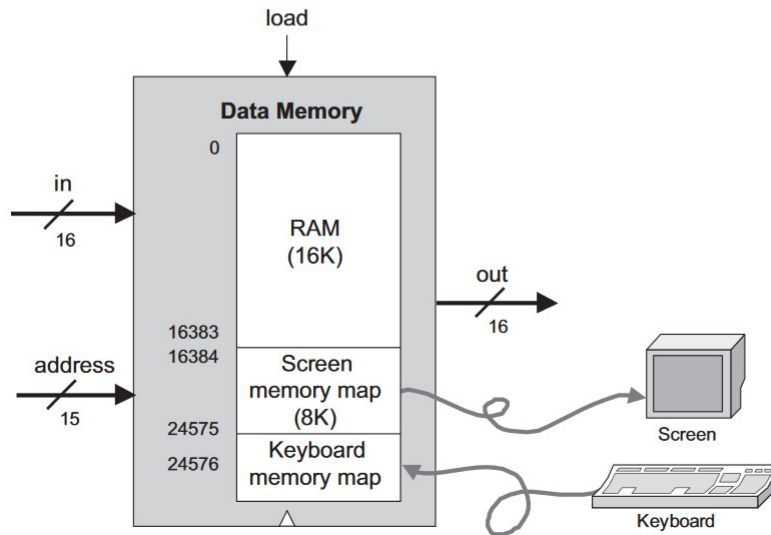
1. להבין את הקטע עם המשתנה הסטטי בחילוק
2. כתובות של מסך ומקלדת, איך משנים משהו ספציפי במסך...
3. האם הקובץ של המחלקה point ב-jack חייב להיקרא point,jack?
4. מתי אנחנו הולכים ב-goto – if ומתי לא?
5. שאלות כמו 2018 מועד ב' עם חילוק של מספרים בינאריים - להבין איך אני אמורה לפתור את זה!
6. לוודא את הקטע של הכתובות שלא קונסיסטנטי אצלי בספר
7. האם מותר לי לעשות פונקציה ריקה לגמרי ללא ריטורן?
8. void נחשב לפרימיטיבי - מתי וכיצד נוכל לעשות בו שימוש למעט בפונקציות?

אי תאימויות בספר

אי תאימויות בכתובות של דברים:

RAM addresses	Usage
0–15	Sixteen virtual registers, usage described below
16–255	Static variables (of all the VM functions in the VM program)
256–2047	Stack
2048–16483	Heap (used to store objects and arrays)
16384–24575	Memory mapped I/O

Label	RAM address	(hexa)
SP	0	0x0000
LCL	1	0x0001
ARG	2	0x0002
THIS	3	0x0003
THAT	4	0x0004
R0–R15	0-15	0x0000-f
SCREEN	16384	0x4000
KBD	24576	0x6000



אי תאימויות לגבי return:

ReturnStatement 'return' expression? ';' ;

Return expression;

or

return;

למידה למבחן הארור בנאנד

סוכס ע"י תמר פאר

מייל להערות: tamar.peer@mail.huji.ac.il

עוד ריכוז של טריקים בנאנד

מסקנות כלליות חשובות:

- גם אם נראה לכם שהבנתם את העיקרון של הלולאה, גם אם נראה לכם שזיהיתם תבנית, גם אם נראה לכם שהלולאה נגמרה ואין מה להמשיך להריץ את הקוד את המעט שורות שנשארו - **תהיו יסודיים**, תריצו עד הסוף, יש מצב שתגלו שזה בדיוק המלכודת שטמנו בשאלה הזאת.

- אמת ושקר ($True/False$)

- בכל השפות, $False = 0$

- אם יש בשפה מספרים שליליים, אז: $True = -1$ (לדוגמא ג'ק) ואם אין אז $True = 1$ (לדוגמא בצ'יפים)

- $Not(-1) = 0$ ולהיפך $Not(0) = -1$

* זה בעצם אומר שלא אמת שווה לשקר ולא שקר שווה אמת.

- שיפט ימינה:

- בייצוג בינארי הוא מזיז את כל הביטים ימינה, שם 0 איפה שנותר "ריק" (אבל שומר על ביט הסימן, מספר שלילי נשאר שלילי וחיובי נשאר חיובי)

- בייצוג מספרי הוא בעצם מחלק את המספר ב2, בלי שארית (כמו $//$ בפייתון, $10//2 = 5$ וגם $11//2 = 5$)

* כשהמספר שלילי, הוא **יעגל למטה** (אבל ללמטה ה"לא נכון" - $11\#$ יהיה שווה ל-6 ולא -5)

- באסמבלי: $>>$, VM: $shiftright$, ב-JACK: $\#$

- שימושי לעשות חילוק בלי להשתמש בפעולת החילוק באמת

- שיפט שמאלה:

- בייצוג בינארי הוא מזיז את כל הביטים שמאלה, שם 0 איפה שנותר "ריק"

- בייצוג מספרי הוא בעצם מכפיל את המספר ב2

- באסמבלי: $<<$, VM: $shiftright$, ב-JACK: $\wedge$

- שימושי לעשות כפל בלי להשתמש בפעולת הכפל באמת

- לא שומר על הסימן!

עוד מסקנות כלליות:

- הפוינטר לסטאק לא מצביע לערך העליון ביותר בסטאק, הוא מצביע למקום הפנוי הבא בסטאק
- שערים שמחוברים לשעון:
 - PC, CPU, DFF, RAM64
 - DFF הוא השעון עצמו. נשים לב ש ALU ו Full Adder לא.
 - צ'יט למבחן: זה שקול לזה שהצ'יפ מקבל את הארגומנט load או משתמש ברכיב שמקבל אותו.
- “כמה פקודות בשפת האק יתקבלו מקוד האסמבלי הבא” שקול לשאלה “כמה שורות מקוד האסמבלי באמת צריך לתרגם להאק” (שורה של הכרזה על תוית (לא טעינה שלה) - לא מתרגמים, כנ”ל הערות)
- מסך בזיכרון:
 - המסך מורכב מ8192 רגיסטרים, שכל אחד מהם מחזיק 16 ביטים וכל ביט מייצג פיקסל במסך. המסך בגודל $256 * 512$ פיקסלים בסך הכל
 - בכל שורה במסך יש 32 באסים (שמוחזקים ב32 רגיסטרים) שמייצגים כל אחד 16 ביטים.
 - יש סך הכל 256 שורות במסך.

פישוט ביטויים לוגיים:

- חוקי דה מורגן (חיזוק לקונספירציה שטוענת לקשר בין נועה ניצן לנועם ניסן):

$$\text{Not } (A \text{ and } B) \Leftrightarrow \text{Not } A \text{ or Not } B -$$

* “מכניסים את השלילה פנימה”, אז שני המשתנים נהפכים לשלילה, ושלילה של and נתנה לנו or

$$\text{Not } (A \text{ or } B) \Leftrightarrow \text{Not } A \text{ and Not } B -$$

* “מכניסים את השלילה פנימה”, אז שני המשתנים נהפכים לשלילה, ושלילה של or נתנה לנו and

- חוקי פילוג:

$$A \text{ and } (B \text{ or } C) \Leftrightarrow (A \text{ and } B) \text{ or } (A \text{ and } C) -$$

* ממש כמו חוק פילוג של מספרים, כש and זה כפל or זה חילוק $(A \cdot (B + C) = A \cdot B + A \cdot C)$

$$A \text{ or } (B \text{ and } C) \Leftrightarrow (A \text{ or } B) \text{ and } (A \text{ or } C) -$$

* לזה אין מקביל במתמטיקה שאני מכירה:)

- זהויות:

$$A \text{ and } (\text{Not } A) = 0 \text{ (משהו וגם השלילה של עצמו זה 0)} -$$

$$A \text{ or } (\text{Not } A) = 1 \text{ (משהו או השלילה של עצמו זה 1)} -$$

$$A \text{ and } A \Leftrightarrow A \text{ or } A \Leftrightarrow A \text{ (משהו וגם עצמו, שקול למשהו או עצמו, שקול למשהו עצמו)} -$$

$$A \text{ xor } B \Leftrightarrow (A \text{ and } (\text{Not } B)) \text{ or } ((\text{Not } A) \text{ and } B) -$$

$$\text{Not}(A \text{ xor } B) \Leftrightarrow (A \text{ and } B) \text{ or } ((\text{Not } A) \text{ and } (\text{Not } B)) -$$

$$\text{Nand}(A,B) = \text{Not}(A \text{ and } B) -$$

- טריק נחמד לגבי Xor:

– לכל A,B,C מתקיים:

$$A \text{ xor } B = C$$

$$\Leftrightarrow$$

$$A \text{ xor } C = B$$

$$\Leftrightarrow$$

$$B \text{ xor } C = A$$

- טריק לגבי Not על מספרים “אמיתיים”:

$$\text{Not } A = -A - 1 \text{ וזה גם שקול ל: } \text{Not}(A - 1) = -A$$

• איך יוצרים ביטוי בוליאני מטבלת אמת?

– עבור כל שורה בטבלה שהפלט שלה הוא 1, ניצור ביטוי בוליאני בצורה הבאה:

* קלט ששווה 1 באותה השורה, נשאר

* קלט שווה 0 באותה השורה, נשלול (נעשה Not)

* נשים And בין כל הקלטים (שאת חלקם שללנו ואת חלקם לא)

– לוקחים את כל הביטויים שקיבלנו (כמות הביטויים שווה לכמות הפעמים שהטבלה מוציאה 1), עושים ביניהם
Or.

– מפשטים אם יש כח.

שיט של ציפים:

- הביטוי הלוגי ל-Mux הוא: $(A \text{ and } (\text{Not}(sel))) \text{ or } (B \text{ and } sel)$, ובכמתים: $(A \cdot \bar{sel}) + (B \cdot sel)$
- תכל'ס, בגלל ששוללים את sel ועושים פעם אחת and איתו ופעם אחת עם השלילה, לא נקבל אף פעם $1 + 1$, אז אפשר לשים שם Xor במקום or.
- כל שער שניתן ליצור משימוש בו (ורק בו, בלי *true* או *false*, אבל בו כמה פעמים שנרצה) את *Not* ואת *And* או *Or* - נקרא "שער אוניברסלי" וניתן לבנות ממנו את כל השערים.
- *Mux* ו-*Dmux* הם אוניברסליים? לא! הם כמעט אוניברסליים, כי נצטרך את *true* ו-*false*.
- נעשה *Not* ב-Mux: $Mux(a = true, b = false, sel = A, out = NotA)$
- נעשה *And* ב-Mux: $Mux(a = A, b = 0, sel = NotB, out = AandB)$

שיט של ניהול זיכרון:

- ניהול זיכרון כללי - דרך טובה לזכור איפה המסך והמקלדת: מהדף נוסחאות, 2 בחזקת 14 זה תחילת המסך (16384) ו-2 בחזקת 13 (8192) זה הגודל של המסך, אם נחבר את שניהם, נקבל 24576, ששם נגמר המסך ונמצאת המקלדת!
- *heap* נמצא ישר אחרי הסטאק (שנגמר ב-2047) ונמשך עד תחילת המסך
- מקום 3 הוא 0 pointer, שמכיל את הכתובת שבו מתחיל הסגמנט This
- מקום 4 הוא 1 pointer שמכיל את הכתובת שבו מתחיל הסגמנט That
- מקום 5 הוא תחילת הסגמנט Temp, כלומר 0 temp.
- מסקנה נכלולית: $pointer\ 2 = temp\ 0$.

שיט של אסמבלי:

- הפקודה $A = -1$ היא חוקית ולא תגרום שגיאה, שגיאה תיגרם רק כשננסה לגשת ל M כש $A = -1$.
- מימוש יעיל ללהוציא ערך מהסטאק (pop): נשתמש ב $AM = M - 1$ כדי בו זמנית להזיז את הפוינטר של הסטאק וגם להשים ב M את הערך האחרון שהיה בסטאק.

– עוד דרך לייעל את המימוש, כשנוציא אל מיקום בתוך סגמנט כמו *this* או *that*:

- * נשמור ב D את הכתובת אליה נרצה להגיע
- * ניגש לערימה ויהיה לנו ב M את הערך שנרצה לשים
- * נעשה $D = D + M$, עכשיו אמנם יש לנו שם ערך זבל, אבל אנחנו יודעים ממה הוא מורכב!
- * נעשה $A = D - M$, שזה בעצם טוען לנו את הכתובת שרצינו אל תוך A
- * ואז נעשה $M = D - A$, כדי לשים שם את המספר שרצינו! שהוא הערך זבל פחות הכתובת! ייא!

- דריכת דברים שהם לא מספרים לרגיסטר A :

– $@LOOP$ ידרוך לרגיסטר A את מספר השורה שבה הוצהר ($LOOP$)

- $@x$ כאשר x הוא משתנה חדש כלשהו, x יקבל את הכתובת 16 (אם הוא המשתנה הראשון שהוכרז בתכנית) והלאה (17 למשתנה השני, 18 לשלישי וכו')

- *באסמבלי $R5 = temp\ 0$

- אפשר לעשות באסמבלי: $@A$. אי אפשר לעשות באסמבלי: $@D$ ו $@M$ והפקודות האלה בעצם יתורגמו כאילו D (או M) זה משתנה חדש שהוצהר (יישמר ביחד עם המשתנים ה"רגילים" החל מ16)
- אם לא נותנים את המשתנים, נסמן x ו y (זו אם צריך עוד אותיות). אם לנו תבנית שבה x ו y מאוחסנים, ואז ניגשים מתישהו לערך של התא x או התא $x + something$, אז מדובר בעצם בייצוג של מערך שאנחנו עוברים עליו ועושים לו משהו (והוא בדרך כלל בגודל y).

שיט של VM:

- בשפת VM, הפקודה `if-goto` "מסתכלת" על הפריט העליון ביותר במחסנית, וקופצת לפיו. היא גם **מסירה אותו מהמחסנית עצמה**
- פקודות `push` ב VM שנעשות לפני פקודת `Call` לפונקציה - מה ששמנו על הסטאק זה הארגומנטים של הפונקציה.
- דבר שיעשו לאחר מכן (שזה טכנית תחת ההכרזה על הפונקציה ולא בקריאה לה) ולא כתוב בקוד עצמו: ידחפו לערימה אפסים כמספר ערכי `Local` שהפונקציה שקראנו לה צריכה להשתמש בהם (בעצם כערך דיפולטיבי למשתנים הלוקאלים, עד שישנו אותם)

שיט של ג'ק:

- אם יש לנו אובייקט, שמאוחסן בתוך איבר במערך, לא נוכל לקרוא למתודות של האובייקט כך: `arr[i].method`, כי ג'ק יתייחס ל`method` כמתודה של מערכים, ואם השם שלה לא קיים - הוא יתקע (ואם הוא קיים אבל עושה משהו אחר - באסה לנו)

– באותו הנושא, אם יש לנו מתודה שהופעלה על x , למרות שהשפה היא `weakly-typed`, מה שיופעל זה המתודה כפי שהיא הוגדרה על האובייקט שלפני הנקודה.

- אפשר שב`field` של מחלקה יהיה אובייקט מסוג המחלקה עצמה

- אפשר לעשות `Output.println()`

- בכל תיקייה שבה קבצי ג'ק, חייב להיות קובץ בשם `Main` ובו פונקציה בשם `main`.

- משתני `field` של אובייקט כלשהו יכולים להשתנות גם אם לא ניגש אליהם ישירות:

– האובייקט הוא בעצם פוינטר לכתובת שלו בזיכרון, ומשם נתחיל לספור את משתני ה`field` שלו (אם הוא מצביע ל1000, אז `field 0` (המשתנה אובייקט הראשון) מאוחסן בכתובת 1000, `field 1` (המשתנה אובייקט השני) מאוחסן בכתובת 1001 וכך הלאה.

– אז אם ניגשתי איכשהו לכתובת 1001 ושיניתי את מה שיש שם, שיניתי את `field 1` של האובייקט שהכתובת שלו מתחילה ב1000, גם בלי גישה ישירה.

פקודות C שהן 16 ביטים ואיך מתרגמים אותן לאסמבלי:

111 $ac_1c_2c_3c_4c_5c_6 \quad d_1d_2d_3 \quad j_1j_2j_3$ נראות ככה:

ומתורגמות לפקודות אסמבלי מהצורה הבאה: $dest = comp; jump$

הביטים שבאות d נותנים לנו את $dest$ (וזה מופיע בצ'יט שיט)

הביטים שבאות j נותנים לנו את $jump$ (גם מופיע בצ'יט שיט)

והביטים a ו- c_1 נותנים לנו את $comp$, שבו אפשר ליצור פקודות שנצטרך לדעת לתרגם והן לא יופיעו בצ'יט שיט איך נתרגם?

דבר ראשון, הביט a , מציין אם הפעולה תתרחש על A ($a = 0$) או על M ($a = 1$).

ששת הביטים $c_1...c_6$, מוקבלים ל-6 הפרמטרים שמקבל ה- ALU : zx, nx, zy, ny, f, no .

נחשוב על ה- D תמיד בתור x שלנו, ו- y שלנו יהיה A או M , בהתאם לביט a .

zx אומר לנו "האם צריך לאפס את x ?" אם התשובה היא כן, זה אומר שלא נשתמש ב- D בחישוב שלנו בעצם.

nx אומר לנו "האם צריך לשלול את x ?" אם התשובה היא כן, נוסף NOT (או $!$) (זה $bit - wise NOT$). אם גם איפסנו וגם נצטרך לשלול, אז נקבל 0! וזה שווה ל-1-

zy ו- ny שואלים אותו דבר על y (A או M)

קיבלנו עכשיו שני משתנים, נעבור לביט הבא:

f , שהוא c_5 , אומר לנו "מהי הפעולה שצריך לעשות ביניהם?" כאשר אם $f == 1$, אז הפעולה היא חיבור (רגיל, בין מספרים) ואם $f == 0$, אז הפעולה היא AND ($\&$), ואותו נבצע $bit - wise$.

הביט האחרון הוא no , שאם הוא שווה ל-0 אז הוא לא מפריע לנו בחישוב, ואם הוא שווה ל-1, אז הוא אומר לנו לשלול (NOT , $!$) (זה $bit - wise NOT$) את מה שקיבלנו עד כה, והתוצאה הסופית היא מה שיצא.

פקודות C שהן 16 בעצם $shiftright$ או $shiftright$:

שלושת הביטים השמאליים ביותר שהם תמיד 111, עכשיו יהיו 101 - נדע שמדובר בפקודת $shift$ כלשהי.

נשאר להבין לאן מזיזים ואת מי מזיזים.

לאן:

נסתכל על c_1 , אם $c_1 = 0$, אז נזיז ימינה $<<$ (ברירת המחדל היא ימין)

אם $c_1 = 1$, נזיז שמאלה $>>$

את מי:

נסתכל על הביט a , אם $a = 1$, קל, מזיזים את M .

אם $a = 0$, זה אומר שזה A או D , איך נדע? נסתכל על c_2 :

אם $c_2 = 0$ אז נזיז את A (הוא הברירת מחדל, כי הוא הרגיסטר ה"יותר חשוב")

אם $c_2 = 1$ אז נזיז את D .

שאר הביטים של c , כלומר c_3, c_4, c_5, c_6 יהיו שווים ל-0.

טריקים של שפת ג'ק *JACK*:

- אם יש לנו מספר, שכתוב כסטרינג, נגיד "5", ונרצה להמיר ל5, נעשה: $48 - 5$ "זה ייתן לנו 5 (המספר) (ראינו בסי טריק דומה אבל עם מספר אחר)

טריקים בינאריים:

- ייצוג בינארי של מספר שהוא חזקה של 2 פחות 1, יהיה רק אחדות (לדוגמא, 3 זה $4 - 1$, 4 הוא חזקה של 2, אז הייצוג הבינארי של 3 הוא 11)
- אם נרצה לפרק לביטים מספר בייצוג רגיל, נעשה and עליו ועל הייצוג המספרי של מספר הביטים שנרצה "להעתיק" – לדוגמא, אם נרצה רק את הביטים הזוגיים של מספר רגיל בייצוג בינארי של 16 ביטים, נעשה $A \text{ and } (-21846)$. למה?
 כי רצף המספרים הבאים: 1010101010101010 בייצוג בינארי, יוצאים המספר 21846 –
- אם נרצה רק את הביטים האי זוגיים של מספר רגיל בייצוג בינארי של 16 ביטים, נעשה $A \text{ and } (21845)$. למה?
 כי רצף המספרים הבאים: 0101010101010101 בייצוג בינארי, יוצאים המספר 21845
- בדיקת שפיות: נשים לב שחיבור של שני המספרים המוזרים האלה נותן לנו -1 , שבייצוג בינארי הוא 1111111111111111

תרגום VM ל-JACK:

- אם ישר אחרי ההכרזה של הפונקציה, יש לנו את השורות:

```
push argument 0
pop pointer 0
```

זה אומר שהפונקציה היא מתודה.

- אם ישר אחרי ההכרזה יש לנו

```
push constant n
Call Memory.Alloc 1
pop pointer 0
```

אז מדובר בפונקציה שהיא קונסטרטור, והיא תחזיר *this*. אם אלה השורות היחידות - פשוט נחזיר *this*, זאת תהיה כל פעולת הפונקציה.

- אם אנחנו רואים את הבלוק הבא:

```
pop temp 0
pop pointer 0
push temp 0
pop that 0
```

אז ראינו עכשיו בלוק שמשים את הדבר האחרון בסטאק, למקום המתאים במערך (יש פה מערך!).

- סתם `pop temp 0` יגיע אחרי קריאה לפונקציה שהיא `void`, כדי "לנקות" את הסטאק.
- המספר שבא אחרי שם הפונקציה בהכרזה, אומר לנו כמה משתנים לוקאים יש לה, כלומר על כמה משתנים הכרזנו ככה: `var int x:`
- המספר הכי גדול שיבוא אחרי `this` יגיד לנו כמה משתני `field` יש לאובייקט שאנחנו פועלים עליו (+1), כי האינדקסים יתחילו מ(0)
- כל מספר שיבוא אחרי `argument` יגיד לנו כמה ארגומנטים הפונקציה קיבלה. אם מדובר במתודה אז זה ממש המספר, אם לא - אז +1 (כי ממש נקבל את `arg 0` והוא לא יהיה אוטומטית שווה לאובייקט)
- פקודת `push` שאחריה פקודת `pop` - זה בעצם השמה של מה שהכנסנו למחסנית ב`push`, למה שנמצא בכתובת שאליה הוצאנו ב`pop` (לא לשכוח לכתוב בג'ק את `let` שלפני ההשמה.)
- אם יש רצף פקודות של `push`, אחריהן רצף פקודות אריתמטיות ואז תווית שהפואנטה שלה היא `IF-TRUE` (אפשר לכתוב אותה איך שהקומפיילר ירצה) - זה מיתרגם ל(`if()` כשבתוך הסוגריים הביטוי שיוצא מכל פקודות ה`push` והאריתמטיקה שראינו.
- מה שבא אחרי תווית עם הפואנטה `IF-FALSE`, יצטרך להיכתב בג'ק בתוך `.else{}`.

- מה שעושים לו push בשורה לפני return יהיה הערך החזרה של הפונקציה, לכתוב גם אחרי return בג'ק וגם את הסוג בהצהרה של הפונקציה בהתחלה
- לא לשכוח ; בסוף כל שורה ושכל הפונקציה צריכה להיות תחת מחלקה גדולה יותר (את השם שלה נבין מהשם של הפונקציה).

תרגום מ-JACK ל-VM:

- לשים לב ששם הפונקציה בהצהרה צריך להכיל גם את שם המחלקה, לא משנה איזה סוג פונקציה היא, עדיין נכתוב "function"

- המספר שבא אחרי שם הפונקציה הוא מספר המשתנים הלוקאלים שהוכרזו בתוך הפונקציה.

- השמה (פקודה שמתחילה בlet) מתבצעת ע"י

push In

pop out

– כאשר In וout הם הכתובות של המשתנים שמעורבים בהשמה:

* אם הם משתנים לוקאלים, אז local

* אם התקבלו כארגומנט לפונקציה\הם האובייקט אז argument (לשים לב לאינדקסים במתודות!)

* אם הם משתנה של האובייקט אז this

* אם משתנה של המחלקה עצמה אז static

- תנאי while: (*if* יושלם בהמשך)

– נכתוב תווית (לא לשכוח לכתוב label לפני השם שלה) של "המשך" (while_true_0 נגיד)

– נכתוב push ופעולות אריתמטיות\השוואה לפי קוד הג'ק

– נכתוב not

– נכתוב if-goto END

– ואז נכתוב את מה שבתוך התנאי\לולאה עצמו

– אחריו: נכתוב goto TRUE (כדי שהלולאה תמשיך, היא תמשיך בעצם לבדיקת התנאי שלה של האם היא ממשיכה)

– לאחר מכן נכתוב את התווית (לא לשכוח לכתוב label לפני השם שלה) של "סוף" END

– נמשיך בכתיבת הקוד שאחרי הלולאה.

- השמה למקום כלשהו במערך:

– נוציא את המיקום של תחילת המערך ל pointer

– נקמפל את הביטוי שנרצה לשים

– נוציא אותו אל x, that, כשהביטוי המקורי היה something = arr[x]

- לפני הריטרן: חייב להיות push לסטאק. אם הפונקציה מחזירה משהו, אז לא לשכוח לעשות לו push ואם היא לא - נעשה push constant 0.

אם היא לא, וקימפלנו **קריאה** לפונקציה ולא רק את הפונקציה עצמה, אז נזכור שאחרי הקריאה לה צריך לעשות pop temp 0, כדי להעיק מהמחסנית את הערך המיותר ששמנו.